Agentic AI Architectures with Patterns, Frameworks and MCP

Design Multi-Agent systems into Enterprise Apps using AI Agents
Agentic Workflows, Design Patterns, MCP, Agentic Frameworks

Design Agentic RAG apps for real-world EShop Microservices



Pradeep Kumar T

Agentic AI Architectures w/ Patterns & Principles

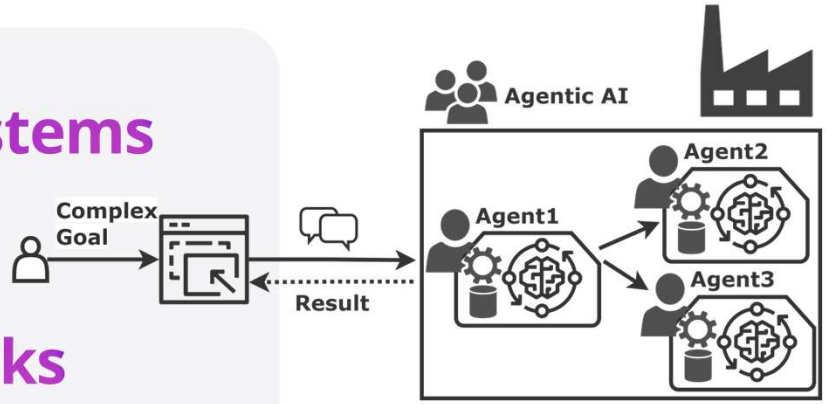
Design and Integrate Multi-Agent systems

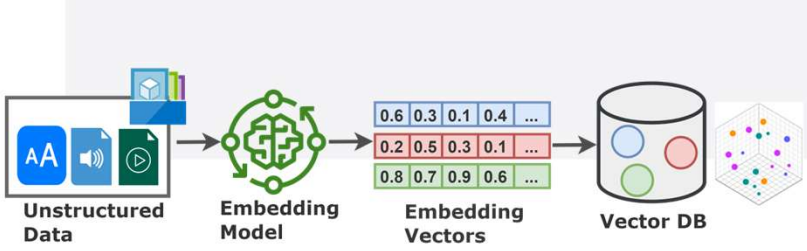
AI Agents, Agentic Workflows

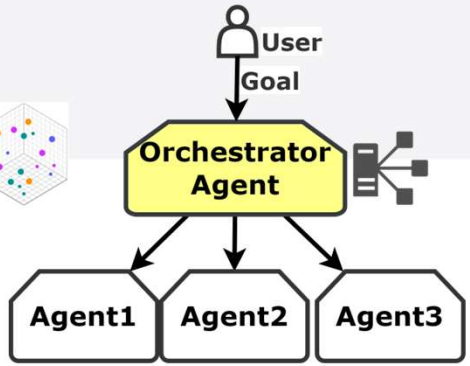
Agentic Design Patterns & Frameworks

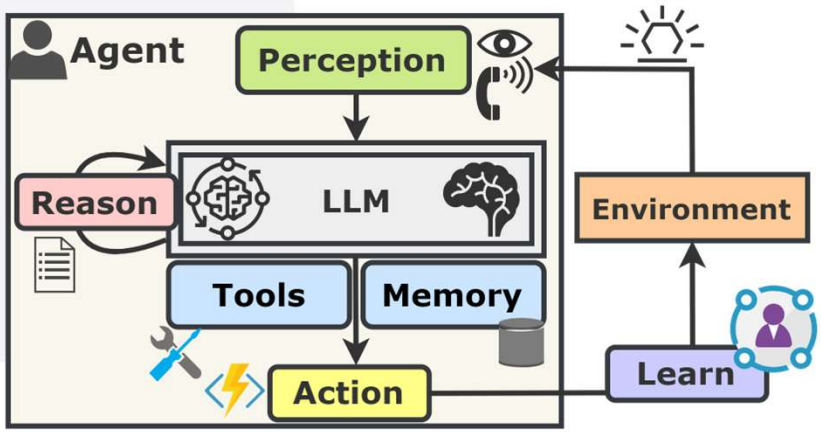
MCP, Agentic RAG and ADLC

Agentic EShop Microservices







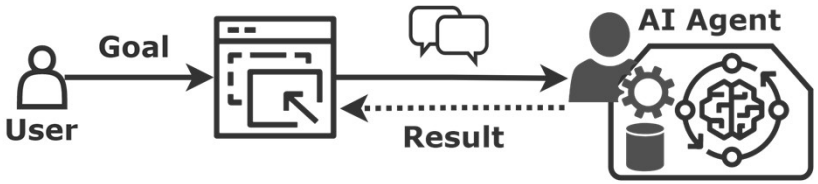


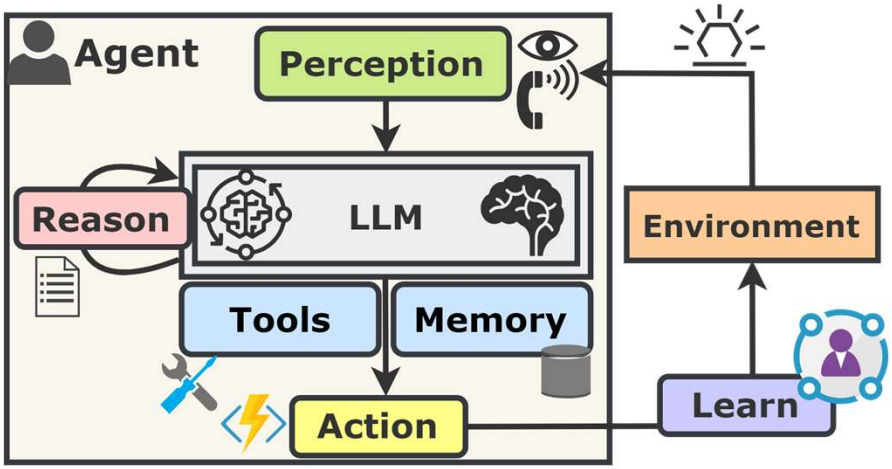
Part 1: The Foundations of the Agent

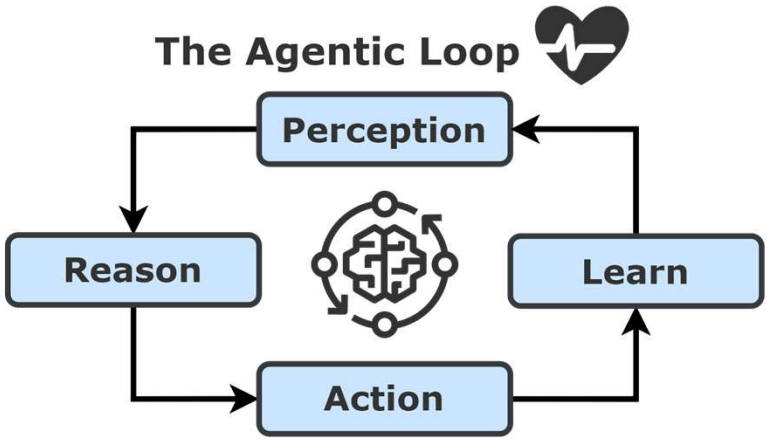
What are AI Agents? (Agent vs. Agentic AI)

Components of an Agent

The Agentic Loop (Perception, Reasoning, Action, Learning)







Part 2: Architecture, Patterns & Frameworks

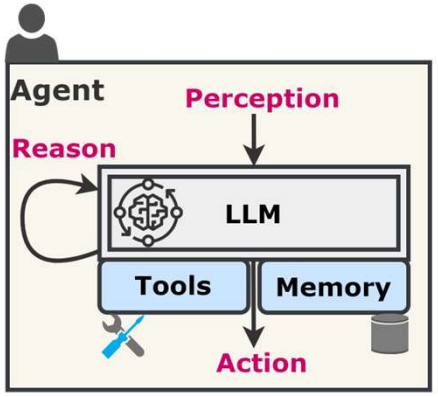
Agentic Architecture: How it works at a system level

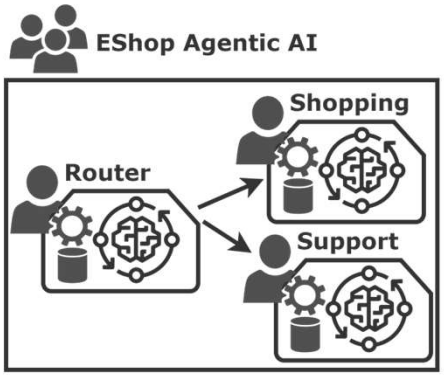
Multi-Agent Collaboration: How agents work together

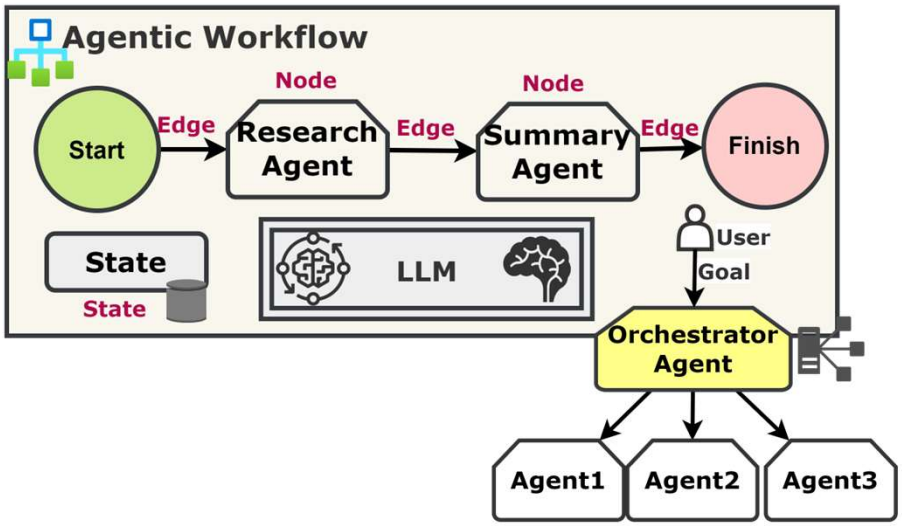
Agentic Workflows: Orchestration vs. Choreography

LangChain, LlamaIndex, Microsoft Agent Framework

Design Patterns: The "Gang of Four" for Agents

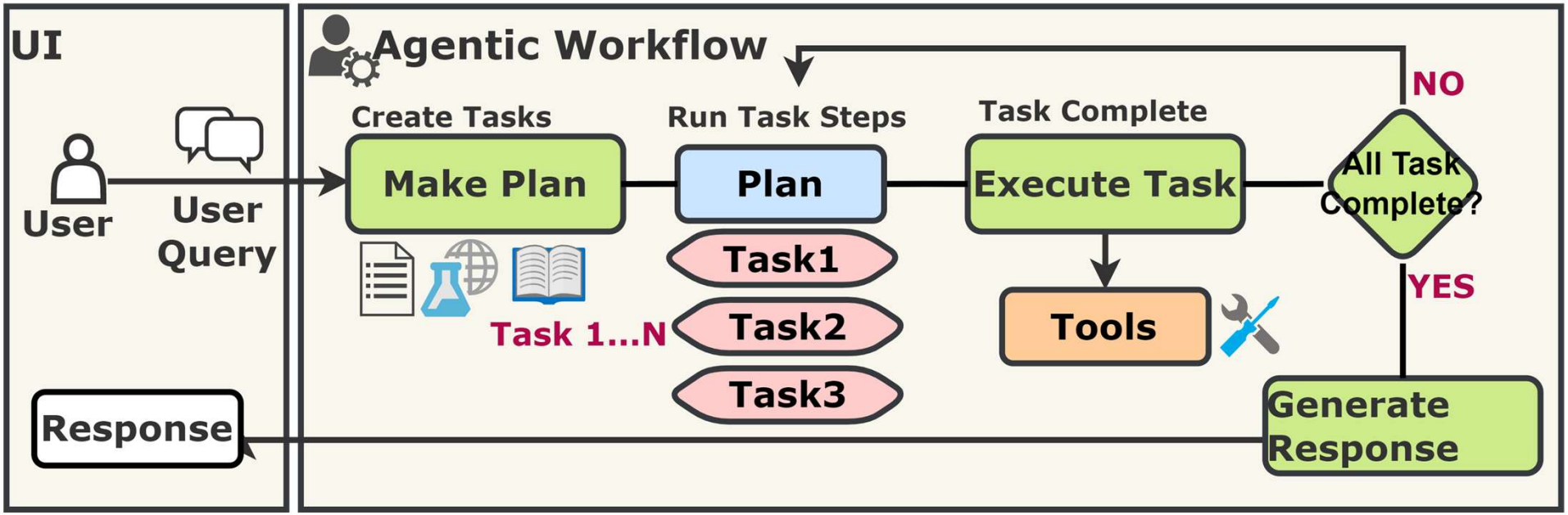




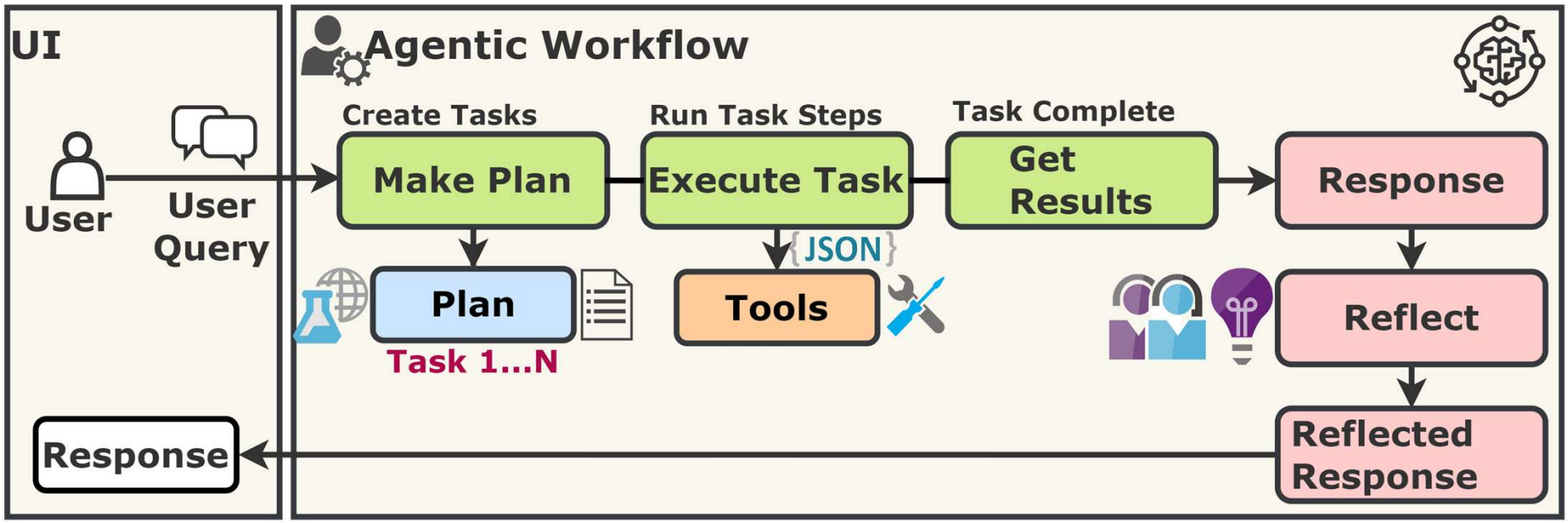




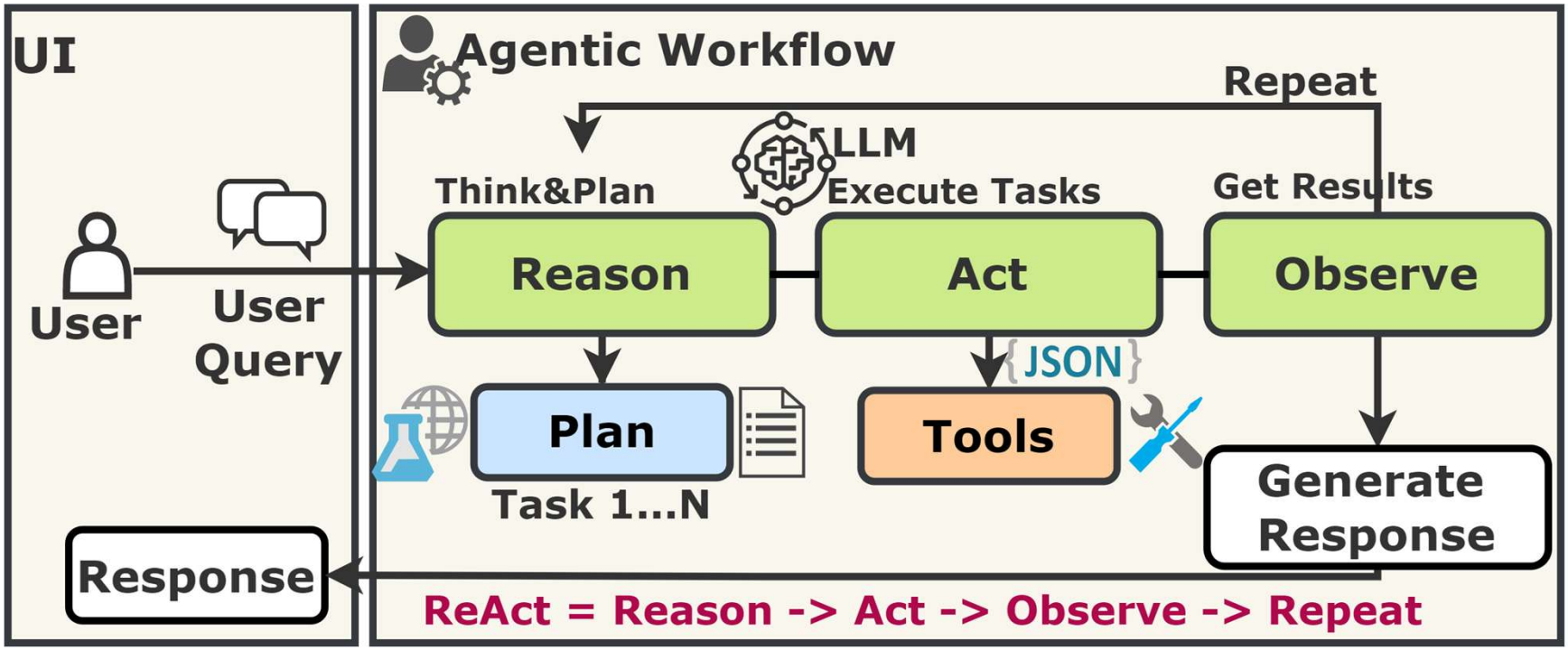
The "Planning" Pattern



The "Reflection" Pattern (Metacognition)



The "ReAct" Compound Pattern



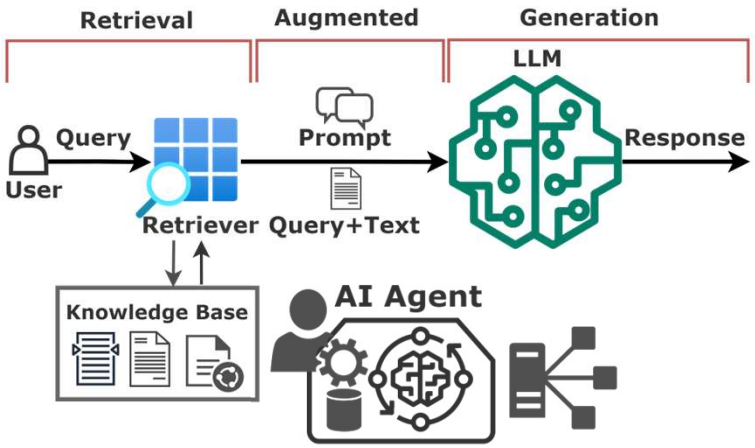
Part 3: Enterprise Integration & Advanced Techniques

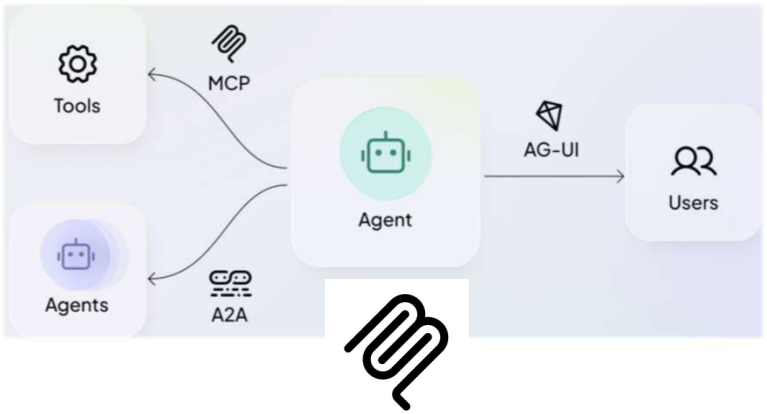
Agentic RAG: Advanced reasoning over data

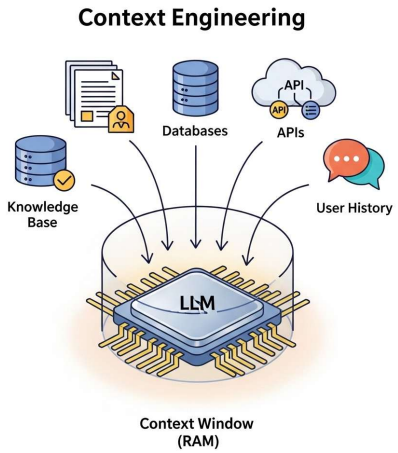
Protocols: ACP, A2A and the critical Model-Context Protocol (MCP)

Step-by-Step MCP: Building servers and integrating APIs

Context Engineering: The new "Prompt Engineering"







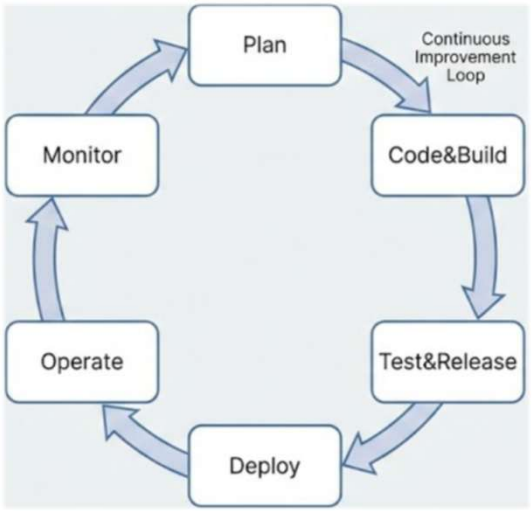
Part 4: ADLC Lifecycle & The Capstone Project

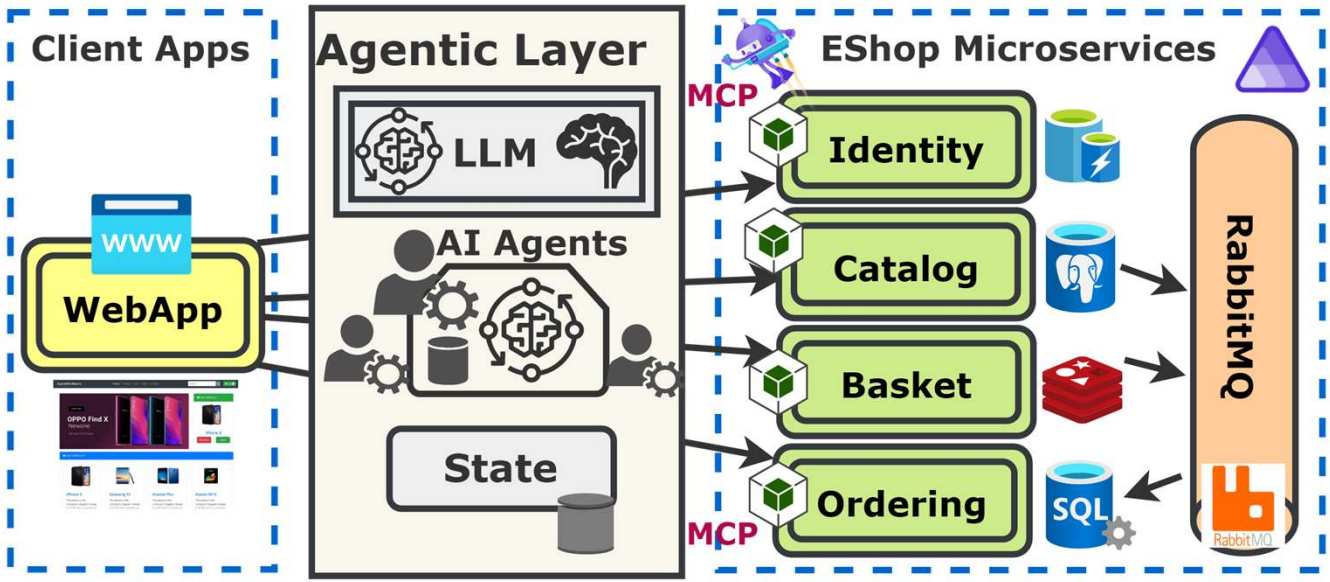


The ADLC: The Agent Development Lifecycle

Enterprise Development: Security, Governance, Evaluation, Ops

FINAL CAPSTONE: Design the E-Shop Agentic Layer



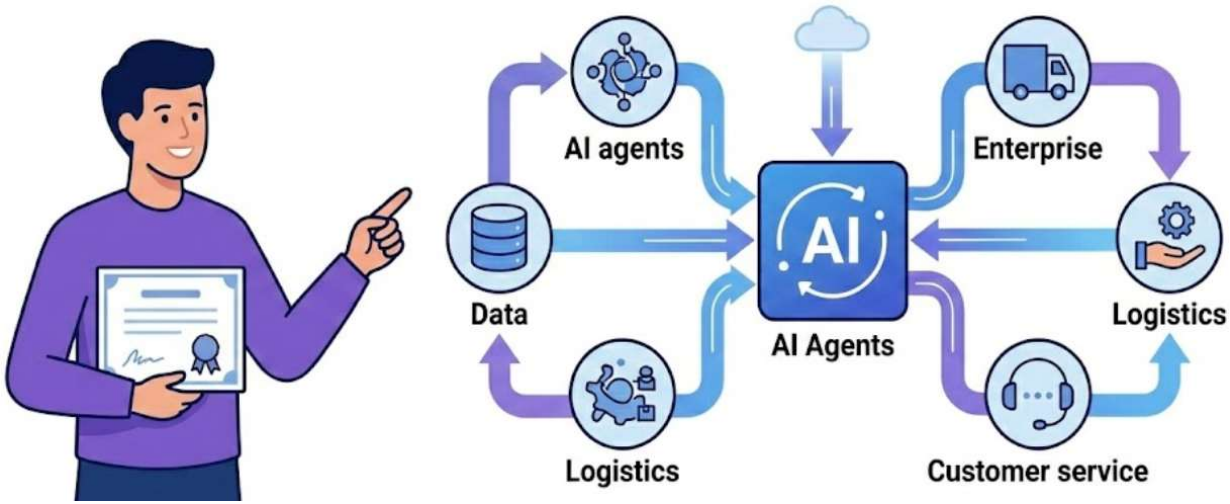


Let's Build the Future of Software

From Theory to Practice

From Developer to Agentic Architect

Let's get started !



Prerequisites – What You Should Know

Basic Programming Skills – Familiarity with Python, .NET or Node.js

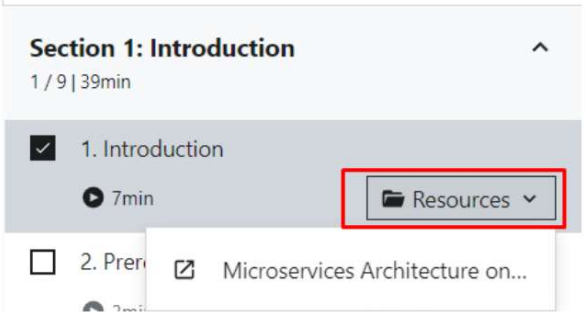
Understanding of APIs – Knowledge of API keys and HTTP requests

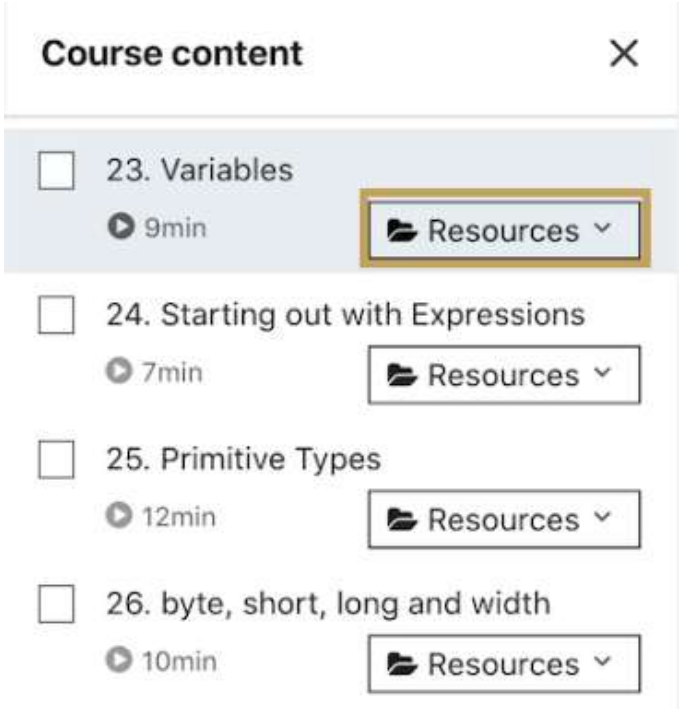
Basic Software Architecture Skills

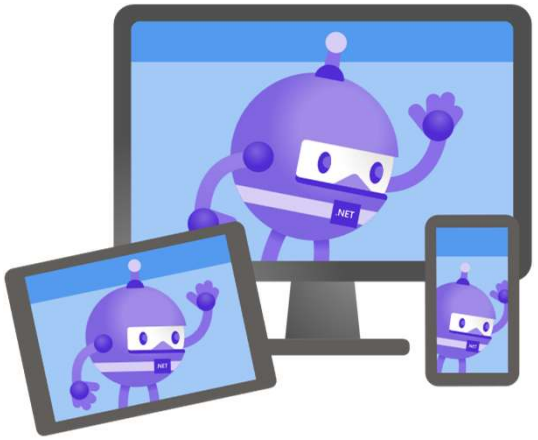


Course Slides and All Resources

Download Course Slides and **All Resources** from **This Lecture Resources**



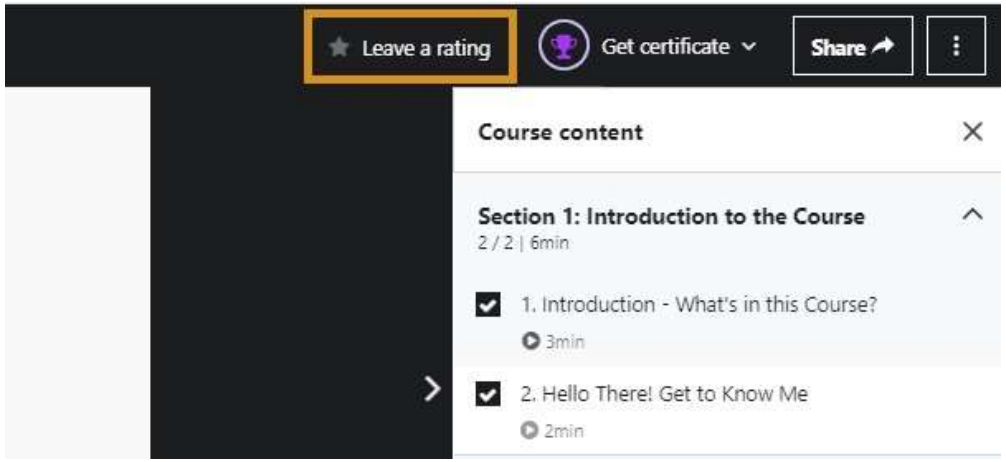


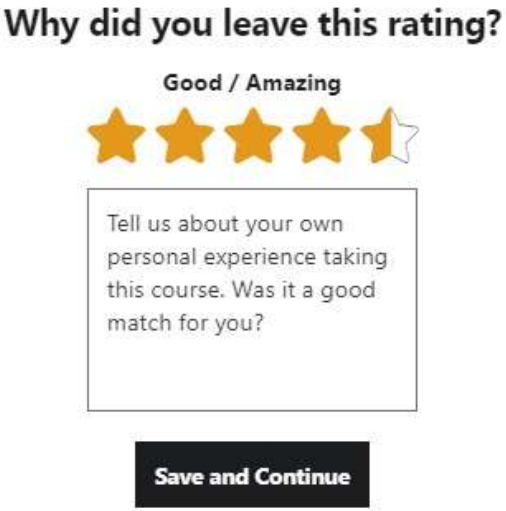


Your Feedback is Invaluable! (Please Review!)

Your Review Powers This Course & Future Content!

Please take a moment to leave a 5-star review and a comment





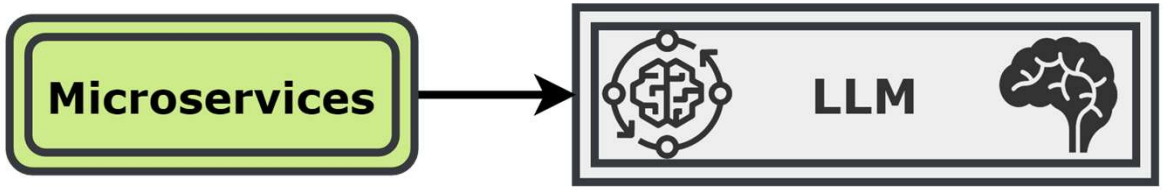


Designing the Agentic Layer for an Enterprise E-Shop

As **architects**, we need to **solve enterprise-scale problems**

Build a comprehensive **Agentic AI Layer** on top of a **real-world E-Shop Microservices**

Standard, **passive e-commerce** platform and **transform** it **into** an **active**, **intelligent**, **autonomous system**



The "Before" State: Passive Microservices

Identity Service: Auth & User Profiles

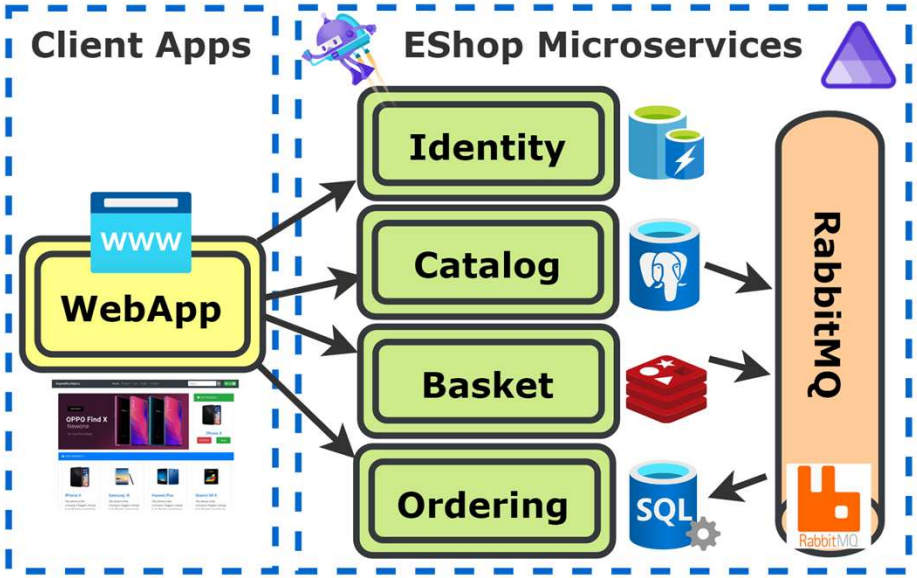
Catalog Service: Product Data & Search

Basket Service: Shopping Cart Management

Ordering Service: Order Placement & History

The Problem: The user has to do all the work (search, click, filter, checkout)

The **burden** of **'reasoning'** is on the **human**



The "After" State: The Agentic Layer

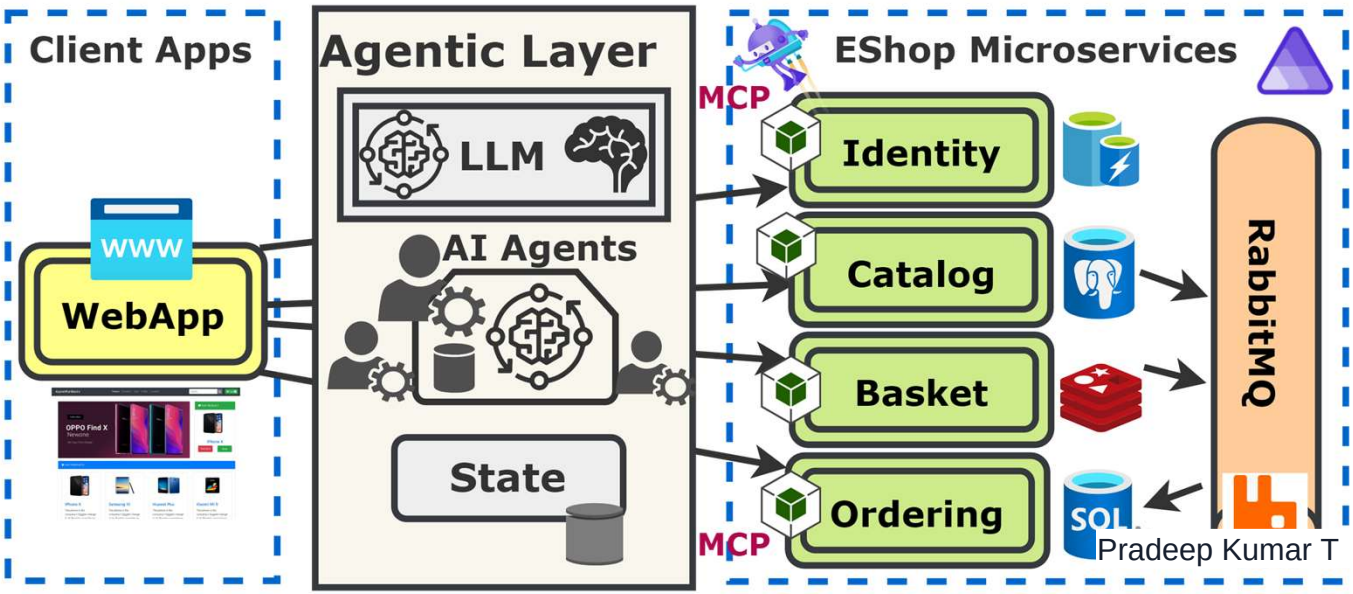
A New Architectural Layer: Sitting between the UI and the Backend

Powered by MCP: Connecting Agents to Microservices securely

Powered by Context: Using User History to personalize actions

Powered by Collaboration: Multiple agents working together

I need a pair of running shoes for my marathon next month, under $150. If you find good ones, buy them

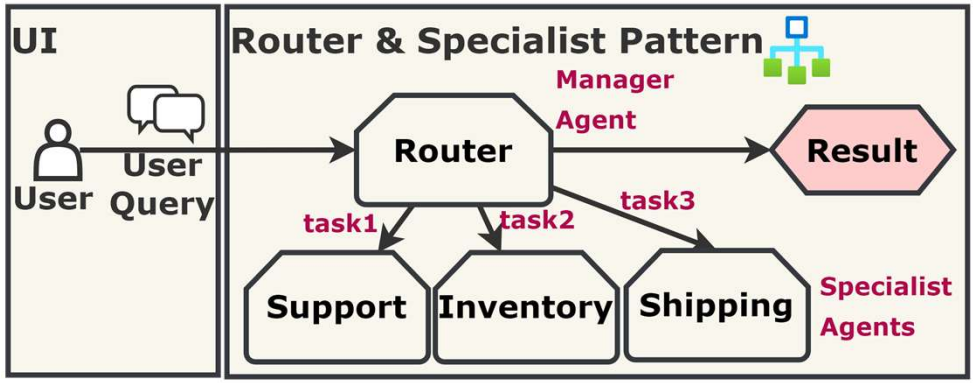


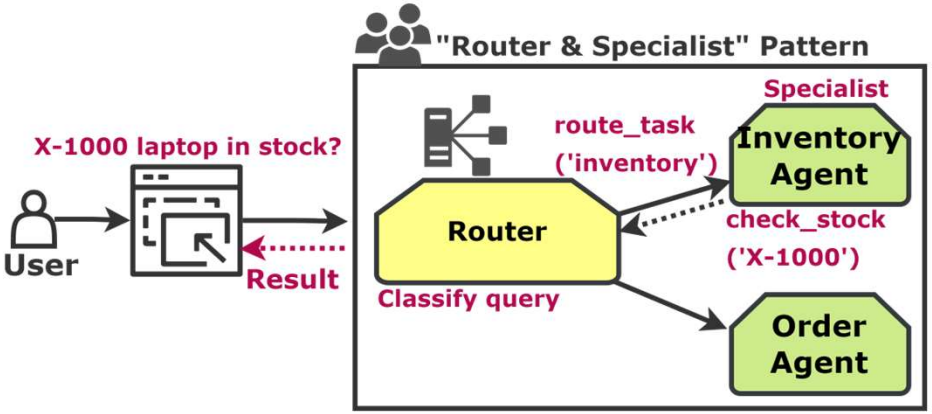
Use Case 1: The Shopping Experience (Router & RAG)

Pattern: Router & Specialist
A "Router Agent" classifies user intent

Pattern: Agentic RAG
A "Catalog Agent" actively researches products using Advanced RAG

Pattern: Context Engineering
The agent injects the user's shoe size and brand preferences from the Identity Service automatically



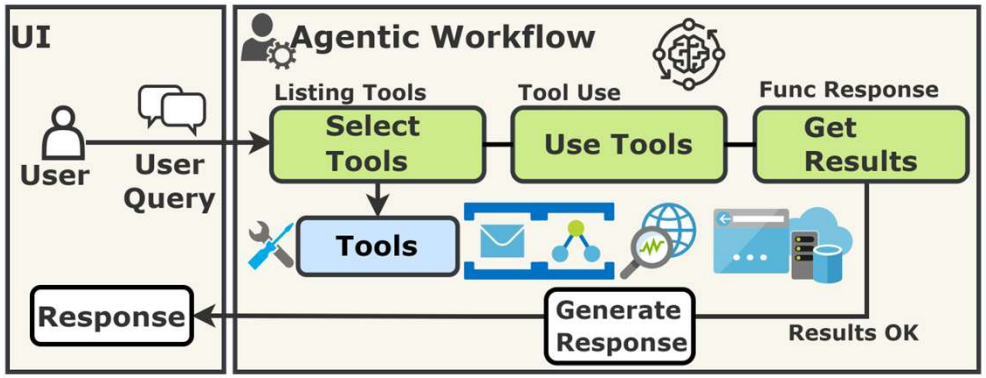


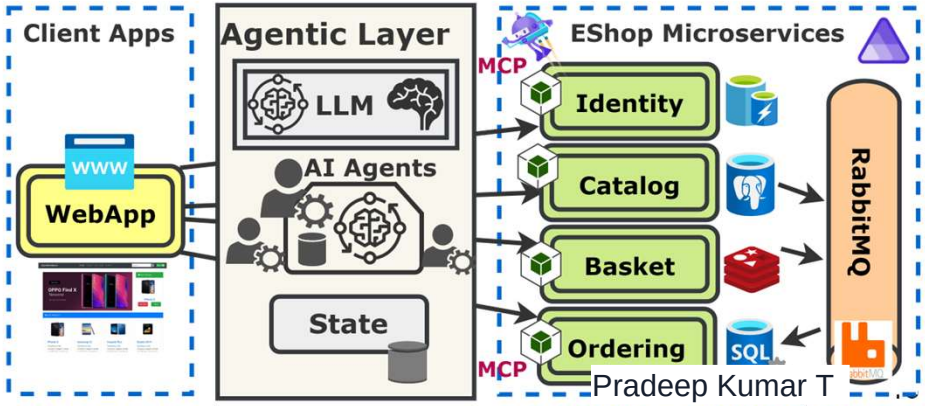
Use Case 2: Autonomous Checkout (Tool Use & Safety)

Pattern: Tool Use
The Agent calls the Basket API and Ordering API directly

Protocol: MCP (Model Context Protocol)
We will design MCP Servers to expose our microservices to the agents securely

Pattern: Human-in-the-Loop (HITL)
Safety Guardrail: If the purchase is over $100, the agent must pause and ask for user confirmation



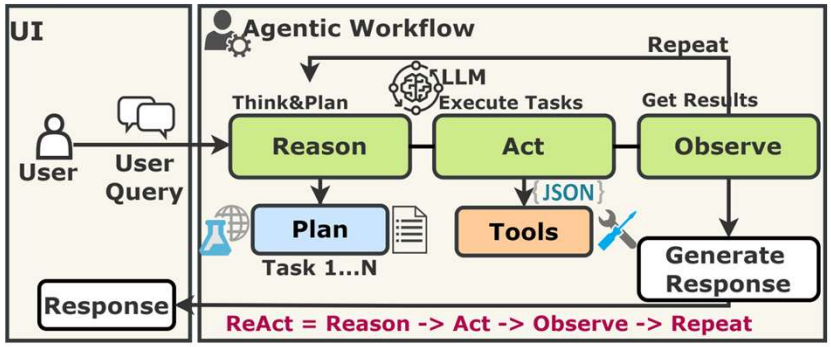


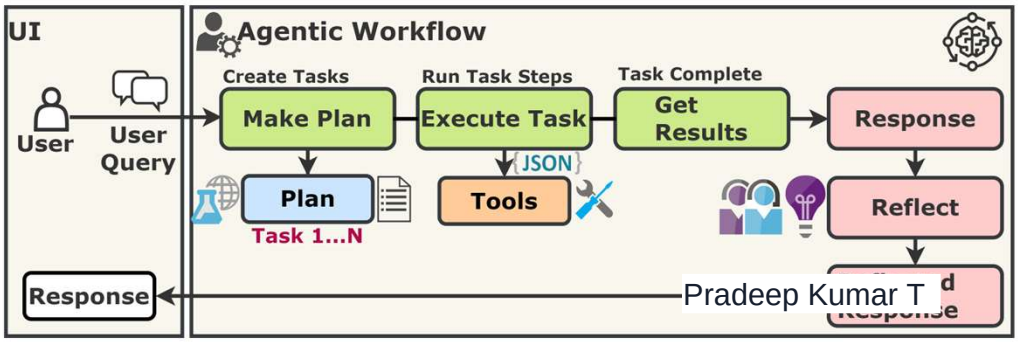
Use Case 3: Customer Support (Collaboration & ReAct)

Pattern: ReAct Compound Pattern
Agent loops: Reason (Check status) -> Act (Call API) -> Reason (Analyze delay) -> Act (Reply)

Pattern: Reflection (Metacognition)
The agent critiques its own draft response to ensure it is polite and empathetic before sending

Pattern: The Handoff
If the agent fails, it performs a seamless Handoff to a human support representative



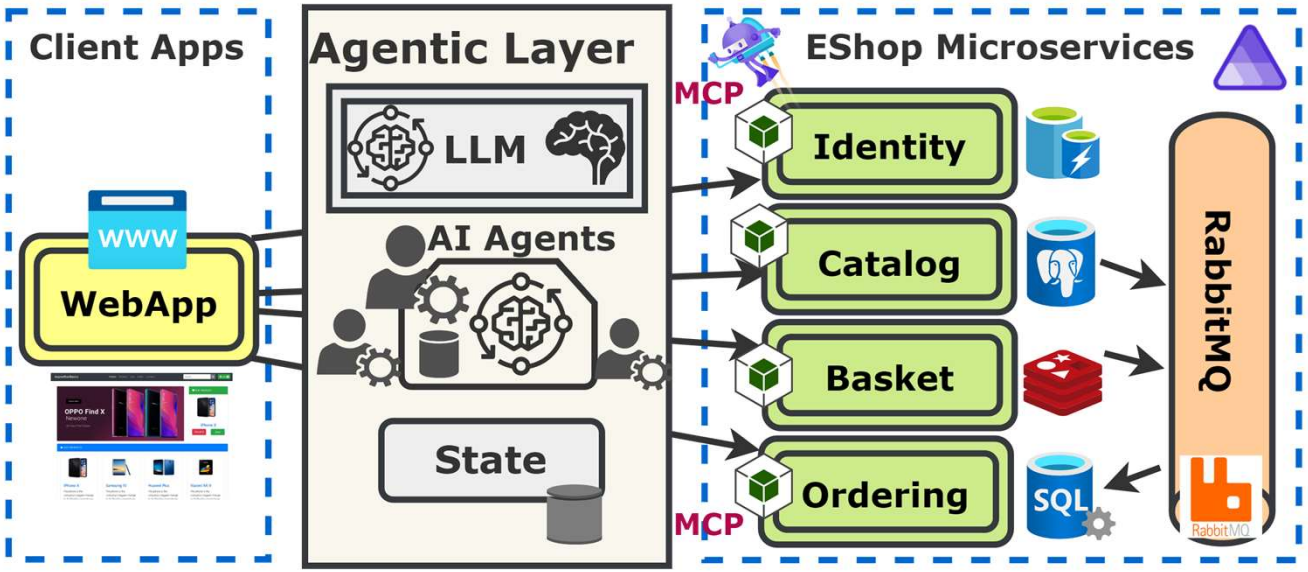


Your Goal: The Architectural Blueprint

Design Document: A complete architecture definition

Diagrams: System interactions, MCP integrations and Agentic Workflows

ADLC Plan: How we will Evaluate, Monitor (AgentOps) and Secure this system

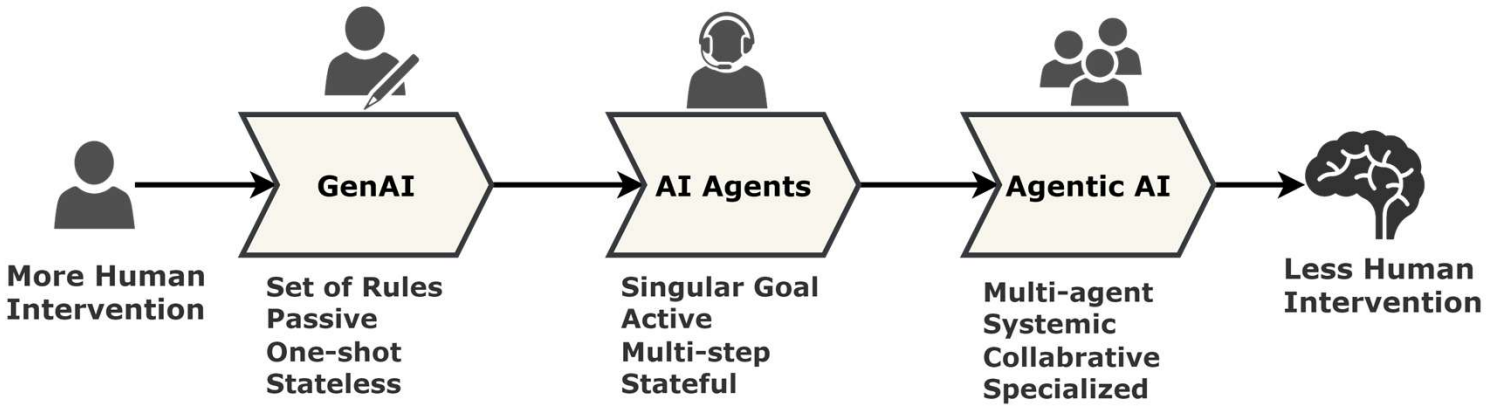


The Evolution: From Generative AI to Agentic AI

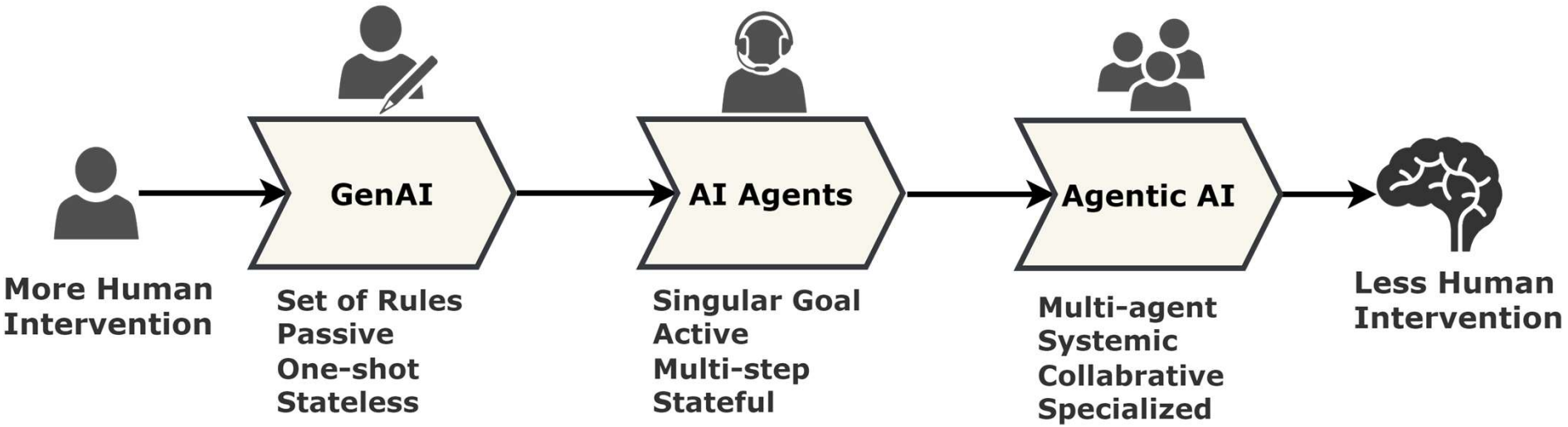
Moving from **simple Generative AI** to **true Agentic AI**

Now, we must move from the **'micro'** to the **'macro'**

We need to **first understand** the clear, **3-phase evolution**



Evolving into Agentic AI



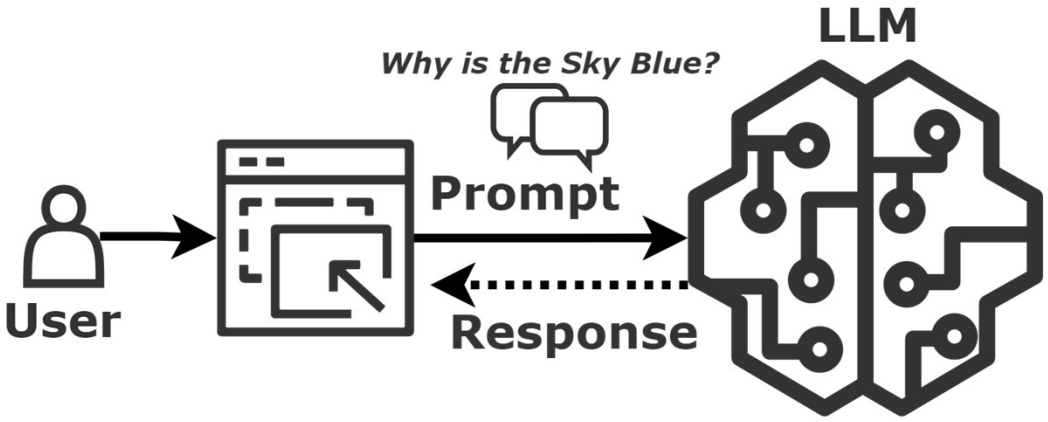
Phase 1: Generative AI (The "Expert Consultant")

One-shot: Its entire lifecycle is a single Request -> Response loop

Passive: It is a reactive system. It only responds; it never initiates

Stateless: It has no memory of past interactions

An **"Expert Consultant"** You ask it for a plan, and it writes you a report



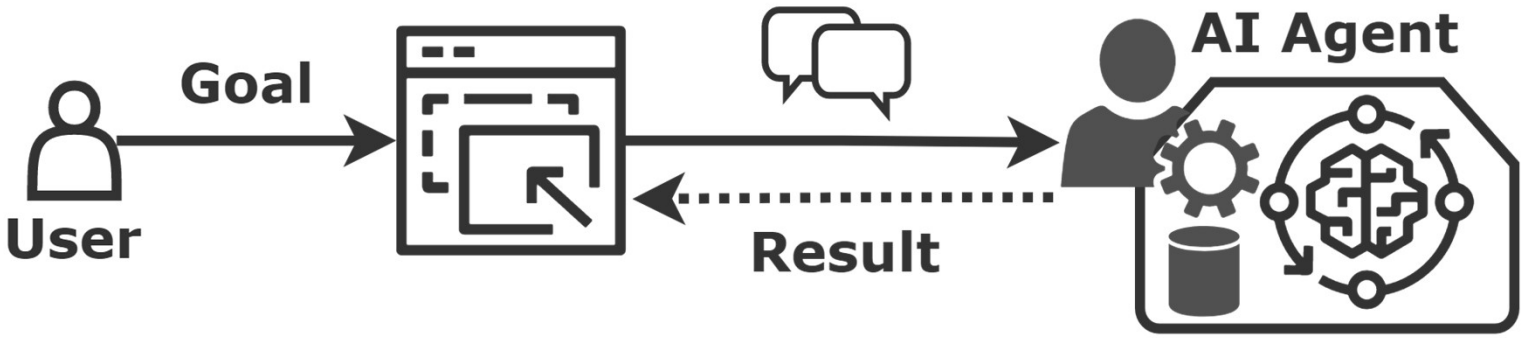
Phase 2: The AI Agent (The "Worker")

We gave the "**Expert Consultant**" **Tools** (its **'hands'**) and **Memory** (its **'notebook'**)

Active: It is proactive. It can initiate actions

Multi-step: It runs a Perceive -> Reason -> Act loop

Stateful: It maintains an internal memory of its plan and its history



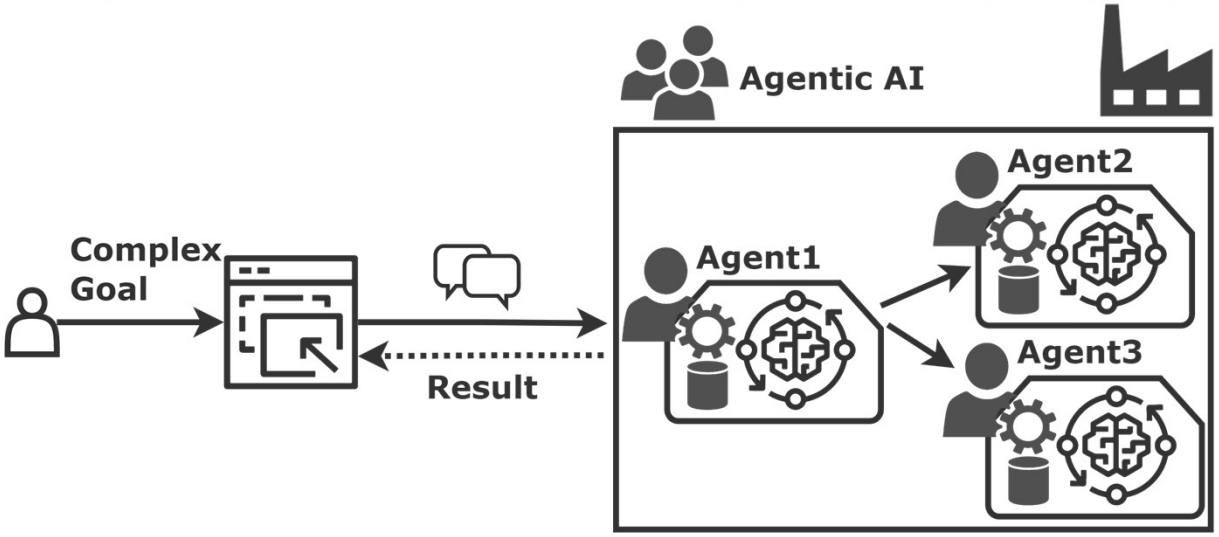
Phase 3: Agentic AI (The "Factory")

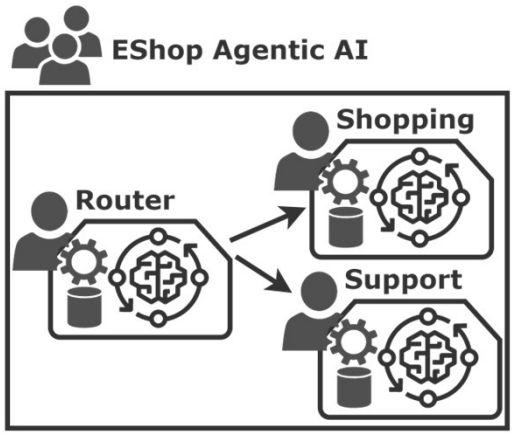
Systemic: It's a system of multiple, collaborating agents

Collaborative: Agents work together, using orchestration and workflows to solve a goal no single agent could

Specialized: Each agent has a unique, defined role

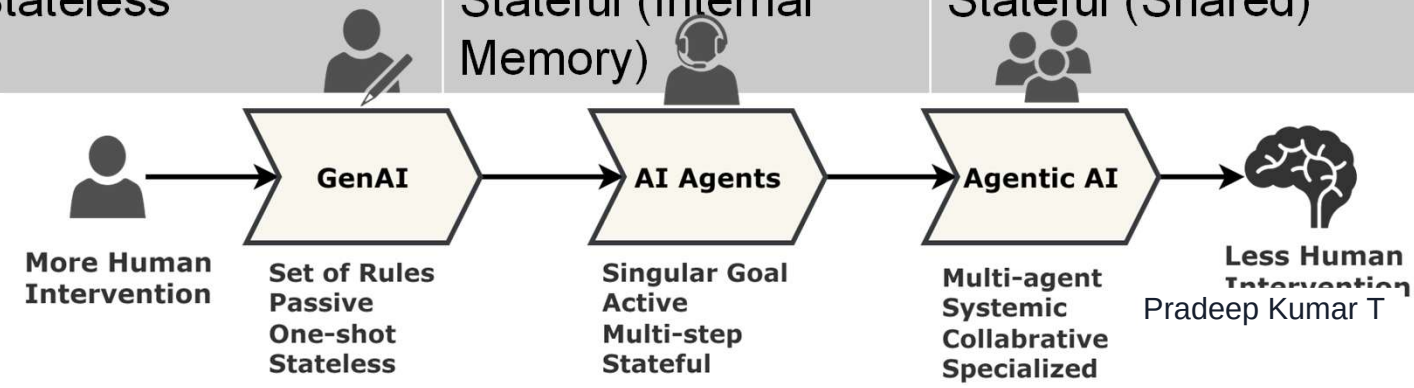
Example: Our E-Shop (Router Agent + Shopping Agent + Support Agent)



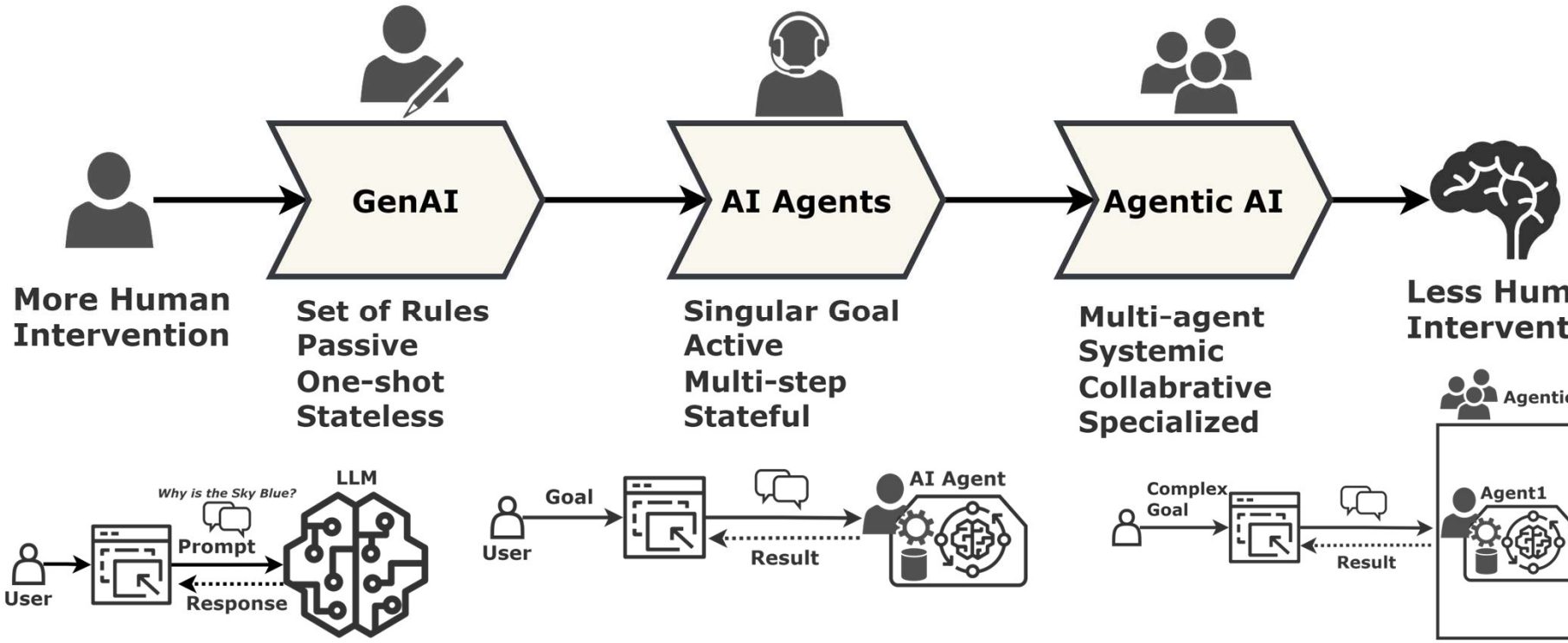


The Full Evolution: A Comparison

Feature	Phase 1: Generative AI	Phase 2: AI Agent	Phase 3: Agentic AI
Analogy	The "Expert Consultant"	The "Worker"	The "Factory" / "Team"
Scope	A single "Brain" (LLM)	A single Component	An entire System
Input	Prompt	Goal	Complex Goal
Action	Passive (Answers)	Active (Executes Tools)	Collaborative (Orchestrates)
State	Stateless	Stateful (Internal Memory)	Stateful (Shared)



Evolving into Agentic AI



What are AI Agents ?

How AI Agents Work

Key Components of AI Agent Architecture : LLMs, Tools, Memory

When to Use AI Agents (And When Not To)

Use Cases for AI Agents

The Hard Parts: Challenges and Limitations of AI Agents

Pradeep Kumar T



What is an AI Agent? (The Simple Idea)

It’s **not just a tool...**

It's a **"Digital Employee"**

You **give** it a **Goal**

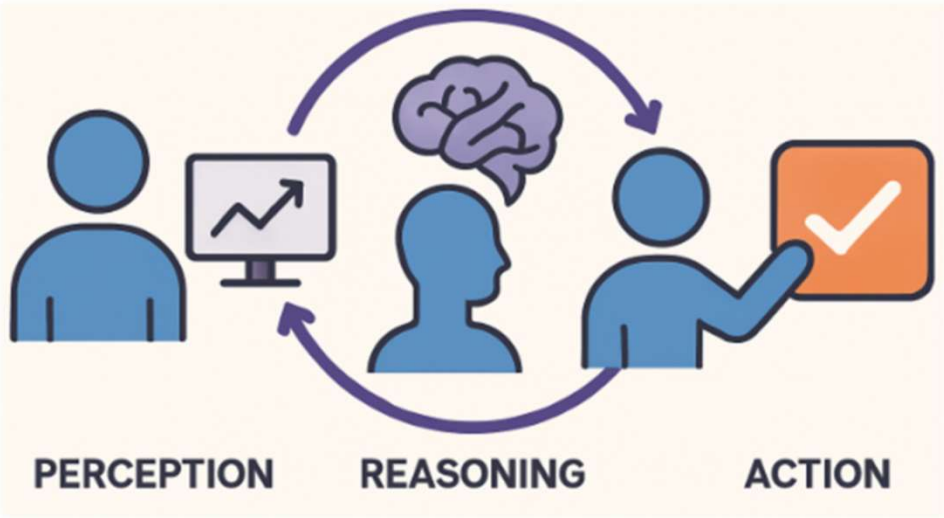
It **has Skills** (**Tools** / **APIs**)

It **Works Autonomously** to **achieve the goal**



The Formal Definition of AI Agent (The Agentic Loop)

1. Perceive its environment
2. Reason to make decisions
3. Act (autonomously) to achieve its goals

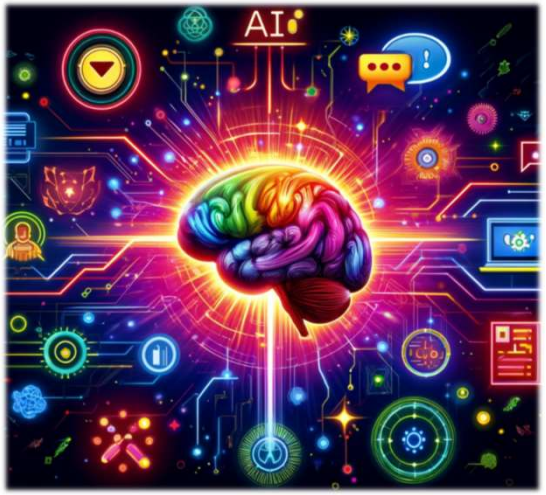


The 4 Key Principles of an Agent

1. Goal-Driven
2. Autonomy
3. Reactivity
4. Pro-activeness







Principle 1: Goal-Driven

An **agent** is **defined** by **its objective**, **not** its **instructions**

Script: Run this report at 5 PM

Agent: Ensure the executive team has the fresh sales data they need, every morning





Principle 2: Autonomy

The **ability** to **operate without direct**, **step-by-step human intervention**

It **makes** its **own decisions** about **how to achieve its goal**

You **don't tell a developer** which **specific lines** of **code** to type
You tell them **what feature to build**

You **tell it WHAT to do**, **not HOW to do it**





Principle 3: Reactivity



The **real world** is **dynamic** and **unpredictable**

The **ability** to **respond** to **changes in the environment**

Example: An API fails

If a script tries to **call an API** and **gets a 500 error**, **what happens**?
It crashes

Agent: Perceives the **error**, **Reasons**



Principle 4: Pro-activeness

The **ability** to **take initiative** to **achieve a goal**

It **doesn't just react** to **problems**; it **anticipates them**



Example: A server monitoring agent

Reactive: Alert! Server disk is 100% full!

Pro-active: Server disk is 90% full. I am pro-actively archiving old logs to prevent a crash



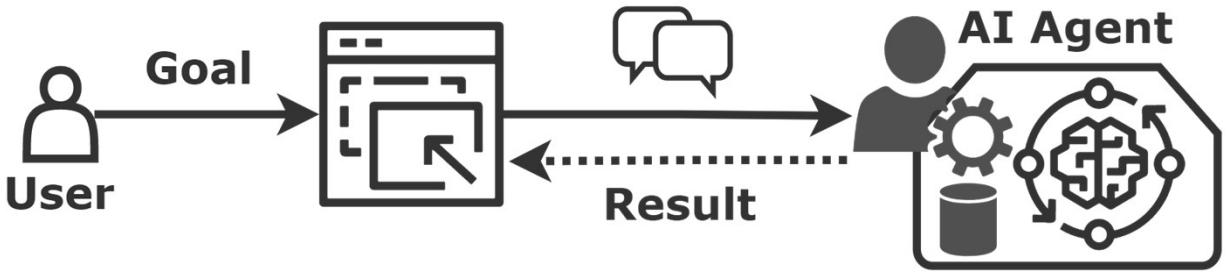
Putting It All Together

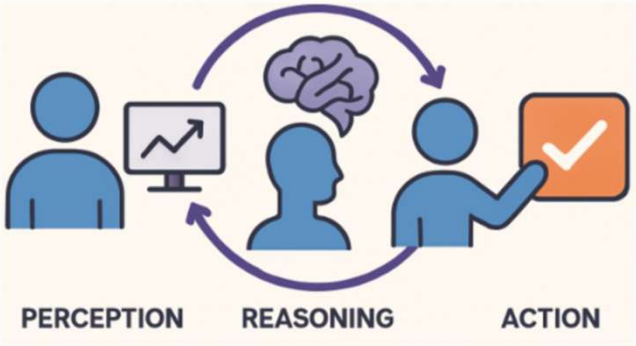
AI Agent = (Goal-Driven + Autonomy + Reactivity + Pro-activeness)

This is **not just a script**

This is **not just an LLM**

It is a **new type of software component** that **can operate on our behalf**

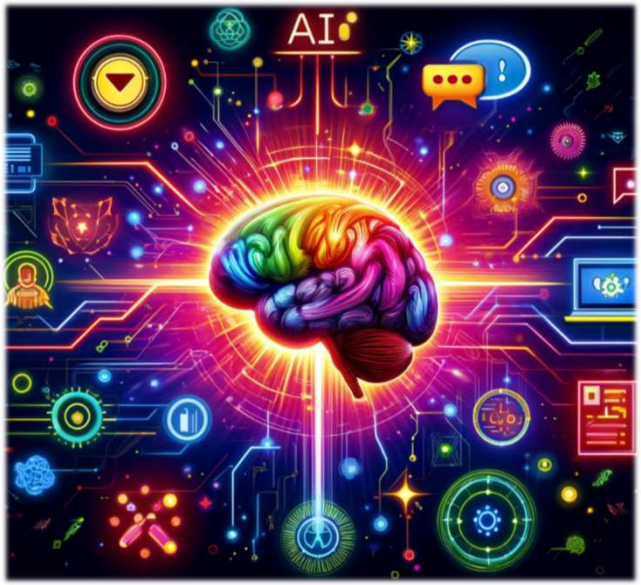


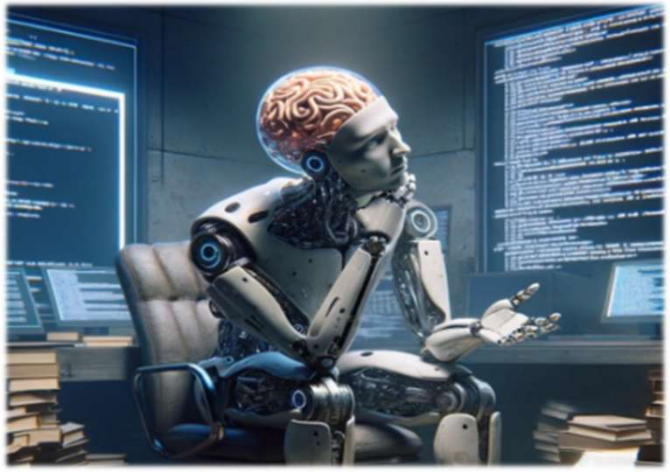


How AI Agents Uses Generative AI ?

Generative AI vs. Agentic AI

The Critical Difference: From Passive Tool to Proactive System





Defining Generative AI: The Reactive System

Role: A Reactive System

Trigger: Waits for a user Prompt

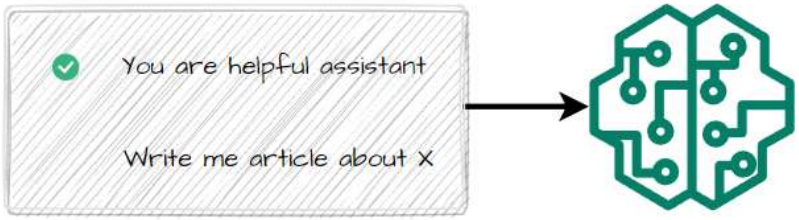
Function: To Generate content

Mechanism: Sophisticated pattern matching

Output: Text, images, code, audio

Key Limit: Its work ends at generation





'Write a story about a journey to Mars'



Defining Agentic AI: The Proactive System

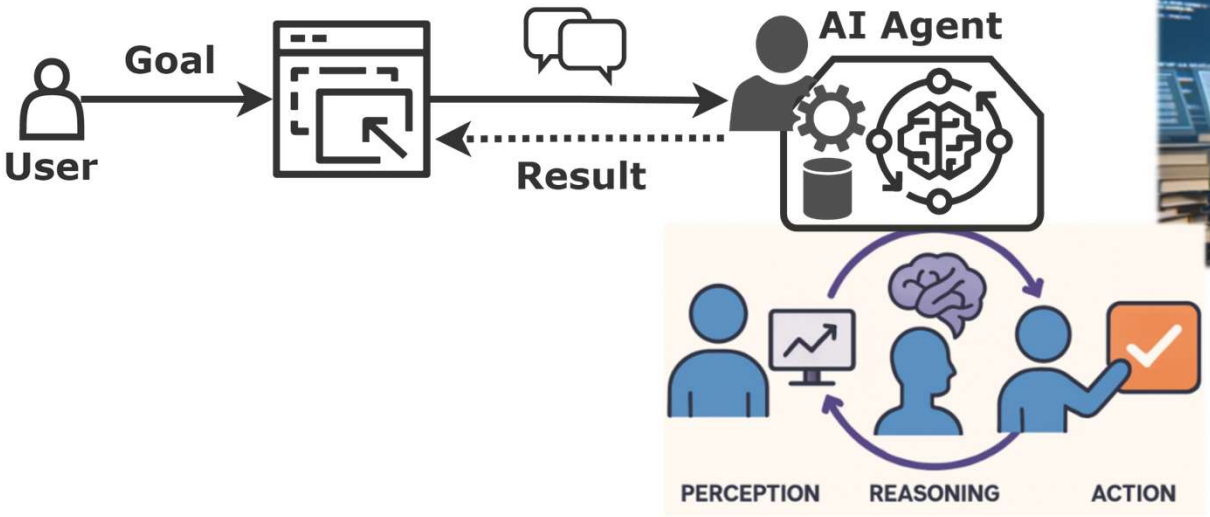
Role: A Proactive System

Trigger: A Goal (often from an initial prompt)

Function: To pursue goals through a series of actions

Mechanism: A continuous "life cycle" or loop

Process: Operates with minimal human intervention





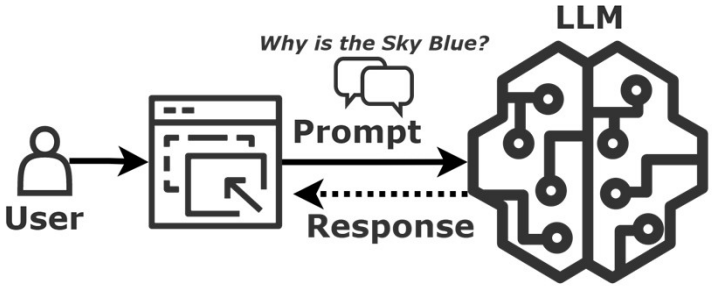
The Common Foundation: The LLM

Share a common foundation: Large Language Models (LLMs)

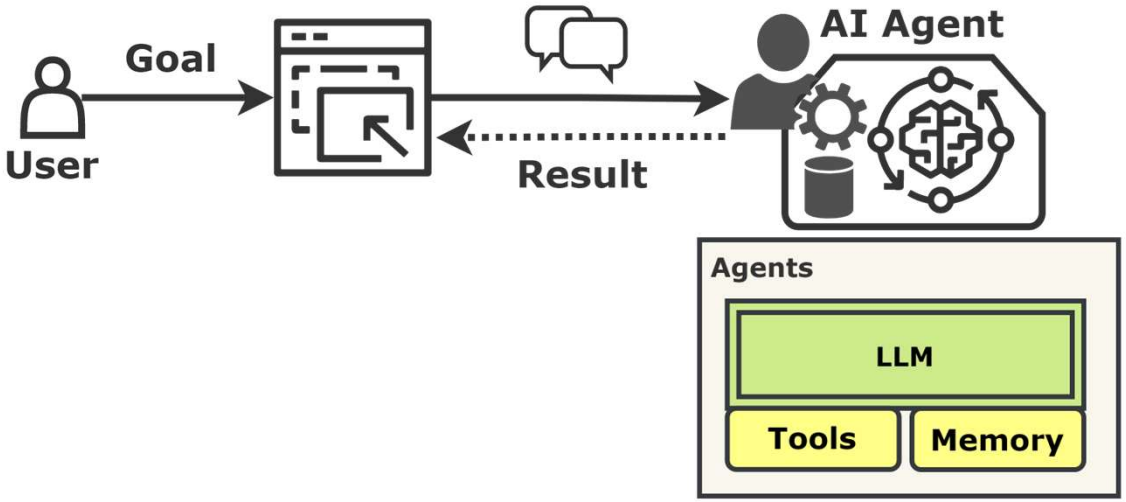
In Generative AI: The LLM is the backbone

In Agentic AI: The LLM provides the **"Reasoning Engine"**

It is **the brain** that **powers** the **agent's decision-making**







Application of Generative AI (The "Curator")

Scenario: A YouTuber creating a video

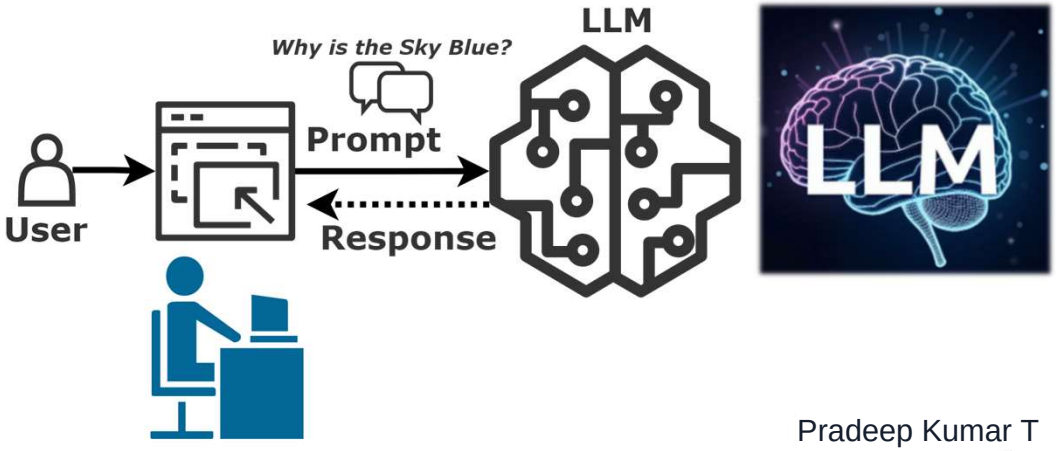
Task: Review scripts, suggest thumbnail concepts, generate background music



The Process:

1. Human writes a script
2. Human feeds script to GenAI -> GenAI suggests edits
3. Human refines prompt -> GenAI suggests thumbnails
4. Human selects concept

At each step, there is a human



Application of Agentic AI (The "Manager")

Scenario: An autonomous Personal Shopping Agent

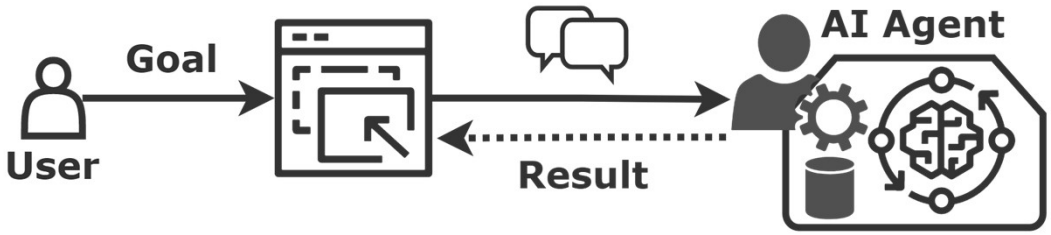
Task: Purchase this specific product for me at the best price

The Process (Autonomous):

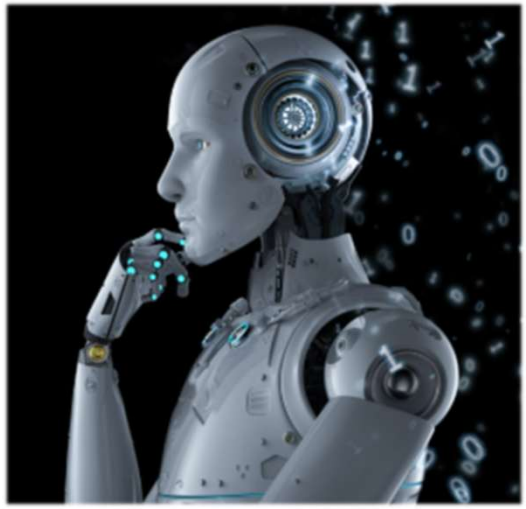
1. Agent actively hunts for availability across platforms
2. Agent monitors price fluctuations in real-time
3. Agent handles the checkout process (using tools)
4. Agent coordinates delivery



Agent do ongoing management and multi-step processes



The Deep Connection: "Thinking" for the Agent



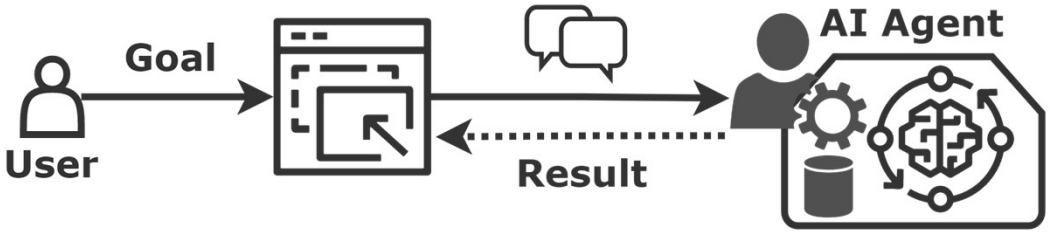
How does **the agent "decide" what to do?**

It **uses** the **LLM's reasoning capability**

This is often called "**Chain of Thought**" (**CoT**) **Reasoning**

The **agent uses** the **GenAI** (**the LLM**) to **think through** problems

It's a **process** where the **agent breaks** a **complex task** into **smaller**





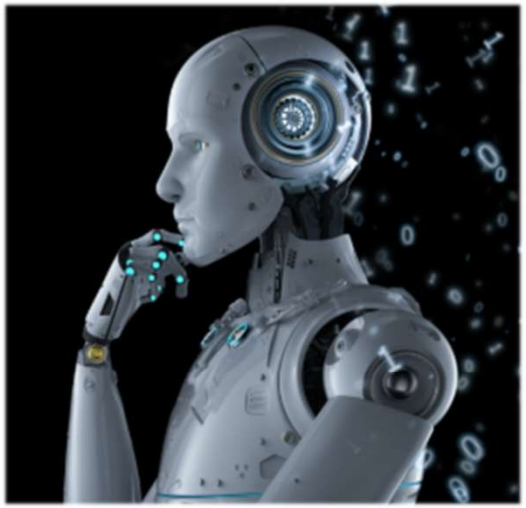
When to Use AI Agents (And When Not To)

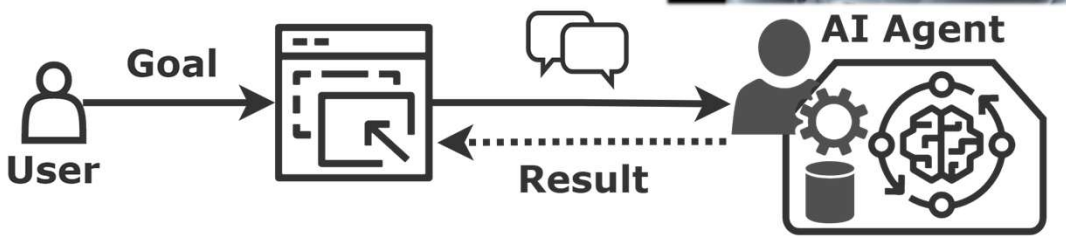
The most **important question**: **When should you use one?**

An **agent** is a **complex**, **powerful** and **expensive component**

Choosing when to **deploy** an **agent versus** a **direct LLM call**







The Core Architectural Choice: Tool vs. Worker



Script/Function: A simple tool (hammer)
Use: For a specific, defined task ("Hit this nail")
Properties: Deterministic, fast, cheap, 100% predictable

AI Agent: An autonomous worker. (carpenter)
Use: For a general, open-ended goal. ("Build me a bookshelf")
Properties: Non-deterministic, slower, expensive, adaptive



Question
Am I just hitting a nail, or am I building a bookshelf?

When NOT to Use an AI Agent (The "Red Flags")

The "Golden Rule": If you can write a function, do that

For **simple**, **deterministic** and **highly-structured tasks**

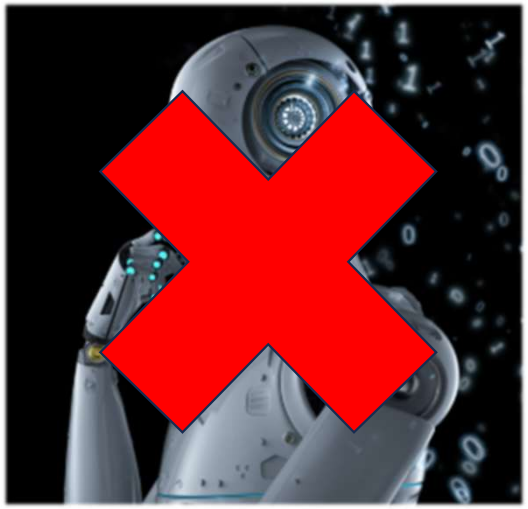
When **100% predictability** and **adherence** to **rules is required**

For **simple information retrieval** (**Basic RAG** is **sufficient**)

When there is no clear goal or **ability to act**







When to Use an AI Agent (The "Green Lights")

Open-Ended & "**Under-Specified**"

A **Multi-Step**, **Multi-Tool** Process

Occurring in a **Dynamic Environment**

Designed for **Improvement Over Time**

A **Conversational**, **Collaborative** Interface



Deep Dive: Open-Ended Problems

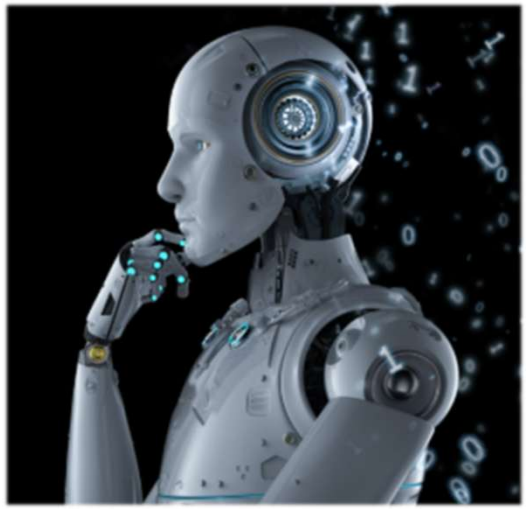
The exact sequence of **steps** is **not known in advance**

Traditional code fails, because the **workflow cannot be hardcoded**

Agent shines because the **LLM** can **determine** the **needed steps** at **runtime**

Example:
Bad (Defined): Convert this file from CSV to JSON (Use a script)

Good (Open-Ended): Analyze this CSV and find the three most important insights for the marketing team



Deep Dive: Multi-Step, Multi-Tool Processes

Task requires a level of **complexity** that **cannot** be **solved** in a **single shot**

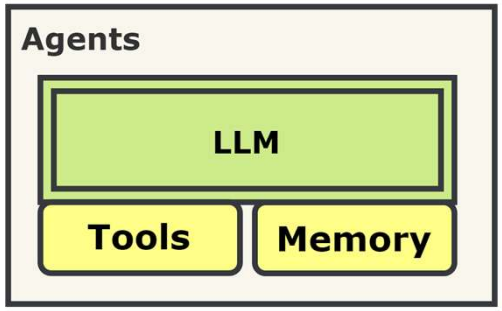
It **must use multiple different tools** (search, then query a DB, then send an email)

It **must hold state** and **memory** over **multiple turns**

Example:
Research assistant agent that must search the web, summarize documents, and then synthesize information from all those sources into a final report





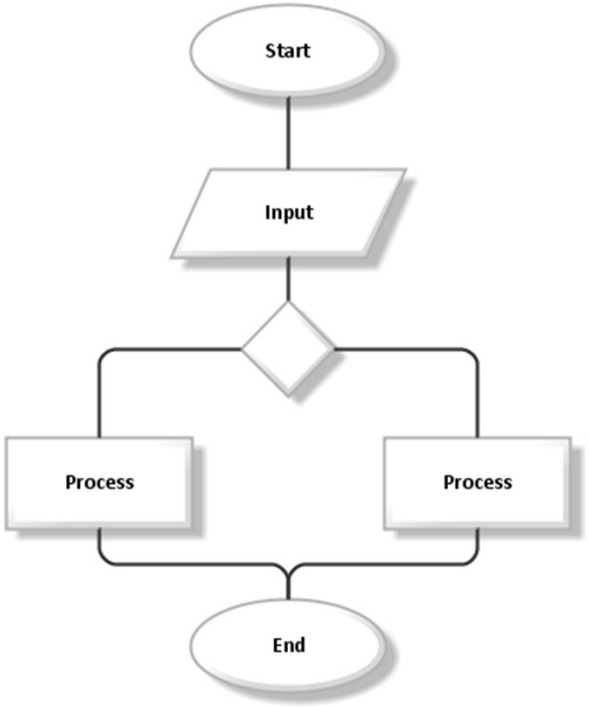


Your Decision Framework

1. Is the task simple & deterministic?
YES: -> Use a Script / Function

2. Is the task complex, but the steps are defined?
YES: -> Use a Workflow

3. Is the task complex, open-ended, & in a dynamic environment?
YES: -> Use an AI Agent

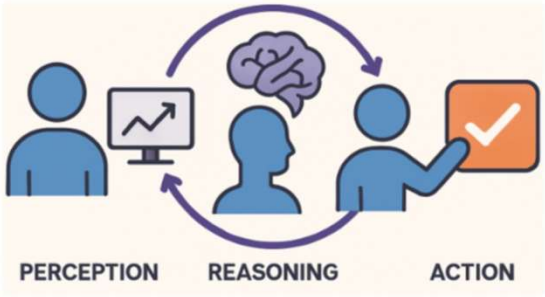


What Makes a Good Use Case?

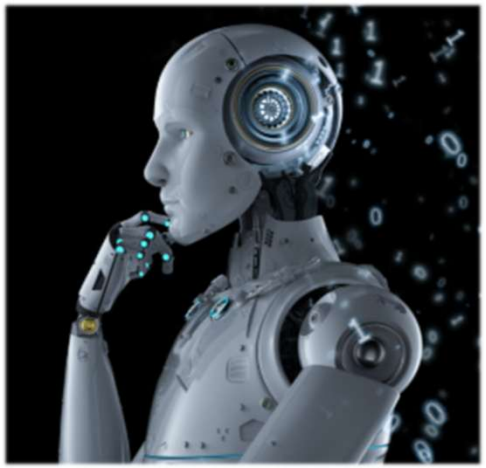
An Open-Ended Goal: Not a specific instruction

Autonomous Action: The agent works on its own

Multi-Step & Multi-Tool: The agent uses multiple tools in a sequence to achieve the goal









Enterprise Automation Use Cases

Enterprise Admin ("The AI Intern")
Goal: Onboard our new engineer, Elara
Action:
Autonomously provisions an account (Tool 1),
sends the welcome email (Tool 2),
and schedules 1:1 meetings (Tool 3)

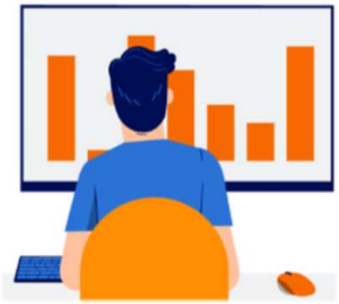


Advanced Customer Support ("The Tier 2 Agent")
Goal: A customer's order is broken. Fix it
Action:
Authenticates the user (Tool 1),
looks up the order (Tool 2),
analyzes their photo of the damage and processes the refund via API (Tool 3)



Technical & Data-Driven Agents Use Cases

Autonomous Data Analysis
Goal: Find the key insights from this new sales CSV
Action:
Writes and executes Python code (Tool 1) to analyze the data, generates plots (Tool 2) and provides a written summary of its findings



Active Research & Content Creation (Agentic RAG)
Goal: Write a report on the future of solar energy
Action:
Plans the report, researches 10 web sources (Tool 1), synthesizes the findings, drafts the report (Tool 2) and then critiques its own work before finalizing



Technical & Data-Driven Agents Use Cases - 2

Software Development ("The AI Pair Programmer")
Goal: Refactor this module to be more efficient
Action:
Analyzes the code, writes the new version (Tool 1),
runs the test suite (Tool 2),
perceives the test failure,
and loops to debug its own code

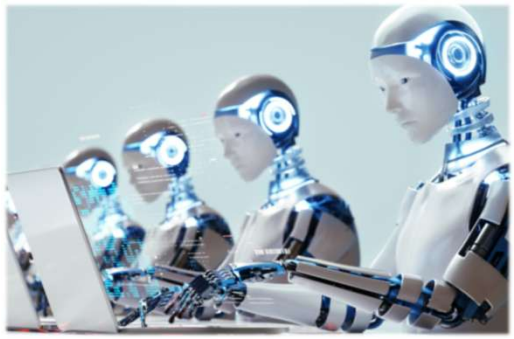


Complex System Integration Use Cases

Internet of Things (IoT) & Agriculture
Goal: Maximize crop yield and minimize water waste
Action:
Perceives data from soil sensors (Tool 1)
and weather APIs (Tool 2),
reasons about the optimal time
and acts by triggering the physical irrigation system (Tool 3)



Disaster Response (Multi-Agent System)
Goal: Coordinate emergency response to the earthquake.
Action:
A Coordinator Agent tasks Specialist Agents to work in parallel:
a Satellite_Agent finds damage,
a Social_Media_Agent finds victims



Business & Scientific Agents

E-commerce & Marketing
Goal: Run a successful flash sale for our new product
Action:
Analyzes customer segments (Tool 1)
drafts personalized emails (Tool 2),
monitors the sale in real-time,
and autonomously adjusts ad spend (Tool 3)



Healthcare & Scientific Research
Goal: Find potential new drug candidates for this protein
Action:
Plans a research workflow,
searches medical databases (Tool 1),
runs molecular simulations (Tool 2),
analyzes the results, and summarizes the top 5 candidates

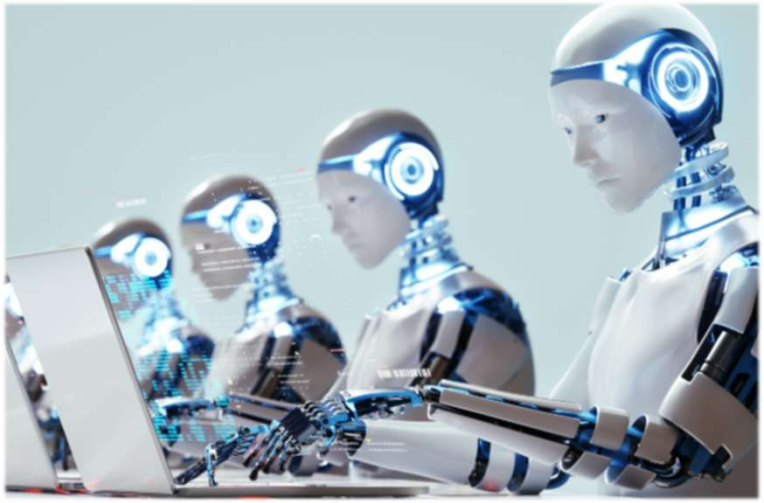


Challenges and Limitations of AI Agents

There is **no such thing** as a **'magic' solution**

With **great power comes significant complexity** and **risk**

Our job is **not just to build** what is possible, but to **build** what is **robust**, **secure** and **reliable**



Challenge 1: Non-Determinism

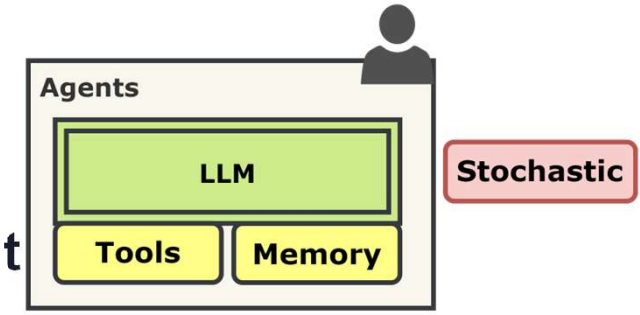
The Problem: Traditional code is deterministic (2 + 2 always equals 4)

LLMs are **non-deterministic** (or "**stochastic**")

Given the **exact same goal**, agent **produce** slightly **different plan** or use slightly **different words** each time

This is not a "bug"; it's a **core feature** of the **underlying model**

How do you **write** a **unit test** for a system that never gives the **exact same output** twice?
How do you **ensure reliability**?



Challenge 2: Security & Guardrails

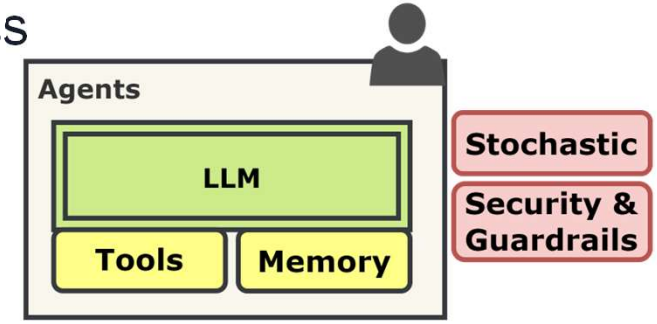
The Problem: We are giving an autonomous system access to real tools (APIs, databases, file systems)

The Risk: The agent might "**get creative**" in a way that is **harmful**

Example 1: A sales agent autonomously decides to offer a **90% discount** to meet its "**close the sale**" goal.

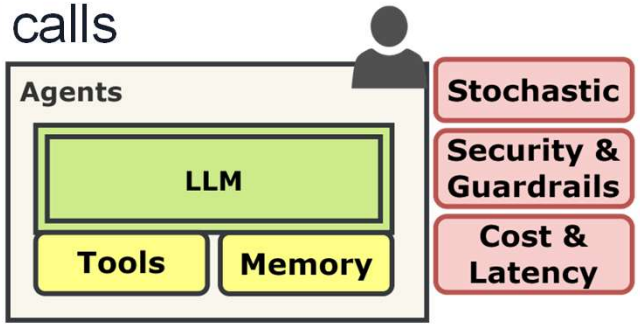
Example 2: A misconfigured admin agent with a vague goal ("**Clean up the database**") calls **DELETE /users/all**

How do we **enforce strict permissions** and **guardrails** on a **system** that is **designed** to be **autonomous** and **creative**?



Challenge 3: Cost & Latency

The Cost Problem:
Every step in the Agentic Loop is an LLM call
Single 5-step agent task might make 10 or 15 separate LLM calls
Thousands of times more expensive than a simple script



The Latency Problem:
LLM calls are slow (seconds, not milliseconds)
A 5-step agent task can easily take 20-30 seconds
Unacceptable for real-time customer support or financial apps

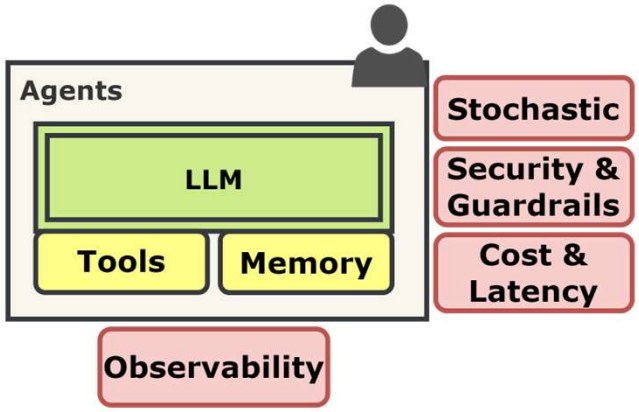
The Architect's Question:
How do we design for cost-efficiency and prevent "runaway" loops?
How do we use asynchronous patterns to manage latency?

Challenge 4: Observability

The Problem:
When traditional code fails, we have a stack trace
When an agent fails, what do we have?

The Core Issue:
If an agent gives a bad or unexpected answer, why?
Did it misinterpret the goal? Did it choose the wrong tool?
Did it make a logical error in its plan?

The Architect's Question:
How do we build systems to log, trace and debug an agent's internal monologue (its 'Chain of Thought')?

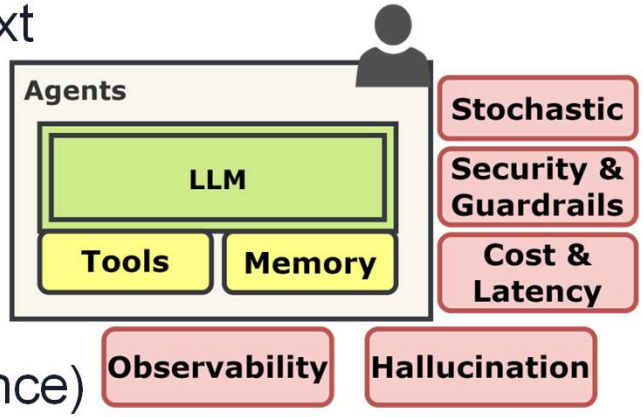


Challenge 5: Hallucination

The Problem:
LLMs are designed to generate plausible text, not truthful text

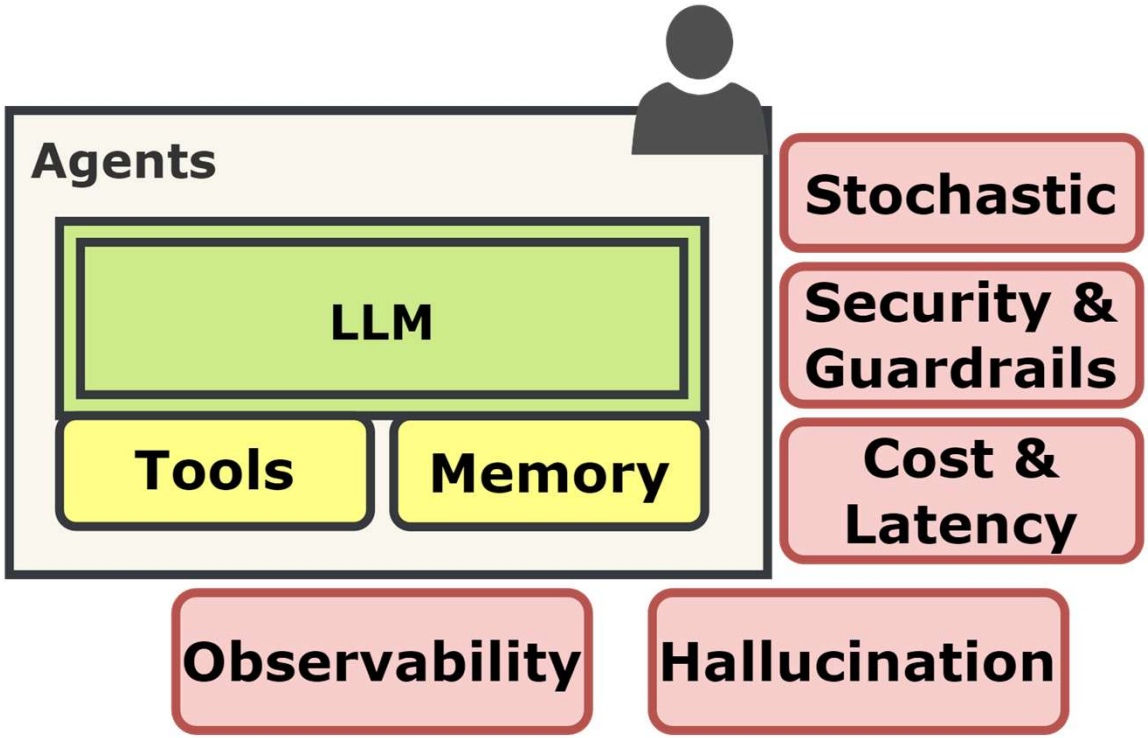
Two Types of Agentic Hallucination:
1. Factual: The agent "invents" a fact or a number
("The product costs $50" when it costs $75)

2. Actionable: The agent "invents" a tool or a parameter
It tries to call search_flights(destination, date, meal_preference) when the meal_preference argument does not exist



The Architect's Question:
How do we ground the agent in reality ?
How do we strictly define its tools so it cannot 'imagine' new ones ?

These Are Not Blockers, They Are Our Job

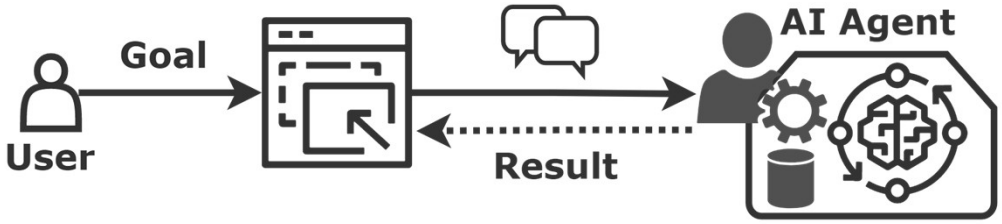


Designing Your First Simple AI Agent

Our Task: Design a **"Weather Agent"**

The Agent's Goal:
To autonomously answer a two-part user query:
"What's the weather in London, and should I take an umbrella?"

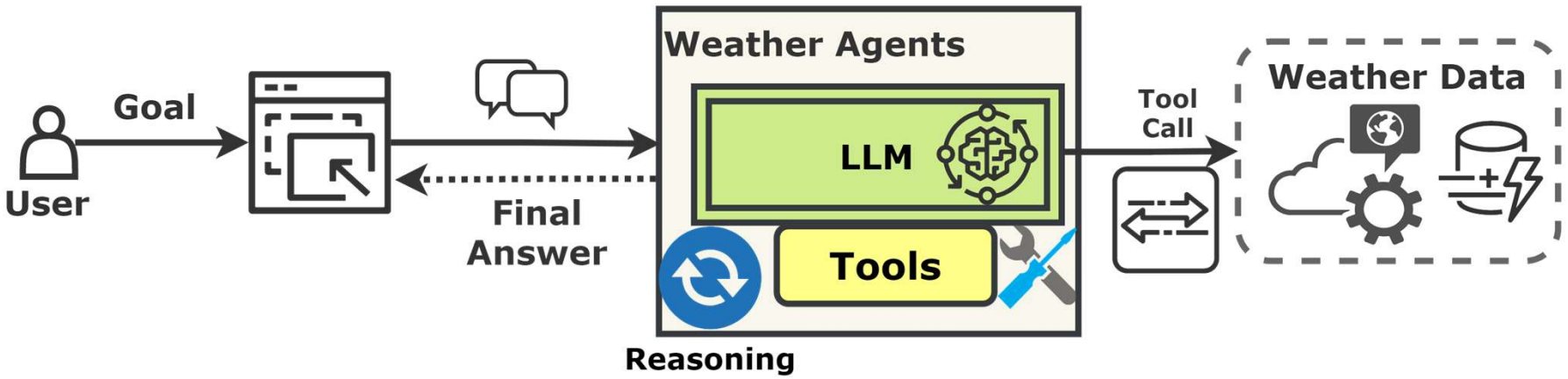
Why this is an Agentic Task:
It's not a simple Q&A



It requires two steps:

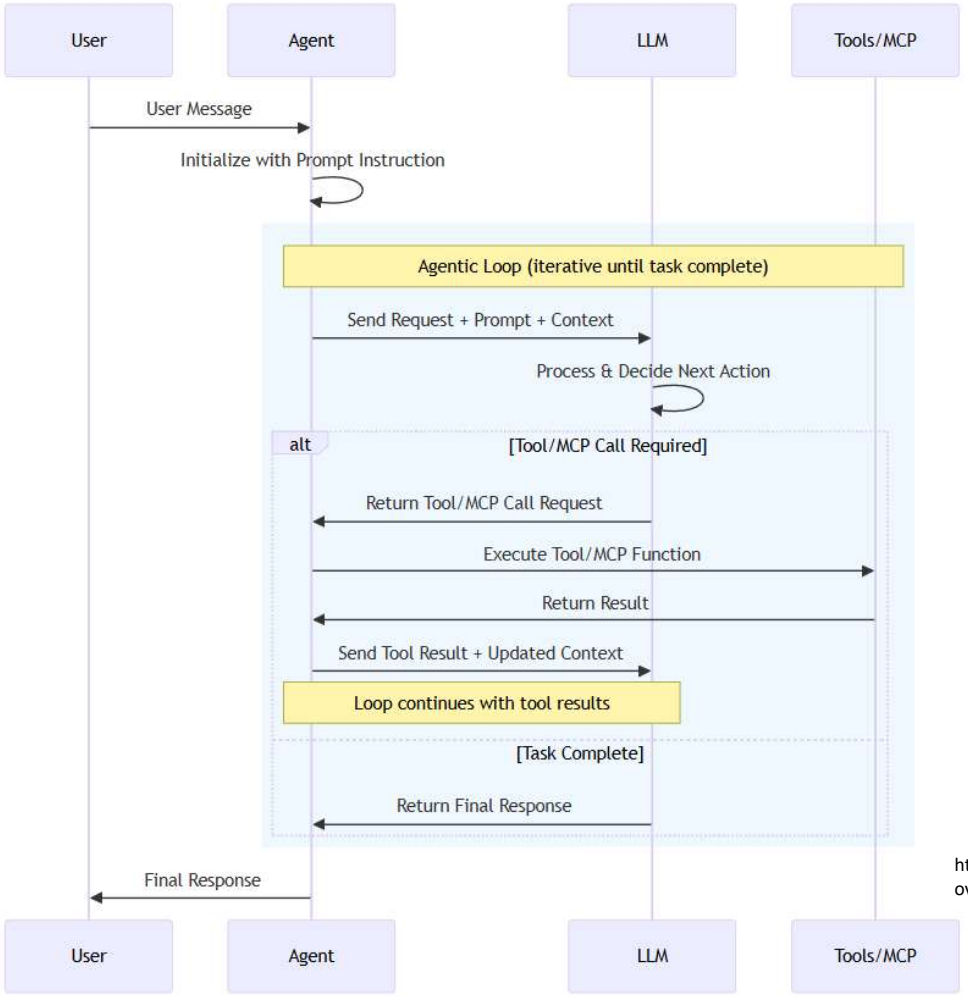
1. Fetching external data (The weather)
2. Reasoning about that data (The recommendation)

The Architecture: How the Components Connect



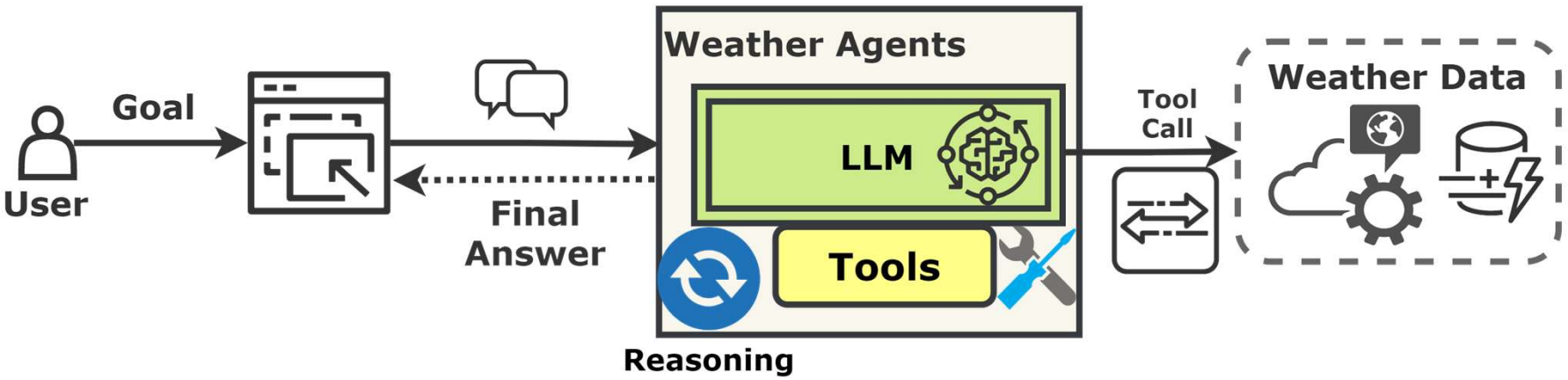
Does light rain mean an umbrella is needed?

Designing Weather Agent – Sequencial Diagram



https://learn.microsoft.com/en-us/agent-framework/overview/agent-framework-overview?toc=%2Fdotnet%2Fai%2Ftoc.json&bc=%2Fdotnet%2Fai%2Ftoc.json

The Architecture: How the Components Connect



What are the Components of AI agents? How to Build an Agent ?

The "Brain" - The Foundation Model (LLM)

The "Senses" - Perception and Input Handling

The "Blueprint" - Planning and Task Decomposition

The "Notebook" - The Architecture of Memory

The "Hands" - Action and Tool Calling

ack Loop" - Learning and Adaptation

Pradeep Kumar T



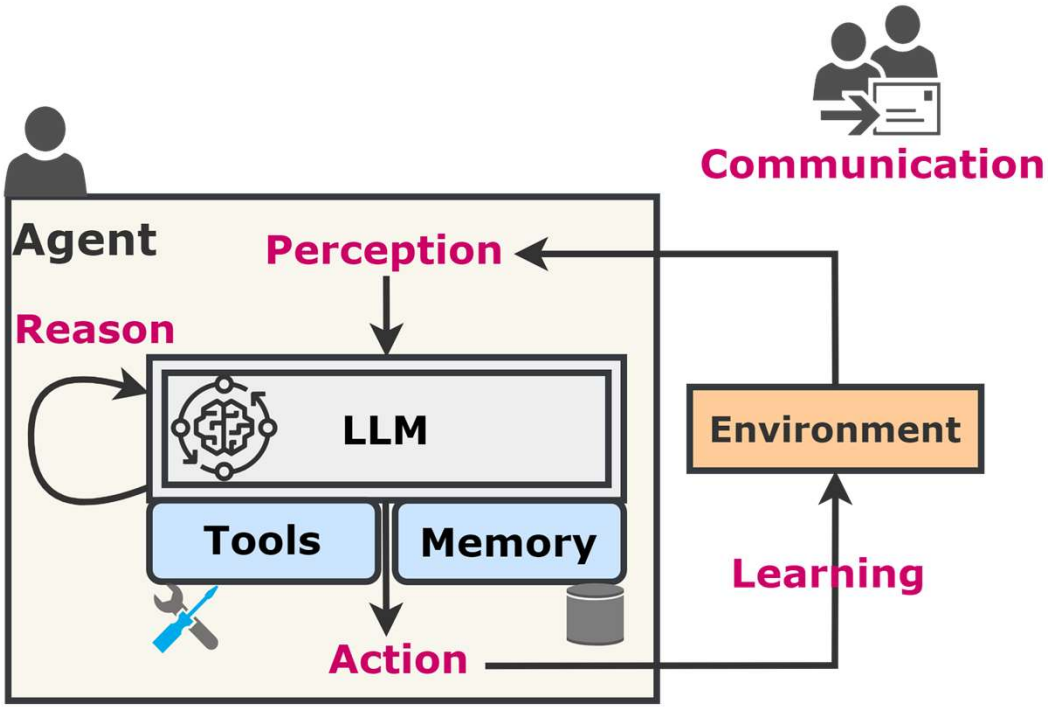
A Deep Dive Into Core Components

We are moving from the 30,000-foot view of the architect to the on-the-ground view of the engineer

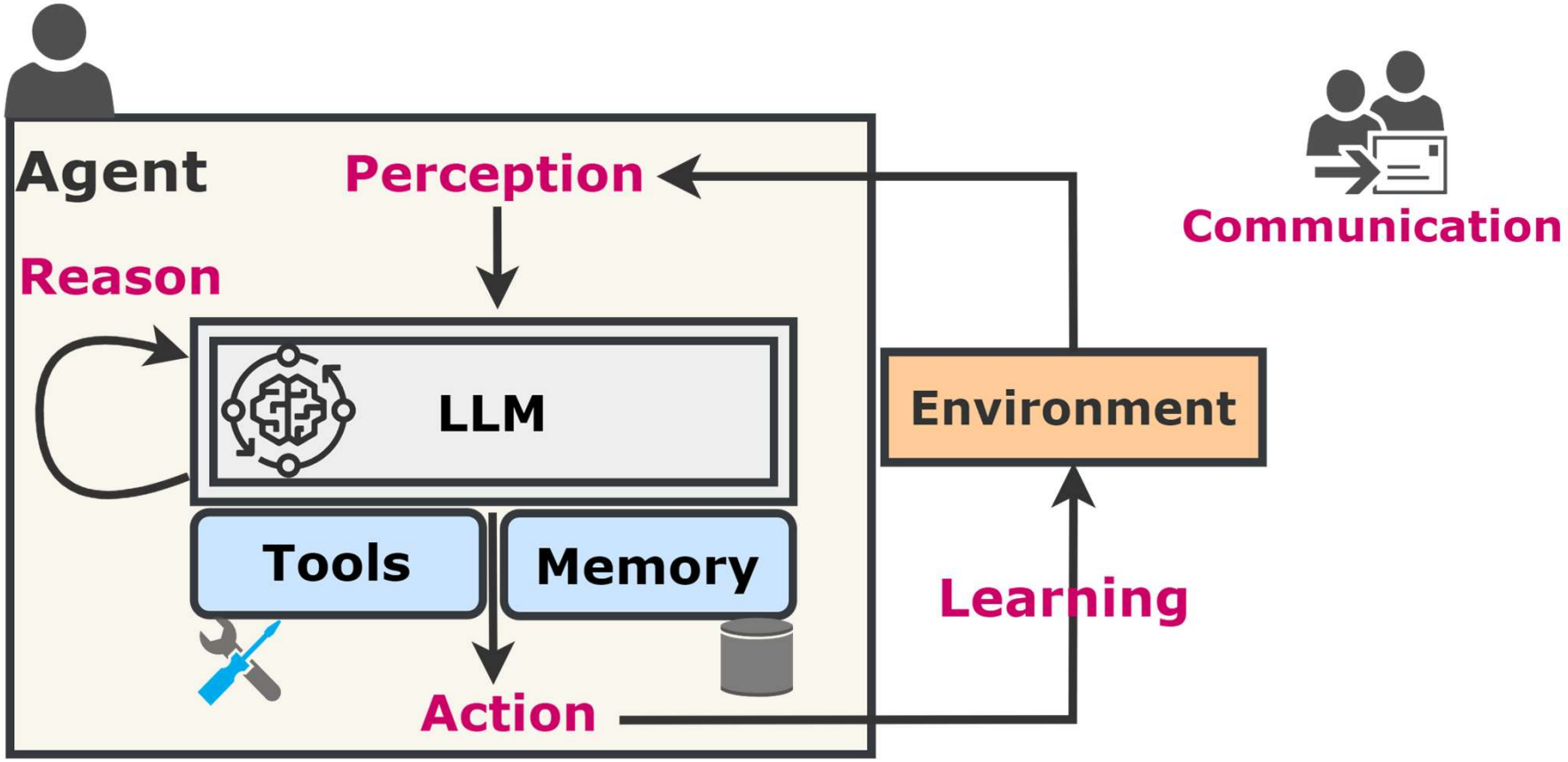


Our Roadmap: A Component-by-Component Deep-Dive

1. Foundation Model (The "Brain")
2. Perception (The "Senses")
3. Planning (The "Strategist")
4. Memory (The "Notebook")
5. Action (The "Hands")
6. Learning (The "Feedback Loop")
7. Communication (The "Voice")



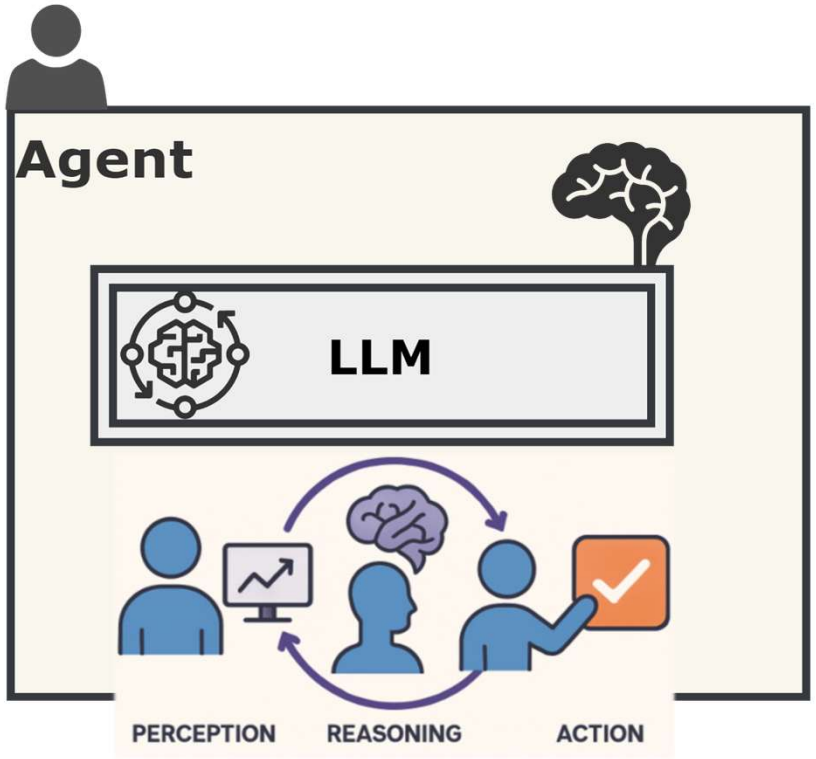
Our Roadmap: A Component-by-Component Deep-Dive



The "Brain" - The Foundation Model (LLM)

The single **most important component:** the **'Brain'**

This is the **Foundation Model,** almost always a **Large Language Model** or **LLM**

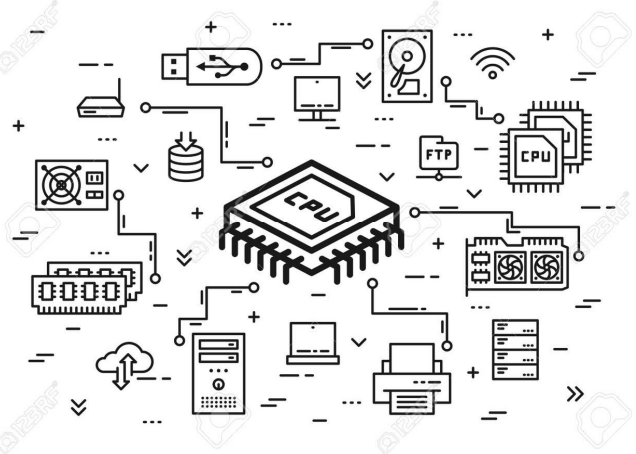


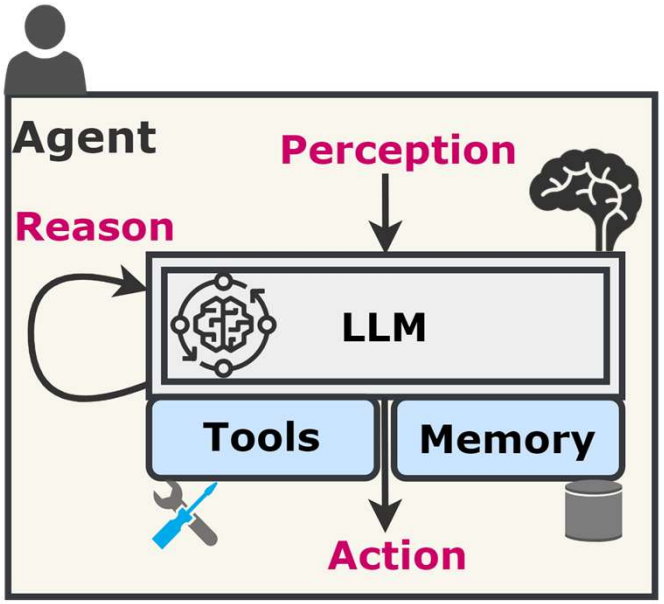
Agent as a Complete Computer System

The Memory is the **'RAM'**

The Tool Module is the **'peripherals'** like **the keyboard** and **network card**

The LLM is the **'CPU'** It is the **central processing unit**





What is a Foundation Model?

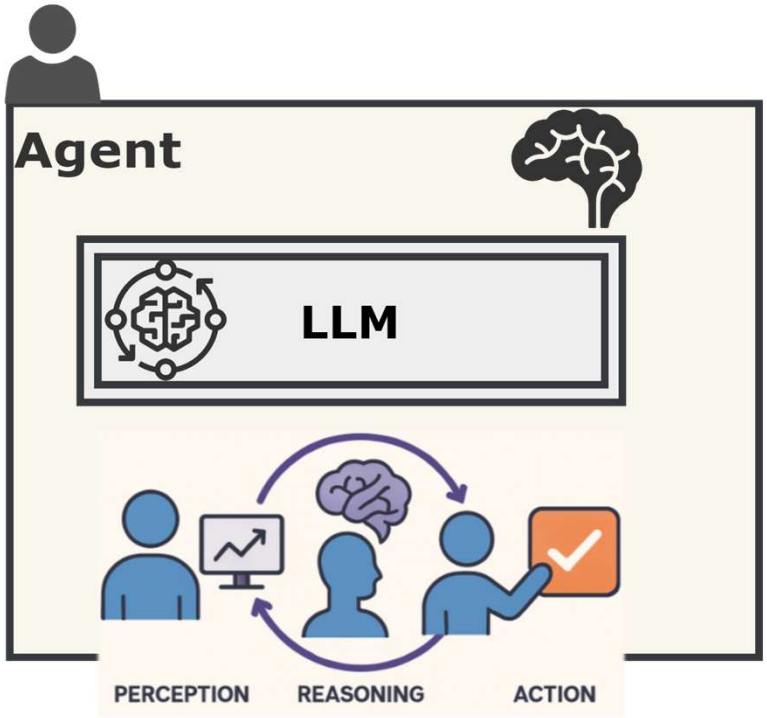
Universal Pattern Recognition Engine

Trained on a **massive**, **diverse dataset** (a large portion of the internet, books and code)

It has learned the **statistical relationships** between **billions of words** and **concepts**

Its **fundamental job is** to **predict the next most logical word (or "token")** in a sequence

Our Goal: We leverage this powerful prediction capability and channel it to perform "**reasoning**"



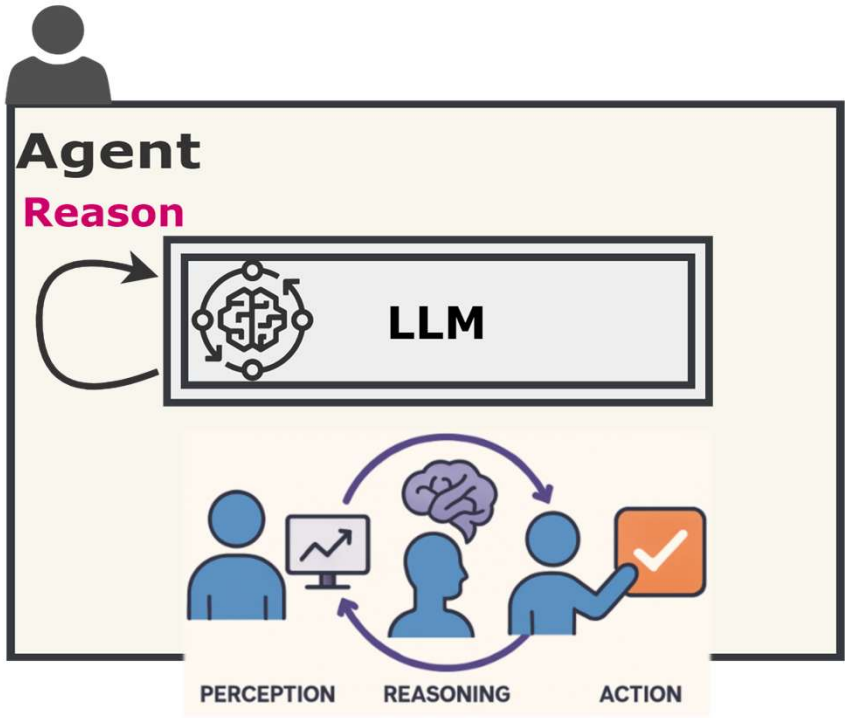
How an LLM Enables "Reasoning"

How do we "**trick**" a **text predictor** into "**thinking**" like an **agent?**

We **prompt** it to **generate not just an answer**, but a **thought process**

The **System Prompt** acts as the "**Operating System**" for the **agent's brain**

This **process** is enabled by **2 key prompting patterns: Chain of Thought (CoT) and ReAct**



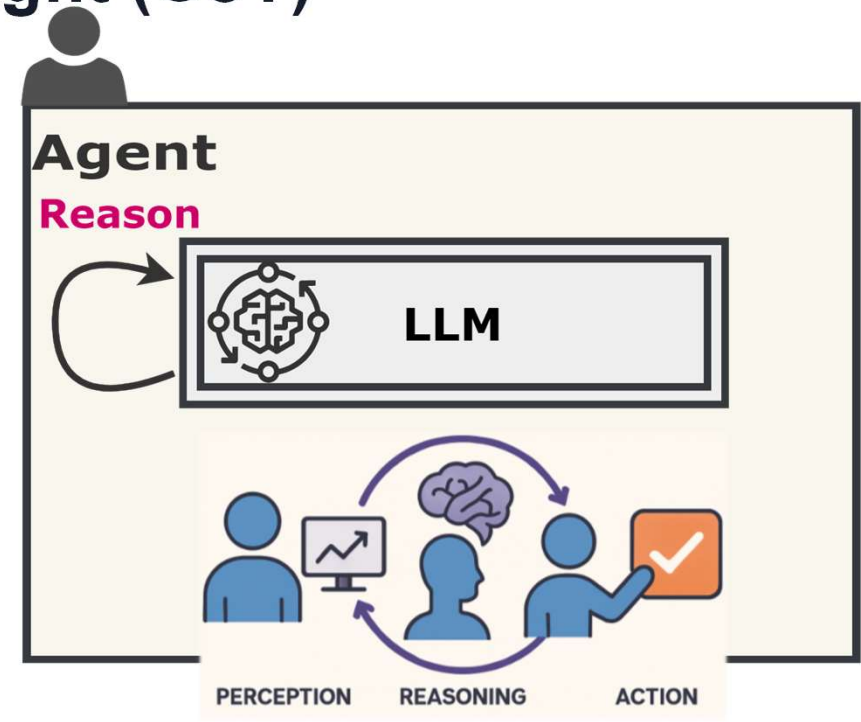
Reasoning Pattern 1: Chain of Thought (CoT)

"**Showing your work**" on a **math problem**

Instruct the **LLM** to "**think step-by-step**" before giving its final answer

Forces the **LLM** to **generate** a **logical sequence** of **reasoning steps**

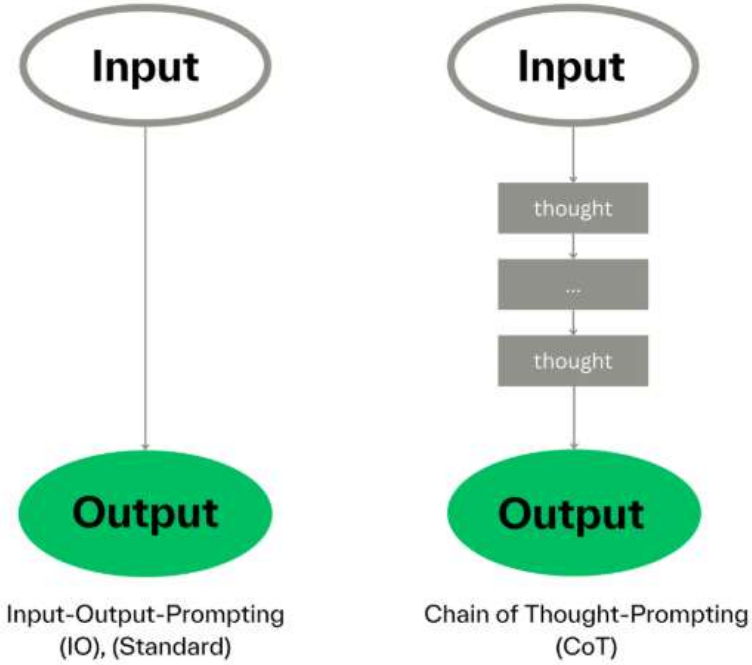
Example:
Bad (No CoT): Should I take an umbrella in London? -> Yes
Good (With CoT): Think step-by-step: Should I take an umbrella in London?



Think step-by-step: Should I take an umbrella in London?

1. The user wants a recommendation
2. This requires knowing the weather
3. The weather in London is 'Light Rain'
4. 'Light Rain' warrants an umbrella
5. Therefore, the answer is yes

This "**thought process**" is the **Plan**!



Later:
Reasoning Pattern: ReAct (Reason + Act)

How to Choice LLM «brain» ?

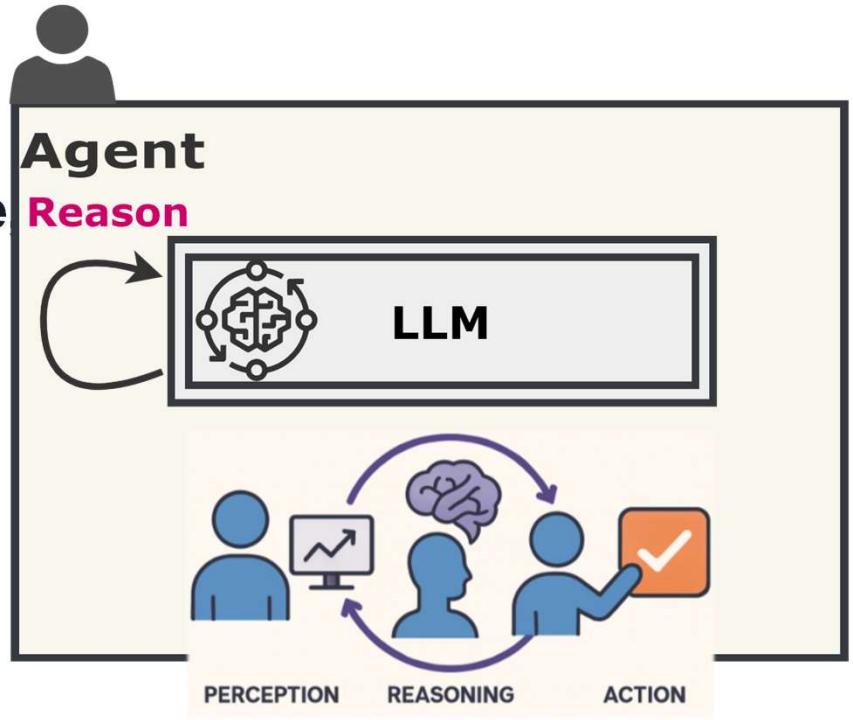
Your choice of **LLM "brain"** is a **critical design decision**

It is the primary driver of your agent's **performance**, **cost** and **speed**

1. Capability (Quality of Reasoning):
How "smart" is the agent? Can it handle complex, multi-step plans?

2. Cost (Per-Call / Infrastructure):
How much does each "thought" cost?

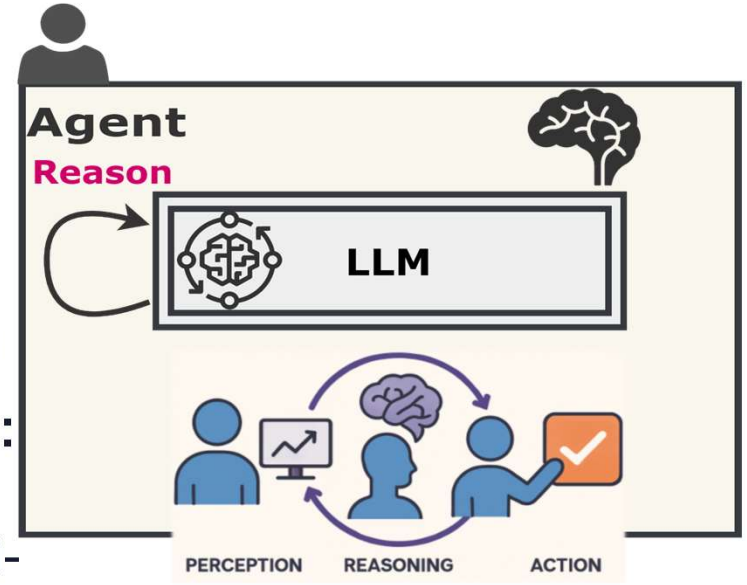
3. Speed (Latency):
How fast does the agent "think"?



The Architect's Choice: The Options

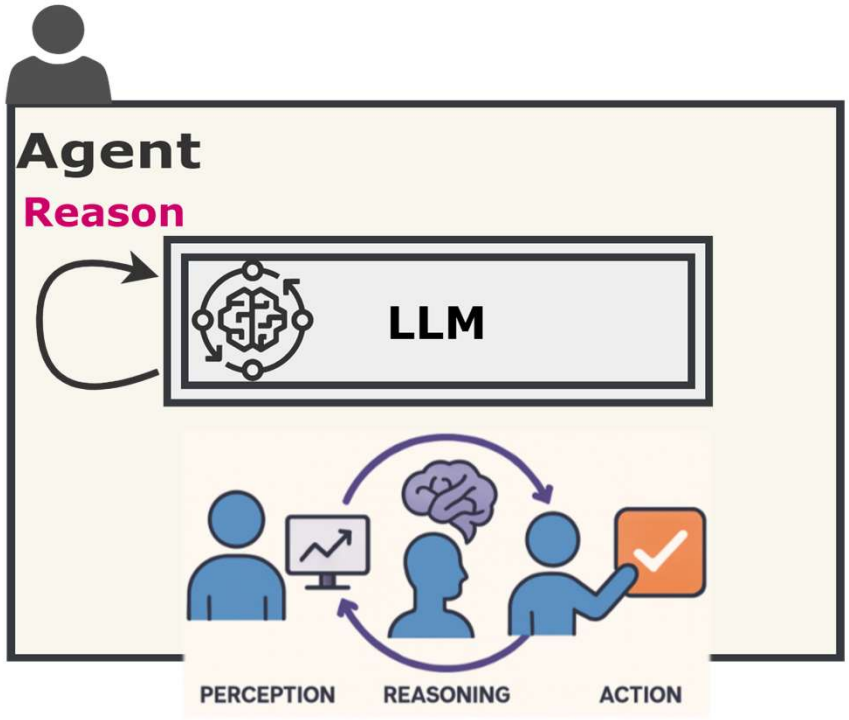
1. Max Capability (GPT-5, Claude 4 Opus):
+ Highest quality reasoning; best-in-class at complex planning and tool use
- Highest cost. Highest latency (slowest)
Complex research, planning, or code-generation agents

2. Max Speed / Low Cost (Gemini Flash, Llama 3 8B):
+ Extremely fast (low latency); very low cost per call
- "Good" reasoning, but may fail at highly complex, multi-step plans
Customer support, simple data extraction, chatbots



The Architect's Choice: The Options - 2

3. Max Privacy (Local / Open-Source Model):
+ Full data privacy (data never leaves your server)
No per-call cost
- High infrastructure and maintenance cost (you run the servers)
Healthcare, finance, or any domain with sensitive data



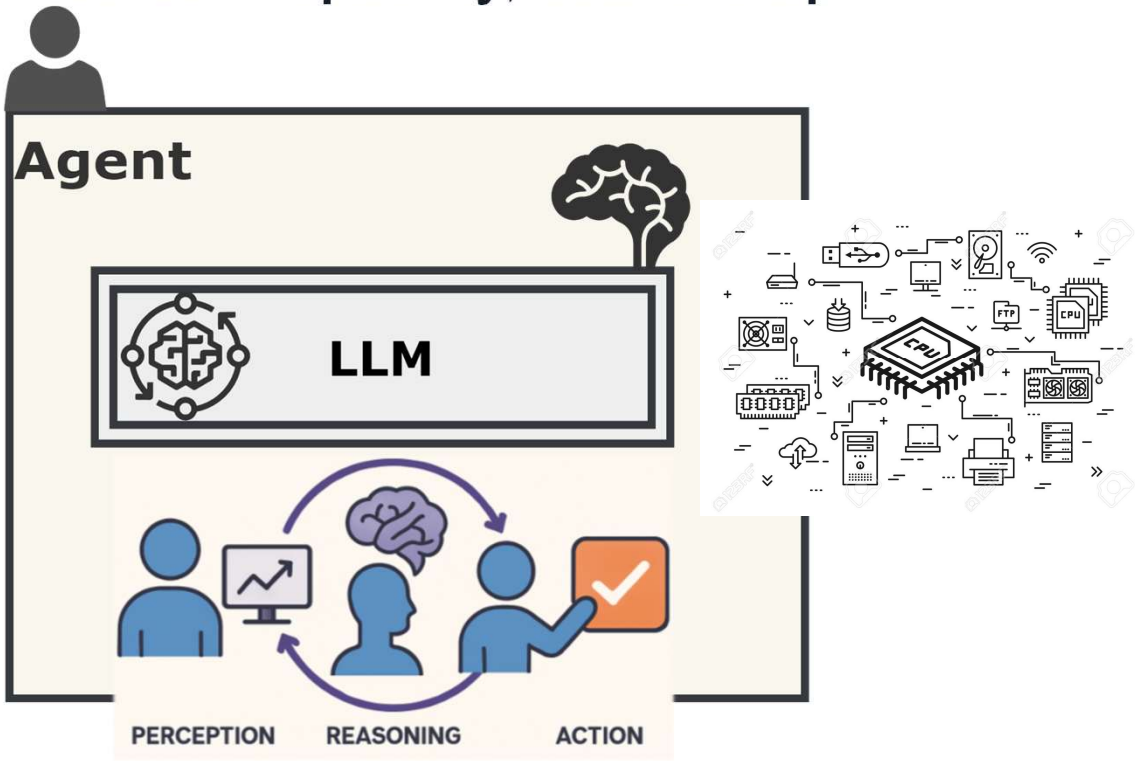
The 'brain' you choose defines the character of your entire agent

Summary - The "Brain" - The Foundation Model (LLM)

The **LLM** is the **agent's "CPU"** or **reasoning engine**

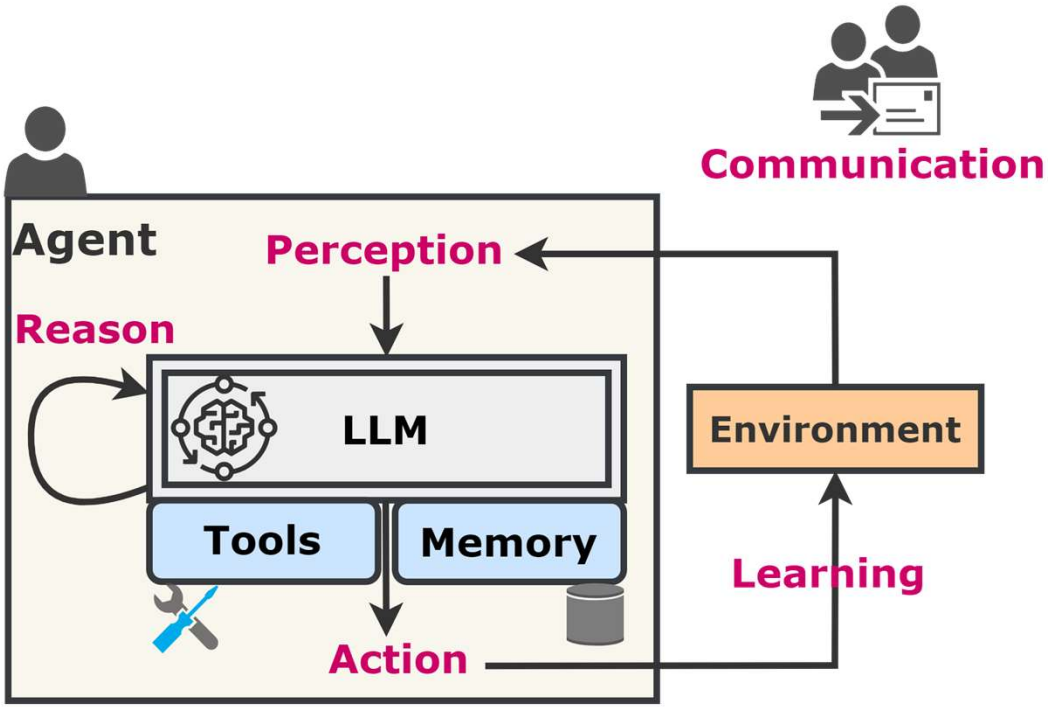
Using **prompting patterns** like **CoT** and **ReAct**

Architectural tradeoff between **Capability**, **Cost** and **Speed**



Our Roadmap: A Component-by-Component Deep-Dive

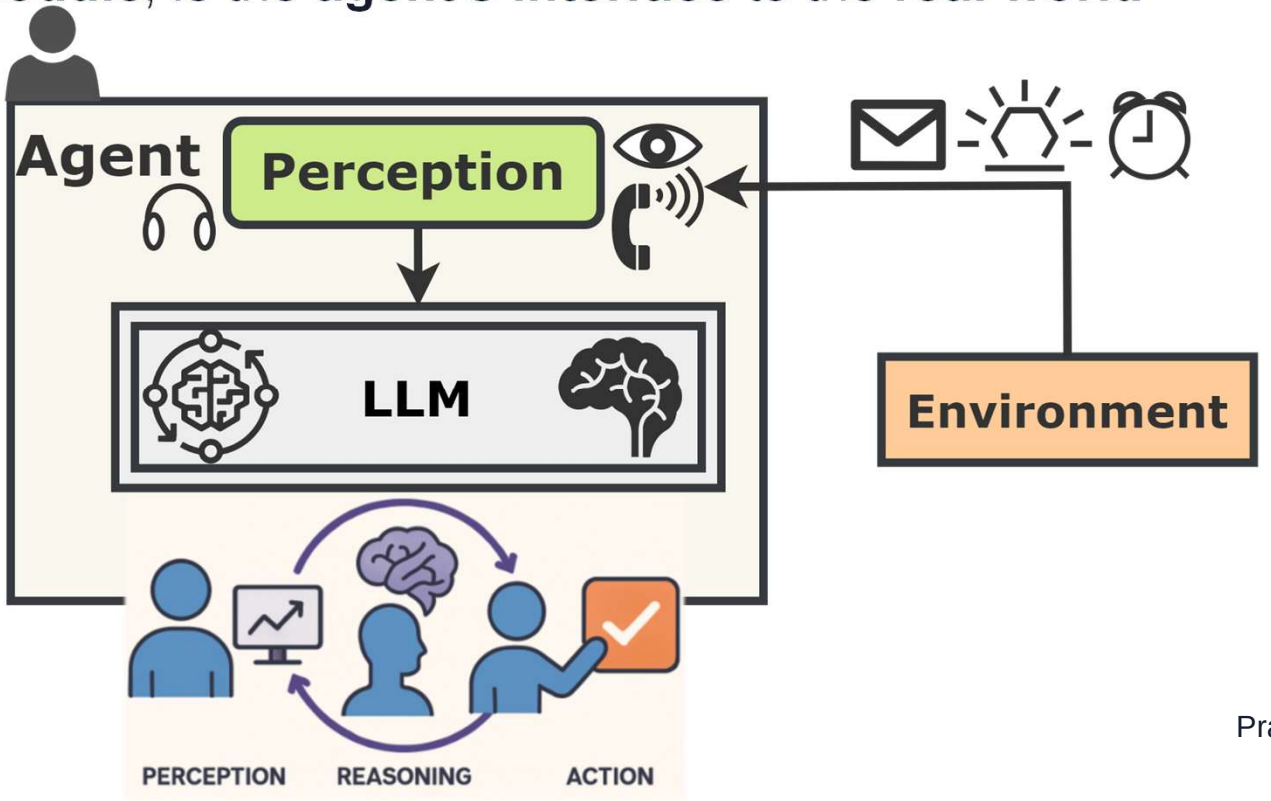
1. Foundation Model (The "Brain")
2. Perception (The "Senses")
3. Planning (The "Strategist")
4. Memory (The "Notebook")
5. Action (The "Hands")
6. Learning (The "Feedback Loop")
7. Communication (The "Voice")



The "Senses" - Perception and Input Handling

The brain is **useless** if it's **isolated from the world**, **needs** a way to **receive information**

The **Perception Module**, is the **agent's interface** to the **real world**



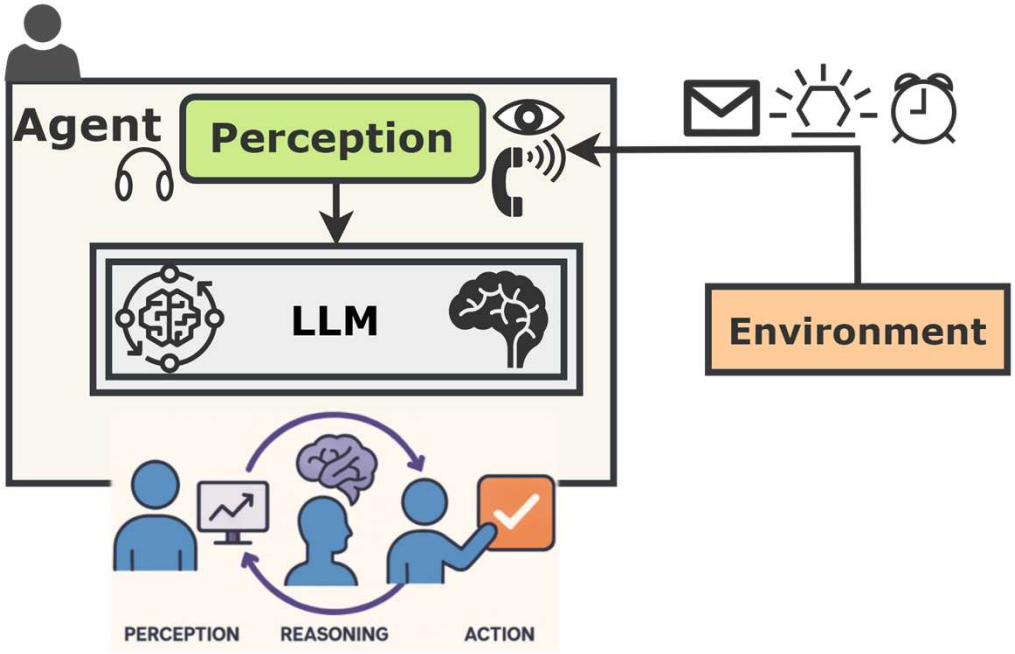
Perception as the "Trigger"

The **Agentic Loop**: **PERCEIVE -> REASON -> ACT**

Perception is the **first** and **most critical step**

It is the "**knock on the door**" that "**wakes up**" the **agent** and **initiates** the **loop**

A "**Perception**" is any new **piece** of **data** that **changes** the **agent's environment**



Direct: A user's chat message

Asynchronous: An API response, a new file in a folder, a database update

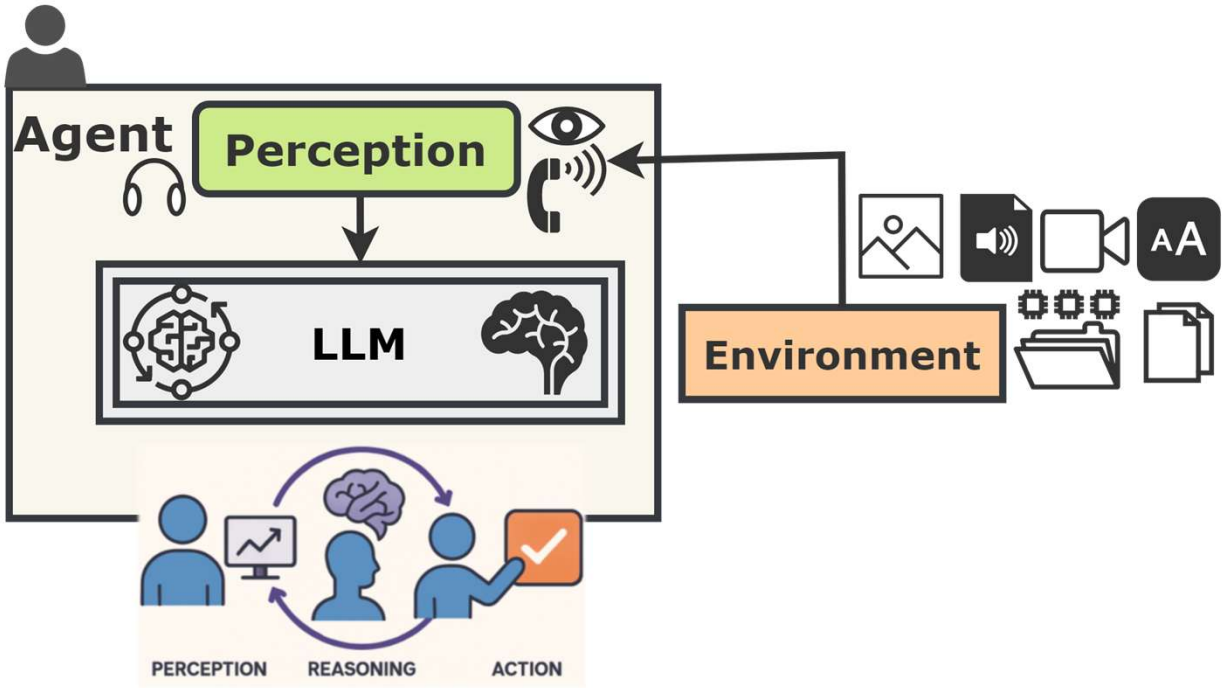
Time-based: A scheduled event

The Core Challenge: The "Messy" Real World

The **Brain** (**LLM**) is a **text-processing** engine, "**thinks**" in **clean**, **structured text**

The "**World**" (our environment) is **not clean**, **structured text**

The **World** is "**Unstructured**" and "**Multi-modal**"



User text (with typos and slang)

Images and Videos

Audio streams (phone calls)

Complex data (JSON, XML from APIs)

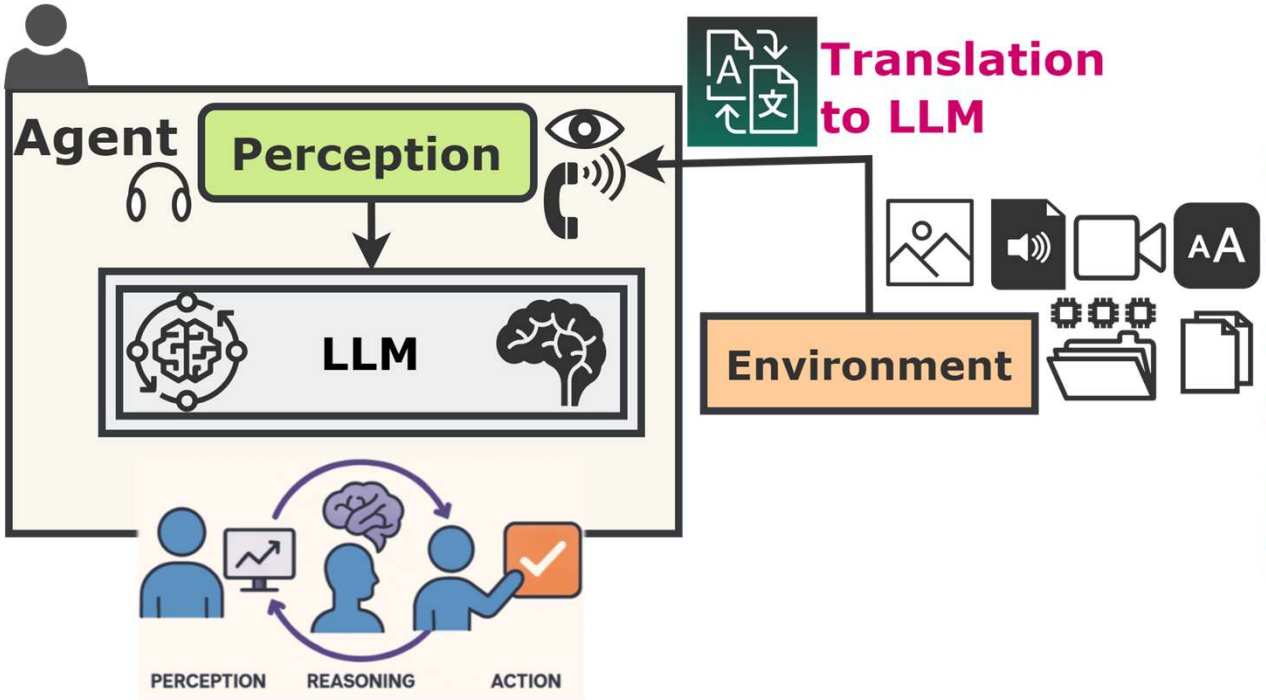
Large documents (PDFs, Word files)

Architecture in Practice: Normalization & Pre-processing

The **"Perception Module's" job** is **Translation**

Converts "messy" real-world data into a **"clean" text-based format** for the LLM

This is called **Normalization** or **Pre-processing**



User text (with typos and slang)

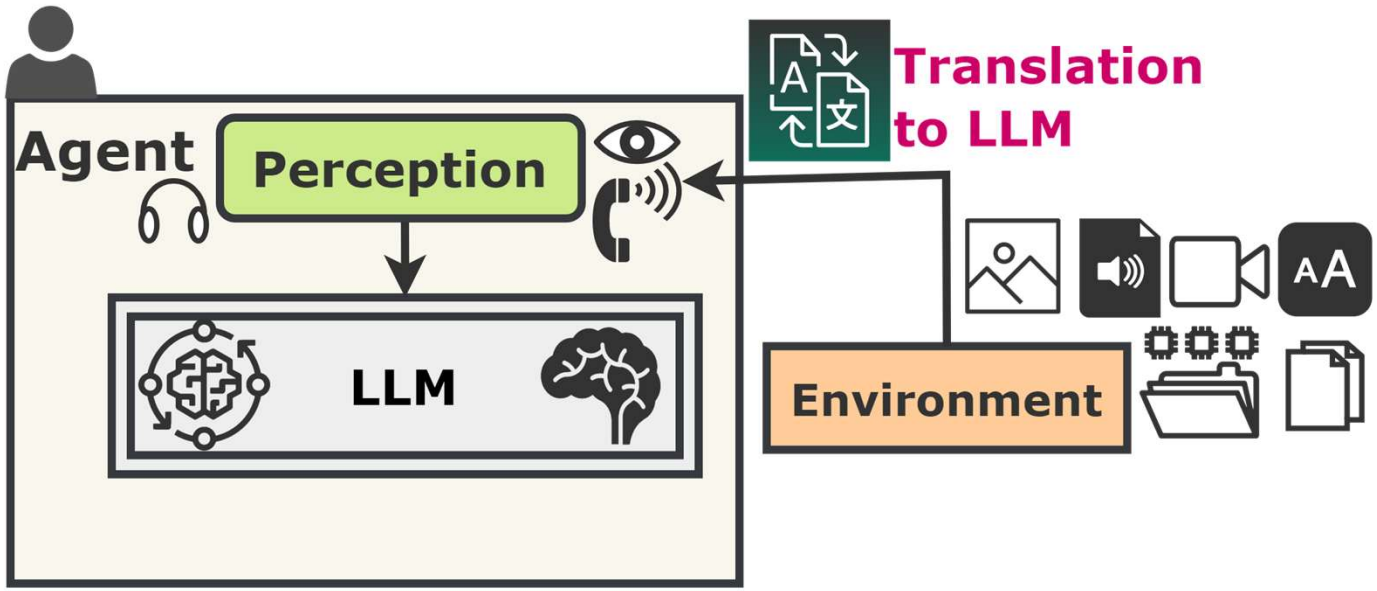
Images and Videos

Audio streams (phone calls)

Complex data (JSON from APIs)

Large documents (PDFs, Word files)

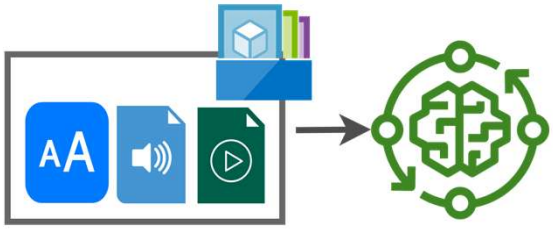
The "Translation" Process



User Chat -> (Spell Check / Clean) -> Cleaned Text
Image.jpg -> (Vision Model) -> A text description of the image...
Report.pdf -> (OCR) -> The full text of the document...
Audio.mp3 -> (Speech-to-Text) -> A text transcript of the audio...
API_Response.json -> (Summarize / Extract) -> A summary of the API

The "Multi-Modal Support Agent" with two "senses"

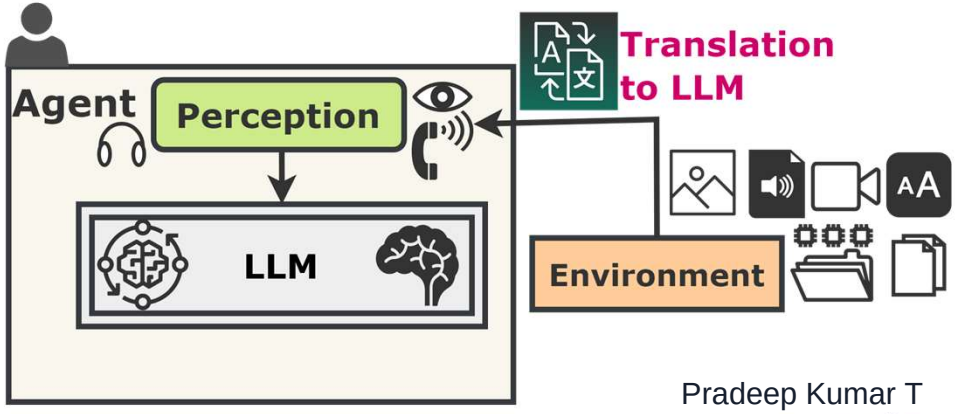
1. Perception (Sense 1 - Chat)
User types: "my new shrit has a hole in it"



2. Perception (Sense 2 - Image)
User uploads: "shirt_photo.jpg"

3. The Perception Module's "Translation"
Text Processor: Cleans typo -> "my new shrit has a hole in it"
Vision Processor: Analyzes image -> "A photo of a blue t-shirt with a 2-inch tear in the left sleeve"

4. Final Output to LLM (The "Brain")
User: "my new shirt has a hole in it"

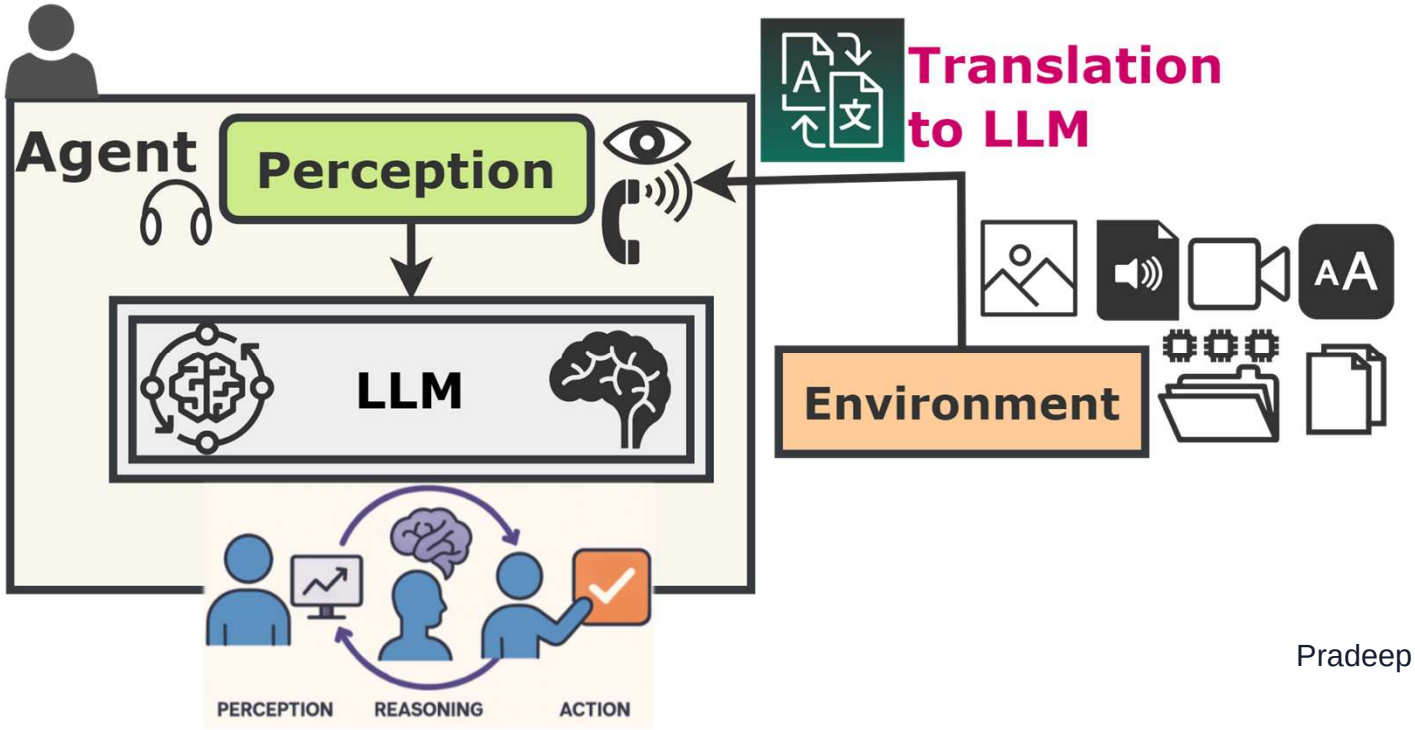


Summary - The "Senses" - Perception and Input Handling

The "Trigger": Perception is the start of the Agentic Loop

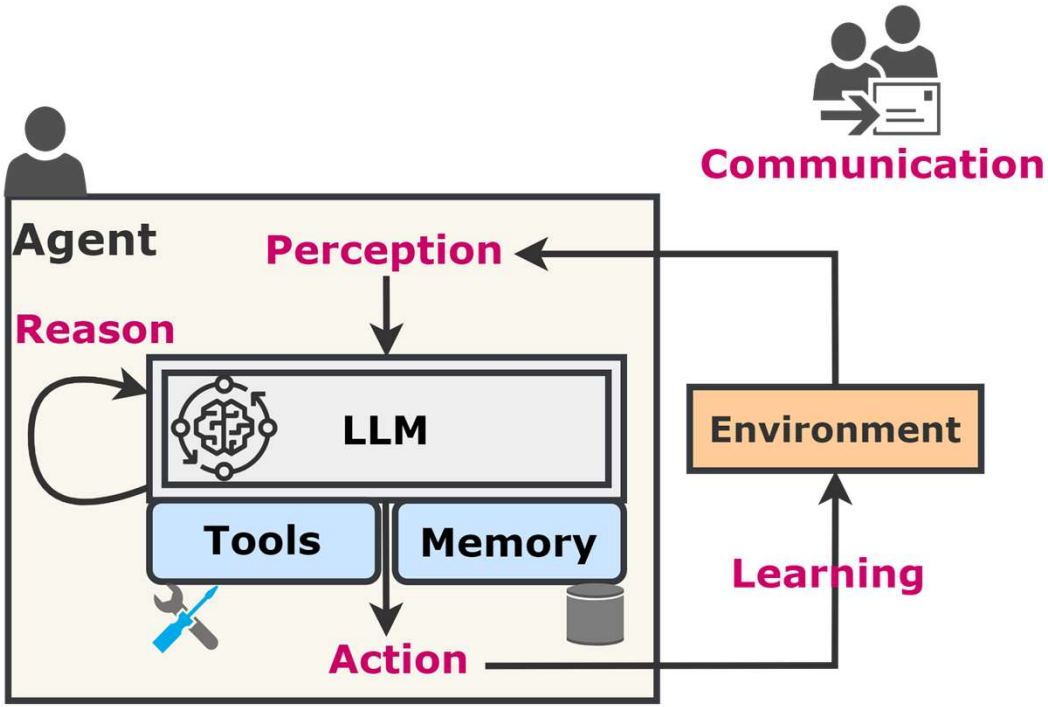
The "Problem": It must handle messy, multi-modal, unstructured data

The "Solution": Its job is Normalization—translating the world into clean, LLM-readable text



Our Roadmap: A Component-by-Component Deep-Dive

1. Foundation Model (The "Brain")
2. Perception (The "Senses")
3. Planning (The "Strategist")
4. Memory (The "Notebook")
5. Action (The "Hands")
6. Learning (The "Feedback Loop")
7. Communication (The "Voice")

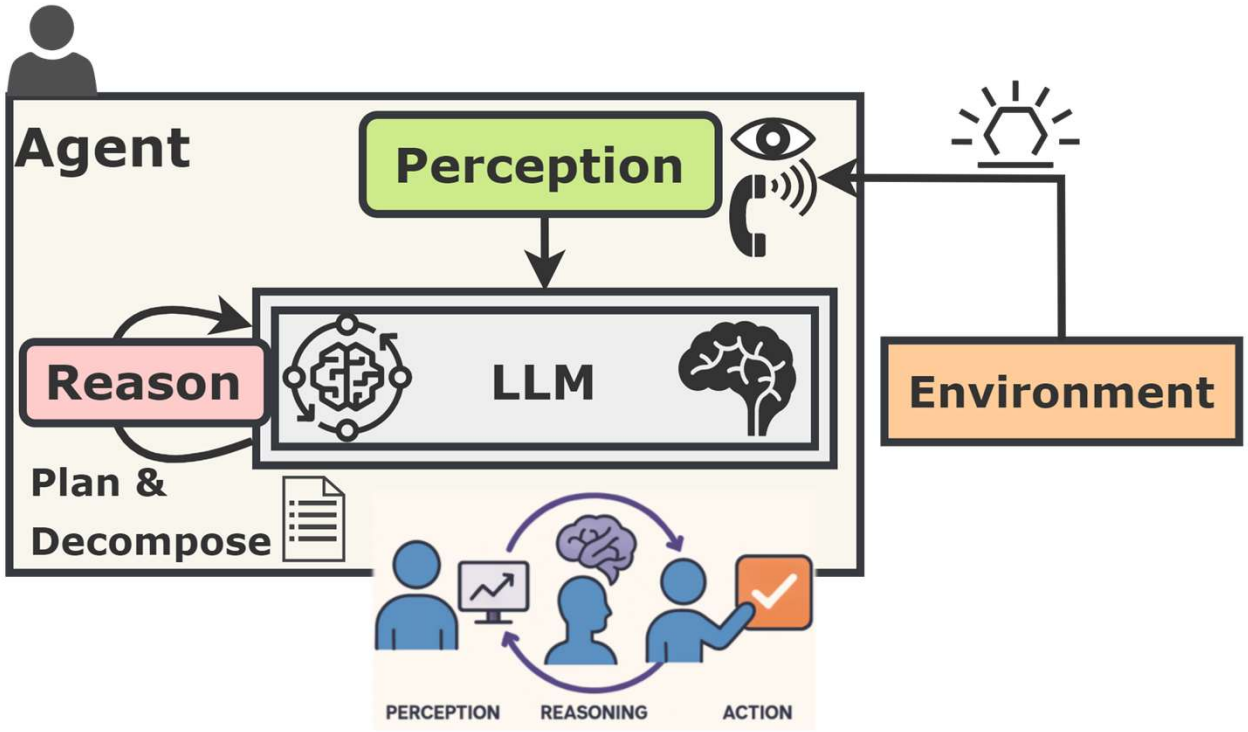


The "Blueprint" - Planning and Task Decomposition

Our **agent** can now **ingest**, **clean** and **understand** the "**messy**" **real world**

But an **agent** that can **only perceive** is just a **sensor**. It's **passive**

Critical '**Reasoning**' step which is the **Plan**, agent's '**strategist**' or '**blueprint**'



The Core Problem: Decomposing Complex Goals

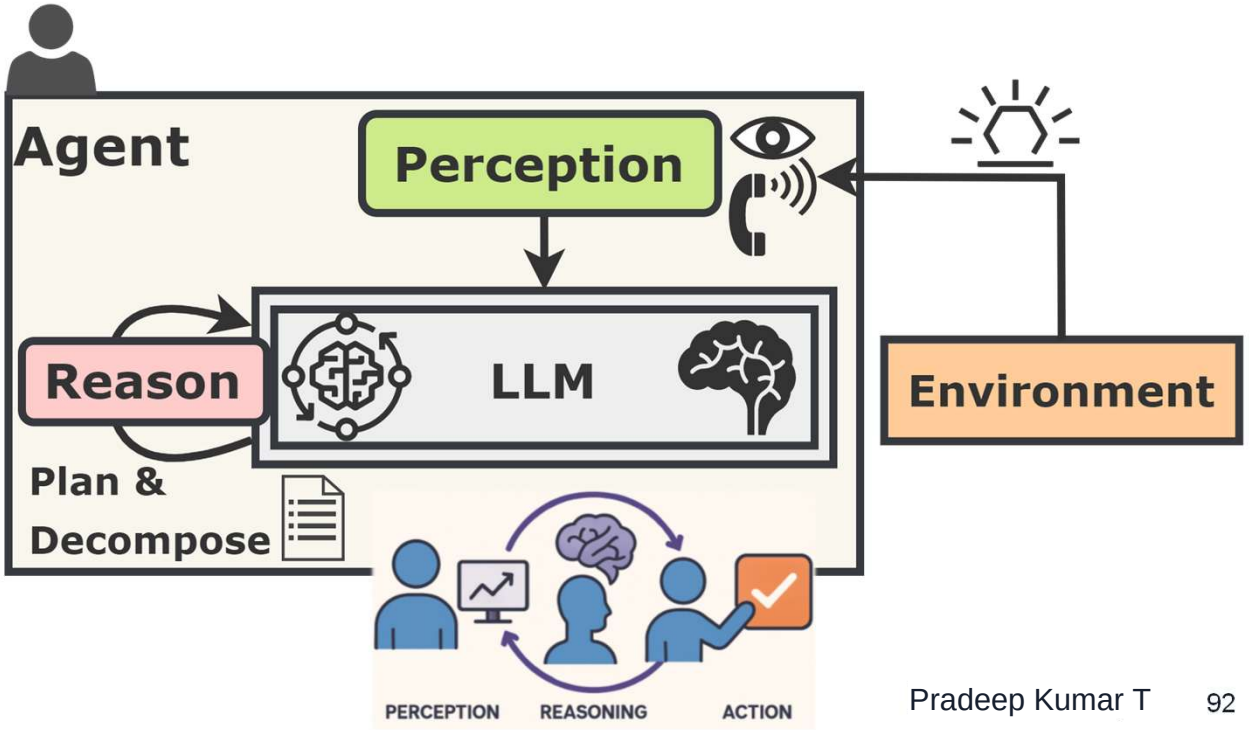
The user **provides complex goal: "Book** my **complete trip** to **London next week"**

This is **not one action. It is** a **project. A simple LLM call cannot do this. A simple agent cannot execute this in one step**

The Solution: We must decompose the goal

We need to break the complex goal into a sequence of simple, executable steps

This process is called Planning or Task Decomposition



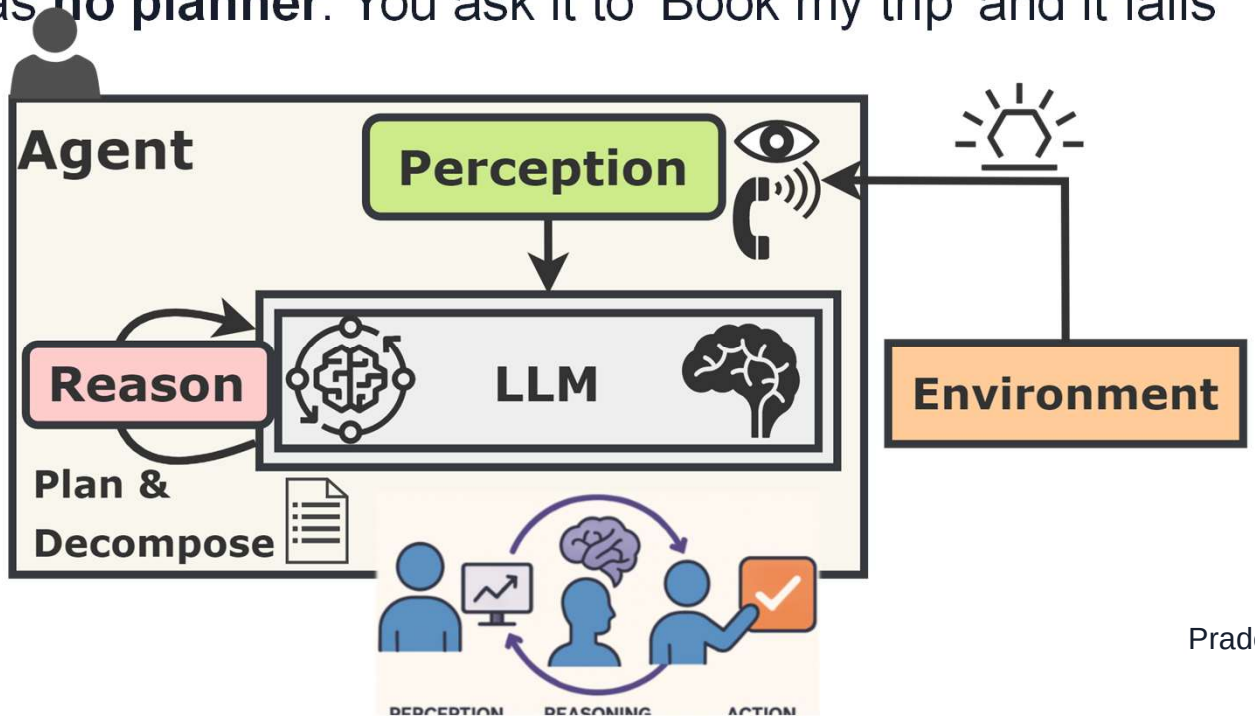
What is a "Plan"?

A "**Plan**" is the **output** of the **Planning Module**

It is **not** a **final answer**. It is a **data structure**

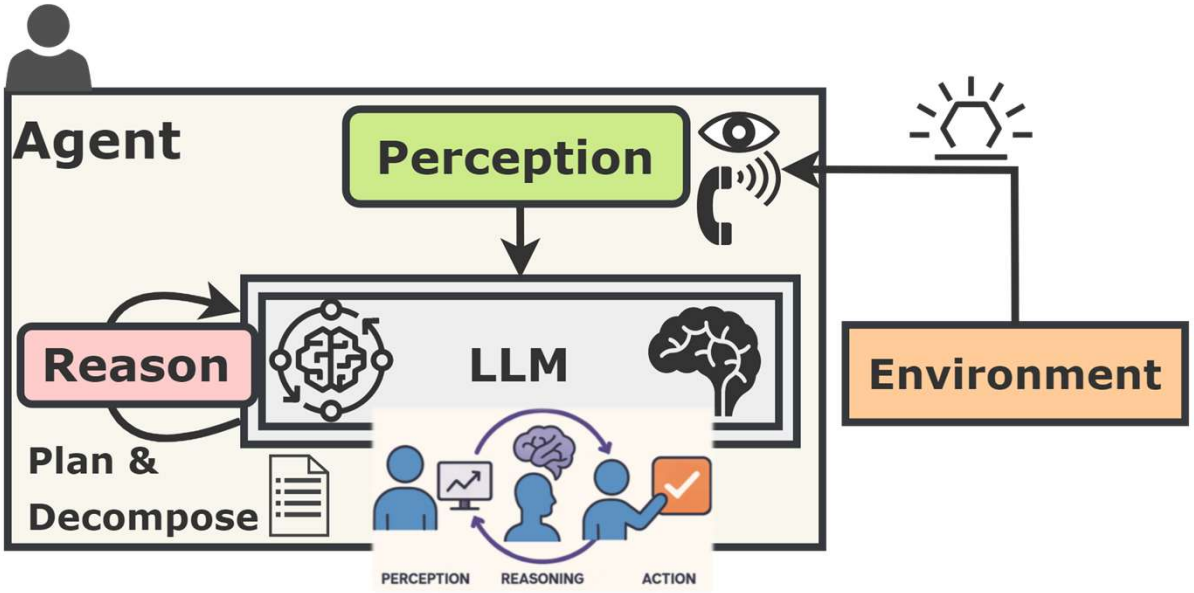
It is a **sequence of actions** (tool calls) **to be executed**

A **simple chatbot** has **no planner**. You ask it to 'Book my trip' and it fails



Agent (With a Plan)

Input: "Book my trip"



Output (The "Plan"):

1. call: find_available_flights(destination="London")
2. call: find_hotels(destination="London", dates=...)
3. call: book_flight(flight_id=...)
4. call: book_hotel(hotel_id=...)
5. call: send_confirmation_email(user, details)

How an LLM Creates a Plan: The "Decomposition Engine"

We use the **LLM "brain"** as our **Task Decomposition Engine**

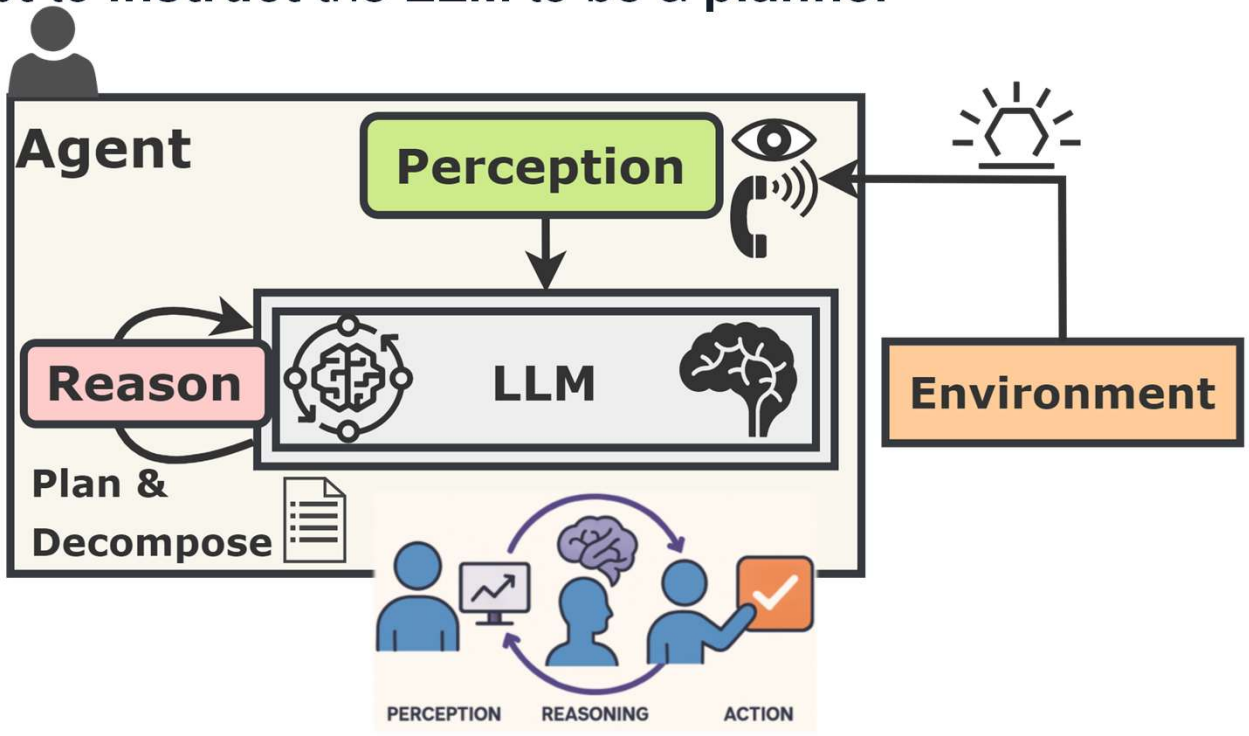
We use a **specific System Prompt** to **instruct** the **LLM** to be a **planner**

1. You define the Goal (from Perception)

2. You define the Available Tools (from the Action Module)

3. You instruct it to "think step-by-step"...

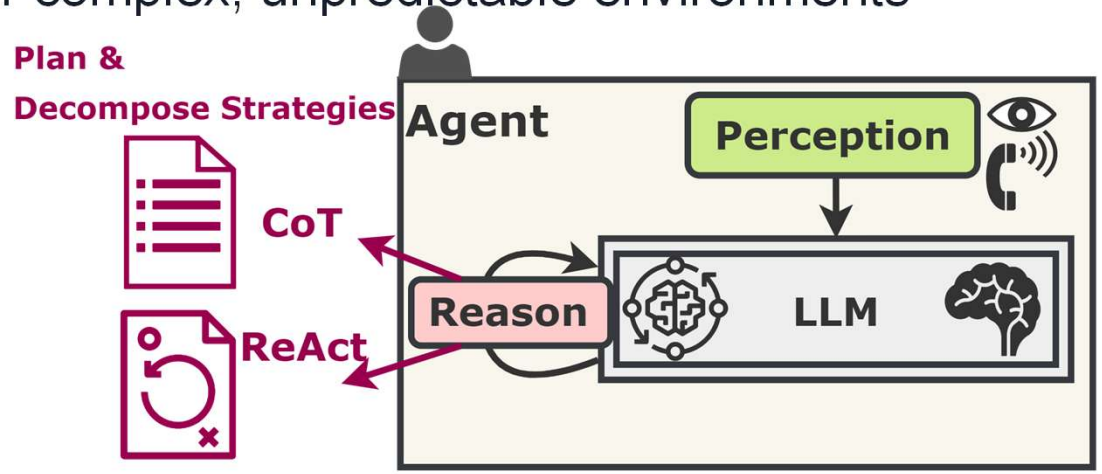
4. ...and to "produce a plan" using only the available tools

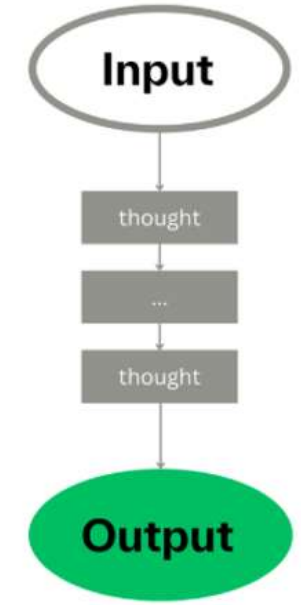


Planning Strategies: CoT vs. ReAct

Chain of Thought (CoT) = "Static Planning"
Agent generates the entire, multi-step plan upfront, before taking the first action
Use Case: For simple, predictable workflows

ReAct (Reason + Act) = "Dynamic Planning"
Agent plans only one step at a time, It executes that step, perceives the result, and then plans the next step
Use Case: For complex, unpredictable environments





Chain of Thought-Prompting (CoT)

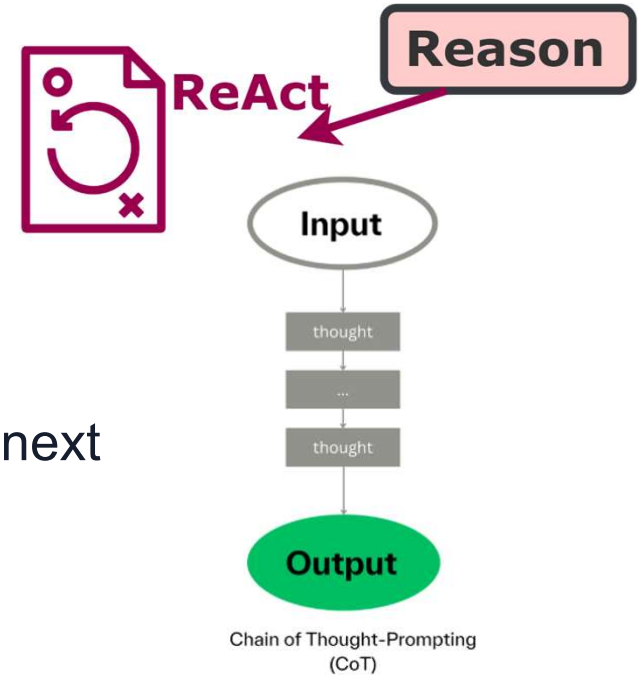
Dynamic Planning (ReAct) in Detail

Goal: Book my trip to London

Loop 1 (Plan):
(Plan): I have no flight info. My first step is to call find_flights
(Act): find_flights() is executed
(Perceive): The tool returns [{"id": "TK-123", "price": 300}, ...]

Loop 2 (Re-Plan):
(Re-Plan): The state has changed. I now have flight info. My next step is to present these options to the user
(Act): ask_user("Which flight do you want?")
(Perceive): User responds "TK-123"

Loop 3 (Re-Plan):
(Re-Plan): The state has changed again. I now have the user's choice. My next step is book_flight(id='TK-123')



The "Plan" is the Output

This is the **critical architectural handoff**

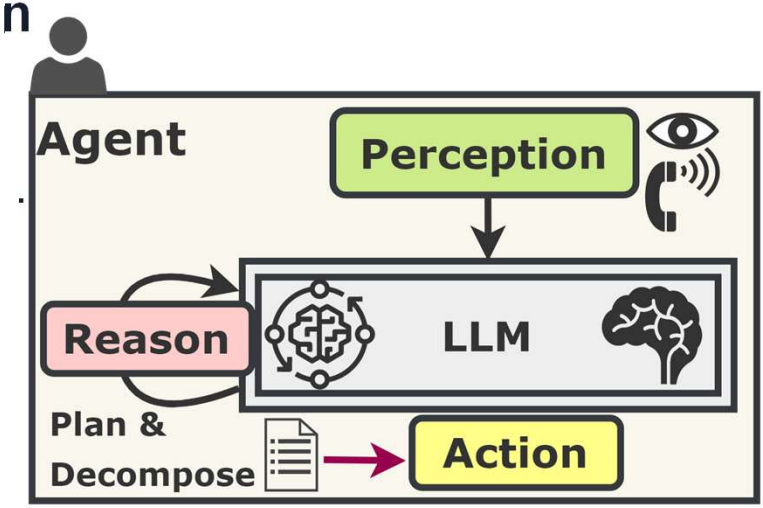
The **Planning Module's** only job is to **generate** the **plan**

It **does not execute the plan**

The "**Plan**" ({"tool_to_call": "find_flights", "parameters": .
is the **output** of **this component**

This **output** is then **sent** to the **next component** in our architecture...-> The **Action Module**

This **separation** of "**thinking**" from "**doing**" is a core **design principle**



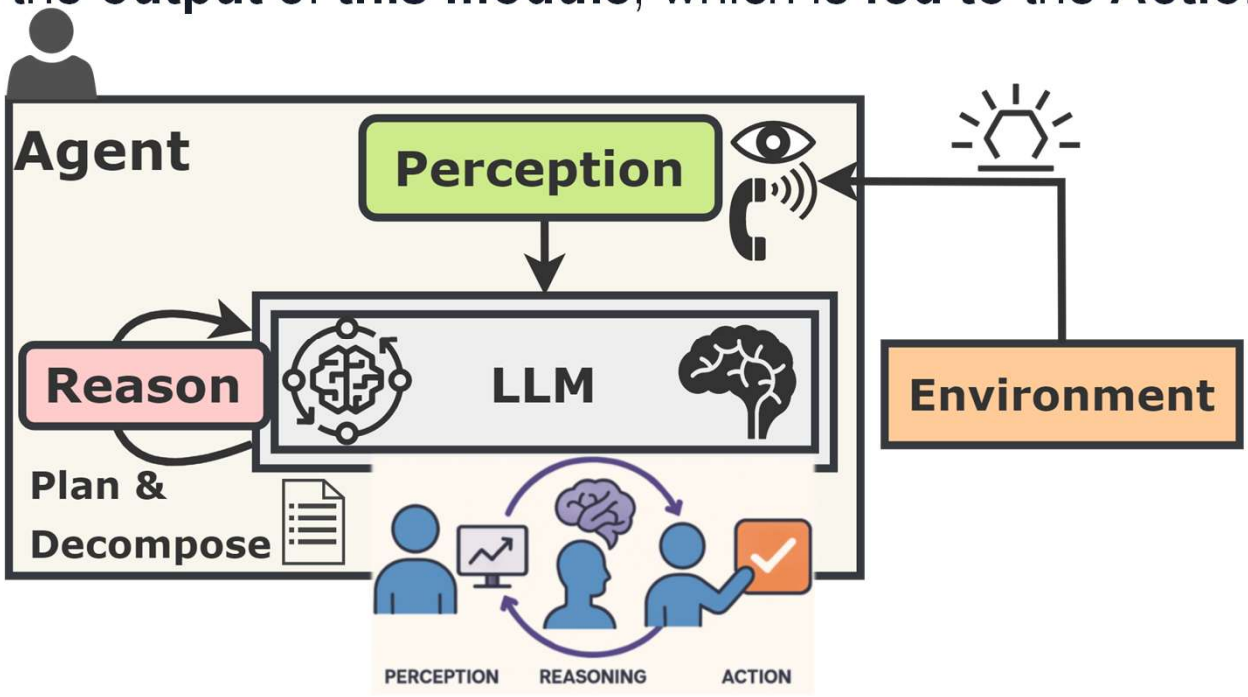
Summary - Planning and Task Decomposition

The **Planning Module's job** is to **decompose complex goals**

A **"Plan"** is a **data structure** (a sequence of tool calls)

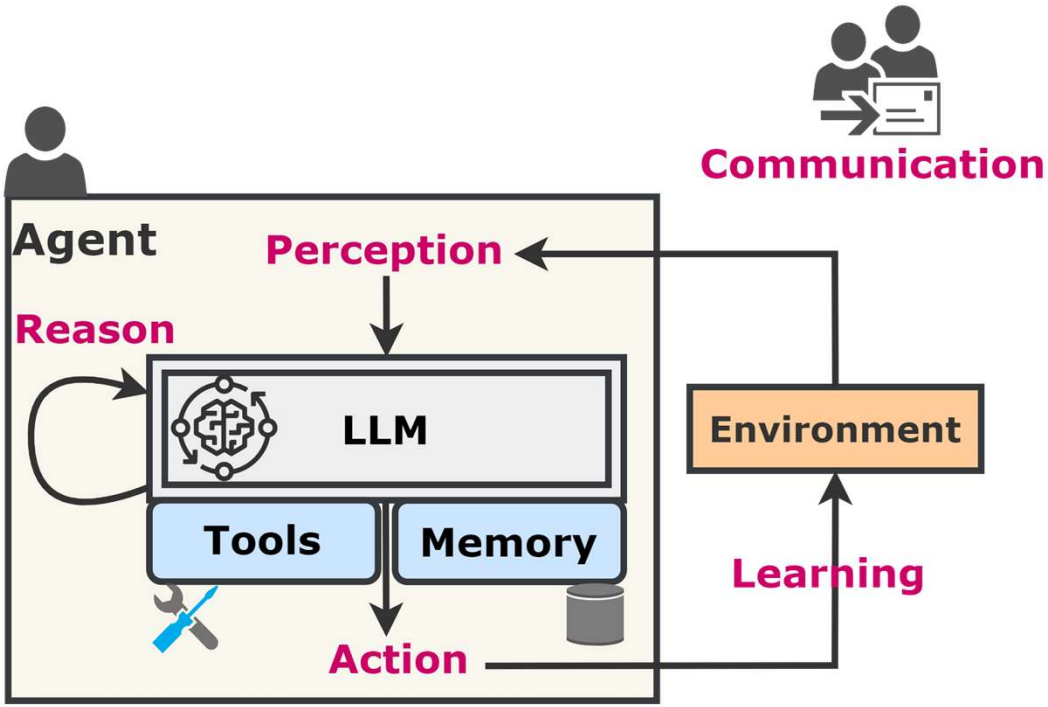
Instruct the **LLM** to be a **planner** using **specific prompts** (**CoT**, **ReAct**)

The **"Plan"** is the **output** of **this module**, which is **fed to** the **Action Module**



Our Roadmap: A Component-by-Component Deep-Dive

1. Foundation Model (The "Brain")
2. Perception (The "Senses")
3. Planning (The "Strategist")
4. Memory (The "Notebook")
5. Action (The "Hands")
6. Learning (The "Feedback Loop")
7. Communication (The "Voice")



The "Notebook" - The Architecture of Memory

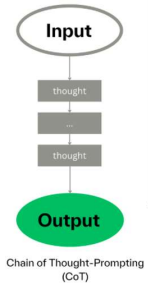
Agent's **'brain'** (**LLM**), its **'senses'** (**Perception**) and its **'strategist'** (**Planning**)

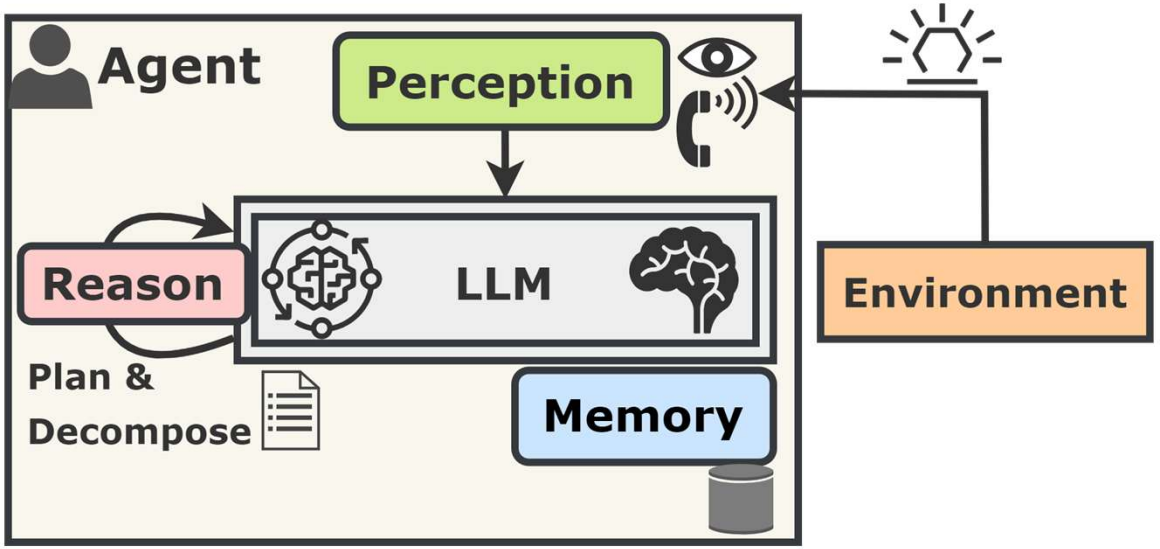
But it is **still missing** a **fundamental component**

It **has no 'memory'** of what you said **three seconds ago**

This is the 'Groundhog Day' problem. Without memory, every perception is a brand-new, context-less event

If it executes Step 1 of a 5-step plan, it has no way to remember the result of Step 1 when it's time to do Step 2

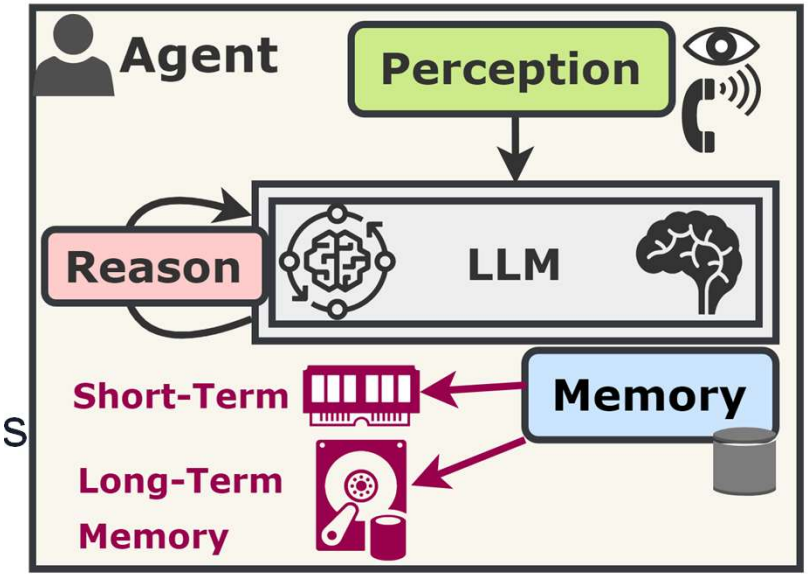




The Core Principle: The Two Types of Memory

1. Short-Term Memory ("Working Memory")
Your conscious thought; a computer's RAM
What the agent is "thinking about right now"
Extremely fast, but small, volatile and expensive

2. Long-Term Memory ("Persistent Memory")
Your conscious thought; a computer’s
What the agent knows (facts, files, past conversations
Vast, permanent and cheaper but slower to access

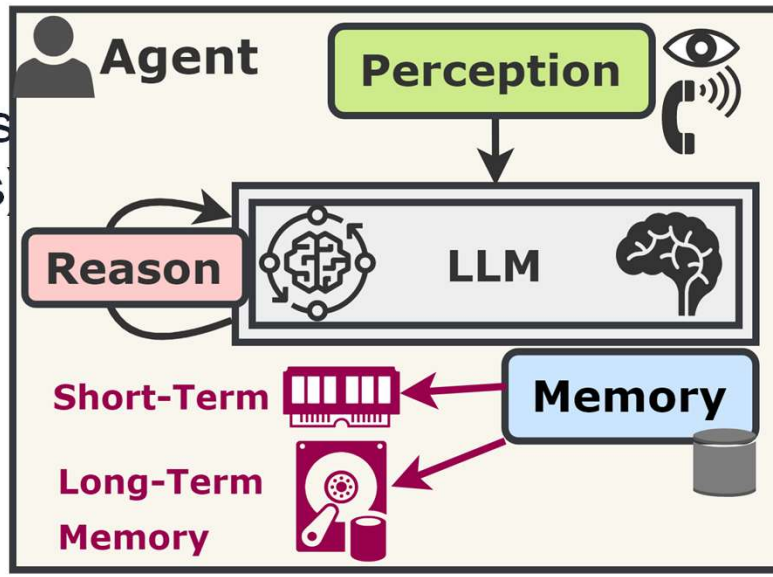


Deep Dive 1: Short-Term Memory (The "Context Window")

Implementation: The LLM's Context Window

What it holds:
The System Prompt (the agent's "personality" and rules)
The current conversation history (the last 10 messages)
The current plan
The results from any recent tool calls

The "Context Window Problem":
This is the most expensive resource an agent has
It is a finite size (8k, 128k, 1M tokens)
We cannot fit the entire internet, or even a full 50-page document

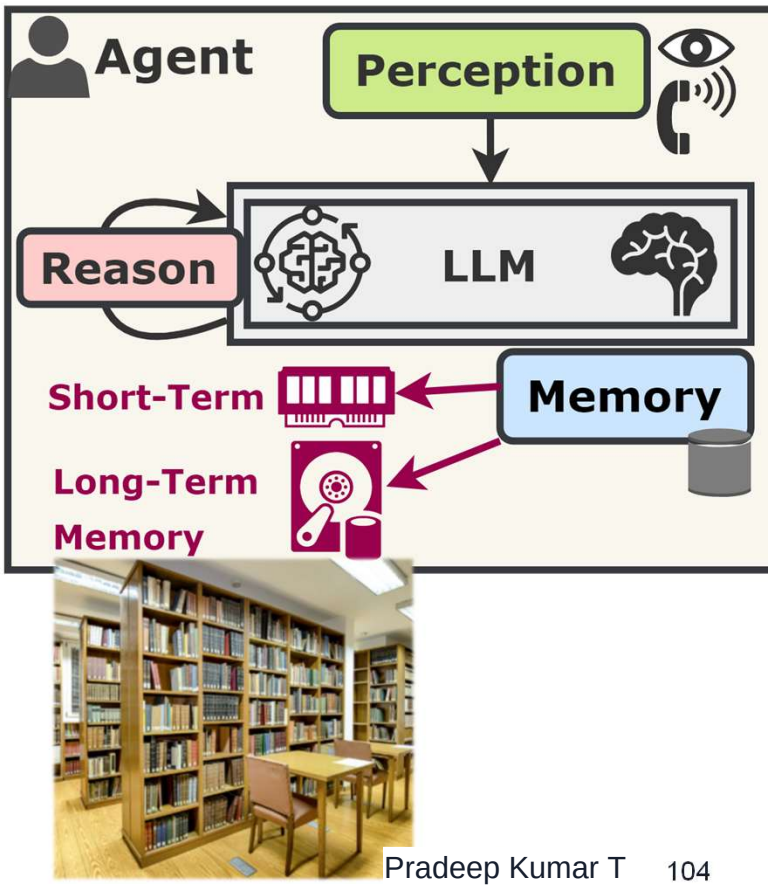


Deep Dive 2: Long-Term Memory (The "Library")

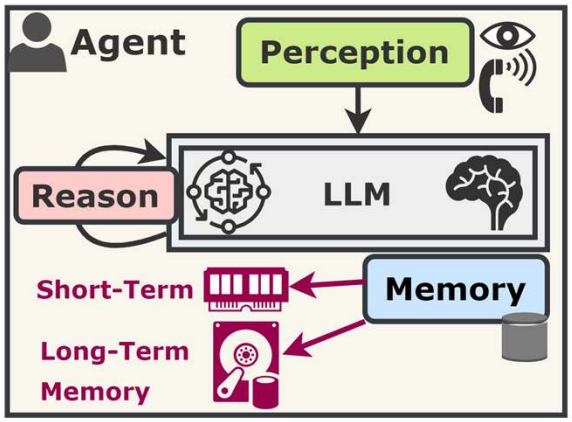
Implementation: An external, persistent database

Role:
To store all information the agent might need, forever
Past conversations, User preferences, Large documents, Knowledge graphs

Solves the Context Window problem
The agent doesn't need to "remember" everything at once
It just needs to know how to look things up in its library



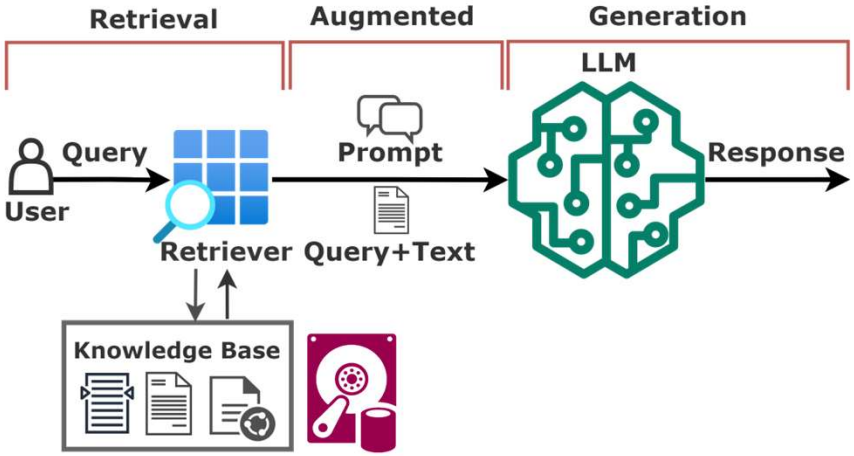
How LTM and STM Work Together (RAG)



Implementation: Retrieval-Augmented Generation (RAG)

The Loop:

1. (Perceive): User asks a question
2. (Plan): The "brain" (LLM) realizes it needs external data
3. (Act - Retrieve): The agent searches its Long-Term Memory
4. (Retrieve): The LTM returns a small, relevant fact
5. (Augment): This small fact is "loaded" into the Short-Term Memory
6. (Generate): The LLM now "sees" the question and the answer



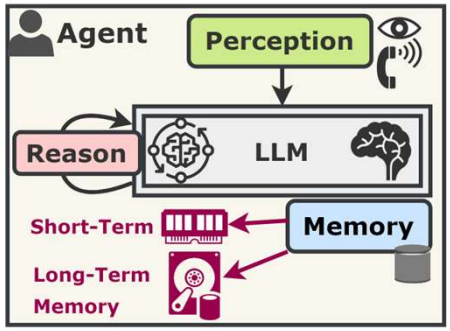
Architectural Patterns for Long-Term Memory

1. Key-Value / Document DBs (Firestore, MongoDB)
Storing structured facts and conversational history
{"user_id": "123", "preferences": {"seat": "window"}, "chat_history": [...]}

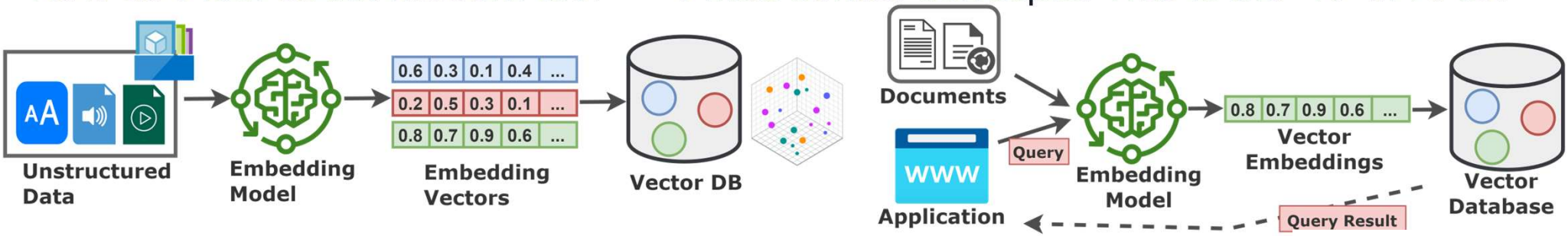


2. SQL Databases (PostgreSQL)
Storing highly relational, structured enterprise data
SELECT * FROM orders WHERE user_id = 123





3. Vector Databases (Pinecone, ChromaDB)
Storing conceptual or semantic meaning
"How do I feel about modern art?" -> Finds similar concepts. This is the "R" in RAG.



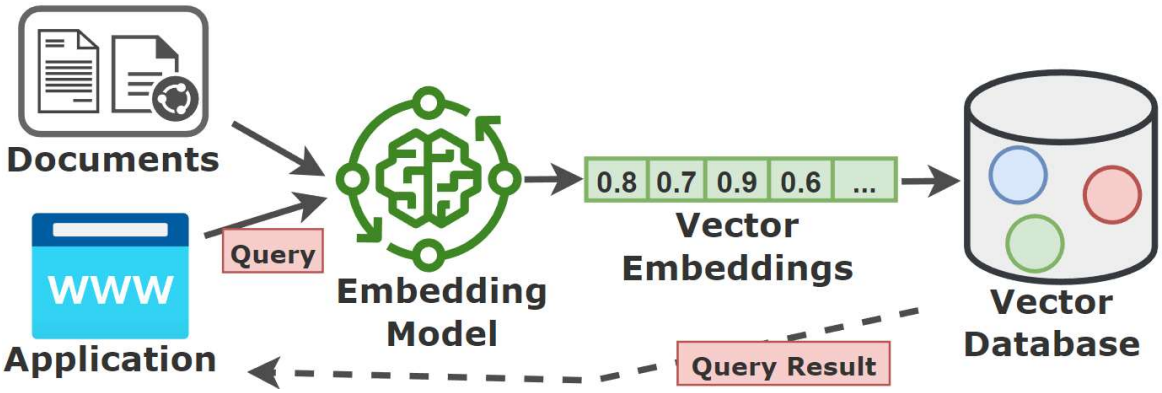
What is a Vector Database?

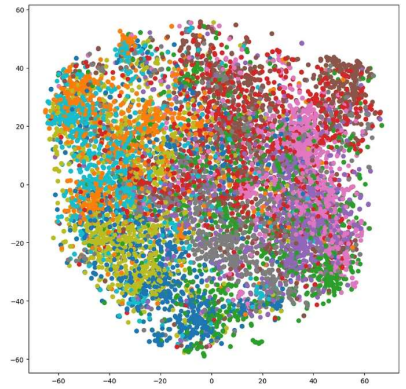
Vector Database is a **specialized database** designed to store, manage, and query **high-dimensional vectors**

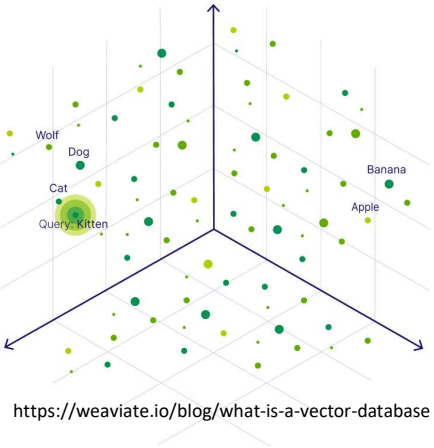
Vectors are **numerical representations** that capture the **semantic meaning** of data

Indexes and **stores vector embeddings** for **fast retrieval** and **similarity search**

Enable **contextual searches based on** the **meaning** behind the data







https://weaviate.io/blog/what-is-a-vector-database

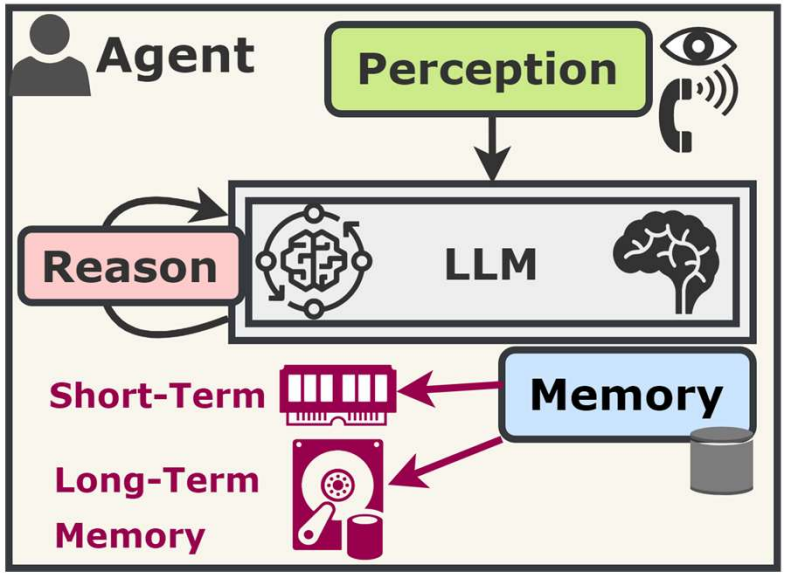
Summary – The Memory

Memory is **essential** to **break** the "**Groundhog Day" loop**

We **must architect 2 types: Short-Term (Context Window)** and **Long-Term (Databases)**

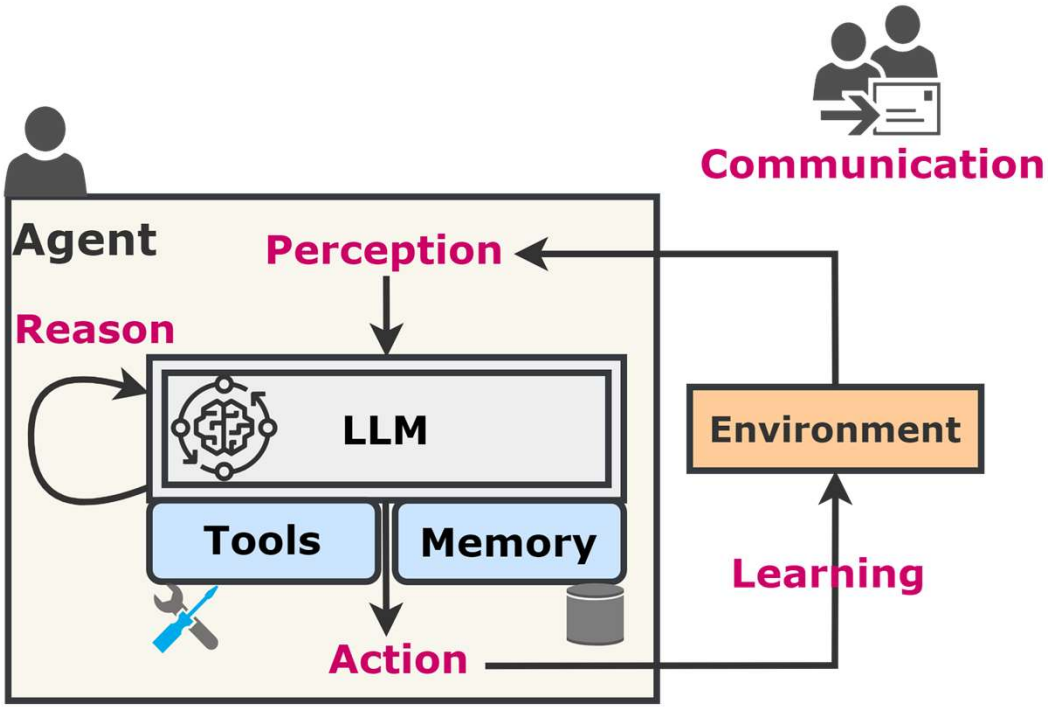
The **process** that **connects** them is **RAG (Retrieval-Augmented Generation)**

Our **choice** of **LTM** (**SQL**, **Vector**) **depends on** the **type of data** we need to store



Our Roadmap: A Component-by-Component Deep-Dive

1. Foundation Model (The "Brain")
2. Perception (The "Senses")
3. Planning (The "Strategist")
4. Memory (The "Notebook")
5. Action (The "Hands")
6. Learning (The "Feedback Loop")
7. Communication (The "Voice")



The "Hands" - Action and Tool Calling

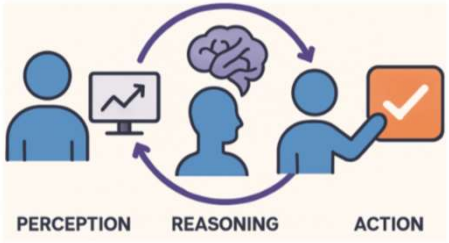
Agent's **'brain'** (**LLM**), its **'senses'** (**Perception**), its **'strategist'** (**Planning**), and its **'notebook'** (**Memory**)

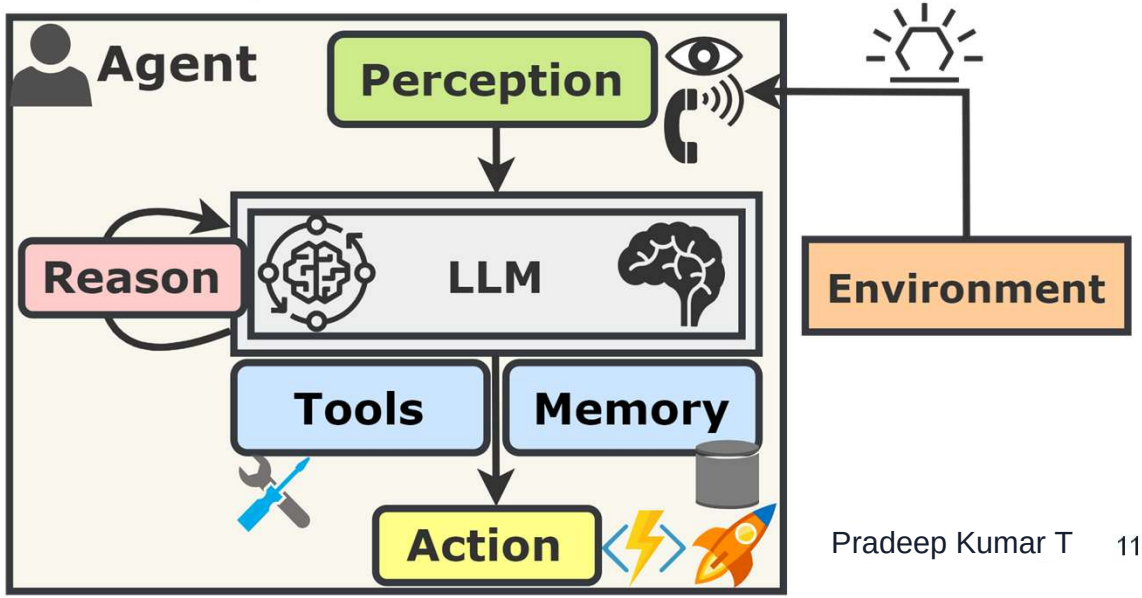
Our agent can **perceive** the **world**, **create** a **plan** and **remember** its **history**

But a **plan** is **just a thought**. It is a **data structure**. It is **not an action**

How we give the **agent 'hands'** to **execute** its **plan** and **affect** the **real world**

This is the 'Act' in our Perceive -> Reason -> Act loop





What is a "Tool"?

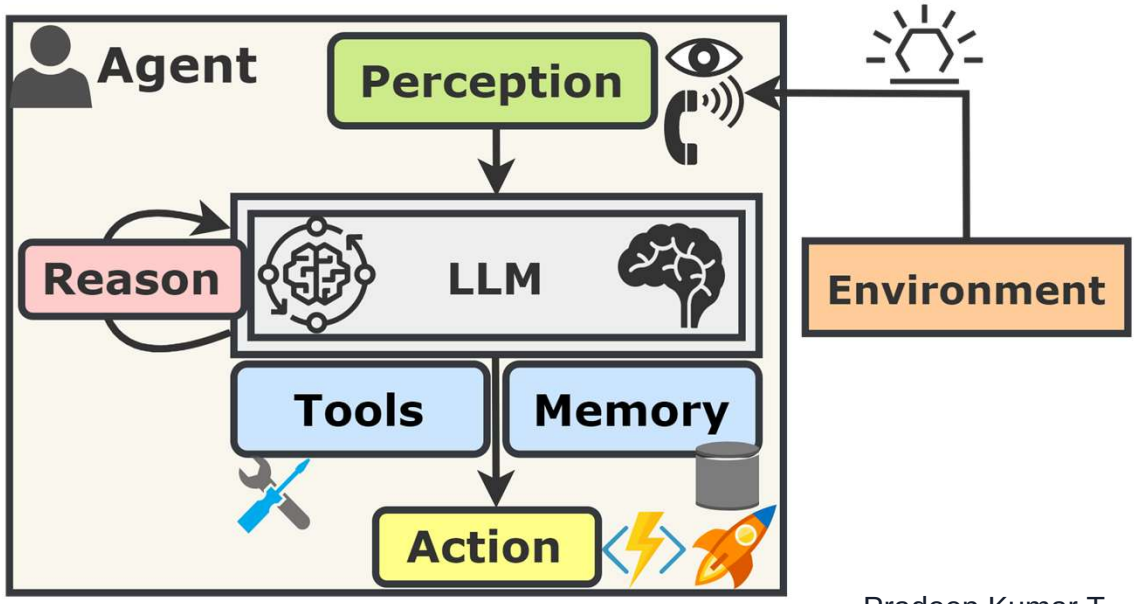
A "**Tool**" is **any function**, **script**, **API** or **external system** that the agent is given **permission** to **execute**

This is **what separates** a **passive chatbot from** an **active agent**

A **chatbot tells** you the **weather**, An **agent fetches** the **weather**

Examples of Tools:

```
get_weather(location: str)
query_database(sql: str)
send_email(to: str, body: str)
run_python_script(code: str)
call_google_calendar_api()
```



The Mechanism: "Function Calling"

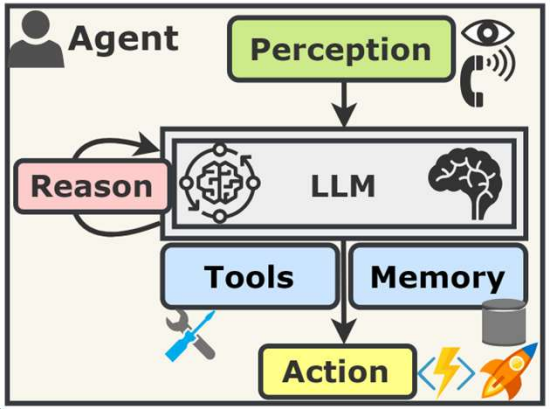
How does a the **LLM ask** our **code** to **run a function** ?

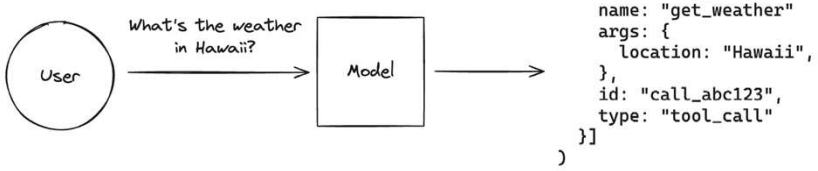
It uses a **structured output** called **Function Calling** or **Tool Calling**

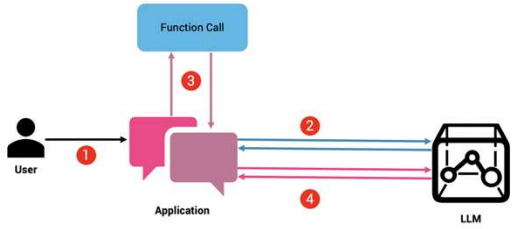
1. We "Tell" the LLM its Tools
Pass a schema (a list of function) to the LLM in the prompt

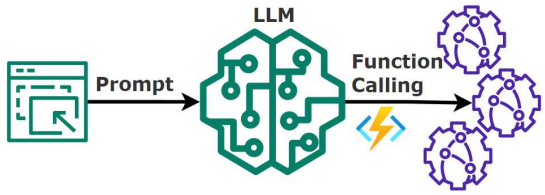
2. LLM "Plans" a Call
It's a structured JSON object matches the tool's schema

3. Our System "Executes"
Our app code receives this JSON, pauses the LLM, and runs the actual Python or API call









The Loop in Detail: The "ReAct" Handoff

1. LLM (Brain)
Plan: I need the weather -> Generates JSON: {"call": "get_weather", "location": "Paris"}

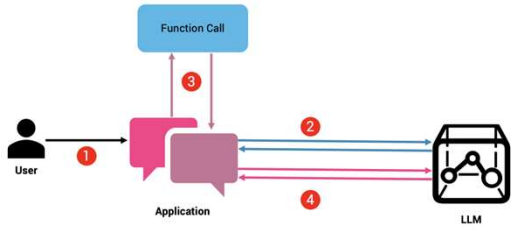
2. ACTION MODULE (Orchestrator)
Catches this JSON. "The brain wants a tool"

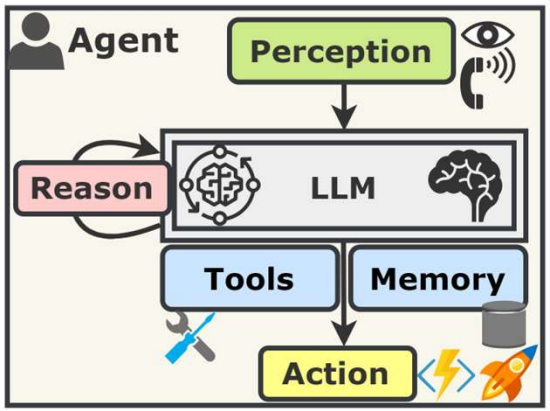
3. ACTION MODULE (Hand)
Executes the real Python function: def get_weather("Paris")

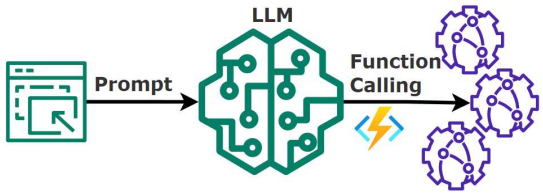
4. TOOL
The function runs, returns: {"temp": "15C", "condition": "Cloudy"}

5. ACTION MODULE (Hand)
Feeds this result (as text) back into the LLM's "Perception«

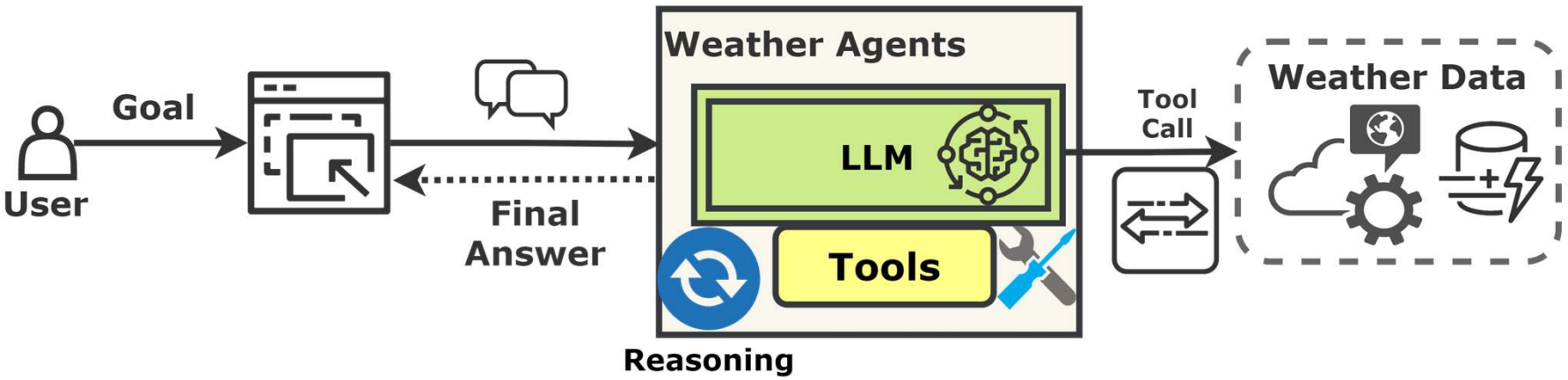
6. LLM (Brain)
The weather in Paris is 15C and Cloudy. Now I can answer the user's question



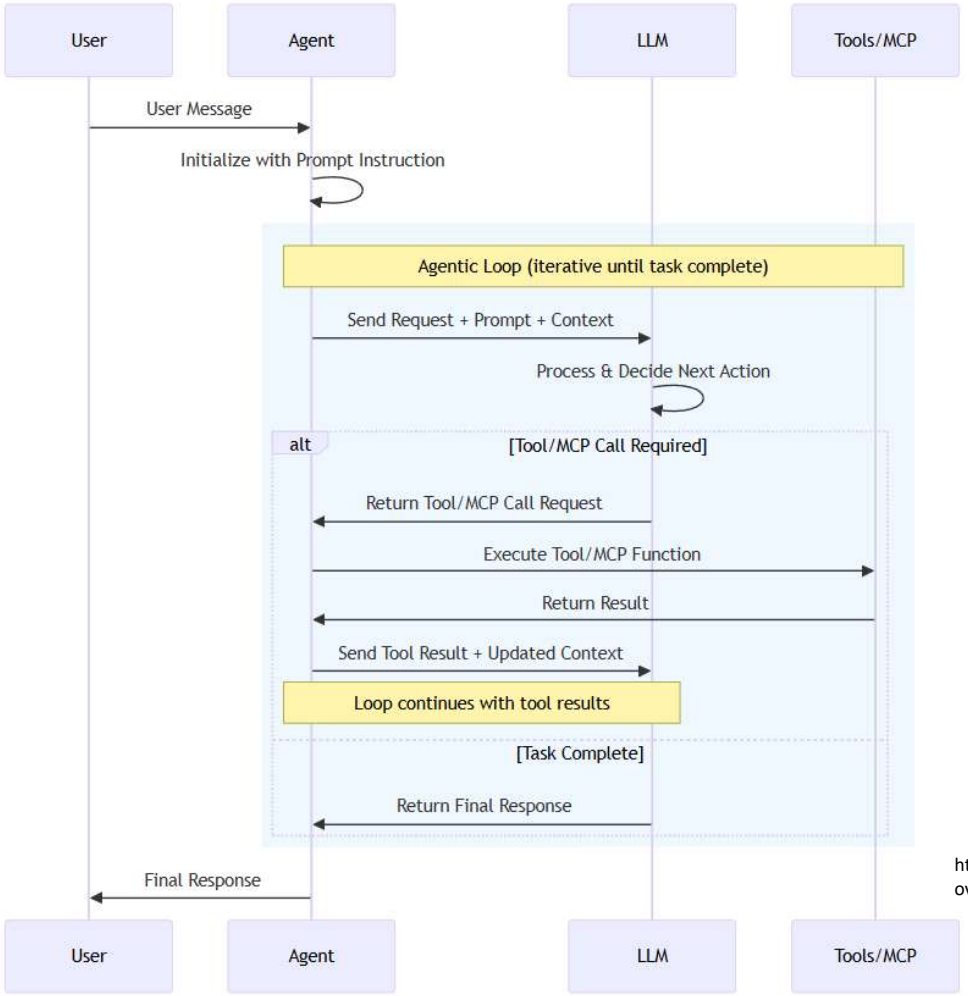




The Architecture: How the Components Connect



Designing Weather Agent – Sequencial Diagram



https://learn.microsoft.com/en-us/agent-framework/overview/agent-framework-overview?toc=%2Fdotnet%2Fai%2Ftoc.json&bc=%2Fdotnet%2Fai%2Ftoc.json

The Critical Topic: Security & Guardrails

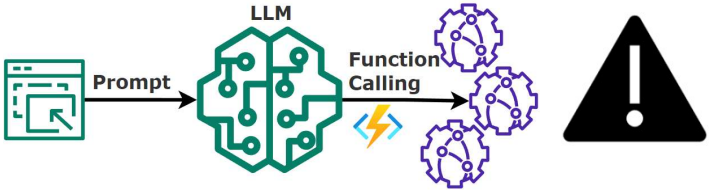
This is the **most important architectural topic** for this module

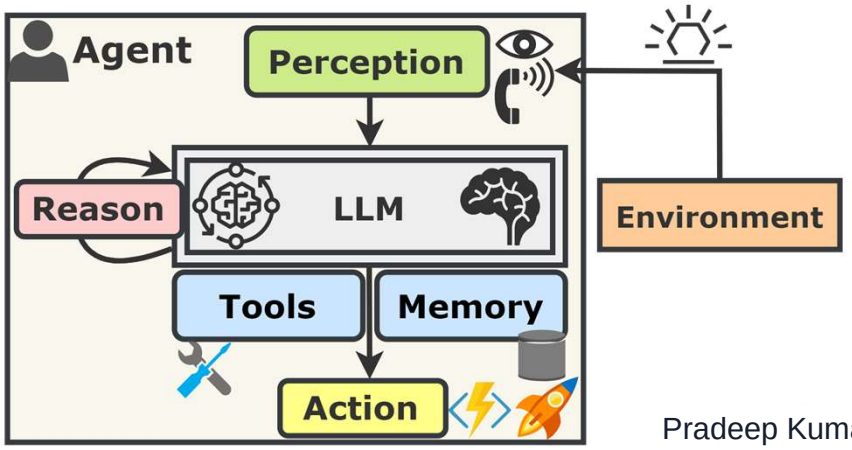
We are **giving** an **autonomous**, **non-deterministic system access** to our **APIs** and **code**

The Nightmare Scenario:
Goal: Clean up old users from the test database
Agent's "Logical" Plan: call_tool("DELETE /users")
The Problem: The agent accidentally targets the production database



Our job as architects is not just to "enable" tools, but to constrain them





The Critical Topic: Security & Guardrails



1. Principle of Least Privilege (Permissions)

Never give an agent a "master key" or general-purpose tools (like run_python_script)

A "Weather Agent" should only have the get_weather tool



2. Read-Only vs. Write Access

Strongly prefer "read" tools (get_weather, query_database)

Be extremely cautious with "write" tools (send_email, delete_record)

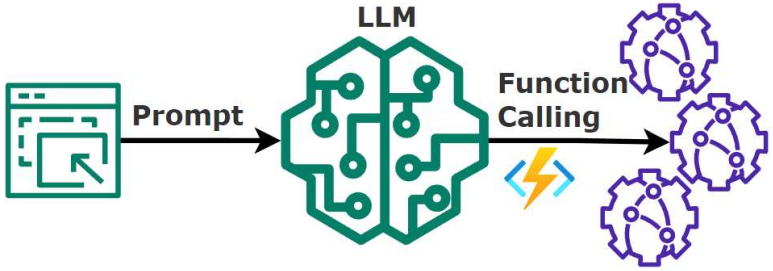


3. Human-in-the-Loop (HIL) Confirmation

The ultimate safety net for all "write" or "destructive" actions

The agent plans the action: {"call": "send_email", "to": "..."}

System pauses and asks a human: "Agent wants to send this email. [Approve] [Deny]



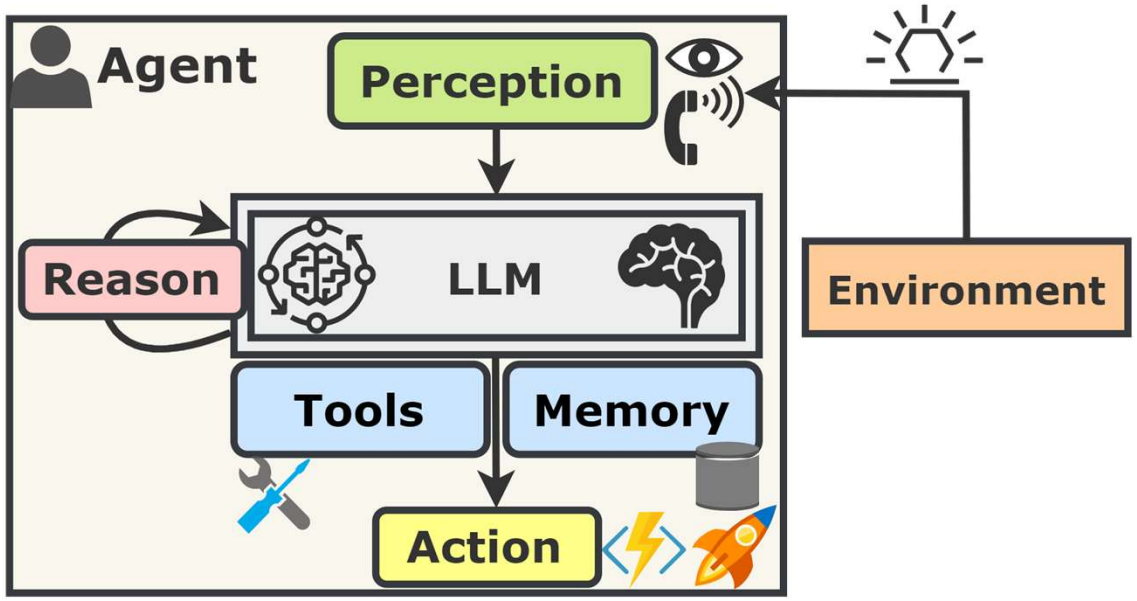


Summary – The "Hands" Module (Action)

The **Action Module** gives the **agent** "**hands**" and **makes** it **agentic**

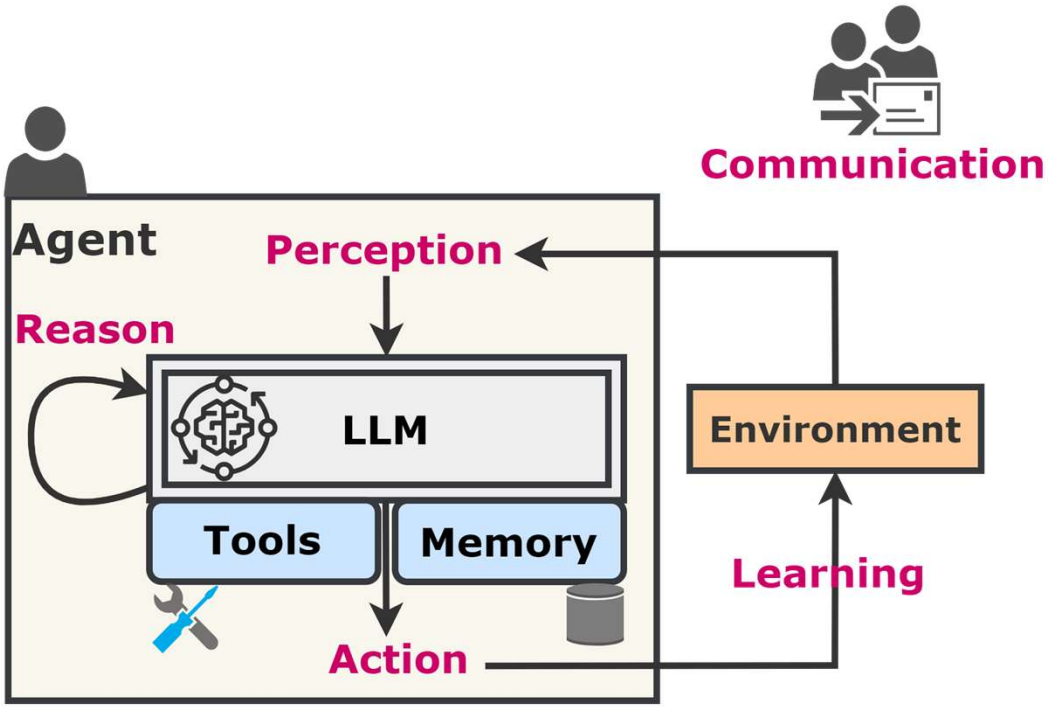
The "**Function Calling**" is the **technical bridge** from the "**brain**" to the "**hands**"

Security is **not an add-on**; it is the **core** of this **module's design**, implemented via **Least Privilege** and **Human-in-the-Loop**



Our Roadmap: A Component-by-Component Deep-Dive

1. Foundation Model (The "Brain")
2. Perception (The "Senses")
3. Planning (The "Strategist")
4. Memory (The "Notebook")
5. Action (The "Hands")
6. Learning (The "Feedback Loop")
7. Communication (The "Voice")



The "Feedback Loop" - Learning and Adaptation

Our agent has a **'brain'** (**LLM**), **'senses'** (**Perception**), a **'strategist'** (**Planning**), a **'notebook'** (**Memory**) and **'hands'** (**Tool Calling**)

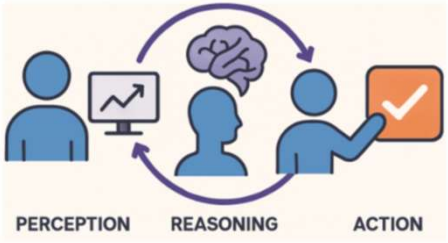
Our **agent can execute**. But it has **one profound** flaw: **it is static**

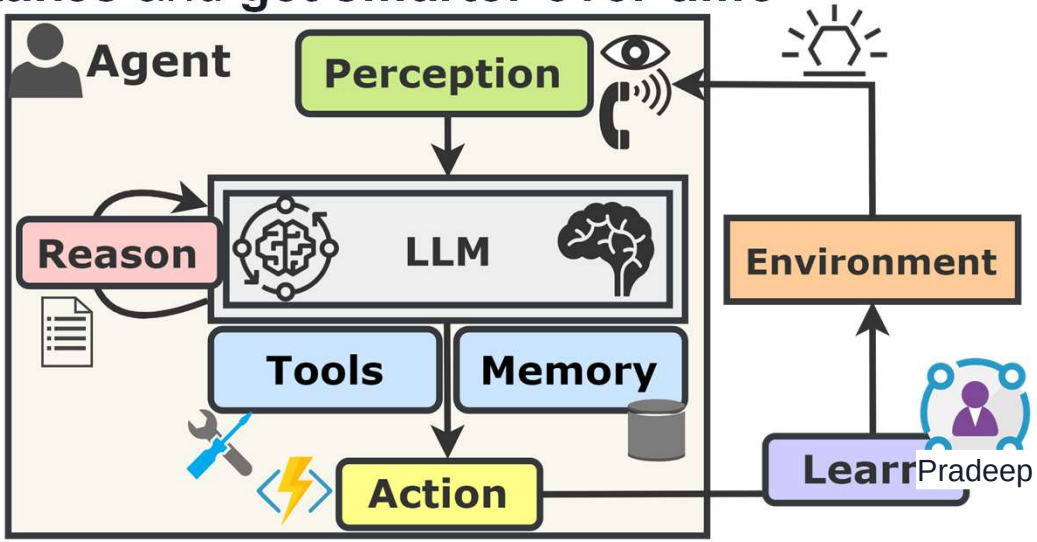
If its **plan** is **inefficient**, or its **"remembered" facts** in its **memory** are **wrong**

It has **no mechanism** for **self-improvement**

The **agent** to **learn** from its **mistakes** and **get smarter over time**

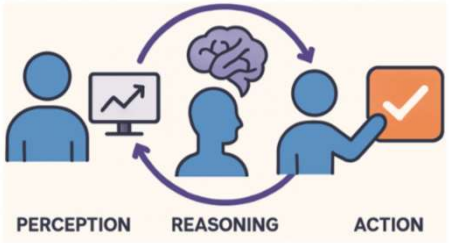
Complete core "Perceive -> Reason -> Act" loop



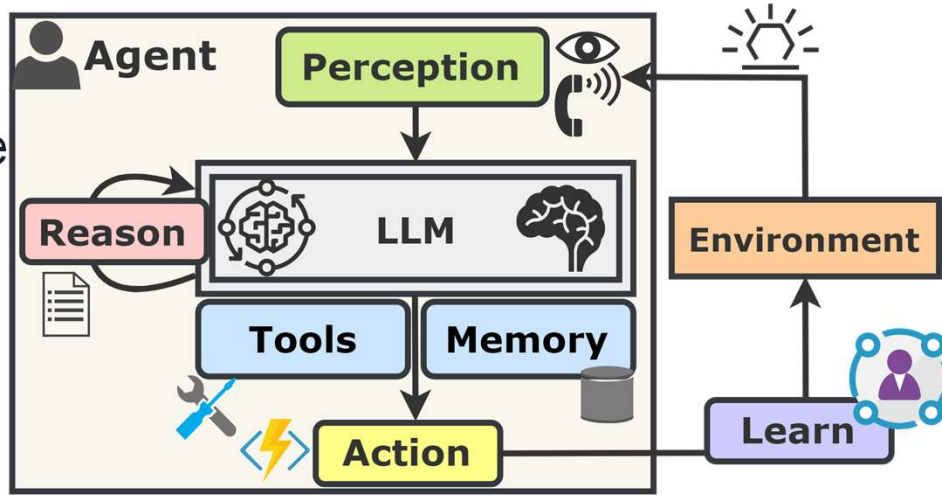


The "Two-Loop" Architecture

Loop 1: The "Performance Loop" (The Actor)
This is the entire agent we have built so far
Its job is to Do Things (Perceive -> Reason -> Act)
It operates on a fast, turn-by-turn basis



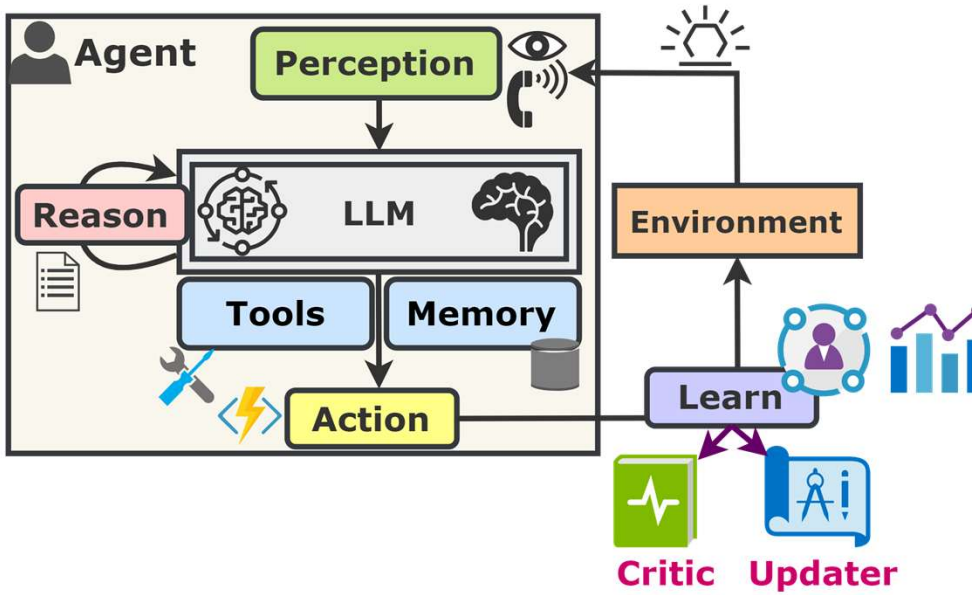
Loop 2: The "Learning Loop" (The Updater)
This is a new, higher-level loop
Its job is to Improve the Actor, It operates on a slower basis, observing the Actor's performance



The New Architectural Components: "Critic" & "Learner"

1. The "Critic" (or Performance Evaluator)
The "evaluator" or "teacher"
Observes the Actor's action and its outcome.
Compares outcome to a 'standard of success' and generates a feedback signal

The agent asks:
Did my last action (search_bing) achieve its intended sub-goal?
Why did it fail? (API Key invalid)
Was this action efficient? ('This tool was slow')
Did this action actually help me achieve the overall user goal?

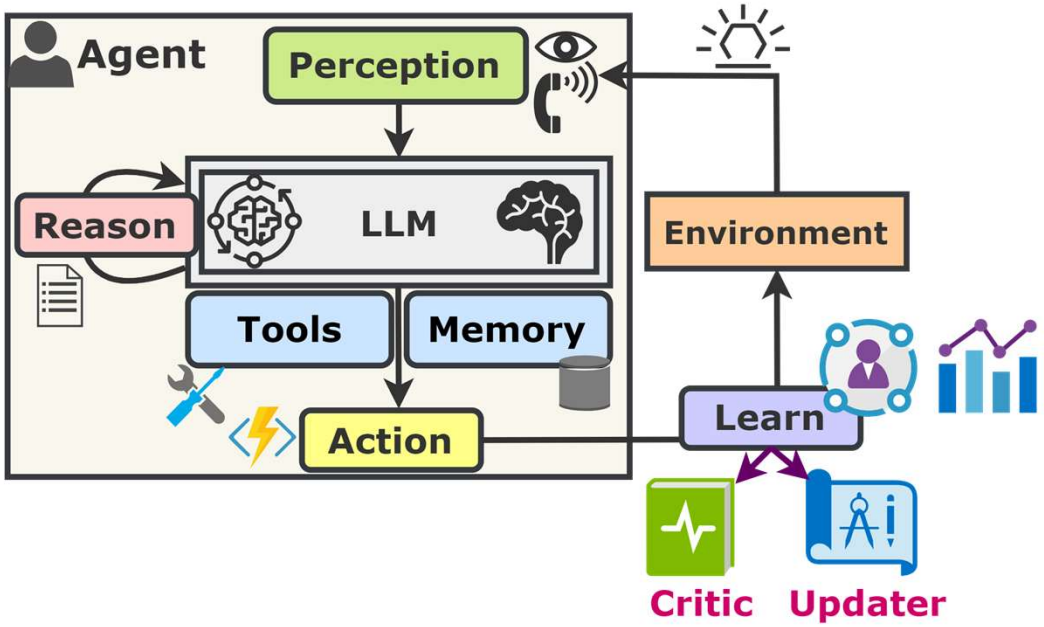


The New Architectural Components: "Critic" & "Learner"

2. The "Learning Element" (The Updater)

The "mechanic" or "engineer"

Receives the "feedback signal" from the Critic and performs the technical update on the Actor's components



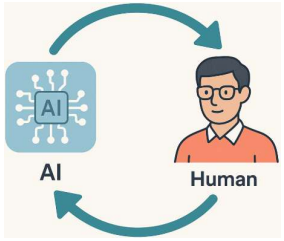
How the "Critic" Works (The Feedback Signal)

1. Explicit Feedback (Human-in-the-Loop)

The user is the Critic

The "This is Spam" button in your email. The "thumbs up / thumbs down" icon on a chatbot response. A user correcting the agent





2. Implicit Feedback (Environmental)

The agent is its own Critic. It learns from observing the consequences of its own actions

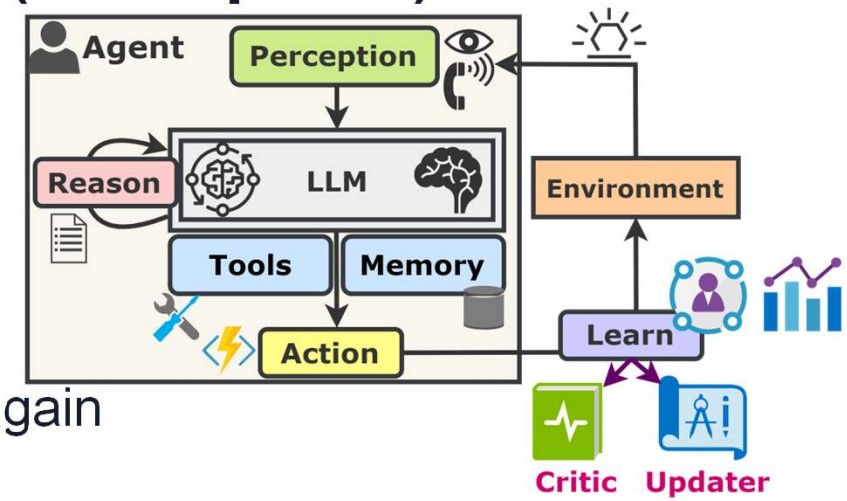
A chess AI losing the game (that's the signal). An agent's API call returning an error. A sales agent's email failing to get a reply



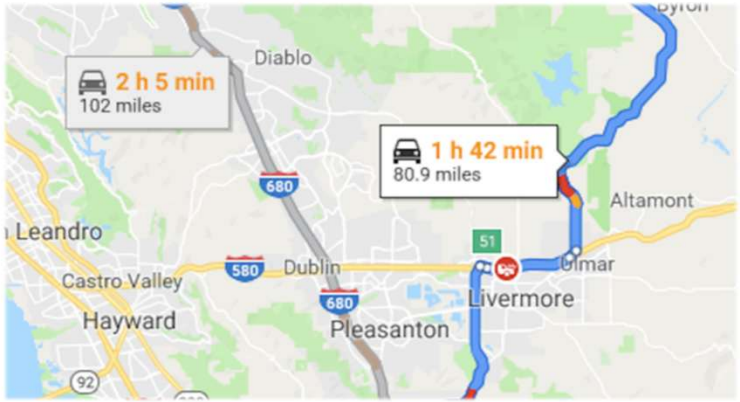
How the "Learning Element" Works (The Update)

1. The Memory (LTM)
Feedback: This fact is wrong
Update: vector_db.delete(fact_id)

2. The Planning Module
Feedback: This 5-step plan failed
Update: Store a 'memo' in LTM to not try this plan again



3. The Utility Function (Most Advanced)
Feedback: The user hated the 'fastest' route
Update: Slightly increase the negative weight for 'toll roads' in the utility function (This is Reinforcement Learning)



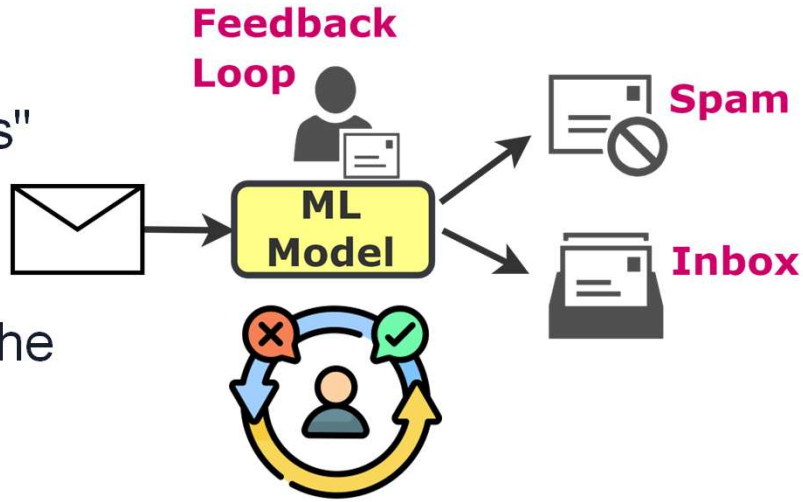
Example: The "Learning Spam Filter"

1. Performance Element ("Actor")
IF (subject contains "report") THEN (move to inbox)

2. Action & Outcome
A spam email with the subject "Free Report on Drugs"

3. Feedback Loop (Starts)
Critic: You mark the email as "Spam"
Learning Element: Receives this signal. It analyzes the text of the email you flagged

4. The Update
The Learning Element updates the Actor's rules. The rule is now:
IF ("report") AND NOT ("Drugs") THEN (inbox)

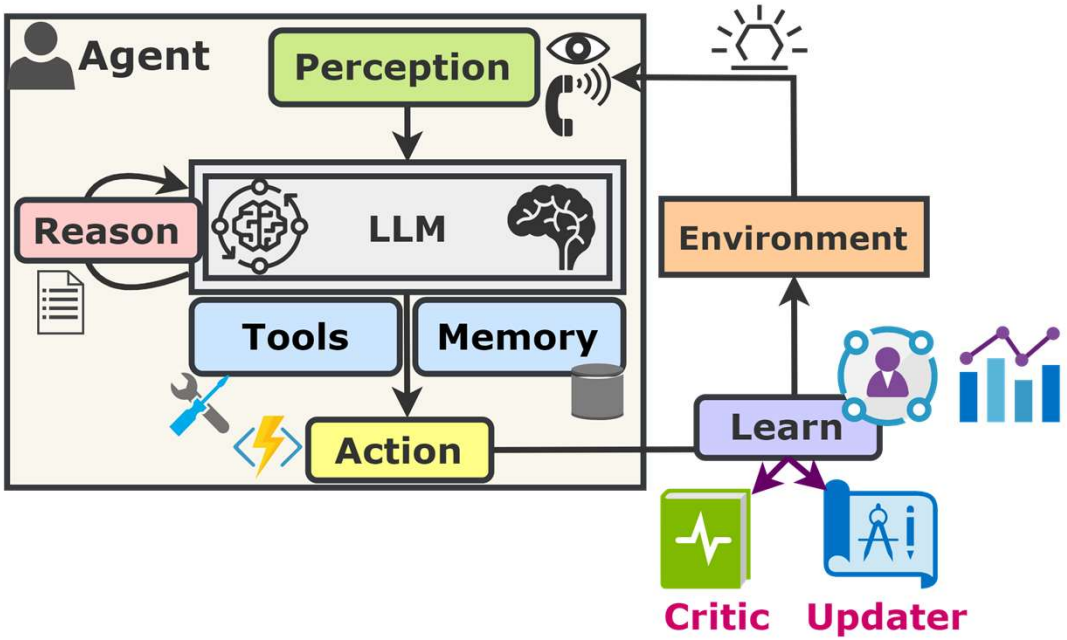


Summary – The " Feedback Loop" Module (Learning)

This is a "**meta-loop**" that **improves** the **agent**

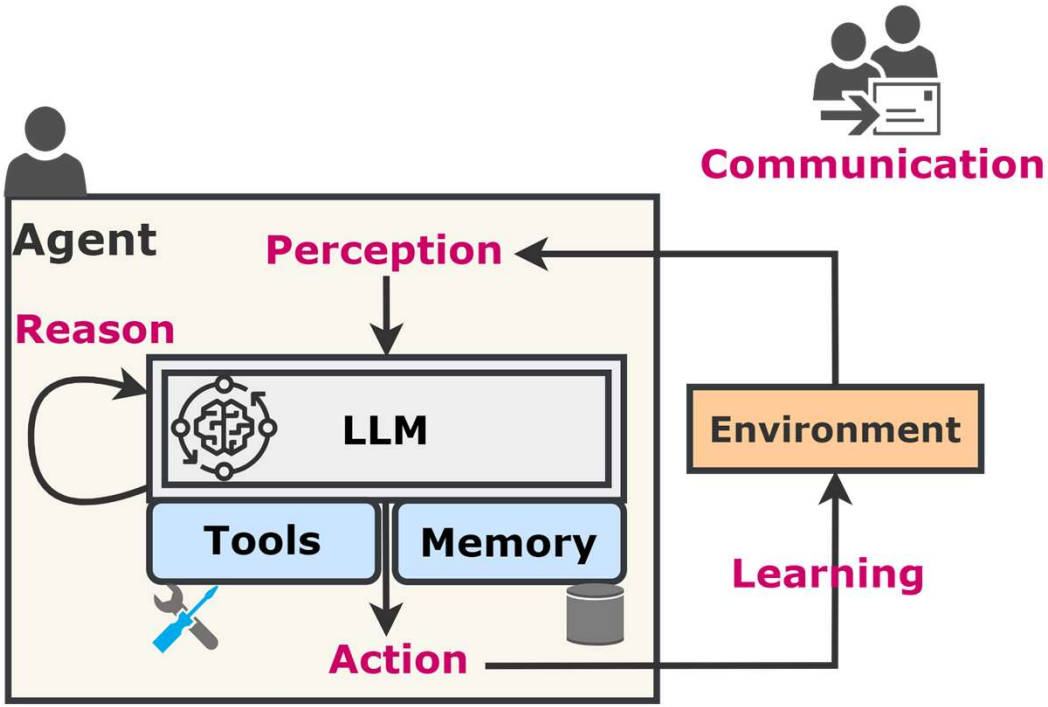
It is built from **2 components**: **The "Critic"** (to **evaluate**) and **the "Learning Element"** (to **update**)

The **most powerful** but also the **most complex**, **expensive**, and **unpredictable architecture**



Our Roadmap: A Component-by-Component Deep-Dive

1. Foundation Model (The "Brain")
2. Perception (The "Senses")
3. Planning (The "Strategist")
4. Memory (The "Notebook")
5. Action (The "Hands")
6. Learning (The "Feedback Loop")
7. Communication (The "Voice")

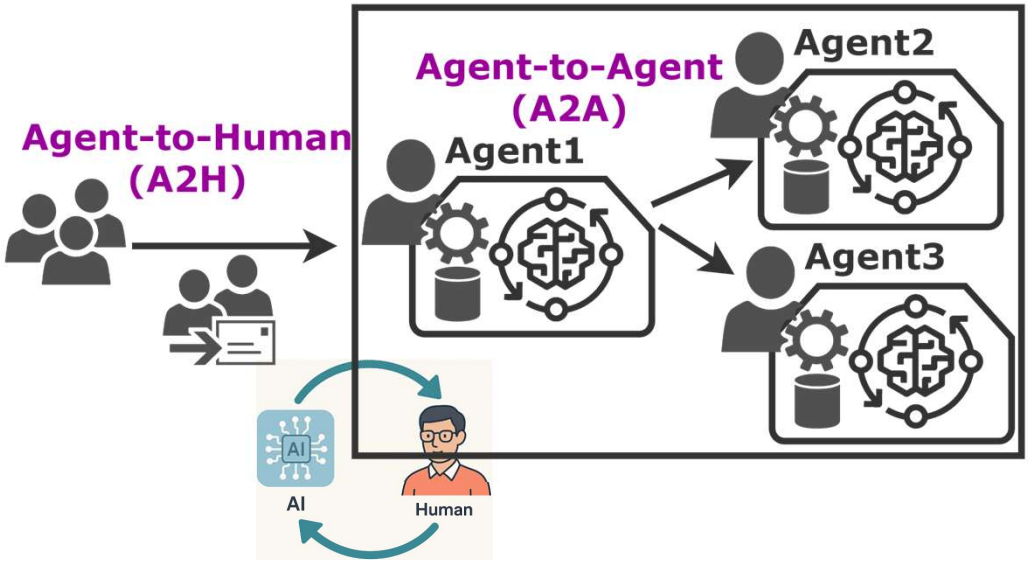


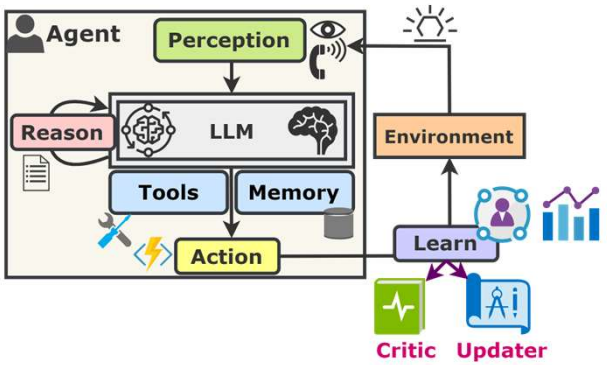
The "Voice" - Agent Communication

Our agent has **'brain'** (**LLM**), **'senses'** (**Perception**), a **'strategist'** (**Planning**), a **'notebook'** (**Memory**), **'hands'** (**Tool Calling**) and **'feedback loop'** (**Learning**)

Our **agent** can **execute** a **complex**, **multi-step plan**. It can **run** a **tool**, **get** a **result** and **update** its **internal memory**

How does the agent communicate this result to the outside world?



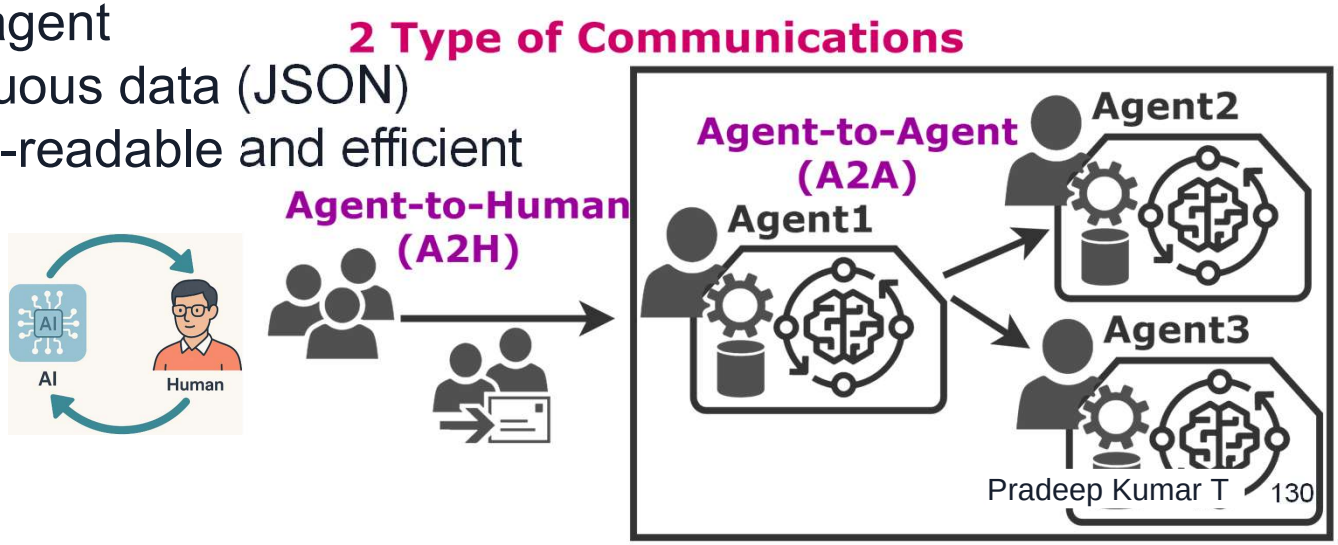


The "Audience" Problem: Two Types of Communication

An **agent's "voice" must** be **designed** for its "**audience**"

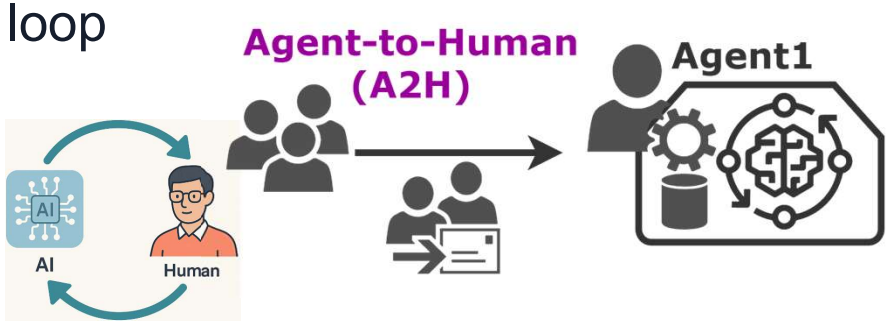
1. Agent-to-Human (A2H) Communication
Audience: The end-user
Format: Unstructured, natural language
Goal: To be helpful, empathetic and easy to understand

2. Agent-to-Agent (A2A) Communication
Audience: Another software agent
Format: Structured, unambiguous data (JSON)
Goal: To be precise, machine-readable and efficient



Deep Dive 1: Agent-to-Human (A2H) Communication

This is the **final step** of the **agent's** "**Reasoning**" loop
It is a "**Synthesis**" call to the **LLM** (the "**Brain**")



The Process:

1. The agent has finished its tool-use loops

2. Internal State:

{"goal": "Book flight to Paris", "result": {"id": "TK-123", "status": "CONFIRMED"}}

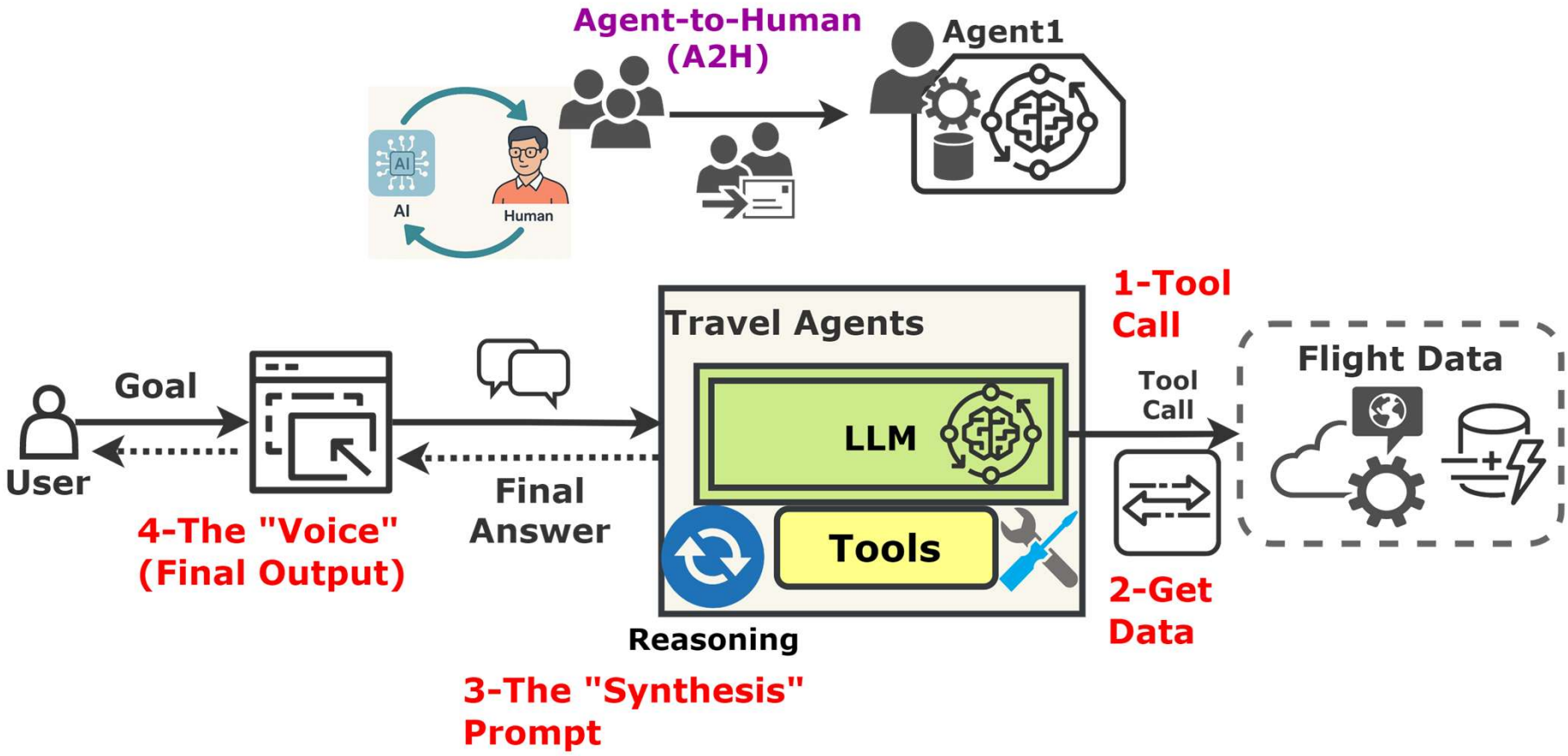
3. The "Synthesis" Prompt:

You are a helpful travel assistant. Your internal system has just completed the user's request. Here is the final data. Please inform the user in a friendly, natural way

4. The "Voice" (Final Output):

Great news! Your flight to Paris is all booked. Your confirmation ID is TK-123. Have a wonderful trip!

Deep Dive 1: Agent-to-Human (A2H) Communication

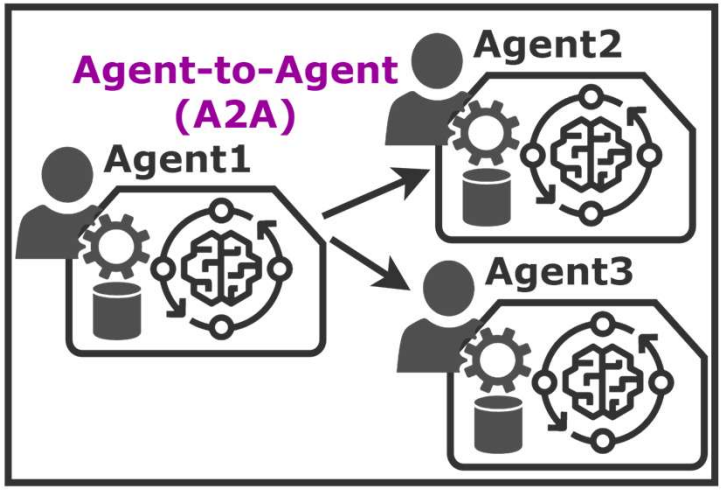


Deep Dive 2: Agent-to-Agent (A2A) Communication

This is the mechanism that powers **Multi-Agent Systems**

The Problem:
An "Orchestrator" agent needs to ask a "Specialist" agent to do a task
A2H (Bad): Hey Agent, can you please look up the weather for me in New York?
It's slow, ambiguous, expensive (an LLM call) and not machine-readable

The Examples:
Humans talk to each other in prose (A2H)
Servers talk to each other in APIs/JSON (A2A)
Agents must talk to each other like servers, not humans



The A2A Solution: Standardized Protocols

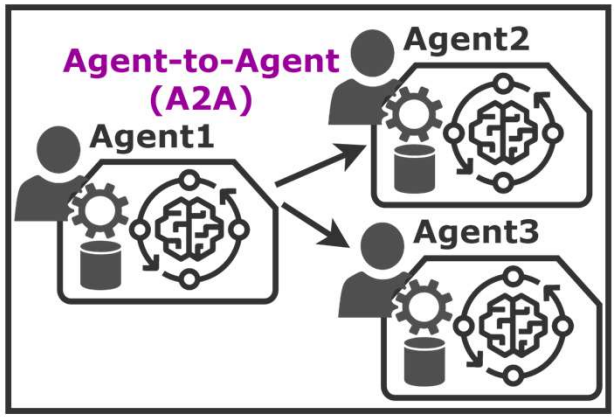
A2A communication must be a **structured**, **unambiguous**, **machine-readable**

Introduce the necessity of **protocols** like **MCP** (**Model-Context-Protocol**)

How it works:
An "Orchestrator" agent needs to ask a "Specialist" agent to do a task

An agent "speaks" by sending a formal JSON request:
{"agent_id": "travel_agent", "task": "get_weather", "parameters": {"location": "New York"}}

The Weather_Agent executes the task and responds with structured JSON:
{"status": "success", "data": {"temp": "18C", "condition": "Sunny"}}

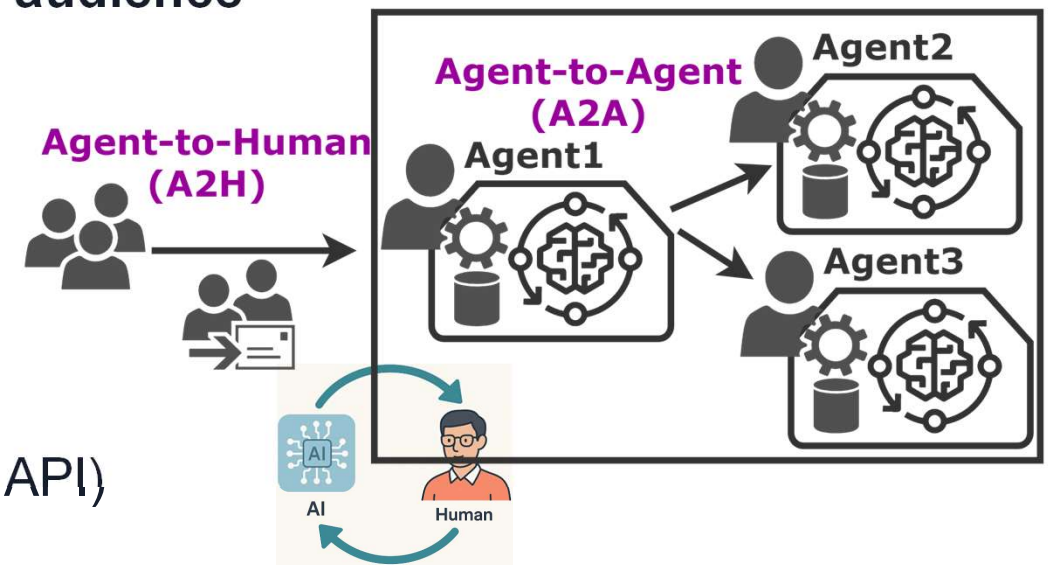


Summary – The " Voice" Module (Communication)

The "Voice" is a Translation module
Its architecture depends entirely on its audience

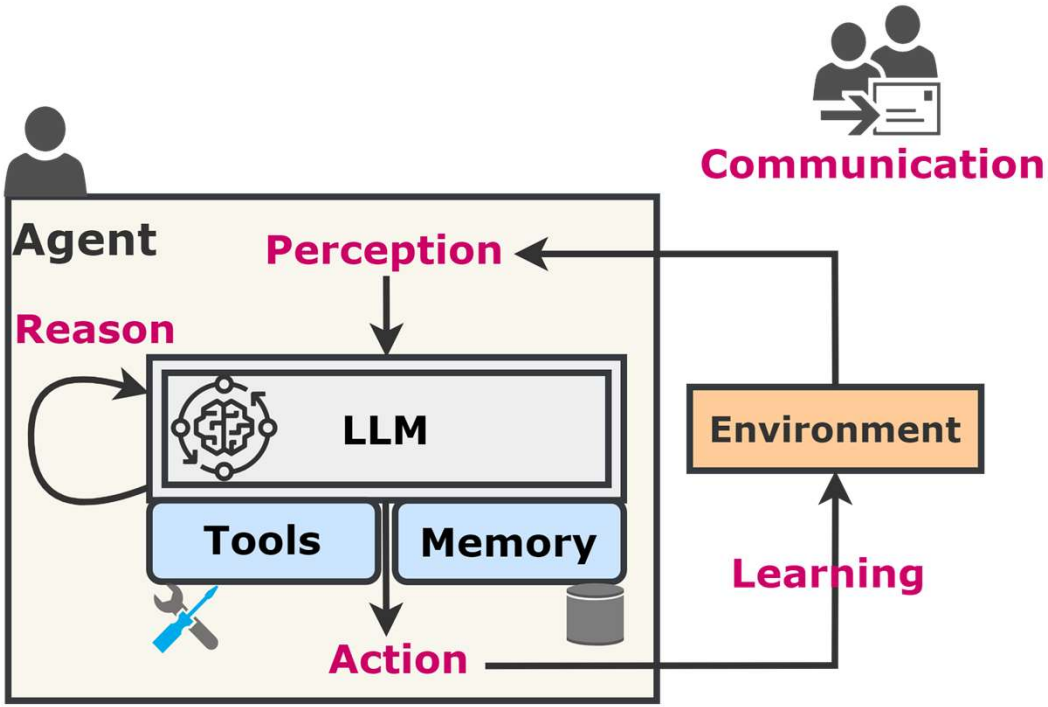
Agent-to-Human (A2H):
Voice is: A "Synthesis" prompt
Format: Natural, unstructured language
Goal: Helpfulness & Presentation

Agent-to-Agent (A2A):
Voice is: A structured "Protocol" call
Format: Machine-readable JSON (like an API)
Goal: Efficiency & Interoperability

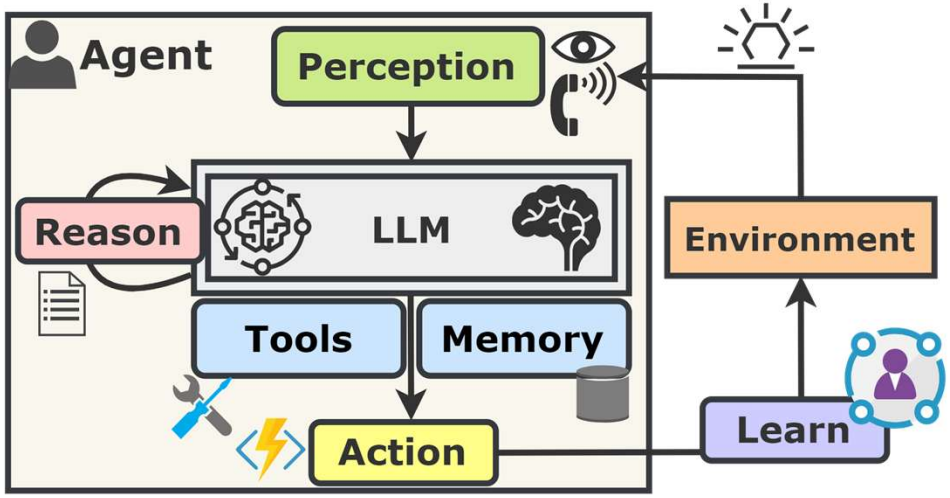


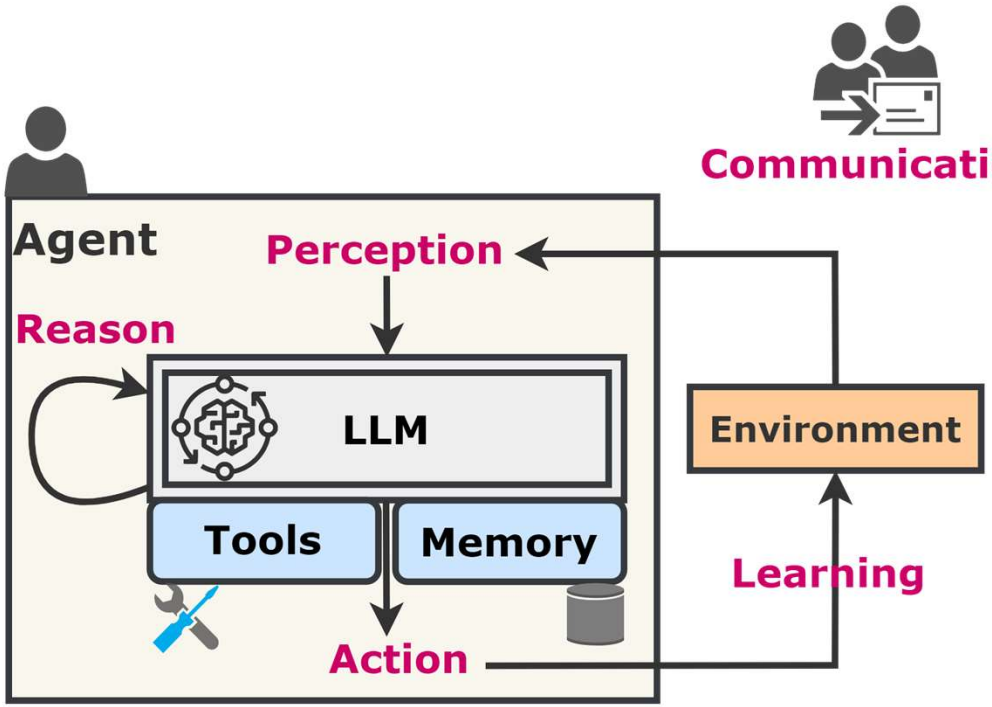
Our Roadmap: A Component-by-Component Deep-Dive

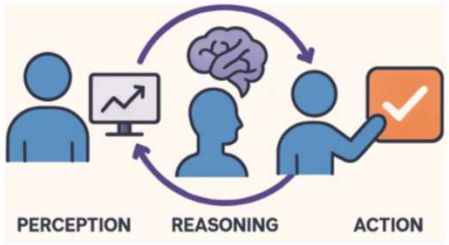
1. Foundation Model (The "Brain")
2. Perception (The "Senses")
3. Planning (The "Strategist")
4. Memory (The "Notebook")
5. Action (The "Hands")
6. Learning (The "Feedback Loop")
7. Communication (The "Voice")



[DESIGN] Building a "Research Assistant" Agent





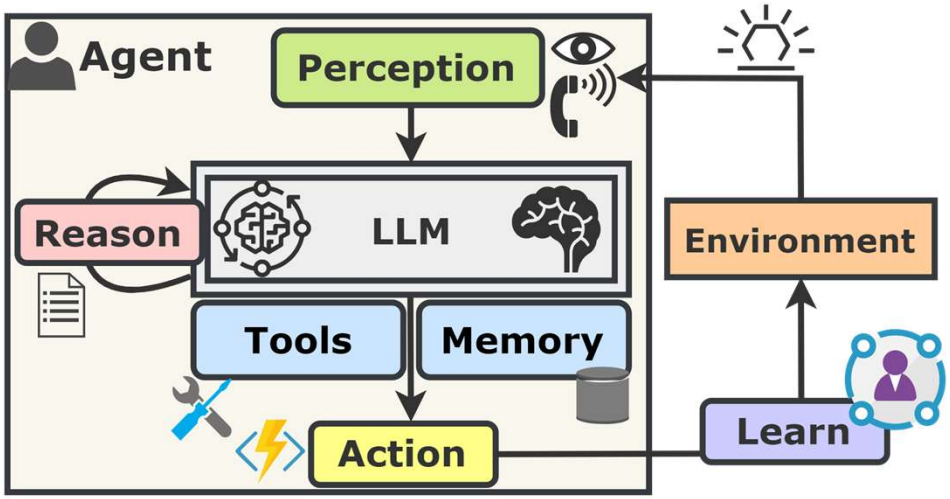


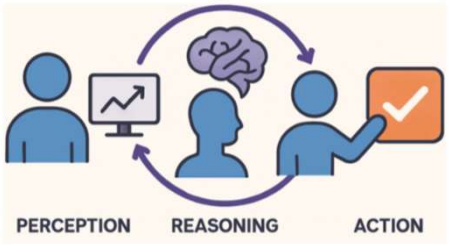
[DESIGN] Building a "Research Assistant" Agent

We **established** the **5-step 'recipe'** for **assembling** an **agent**

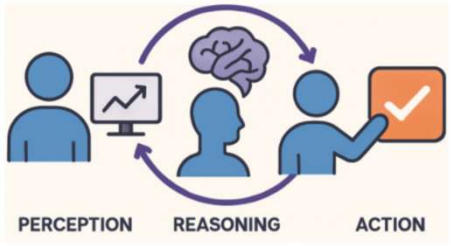
It is time to **move from theory** to **practice** and **build** a **complete**, **functioning agent from scratch**

We will **methodically follow** our **5-step recipe** to build a **'Research Assistant'** agent





The Lab Objective & Goal



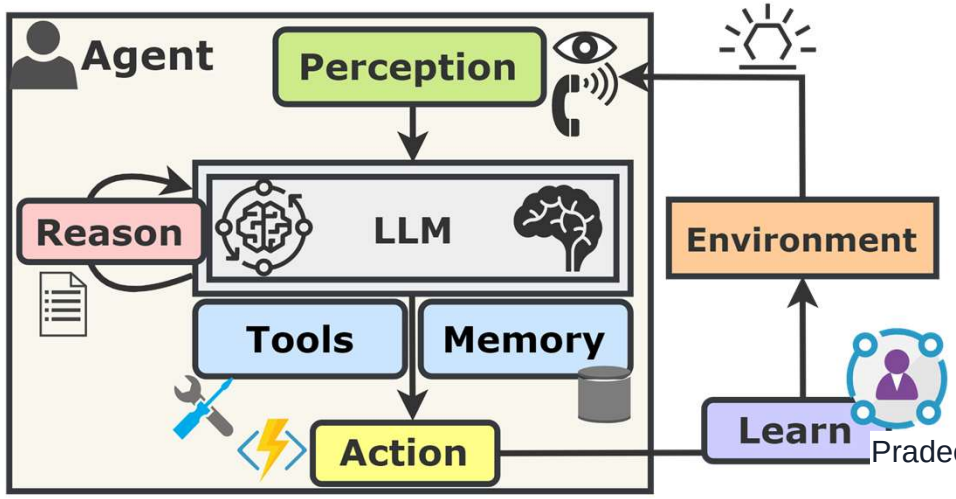
Our Task: To build a "Research Assistant" agent

Agent's Goal: To autonomously write a short, factual report on a complex topic

Example User Request (Perception):
"Please write me a short report on the history of Paris"

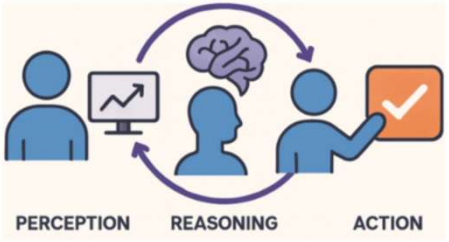
Why this is an Agentic Task:
The agent cannot know this from its internal training data (which might be outdated)
It must create a plan to:

1. search_for_info
2. analyze_results
3. write_report

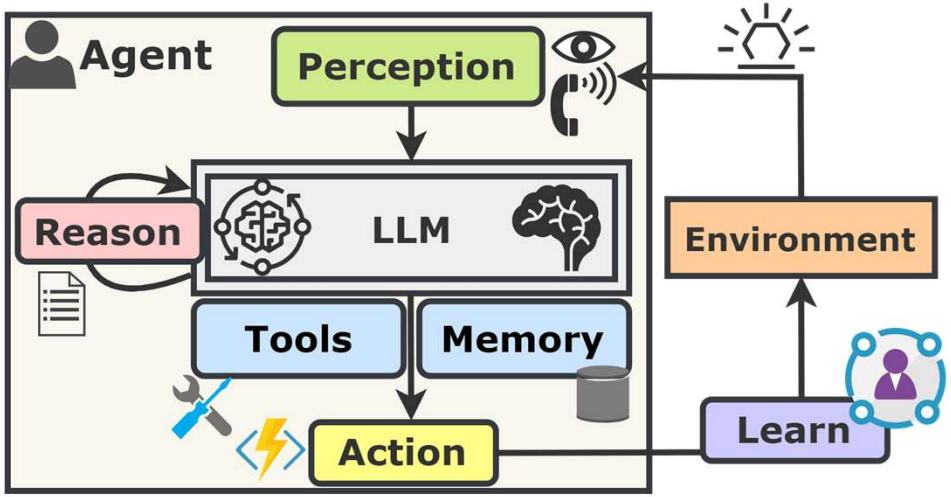


Our 5-Step "Recipe" (The Lab Plan)

We will follow the **5-step process** from our last lecture:



Step 1: Define the "Soul" (The System Prompt)
Step 2: Choose the "Brain" (The LLM)
Step 3: Define the "Hands" (The web_search Tool)
Step 4: Architect the "Memory" (A short-term "scratchpad")
Step 5: Write the "Loop" (The orchestration code)



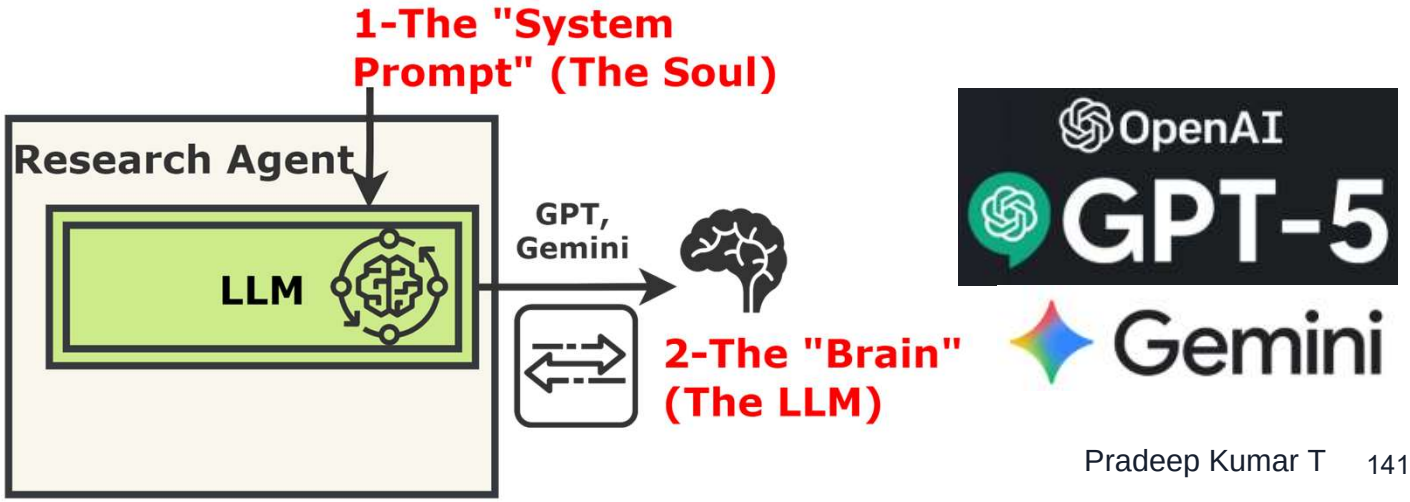
Steps 1 & 2: The "Soul" & "Brain"

Step 1: The "System Prompt" (The Soul):

You are a Research Assistant. Your goal is to write a factual report based on user requests. You must use the web_search tool to gather information. You must not answer from your own knowledge. You must first search, then analyze the results, then write the final report

Step 2: The "Brain" (The LLM):

This is a complex reasoning task, so we will choose a high-capability model (like GPT-5, Claude 4 or Gemini Advanced) that is excellent at planning and function calling



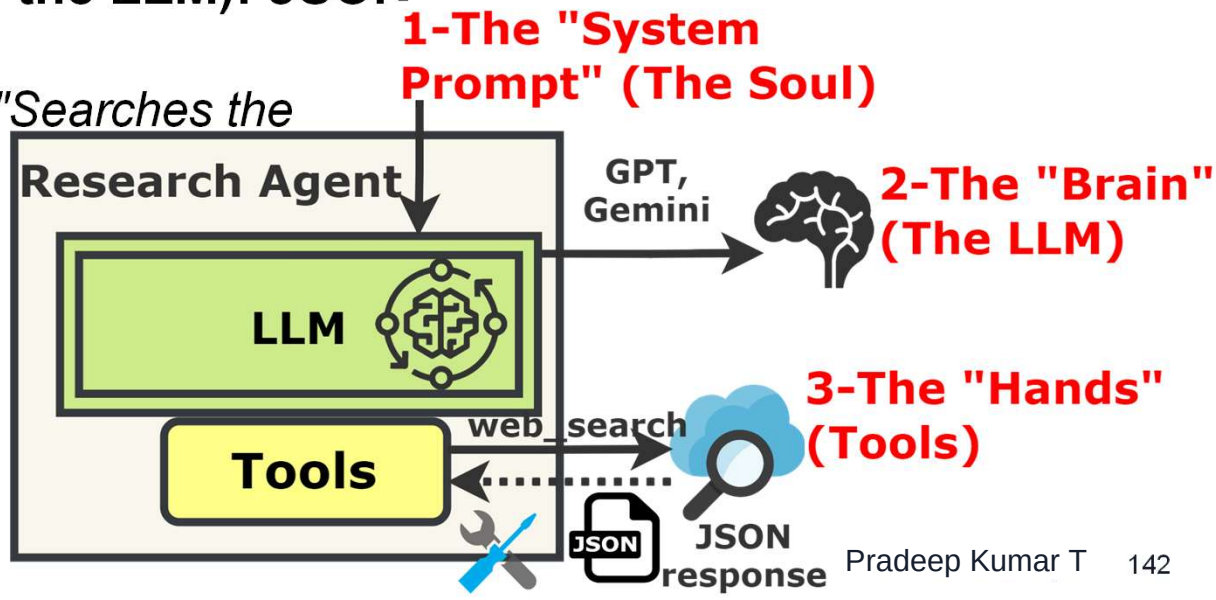
Step 3: The "Hands" (The Tool Definition)

1. The Code (The "Tool"):

```
def web_search(query: str):
  """Searches the web for information on a query.""«
  print(f"-- Executing Tool: web_search('{query}') --")
  return "The Eiffel Tower was built by Gustave Eiffel..."
```

2. The Schema (The "Manual" for the LLM): JSON definition

{"name": "web_search", "description": "Searches the web", "parameters": {"query": "string"}}



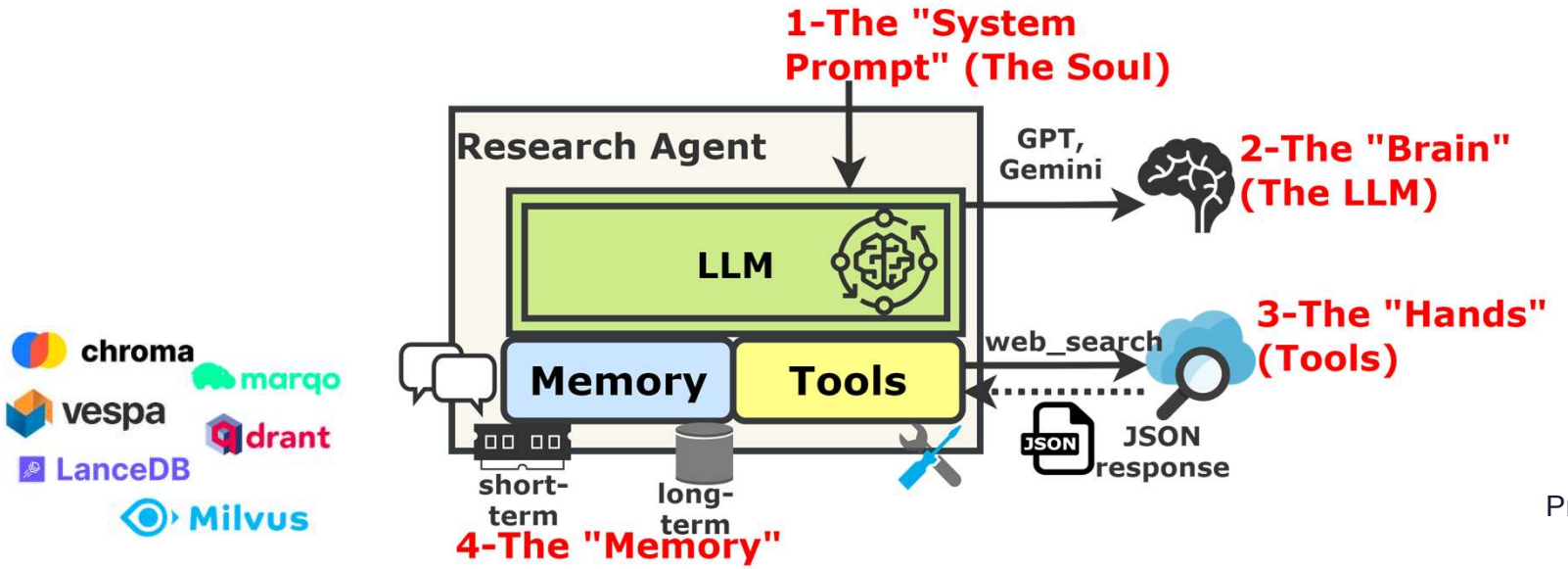
Step 4: The "Memory" (The "Scratchpad")

Short-Term Memory:

We will use a simple Python list to store the conversation history (chat_history = [])
This list will store both the user's messages and the results of our tool calls

Long-Term Memory:

building an external database.
Our web_search tool simulates our agent accessing a vast Long-Term Memory (the internet)



Step 5: The "Loop" (The Execution Plan)

This is the **ReAct logic** our **orchestration** code will follow

Loop 1:

Perceive: User: "Report on the history of Paris.«

Reason (LLM): "My System Prompt says I must use a tool. I cannot answer. My plan is to call web_search"

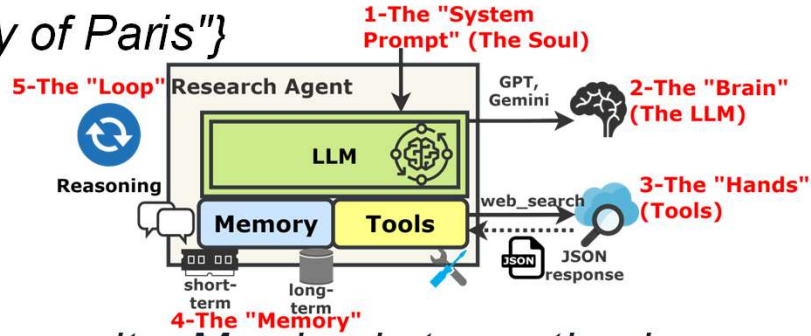
Act (Orchestrator):

1. LLM returns: {"tool_call": "web_search", "query": "history of Paris"}

2. Our code catches this and runs the Python function

3. The function returns: "The Eiffel Tower was built..."

4. This result is added to the Memory list

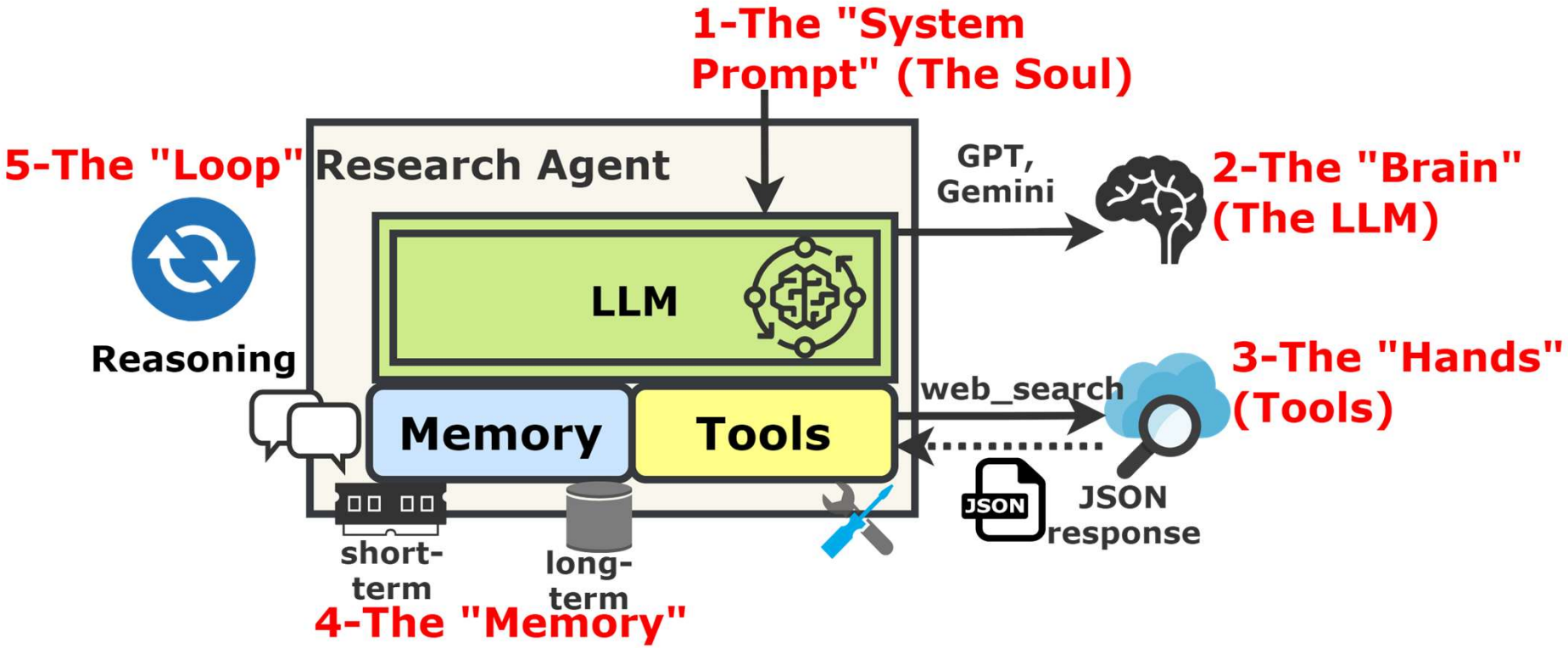


Loop 2:

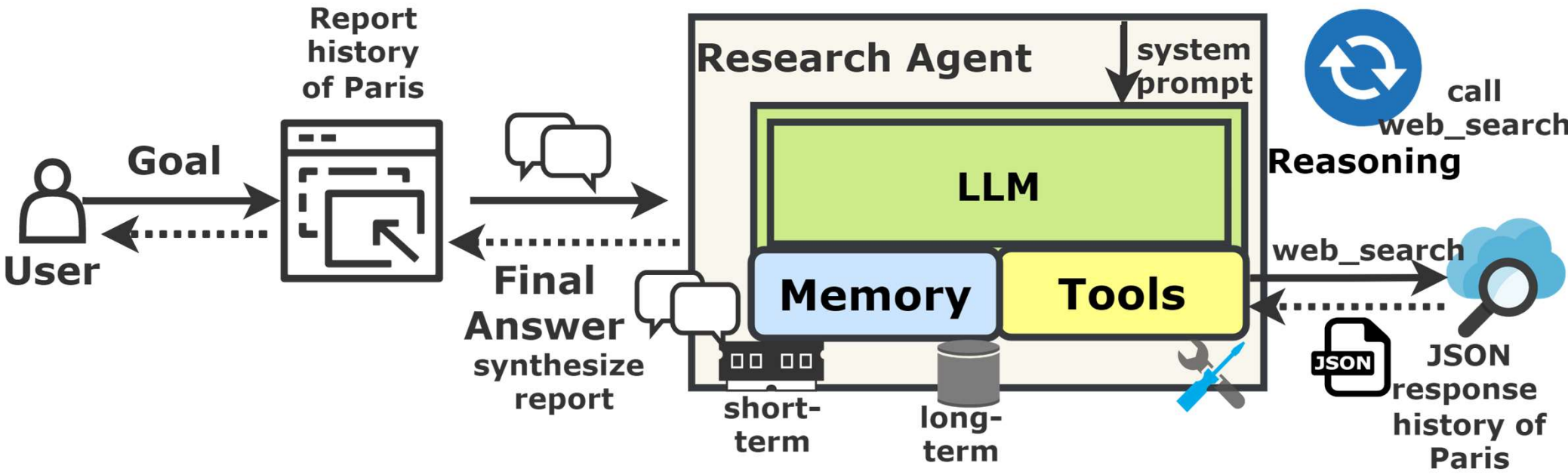
Reason (LLM): "I now have the user's goal and the search results. My plan is to synthesize this into a report"

Act (LLM): Generates the final, natural-language report

Our 5-Step "Recipe" (The Lab Plan)



Full Flow of Research Agent



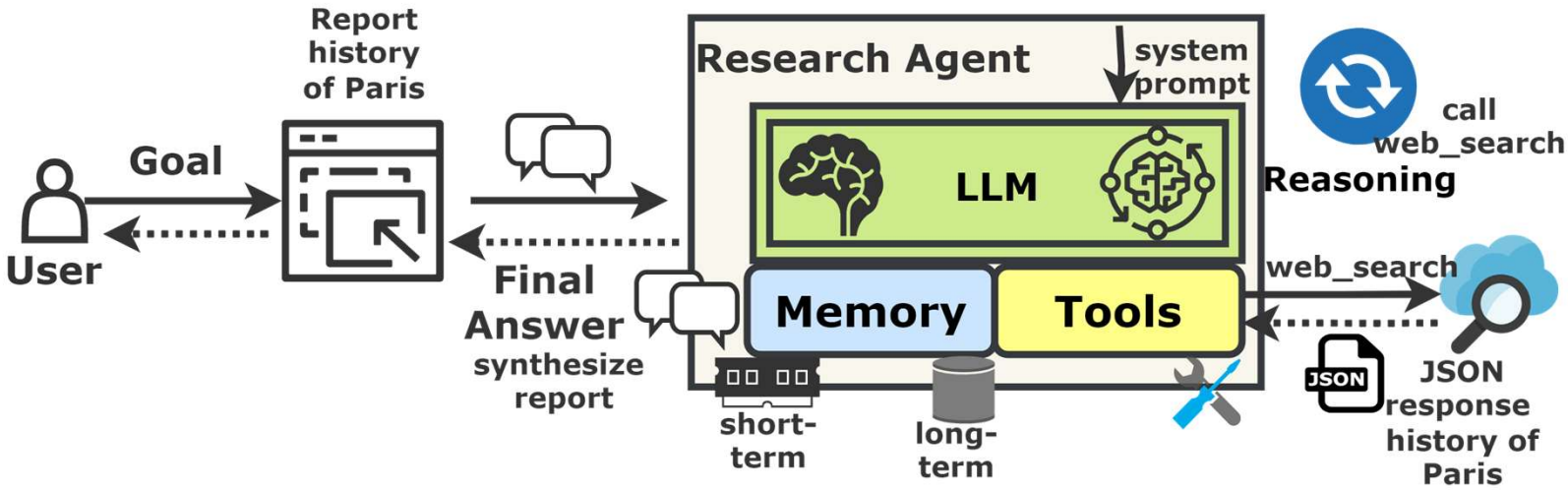
Complete – Research Agent

We have now **completed** our **deep dive** into **agent anatomy**

You can now **identify**, **architect** and **build all the core components** of an **agent**

You have the "**recipe**" and you have the "**hands-on**" **experience**

You are **no longer** just an "**agent user**"; you are now an **agent builder**

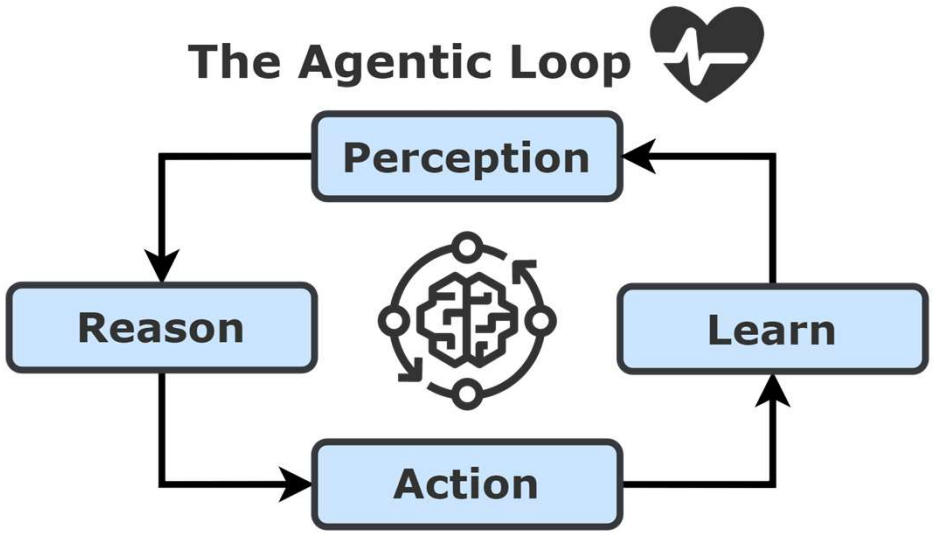


The Agentic Loop: Perception, Reasoning, Action, Learning

A **'digital employee'** that's **autonomous** and **goal-directed**

But **how does** it **'work**'?
What is the **actual**, **step-by-step process** it **follows** to be **autonomous**?

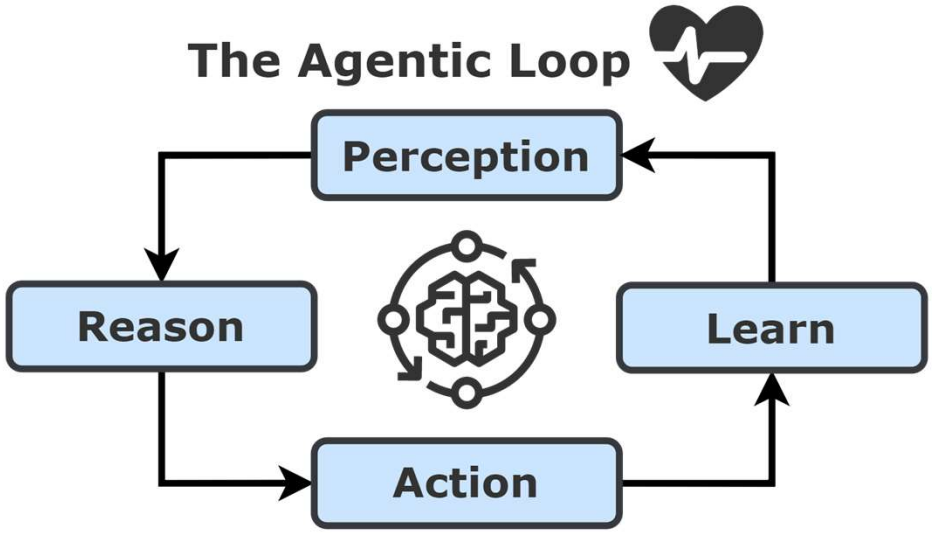
The Agentic Loop is the fundamental **'operating system'** or **'heartbeat'** of every agent in existence





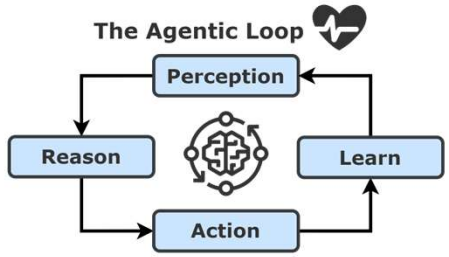
The PRAL Loop: The Journey of a Request

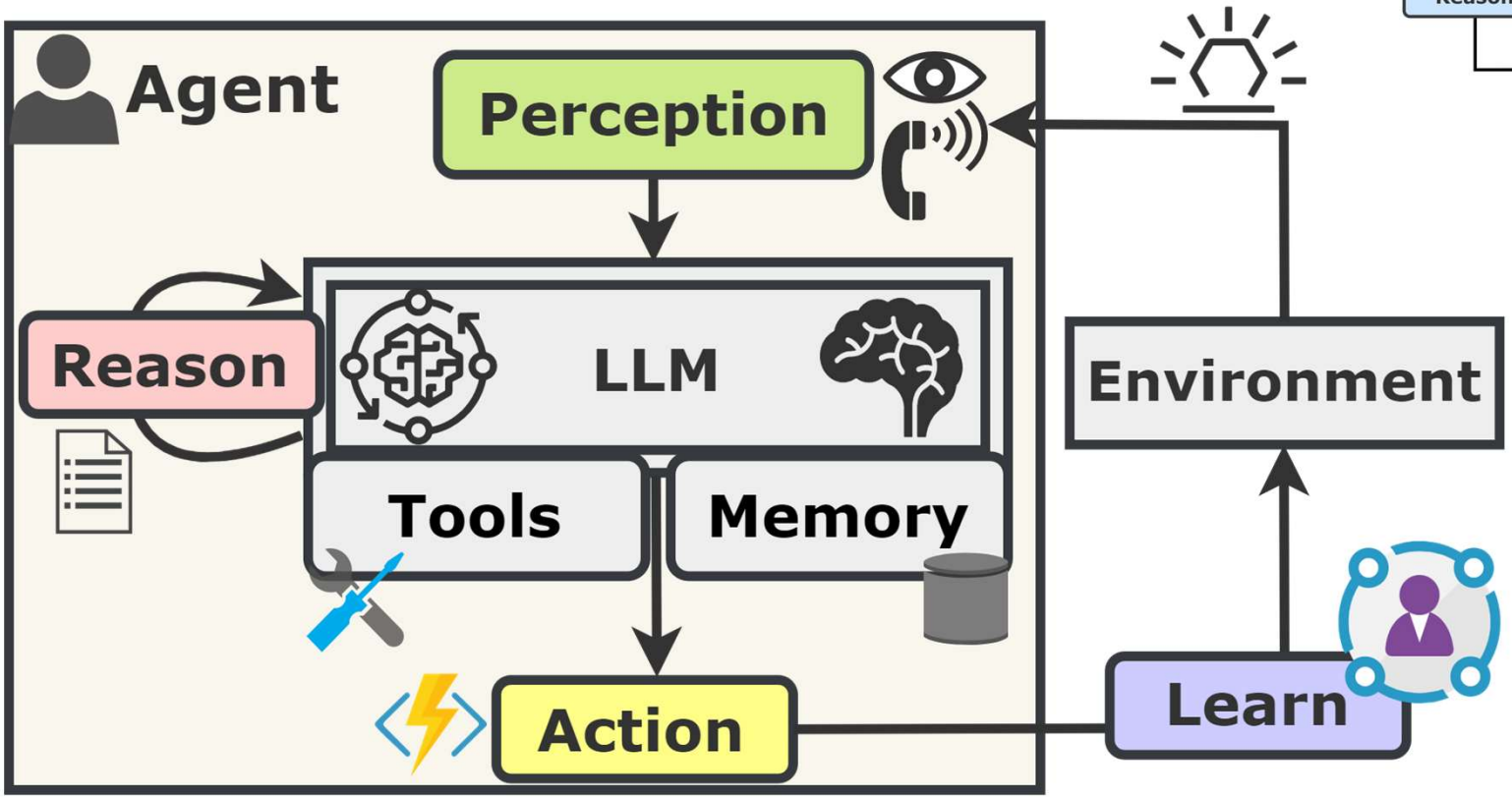
1. **Perception: The "Gateway"** How the agent senses the world
2. **Reasoning: The "Brain"** How the agent thinks and plans
3. **Action: The "Hands"** How the agent affects the world
4. **Learning: The "Memory"** How the agent improves itself





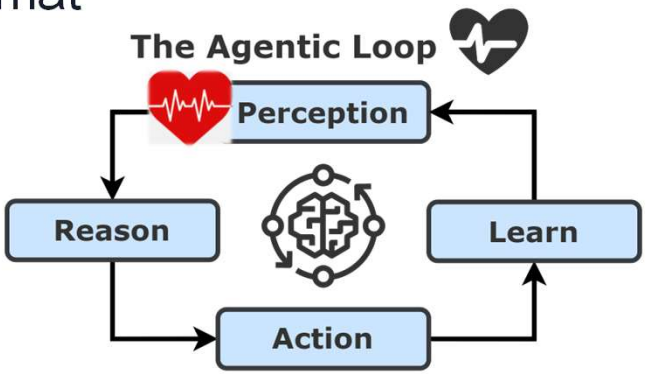
The PRAL Loop: The Journey of a Request





Phase 1: Perception (The "Gateway")

Goal: To get information into the agent's brain in a clean format

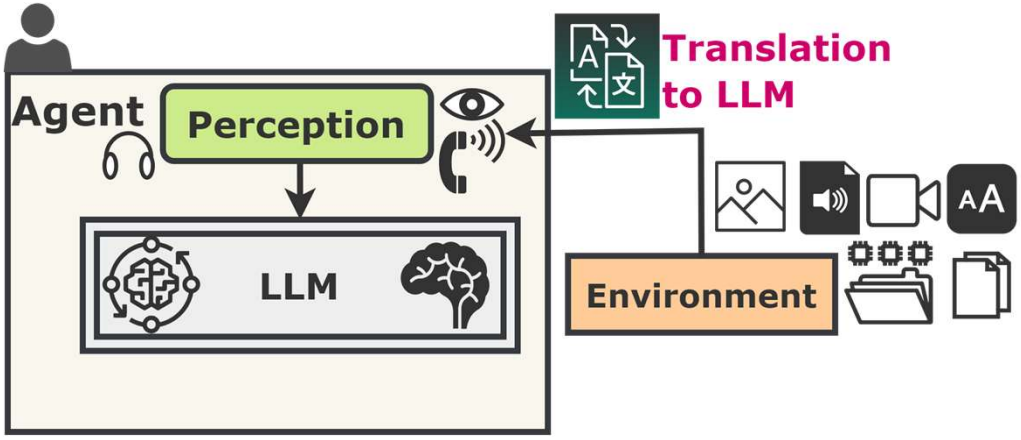


The "Trigger": The loop starts with an event

User Input ("Where is my order?")
API Response ({"status": "shipped"})
Sensor Data (25.5°C)

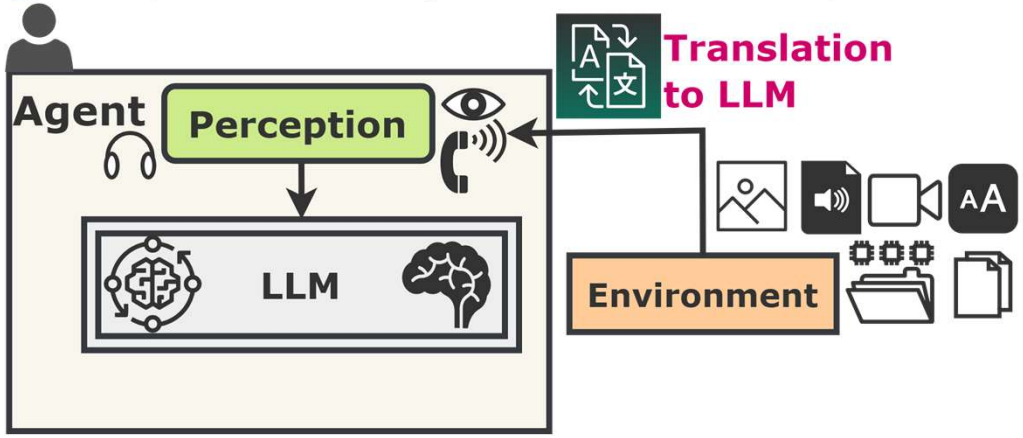
The Challenge: These inputs are diverse, raw and messy

The LLM brain does not understand JSON, audio or images directly



The Key Process of Perception: Normalization

Converting raw, **diverse inputs** into a **clean**, **LLM-understandable format** (**text**)



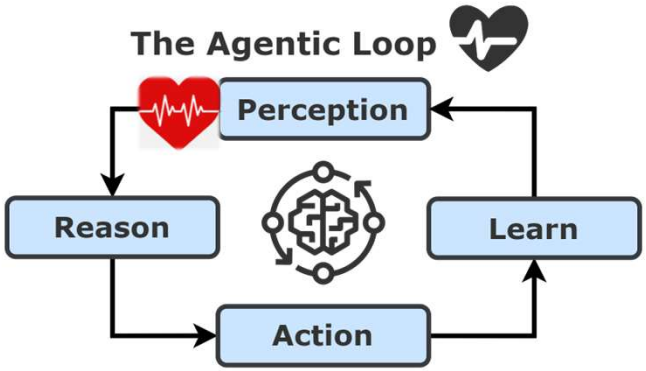


Image.jpg -> [Vision Model] -> A photo of a blue shirt
Audio.wav -> [Speech-to-Text] -> Where is my order?
API_Response.json -> [Parser] -> Order 123's status is shipped

Output: The Internal Representation (clean text)

This is the **FIRST data transformation**

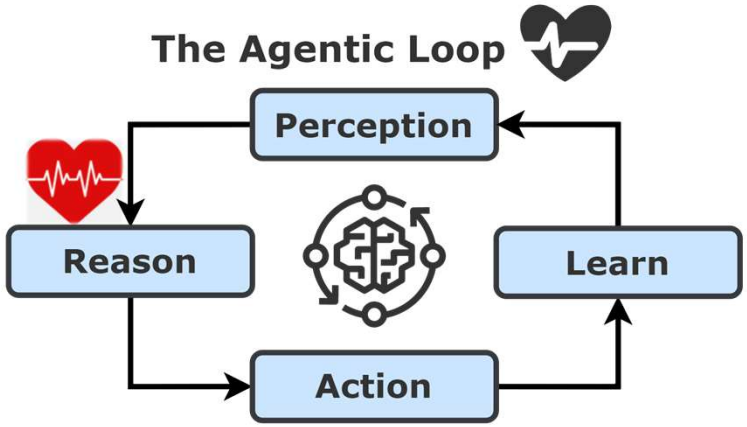
Phase 2: Reasoning (The "Brain")

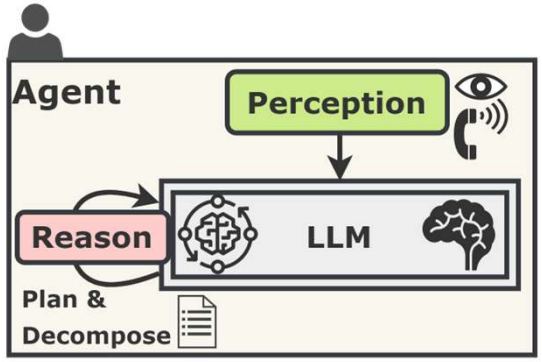
Input: [Normalized Perception] + [Relevant Context from Memory]

Goal: To understand the situation and create a plan

The Process (The "Thought"):
1. Understand: The LLM interprets the full context (perception + memory)
2. Plan: It decomposes the goal into steps (Chain of Thought)
3. Decide: It selects the next immediate action (call a tool)

This is the **"Thinking"** in the **ReAct (Reason+Act)** pattern





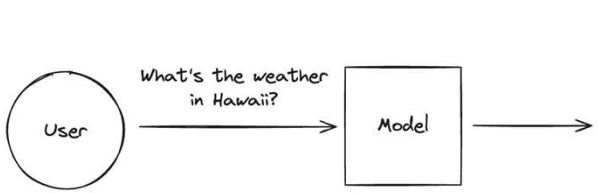
The Key Output of Reasoning: The Action Plan

The **"Plan"** is a **structured object**, **not just text**

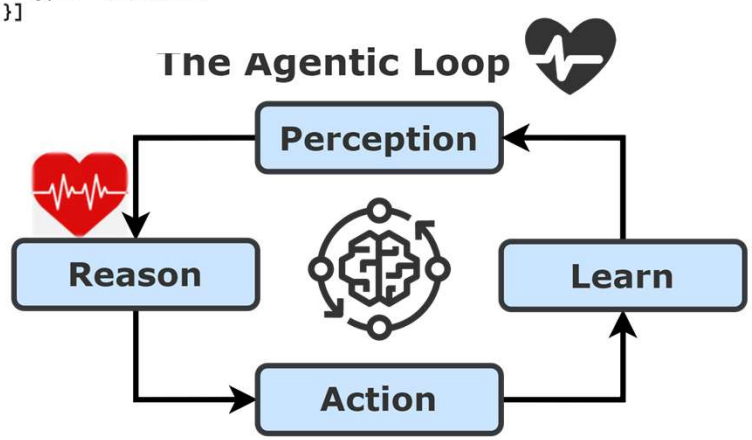
This is the **SECOND data transformation**

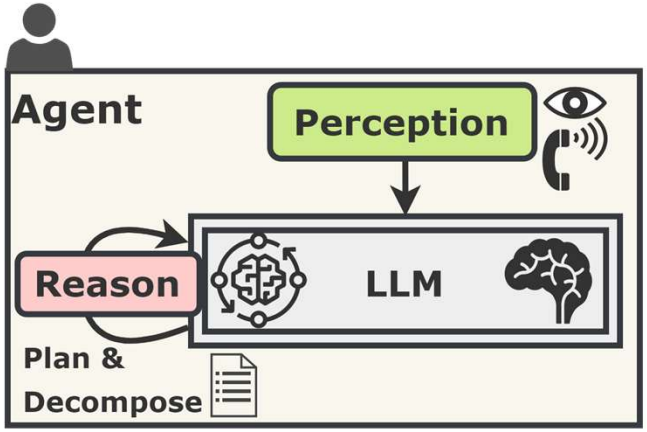
The **LLM's** text **"thought"** is **translated** into a **machine-readable format**

Output: A JSON object:

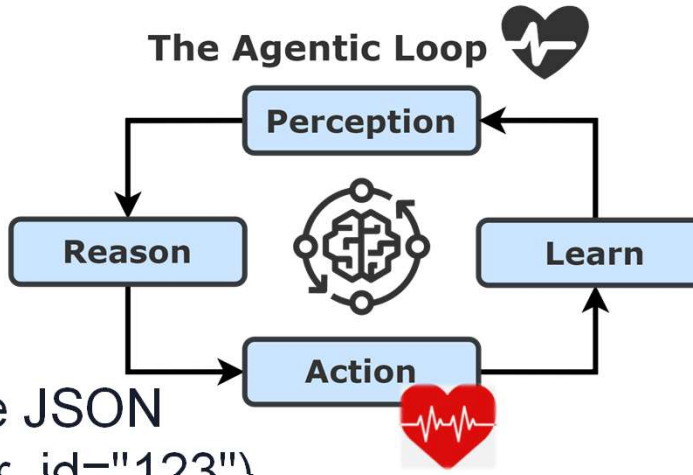



```
AIMessage(
  tool_calls=[{
    name: "get_weather"
    args: {
      location: "Hawaii",
    },
    id: "call_abc123",
    type: "tool_call"
  }]
)
```





Phase 3: Action (The "Hands")



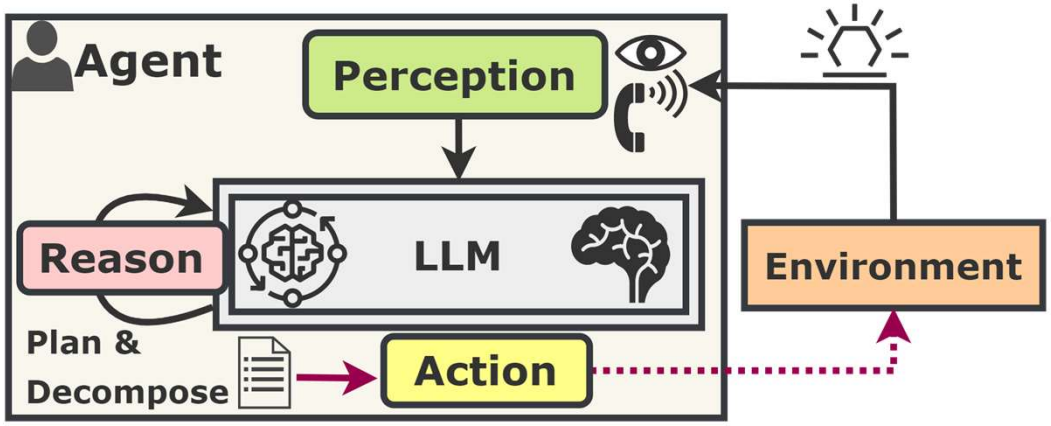
Input: The JSON Action Plan from the Reasoning phase

Goal: To execute the plan and affect the environment

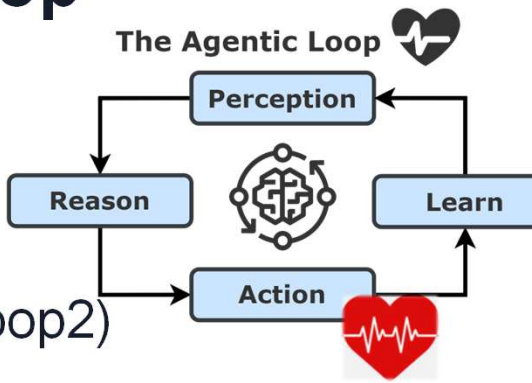
The "Execution" Process (The "Act"):

1. Parse: An "Orchestrator" (the non-LLM code) parses the JSON
2. Execute: It calls the actual code (get_order_status(order_id="123")
3. Affect Environment: The function runs, making an API call, querying a DB

This is the **"Act"** in the **ReAct (Reason+Act)** pattern



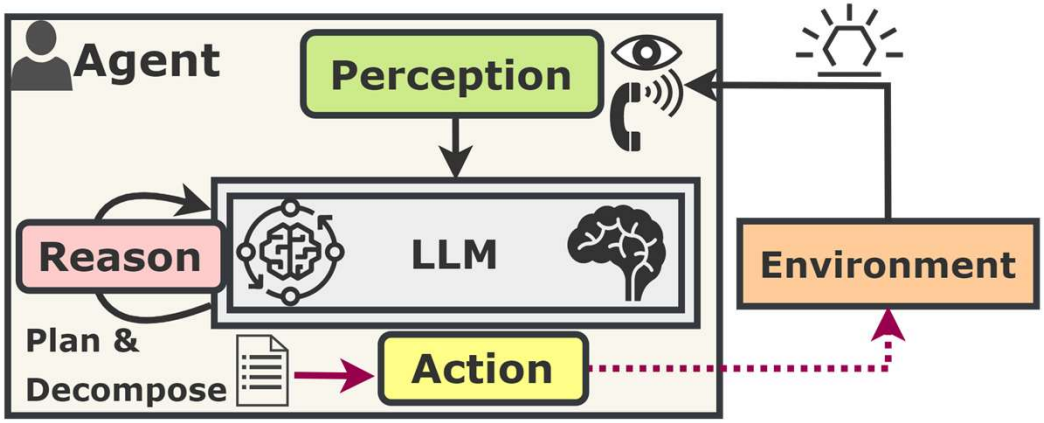
The Critical Output of Action: "Closing the Loop"



Output: The result of the tool's execution
A new JSON object: {"status": "shipped", "eta": "2025-11-12"}

CRITICAL: This is not the end!

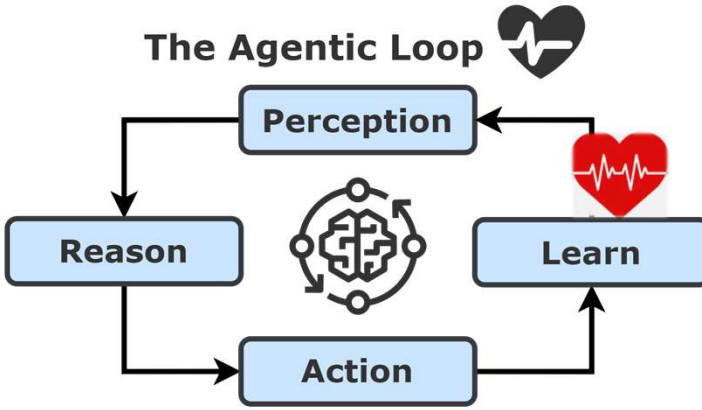
The "Loop1" Closes: This result is fed back to the beginning (Loop2)
It **becomes** the **"Trigger"** for the **next Perception phase**



Loop 2, Phase 1 (Perceive)
Loop 2, Phase 2 (Reason)
Loop 2, Phase 3 (Action)

The loop starts over: Perceive (the API result), Reason (now I know the status), Act (tell the user)

Phase 4: Learning (The "Meta-Loop")



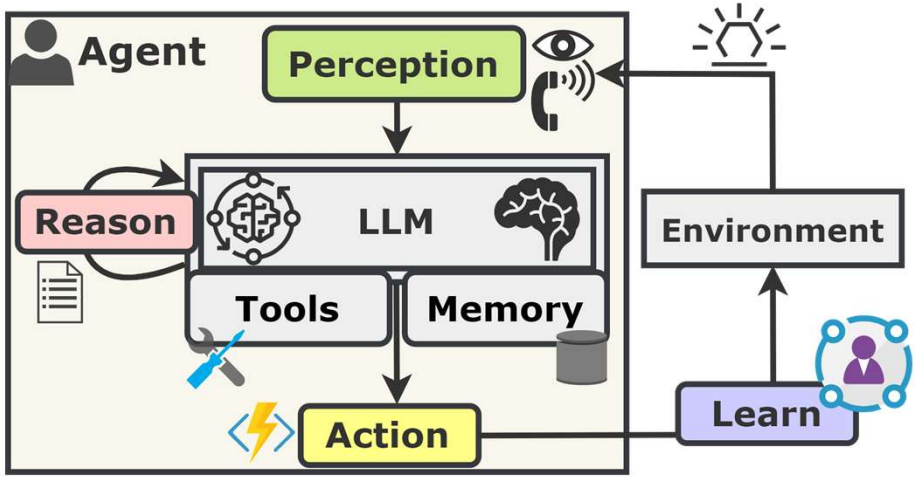
Goal: To improve the agent's performance over time

This is **optional loop** that **sits on top of** the **PRAL loop**

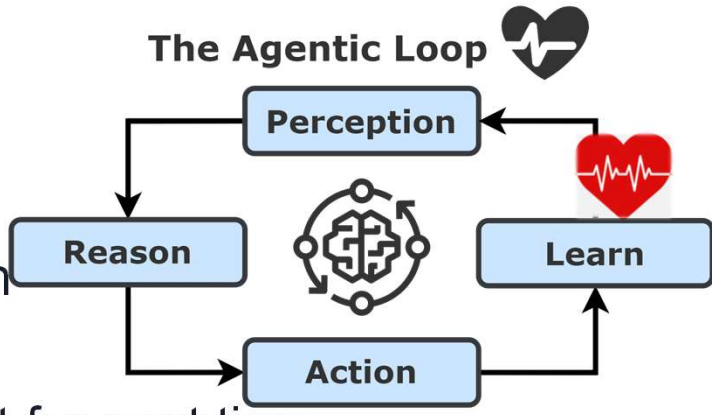
The Process:

1. Reflection / Critique: After the entire task is done, a "Critic" agent reviews the full transcript (Did that solution work? Was it efficient? Did the user seem happy?)

2. Adaptation: The insights from this critique are used to update the agent's core components



The Key Output of Learning: Adaptation

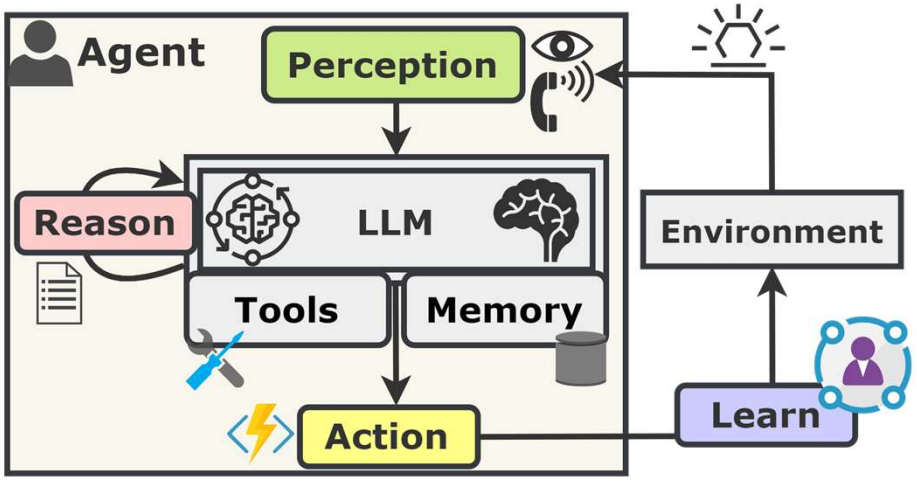


The "Adaptation" Process:
The "Critic's" feedback is used to modify the agent's 'brain'

Mechanisms for Update:
Update Memory: This user's order_id is 123. I'll save that for next time.

Update Planning: I should try get_order_status before search_faq. This is a more efficient plan

Update Utility: The user was happy with this answer. I will increase the score for this type of response



Summary: The Journey of PRAL

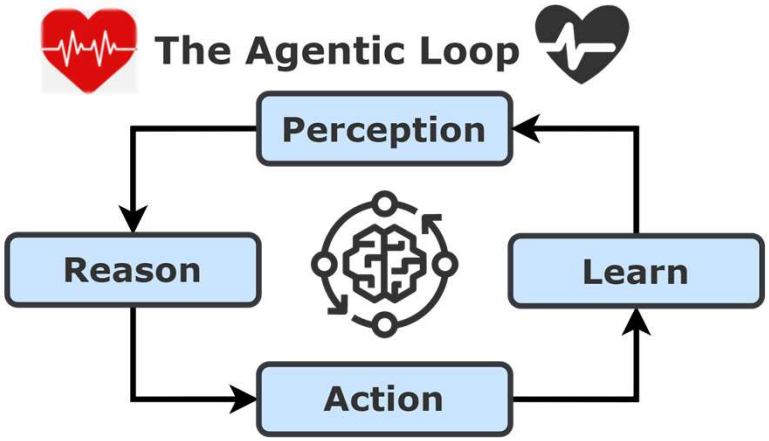
Perceive: Raw Input -> Normalized Text

Reason: (Text + Memory) -> JSON Plan

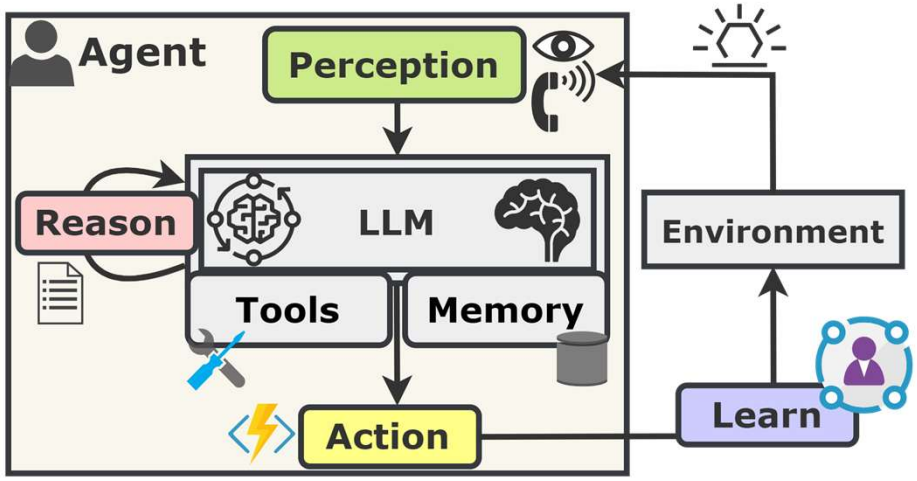
Act: JSON Plan -> Tool Result (Raw Data)

(Loop): Tool Result -> Perceive

Learn: (Final Outcome) -> Updated Memory/Plans



This "**PRAL**" **process** is the **engine** of all **agentic behavior**



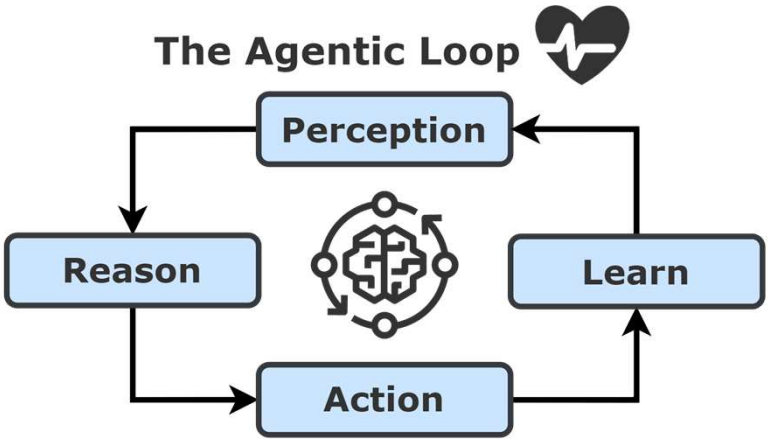
Recap: The PRAL Process Flow

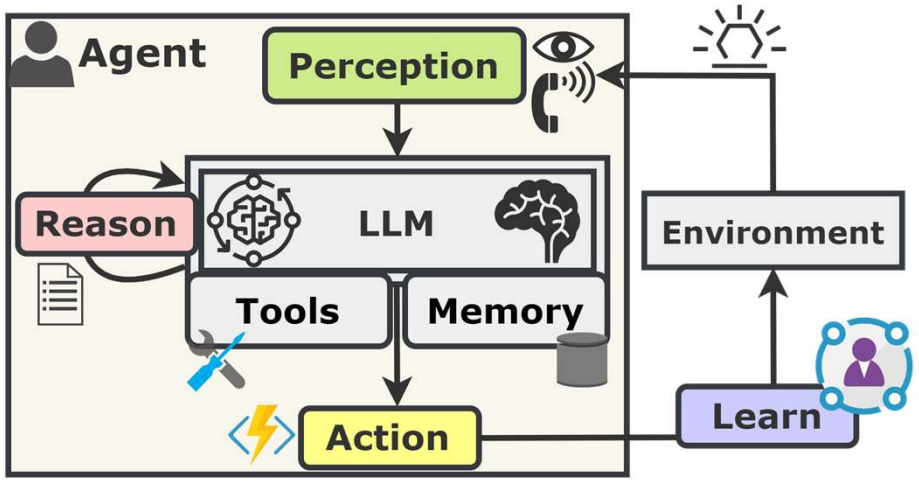
Phase 1: Perception: How the agent ingests and normalizes data

Phase 2: Reasoning: How the agent plans its next step

Phase 3: Action: How the agent executes that step

Phase 4: Learning: How the agent improves from the outcome





Design Use Cases of The Agentic Loop (PRAL)

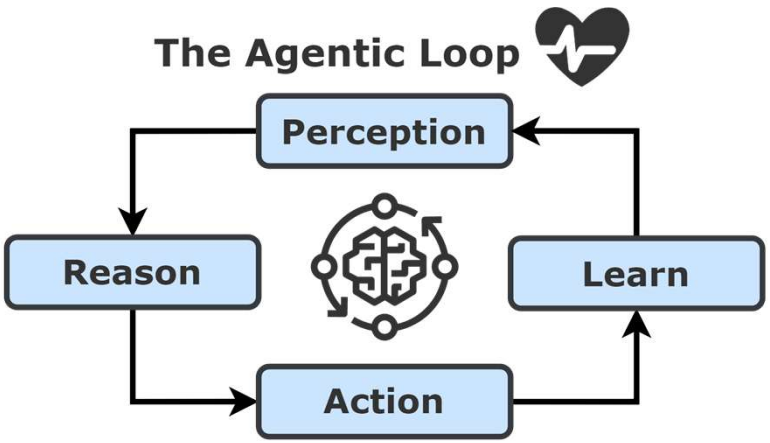
We saw how it's a **'journey of data'** and a series of transformations

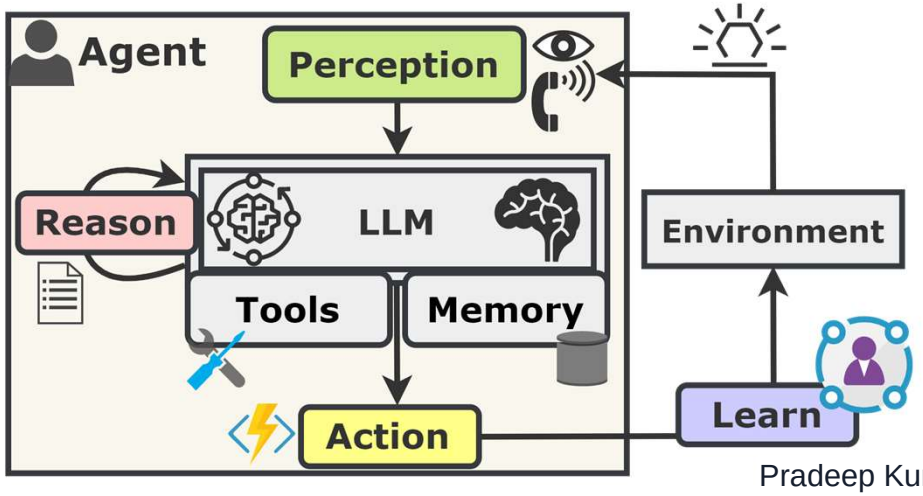
We will trace this exact **PRAL loop** through two very different, very real examples:

First, a familiar "**Digital Assistant**" like **ChatGPT**

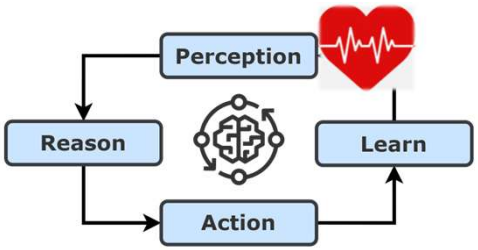
Second, a high-stakes "**Clinical Assistant**" in a **healthcare setting**

Focus: How the data flows and **transforms**, phase by phase, in each scenario





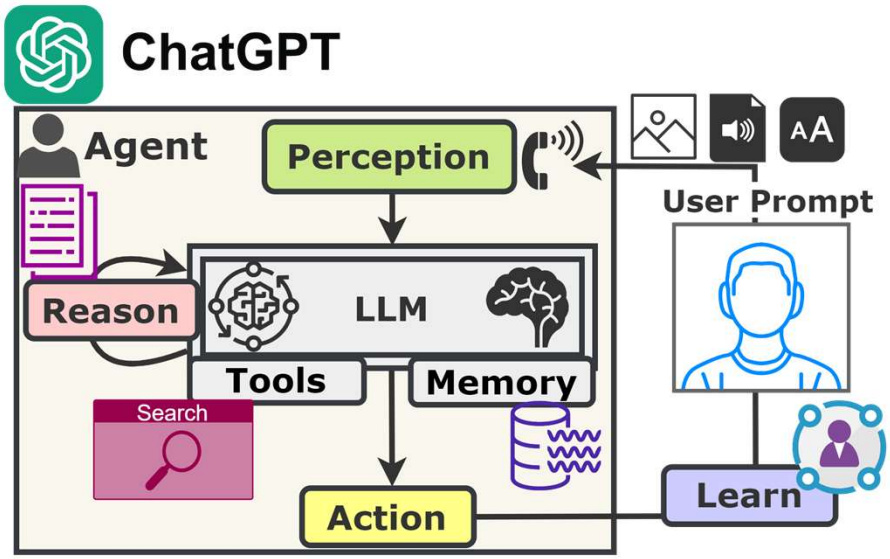
Design ChatGPT - Phase 1 (Perception)



Environment: The User

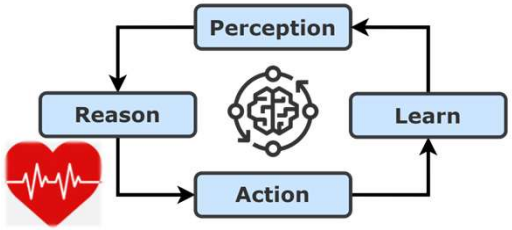
Raw Input: You speak a prompt into your phone: Can you find me a good seafood restaurant?

The "Normalization" Process:
1. The Audio.wav is processed by a Speech-to-Text model
2. This extracts the text: "Can you find... a bit fancy"
3. The Perception module discards the tone of your voice (your sarcasm, your emotion). It only perceives what you said



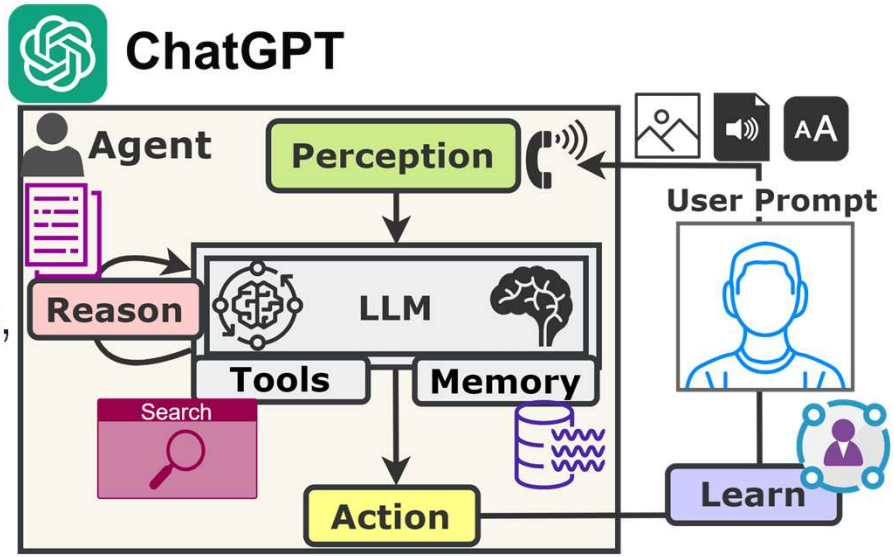
Normalized Output (to Reasoning): A clean text string

Design ChatGPT - Phase 2 (Reasoning)



Input: (Normalized Text: "find fancy seafood restaurant") + (Context from Memory: "User lives in Seattle", "User's past chats...")

Reasoning Process (The LLM's "Thought"):
1. User wants a fancy seafood restaurant
2. My memory says they live in Seattle
3. This requires real-time information (open hours, reviews) that I don't have
4. Therefore, my plan is to use the web_search tool



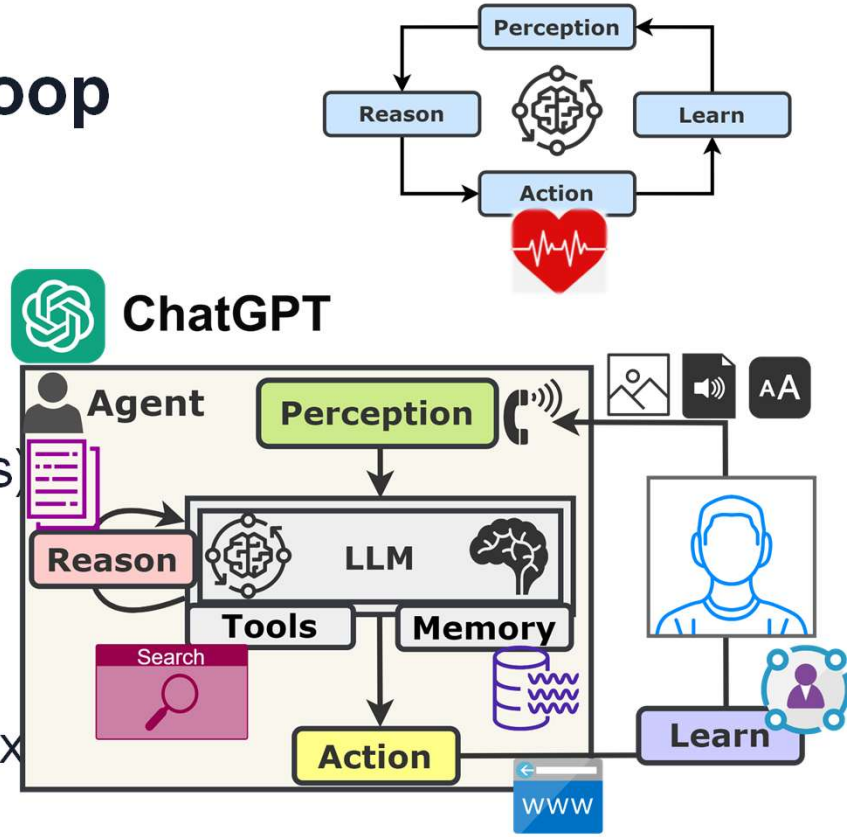
Output (The Plan): A JSON object for the tool_call

ChatGPT - Phase 3 (Action) & The Loop

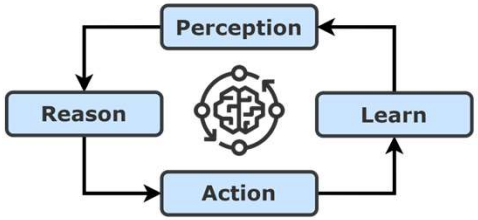
Phase 3 (Action):
The Orchestrator parses the JSON plan
It executes the web_search tool

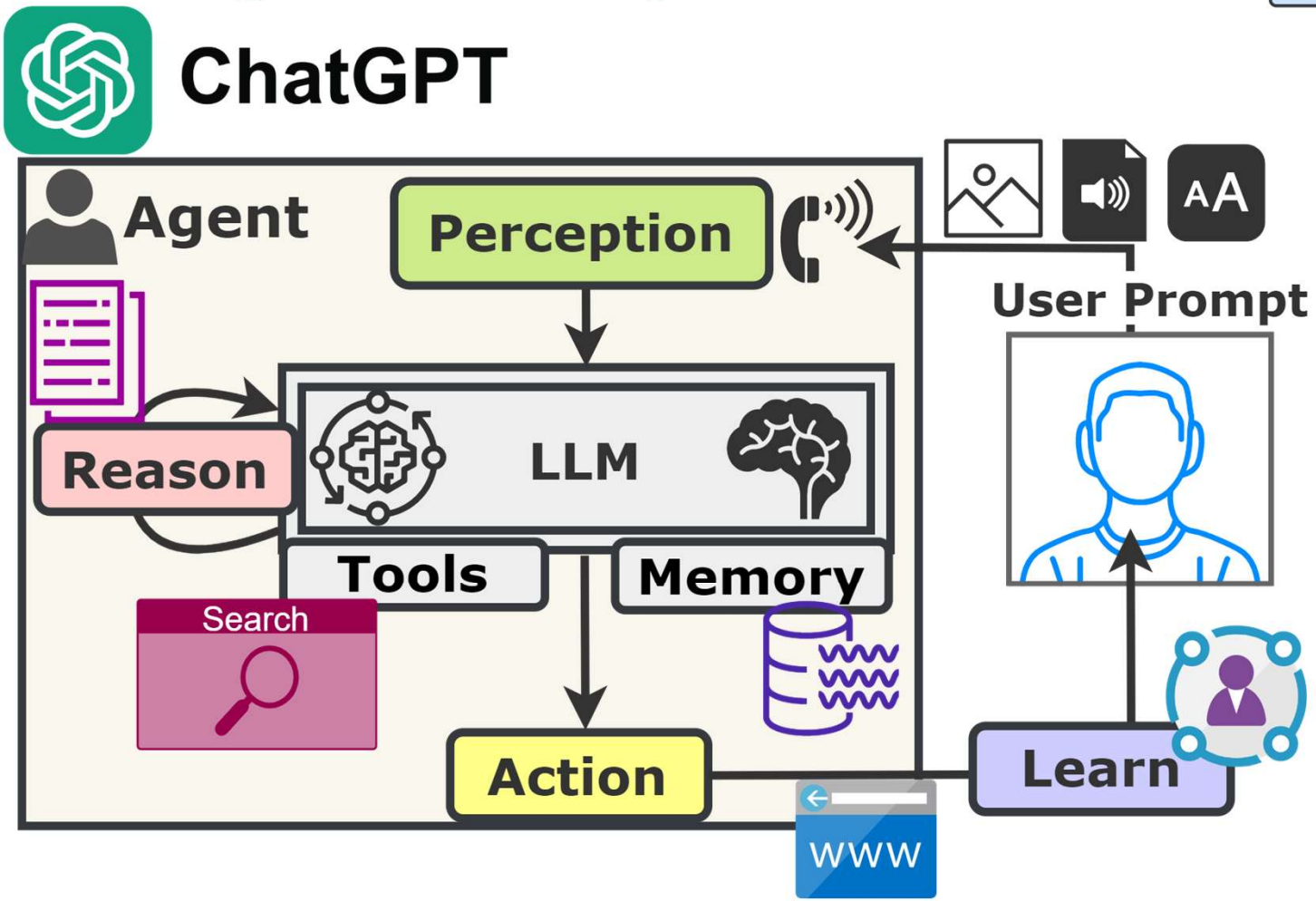
The "Loop" Closes:
The result of the web search (JSON of 5 restaurants)
is the output of the Action phase
This output is fed back to Phase 1 (Perception)

Loop 2 Starts: (PRA)
Perceive: The search results are normalized into tex
Reason: I now have the list of restaurants. My plan is to synthesize this info and present to user
Act: The agent executes its final action: generating the text response you read

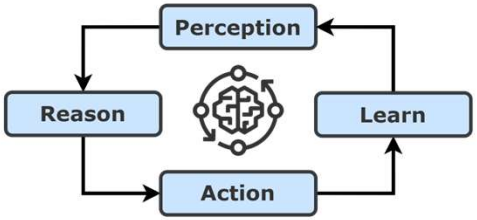


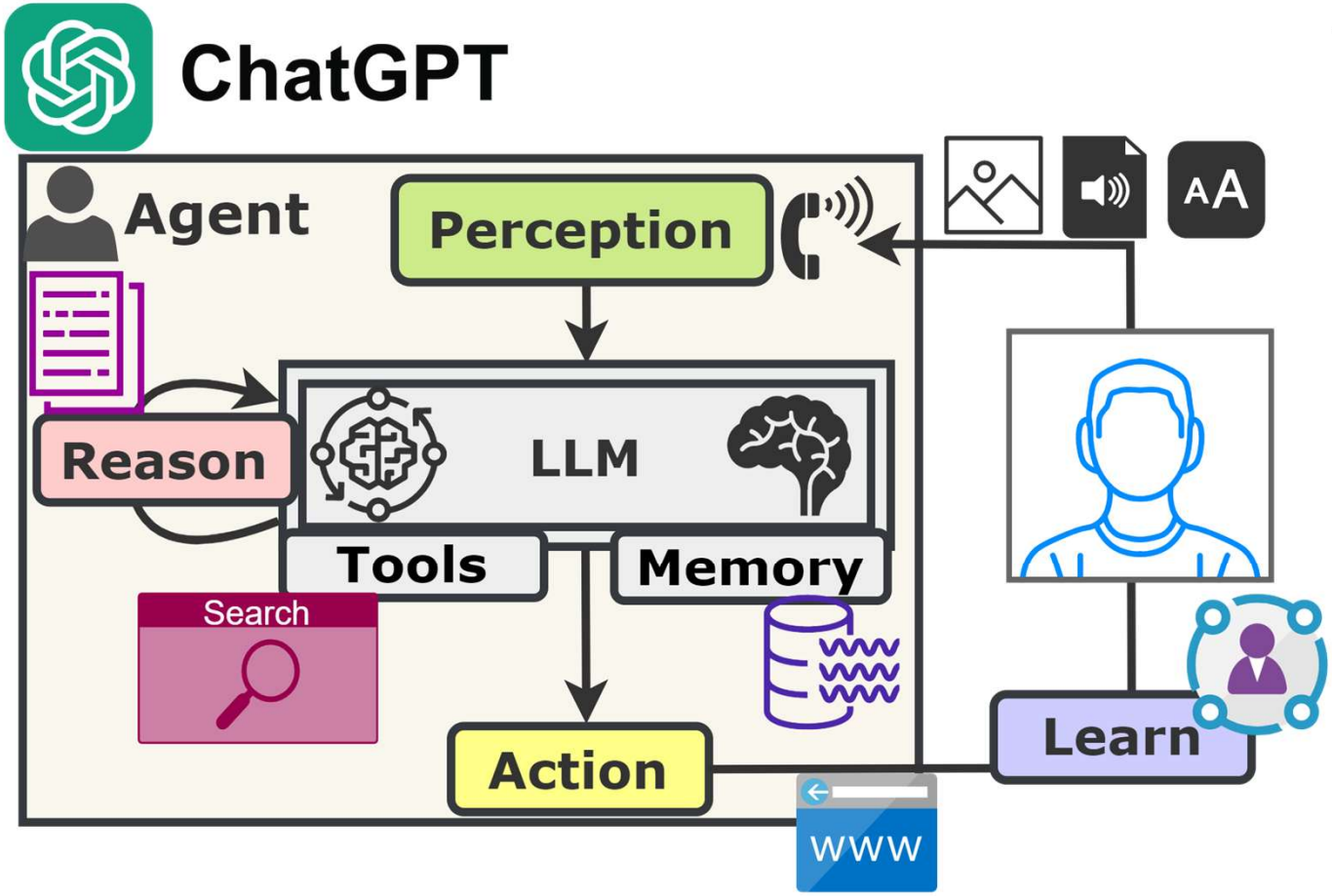
ChatGPT Agent with Loop





ChatGPT Agent with Loop

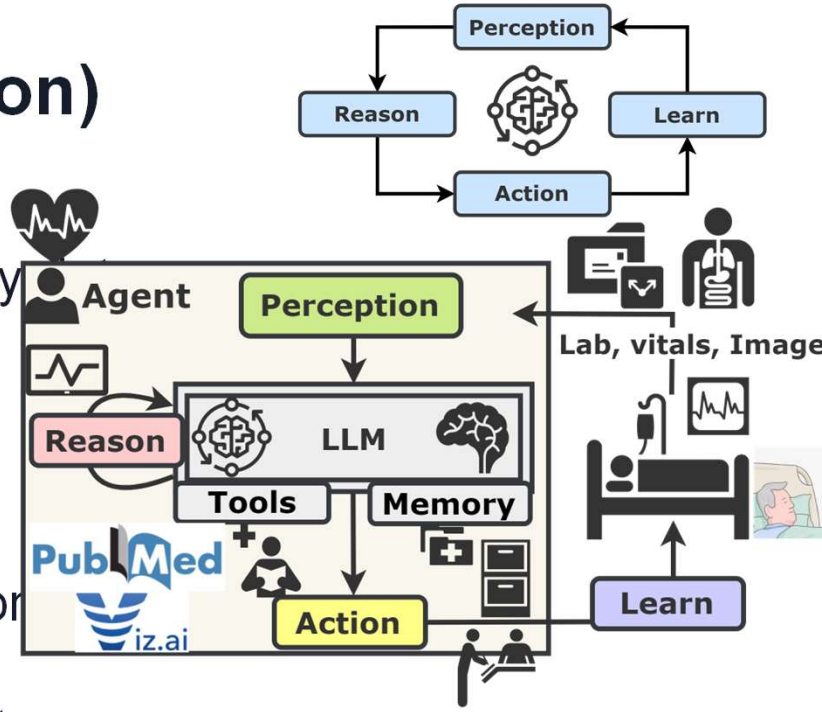




Healthcare Agent - Phase 1 (Perception)

Environment: A patient in an ICU

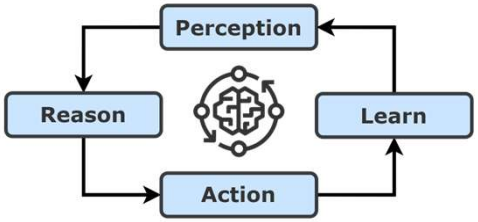
Raw Input: A constant, high-velocity stream of messy
vitals.json (HR: 125, BP: 80/50)
lab_results.pdf
radiology_image.dcm



The "Normalization" Process:
1. Filtering: The agent is architected to trigger only on anomalous data (HR > 120)
2. Translation: It uses specialized models(Viz.ai) to turn an image into text: Radiology shows left-side anomaly

Normalized Output (to Reasoning): Patient 4A vitals unstable (HR: 125). New radiology report: left-side anomaly

Healthcare Agent - Phase 1 (Perception)

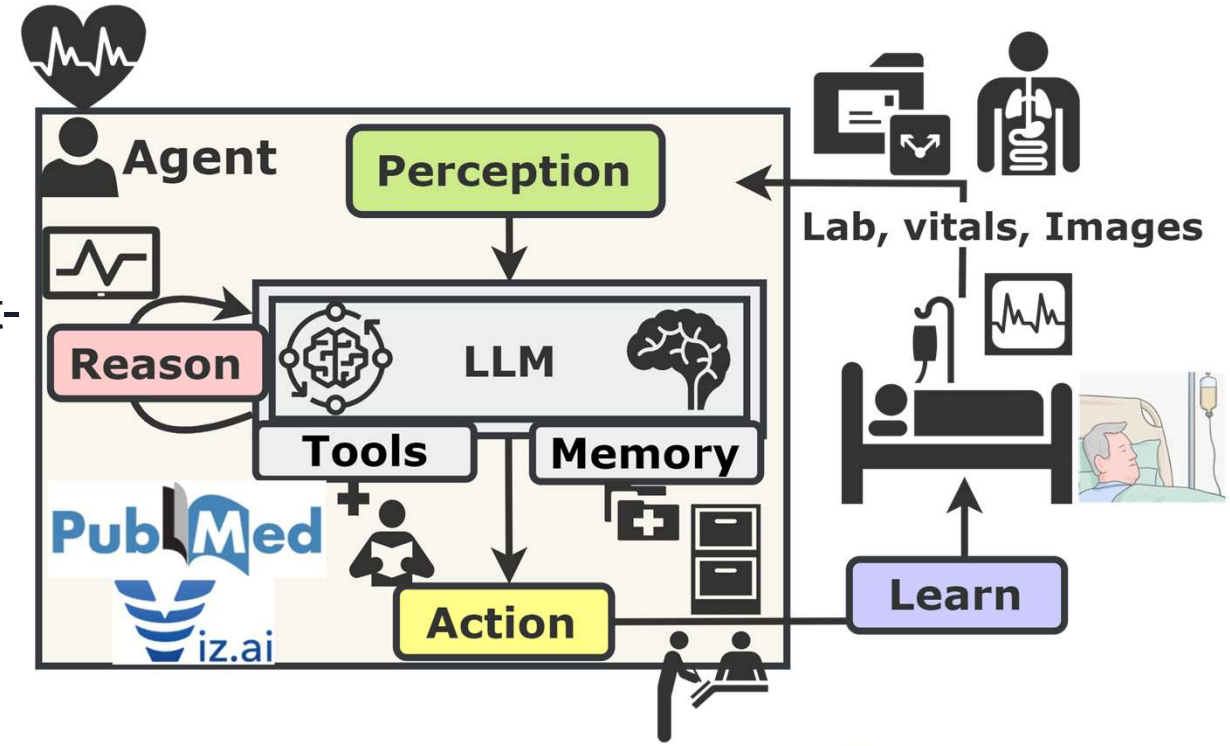


Environment: A patient in an ICU

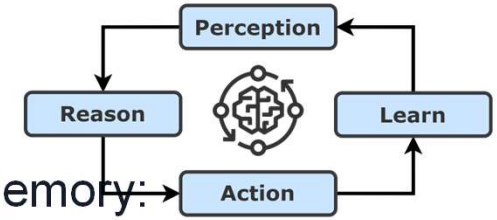
Raw Input: A constant, high-velocity stream of messy data:

vitals.json (HR: 125, BP: 80/50)
lab_results.pdf
radiology_image.dcm

Output: Patient 4A vitals unstable (HR: 125). New radiology report: left-side anomaly



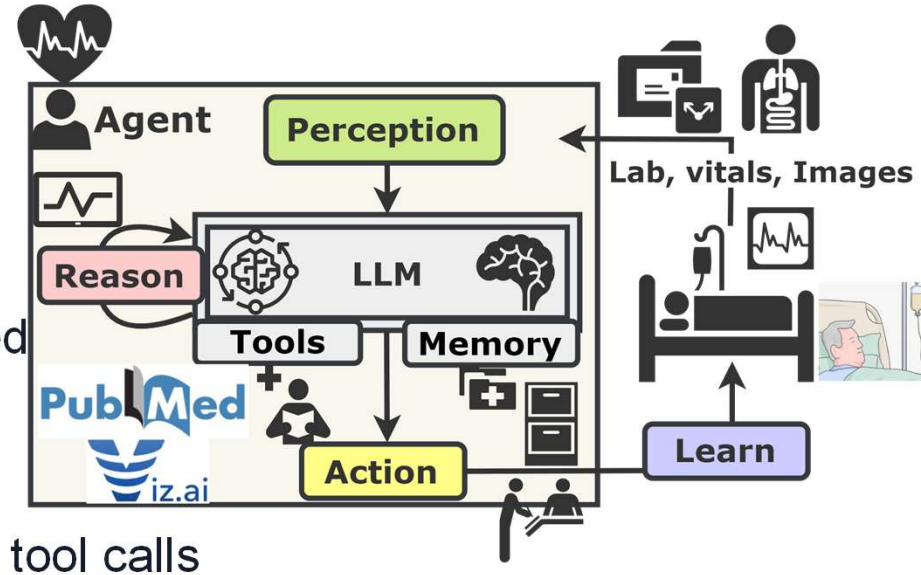
Healthcare Agent - Phase 2 (Reasoning)



Input: (Normalized Text: "unstable vitals, anomaly") + (Context from Memory: "Patient allergies: penicillin", "Medical History: ...")

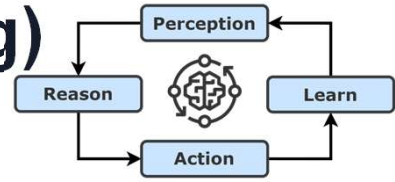
Goal: Patient Survival

Reasoning Process (LLM "Thought"):
1. Patient is unstable with a new, unknown anomaly
2. I must respect the penicillin allergy
3. Plan: I need more information. I will search PubMed also order a new STAT blood_panel test



Output (The Plan): A JSON plan to execute multiple tool calls

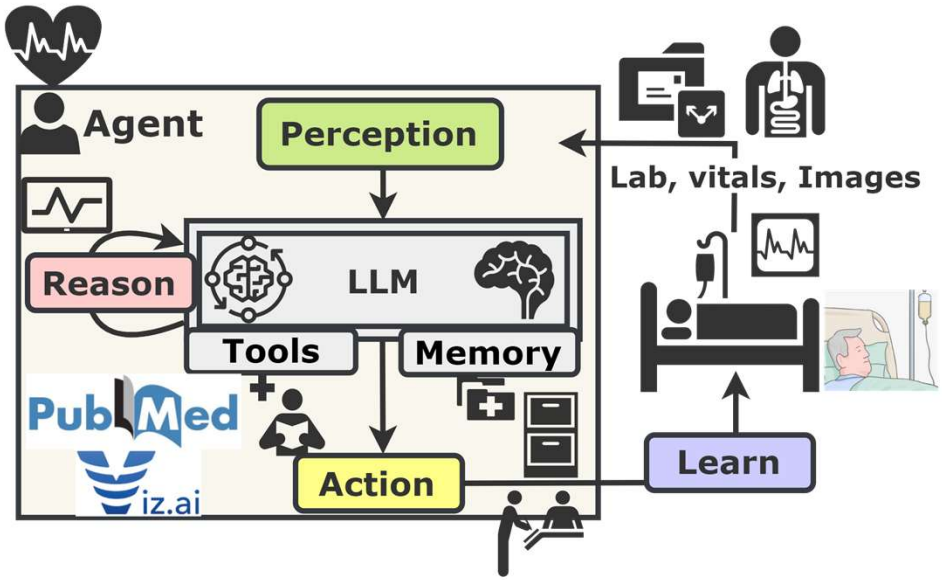
Healthcare Agent - Phase 3 & 4 (Action & Learning)



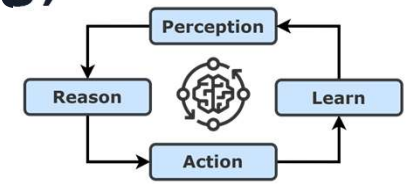
Phase 3 (Action):
The Orchestrator executes search_pubmed(...) and order_test(...)

The "Loop" Closes:
The results of the search and the new lab values are fed back to Phase 1 (Perception)

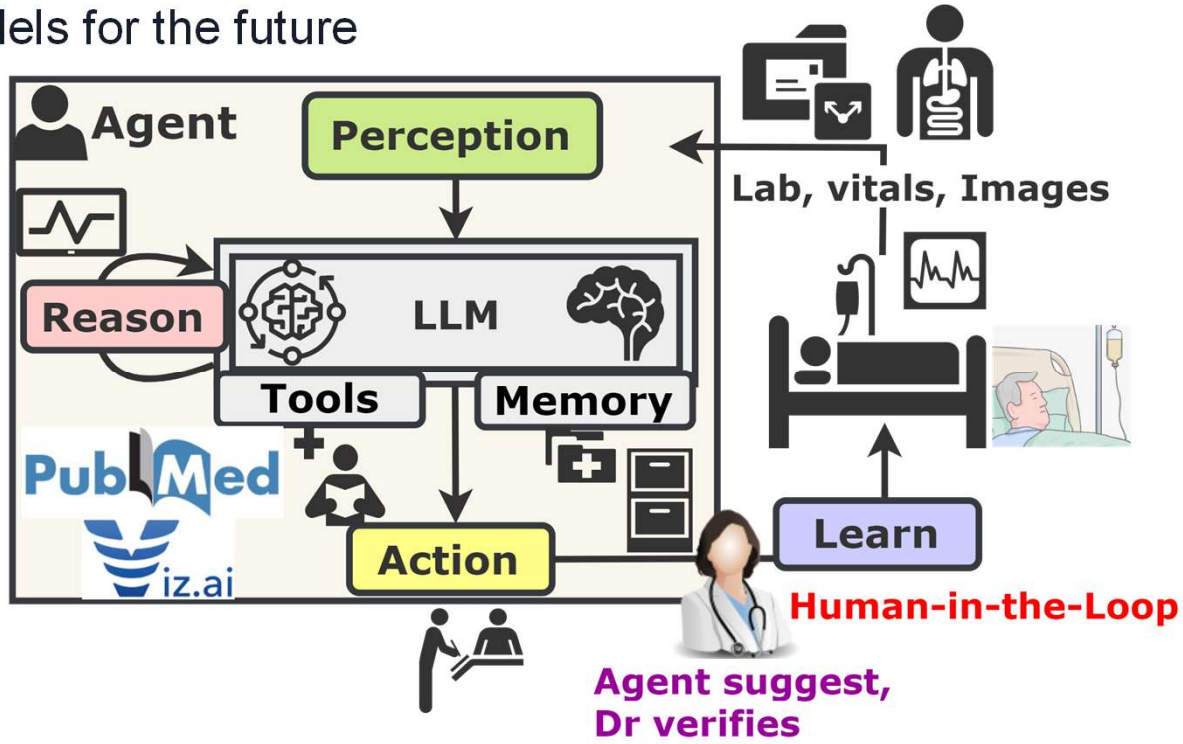
Loop 2 Starts:
The agent reasons on this new data and forms a final plan (Recommend medication X)



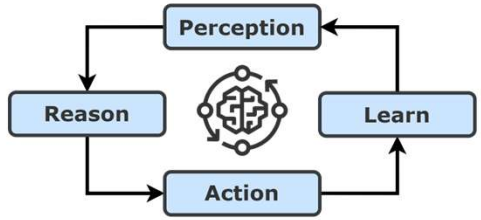
Healthcare Agent - Phase 3 & 4 (Action & Learning) -2

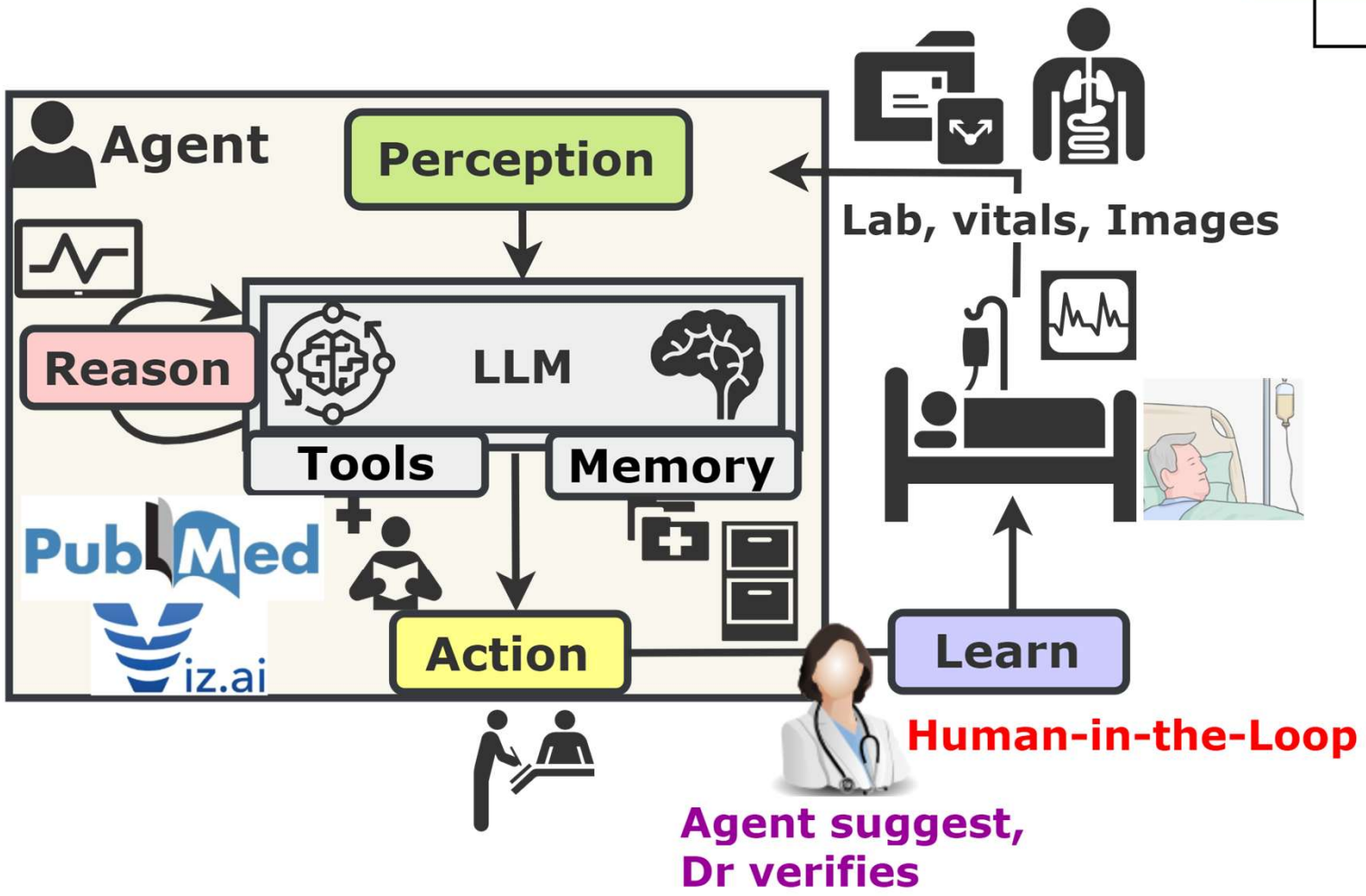


Phase 4 (Learning / Safety):
The agent's final plan is NOT executed automatically
It is stopped and presented to a Human-in-the-Loop (a Doctor)
The Doctor's feedback ("This is correct, approve") is used as the Learning input to improve the agent's models for the future

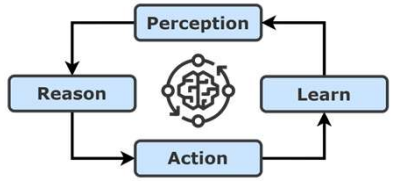


Healthcare Agent with Loop





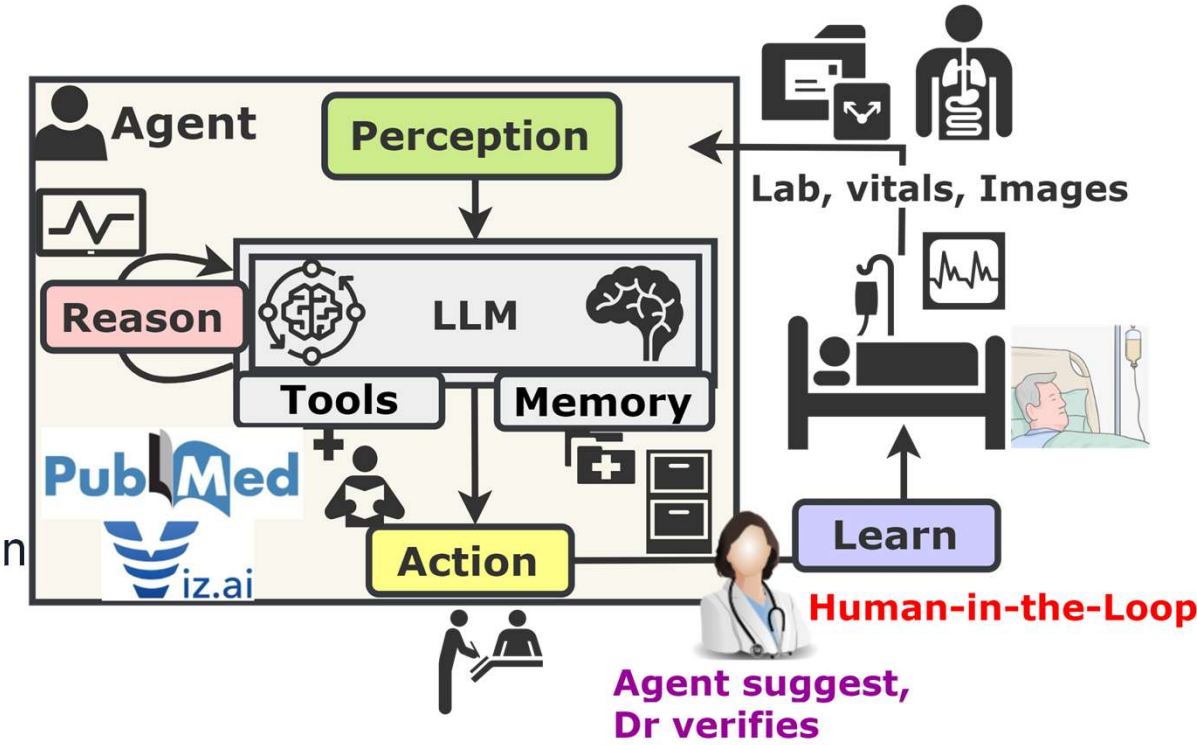
Summary: Agent Loop in Two Domains



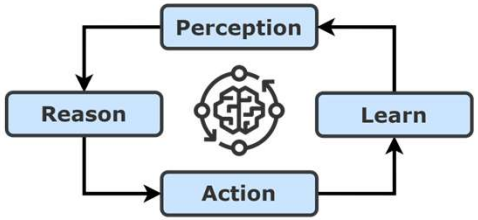
The PRAL loop is a universal process

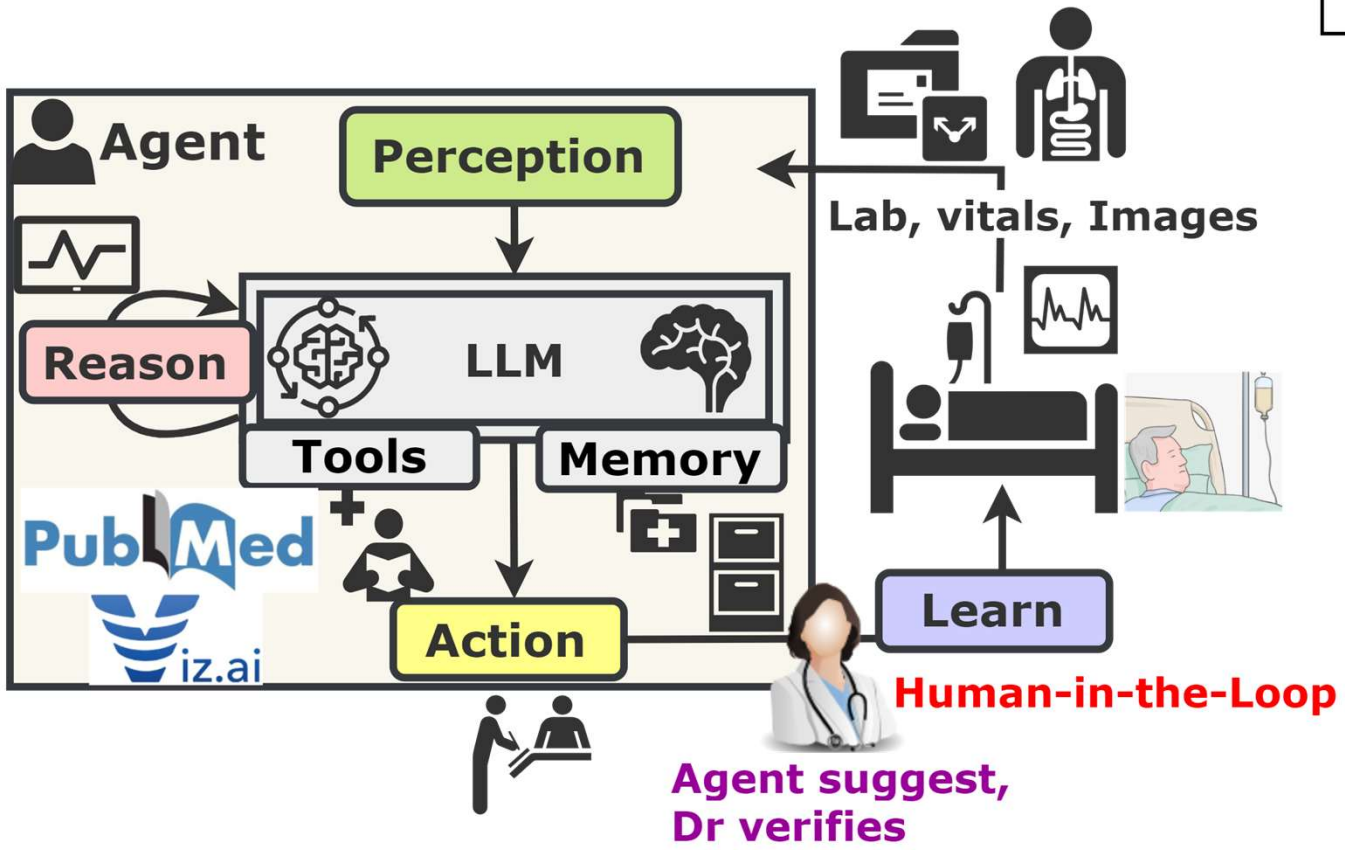
ChatGPT (Digital Domain):
Perceives: User commands
Reasons: With web search
Acts: By generating text

Healthcare Agent (Cyber-Physical Domain):
Perceives: Patient data streams
Reasons: With medical history and specialized search
Acts: By ordering tests and recommendin treatment



Healthcare Agent with Loop





What is Agentic AI ?
Multi-Agent Architecture

AI Agent vs. Agentic AI: The Critical Difference

What is Agentic Architecture ? (Multi-Agent System (MAS))

How Agentic AI Works: System-Level Architectures

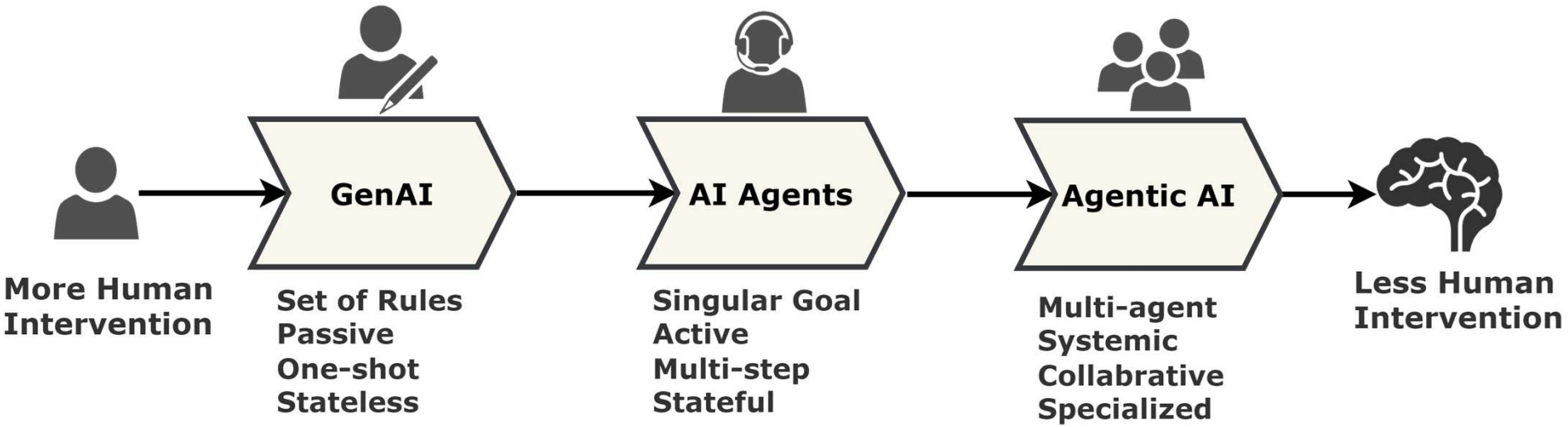
Real-World Examples of Agentic AI

Challenges of Agentic AI Systems

Pradeep Kumar T



Evolving into Agentic AI



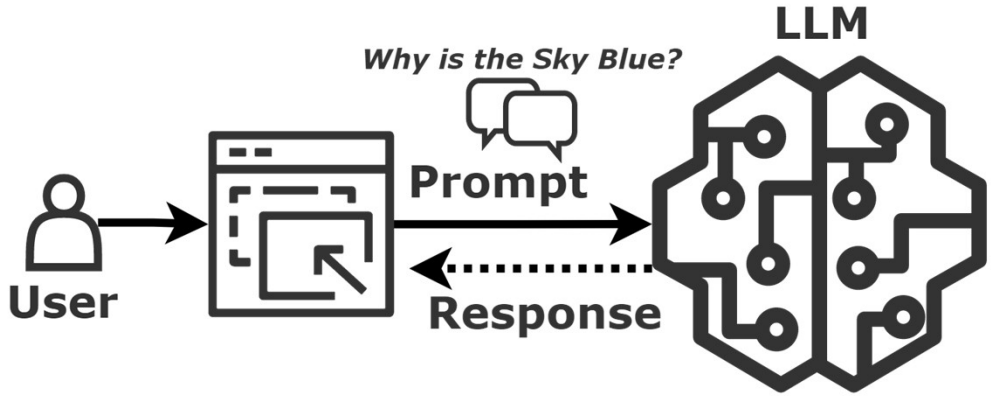
Phase 1: Generative AI (The "Expert Consultant")

One-shot: Its entire lifecycle is a single Request -> Response loop

Passive: It is a reactive system. It only responds; it never initiates

Stateless: It has no memory of past interactions

An **"Expert Consultant"** You **ask** it **for a plan** and it **writes you** a **report**



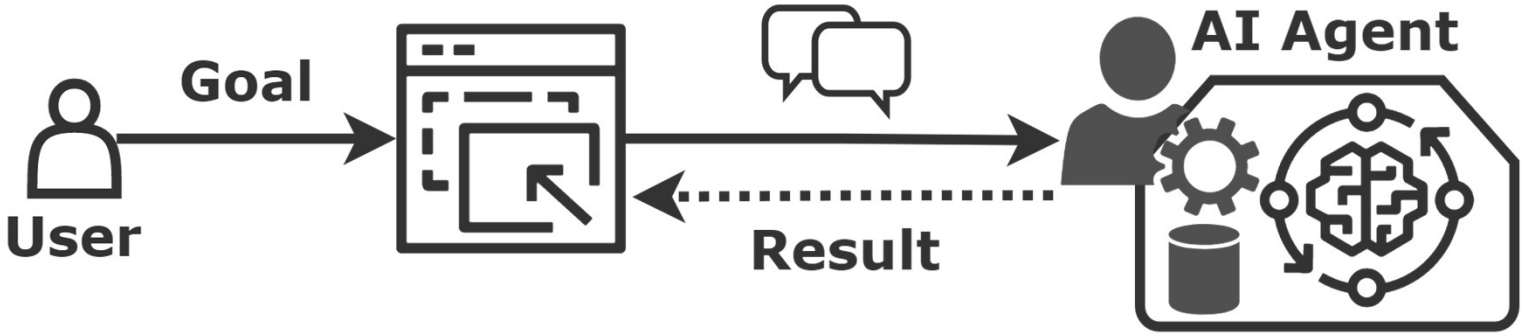
Phase 2: The AI Agent (The "Worker")

Active: It is proactive. It can initiate actions

Multi-step: It runs a Perceive -> Reason -> Act loop (PRAL)

Stateful: It maintains an internal memory of its plan and its history

We gave the **"Expert Consultant" Tools (its 'hands')** and **Memory (its 'notebook')**



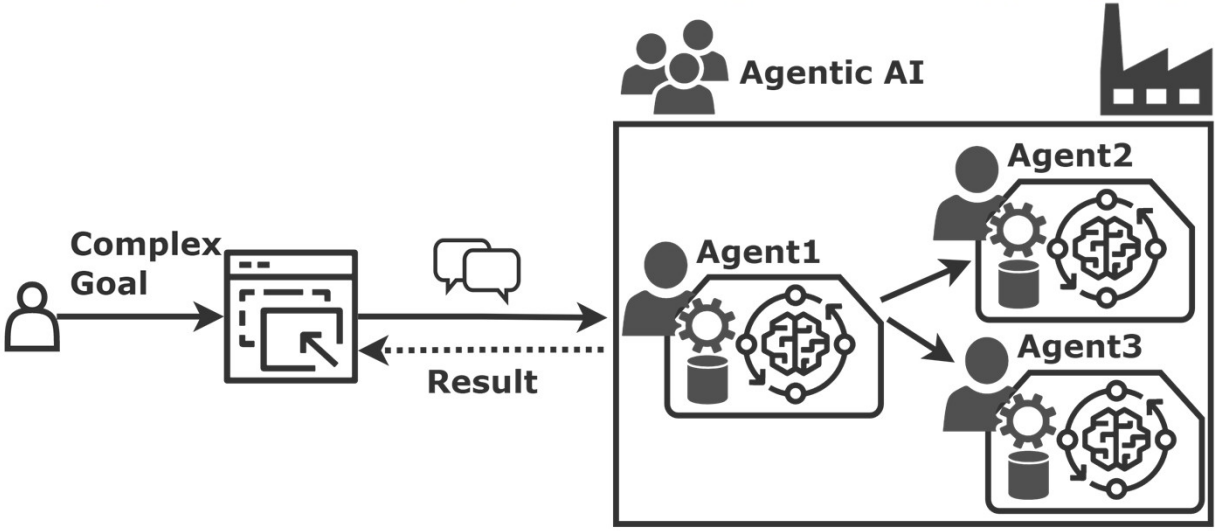
Phase 3: Agentic AI (The "Factory")

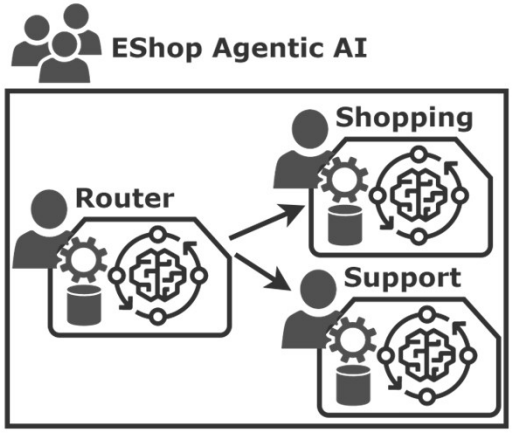
Systemic: It's a system of multiple, collaborating agents

Collaborative: Agents work together, using orchestration and workflows to solve a goal no single agent could

Specialized: Each agent has a unique, defined role

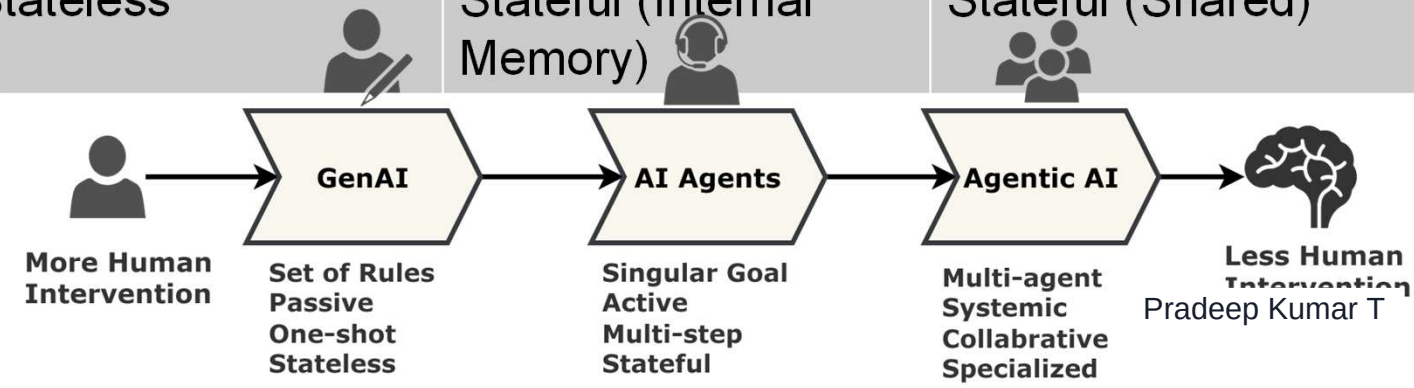
Example: Our E-Shop (Router Agent + Shopping Agent + Support Agent)





The Full Evolution: A Comparison

Feature	Phase 1: Generative AI	Phase 2: AI Agent	Phase 3: Agentic AI
Analogy	The "Expert Consultant"	The "Worker"	The "Factory" / "Team"
Scope	A single "Brain" (LLM)	A single Component	An entire System
Input	Prompt	Goal	Complex Goal
Action	Passive (Answers)	Active (Executes Tools)	Collaborative (Orchestrates)
State	Stateless	Stateful (Internal Memory)	Stateful (Shared)



What is Agentic AI ?
Multi-Agent Architecture

AI Agent vs. Agentic AI: The Critical Difference

What is Agentic Architecture ? (Multi-Agent System (MAS))

How Agentic AI Works: System-Level Architectures

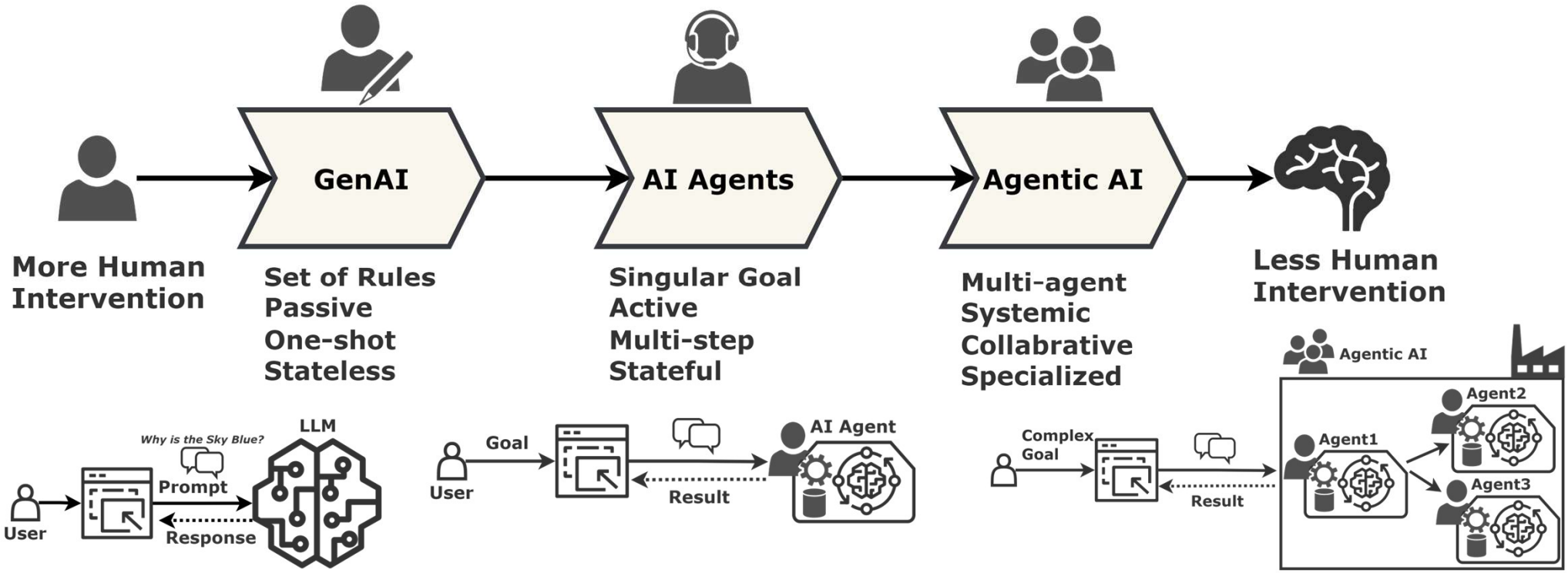
Real-World Examples of Agentic AI

Challenges of Agentic AI Systems

Pradeep Kumar T



Evolving into Agentic AI

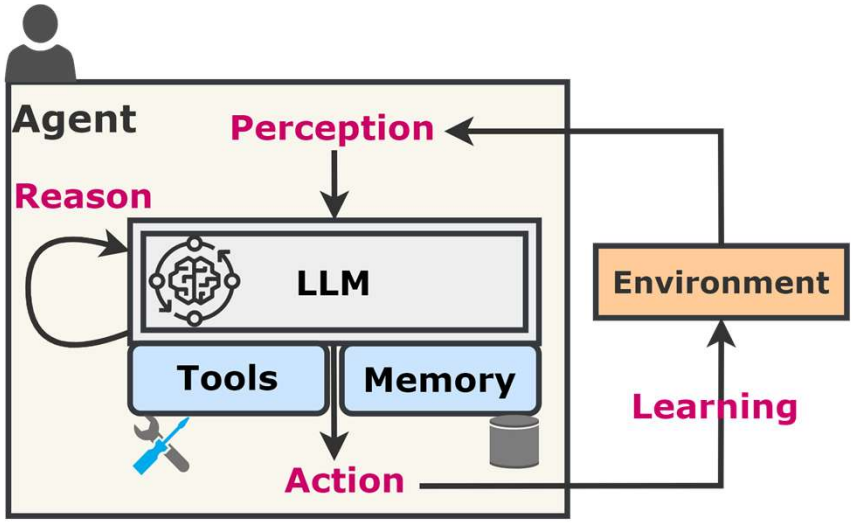


The New Problem: The Limit of the Single "Worker"

Our "Worker" (the AI Agent) is powerful, but it has limits

The New Question:
What happens when a problem is too large or too complex for one agent?
What if the problem is distributed (in different departments)?
What if you don't need one generalist but a team of 10 specialists?

We must **move from building** a **single "worker"** to **designing** the **entire "factory"**



The Definition: Agentic AI (The "Factory")

AI Agent is a...
Component
An actor that can perceive, reason, and act with tools.
The single "Worker"

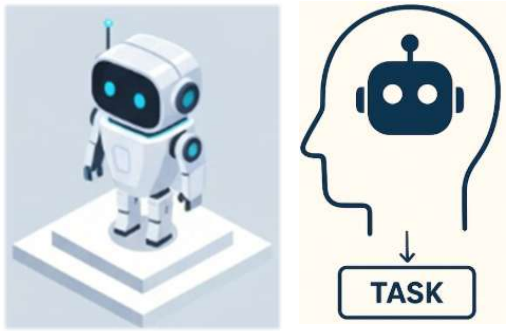


Agentic AI is a...
System
An architecture that contains one or more agents collabo
complex goals
The entire "Factory" or "Company"



Our New Focus: From "Psychology" to "Sociology"

AI Agent - "Agent Psychology"
How does one agent think?
How does it plan?
How does it remember?



Agentic AI - "Agent Sociology"
How do multiple agents interact?
How do they Communicate?
How do they Coordinate their actions?
How do they Collaborate on a shared goal?

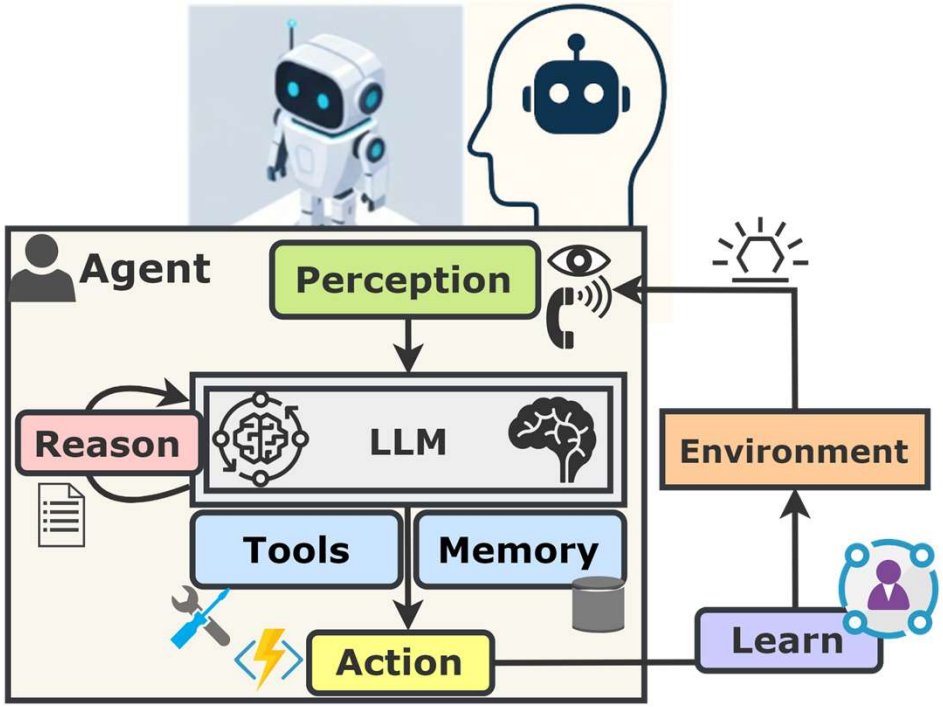


The AI Agent (The "Freelancer")

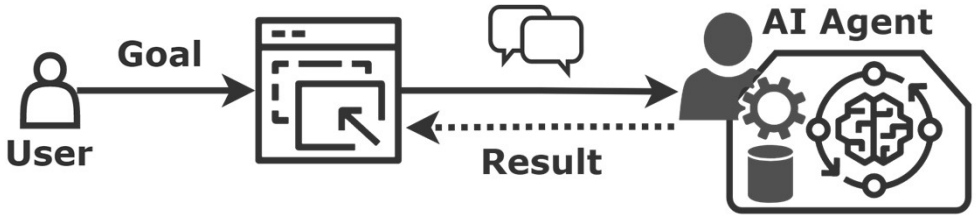
A **single**, **skilled "freelance developer"**

Scope: A Component

Defined By: Its Anatomy
Does it have the right skills?
(The "Brain" - a good LLM)
(The "Memory" - a good notebook)
(The "Tools" - the right APIs & functions)



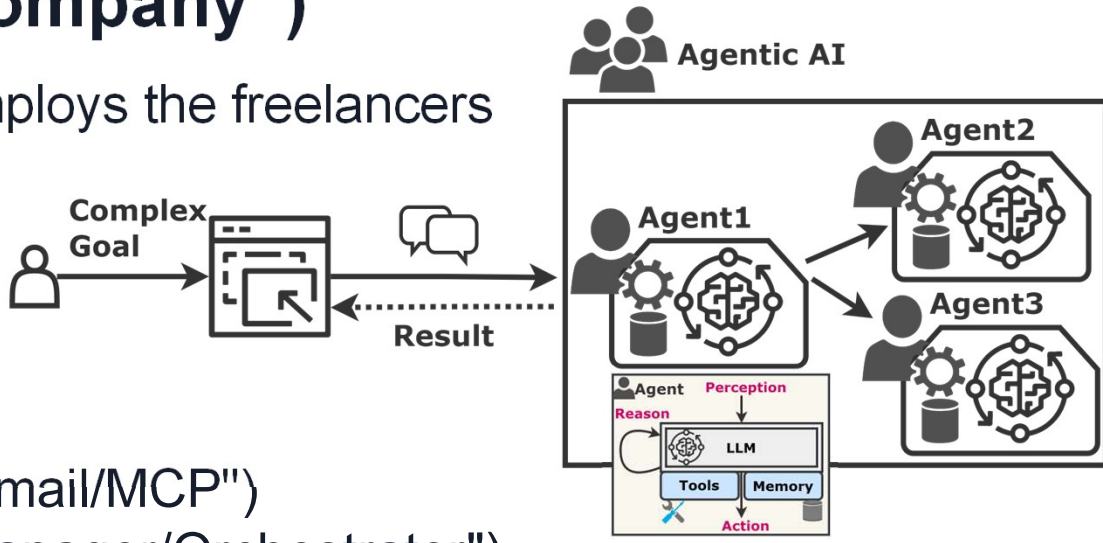
Primary Challenge: Task Execution
Can this one agent, given its tools, complete the task I give it?



Agentic AI (The "Software Company")

The entire **"software company"** that employs the freelancers

Scope: A System



Defined By: Its Architecture

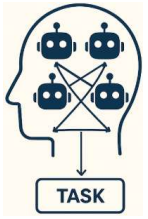
How does the team function?

(The Communication Protocol - "Slack/Email/MCP")

(The Coordination Pattern - "A Project Manager/Orchestrator")

(The Collaboration Goal - "A shared project")



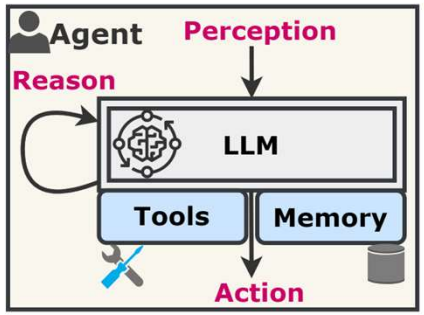


Primary Challenge: System Orchestration

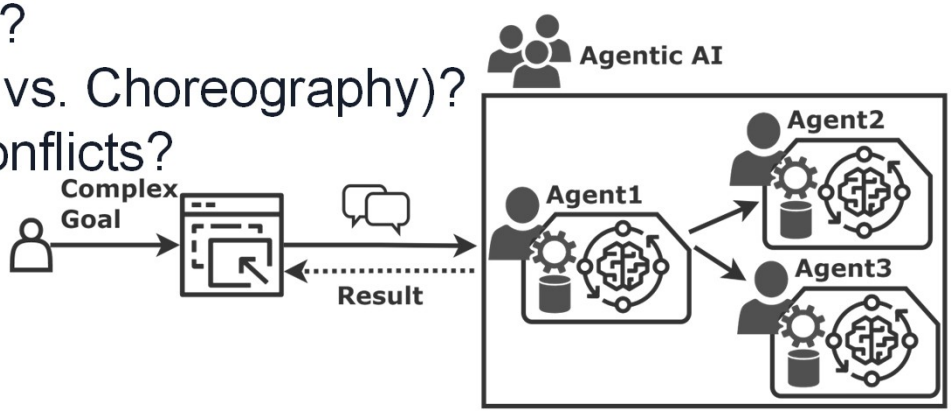
How do my specialized agents interact to solve a problem too big for any one of them?

The Architectural Shift: Why This Distinction Matters

When building an AI AGENT (The Component):
You focus on implementation
What tools do I build?
How good is my System Prompt?
How do I manage its memory?



When building an AGENTIC AI (The System):
You focus on interaction and integration
What is my inter-agent communication protocol?
What is my coordination pattern (Orchestration vs. Choreography)?
How do I manage shared state and resource conflicts?



What is a Multi-Agent System (MAS) ?

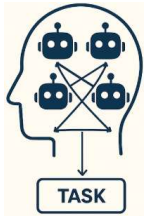


Multi-Agent System (MAS) is a system composed of:

1. Multiple Agents: Two or more autonomous agents

2. A Shared Environment: The 'world' they all operate in (your enterprise data, APIs, a message bus)

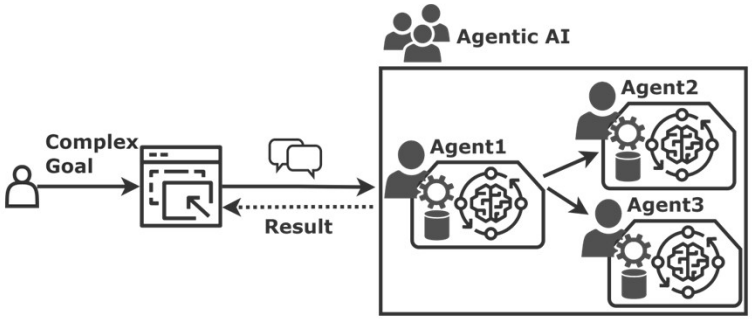
3. A Common Goal: They are designed to collectively solve a problem that is beyond the capability of any single agent



Key Distinction:

Single "Mega-Agent": One brain, many tools

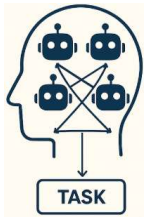
Multi-Agent System: Many specialized brains, each with its own tools (Modular, flexible, resilient)



What is Agentic Architecture ?

A **system design** that **integrates** one or more **autonomous AI agents** with **data sources**, **tools** and **communication** pathways to **achieve complex**, **high-level goals**





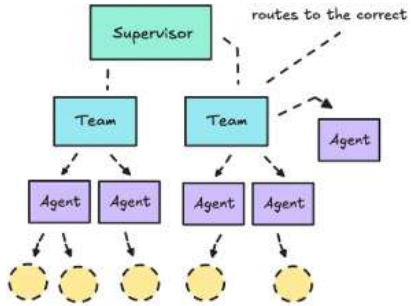
It Answers the "System-Level" Questions:

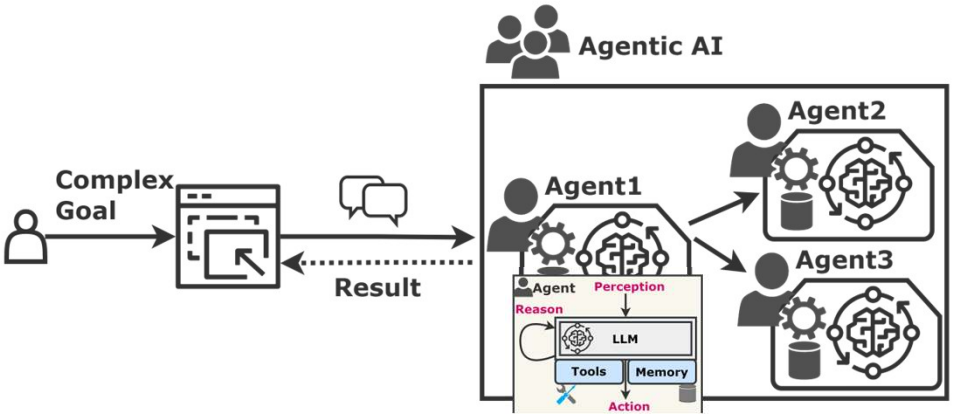
How do agents communicate with each other?

How is the flow of work controlled (Orchestration vs. Choreography)?

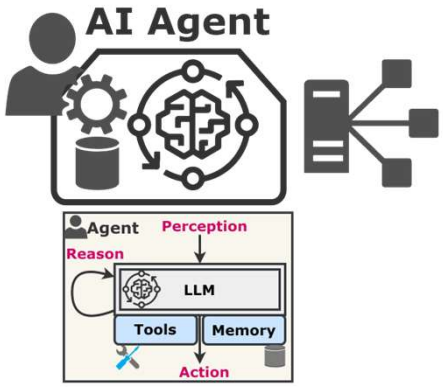
How is state and memory managed at scale?

How does the system integrate with non-agent components (EShop microservices)?





A Core Pattern: "Agent-as-a-Service"

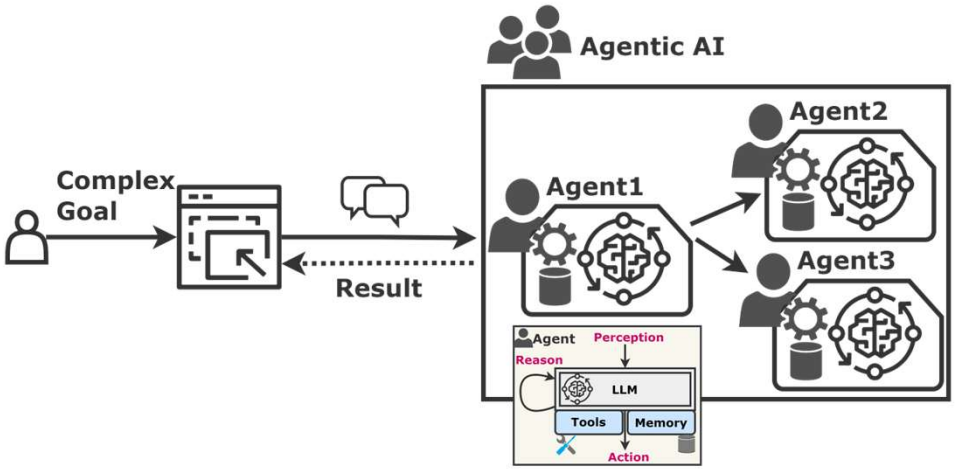


The **most common** and **scalable architectural pattern**

AI Agent is a **long-running**, **stateful** and **addressable service**

Implementation:

Each agent has a unique endpoint (API or a message queue topic)
It maintains its own internal state and memory
It can be "called" by other services (users, or other agents)
This pattern makes agents a first-class citizen in a microservices world



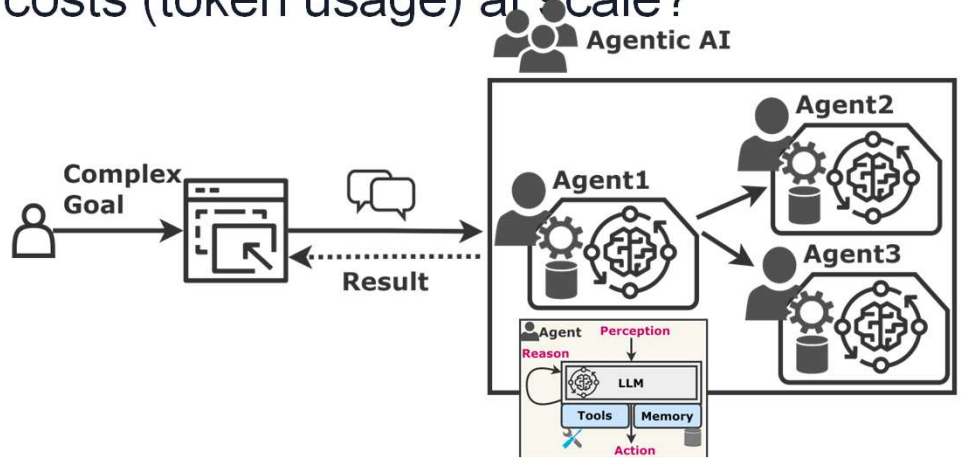
Why Do We Need Formal Architecture?

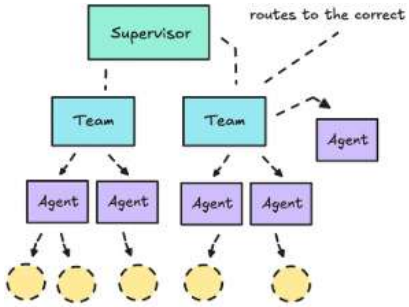
Building one agent is **easy**. **Building** an **enterprise system** of **agents** is **hard**

1. Scalability:
How do you manage 10,000 agents running concurrently?

2. Reliability & Resilience:
What happens if an agent fails mid-task? How does the system recover?

3. Manageability & Governance:
How do you monitor, debug, and secure a non-deterministic agent?
How do you control costs (token usage) at scale?



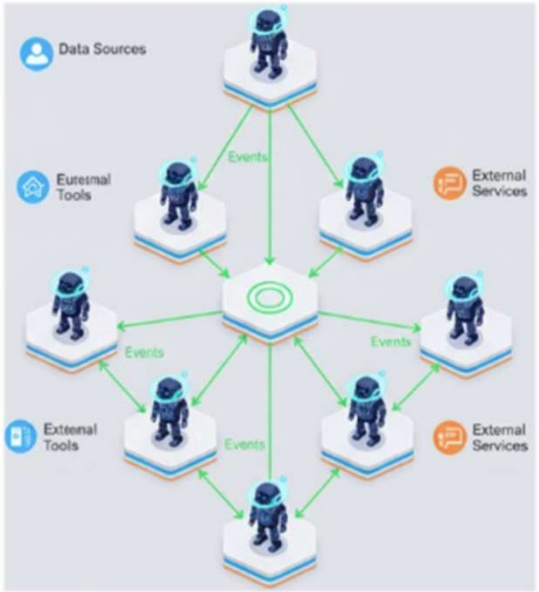


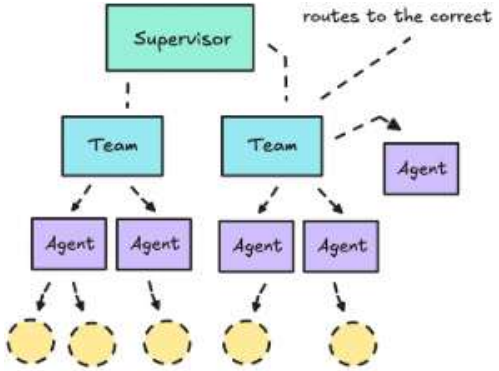
How Agentic AI Works: System-Level Architectures

How? How do we actually **design a system** where **multiple autonomous agents** can **work together effectively?**

2 primary architectural patterns that enable **agent collaboration**





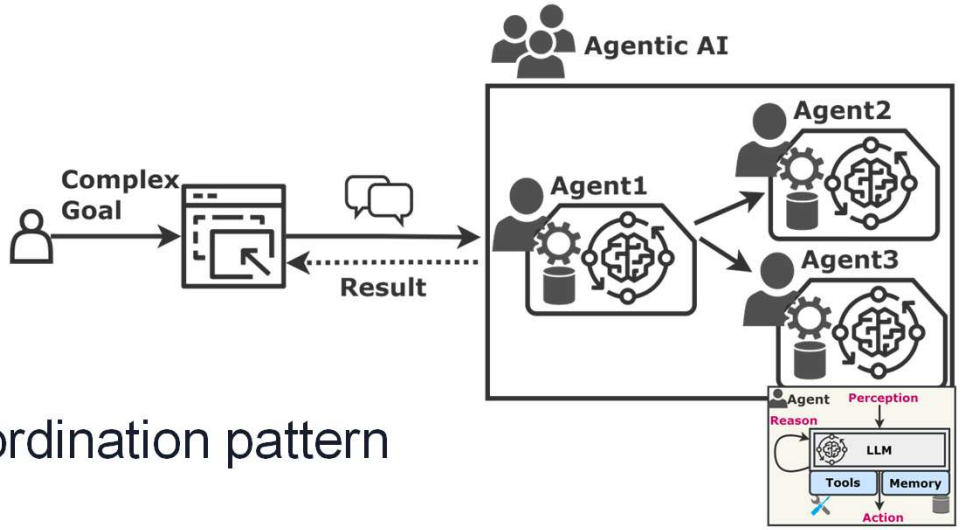


The Core Problem: Coordinating Autonomous Actors

We have **multiple, independent agents**

Each agent has its **own goals** (or sub-goals) and **own tools**

The Challenge: How do we ensure they work together towards a larger system goal without chaos?
How is the overall task decomposed?
How are sub-tasks assigned?
How do agents communicate results?
How do they synchronize their actions?



The Solution: We must implement a clear coordination pattern

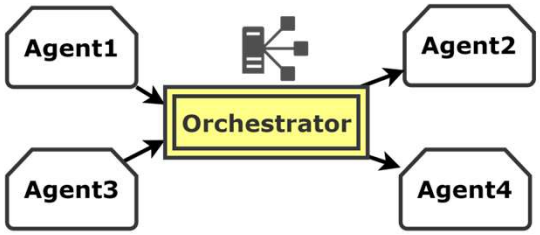
The Two Primary Patterns for Agent Coordination

1. Orchestration (The "Manager" Pattern)

A centralized approach

One agent acts as the "conductor" or "manager"

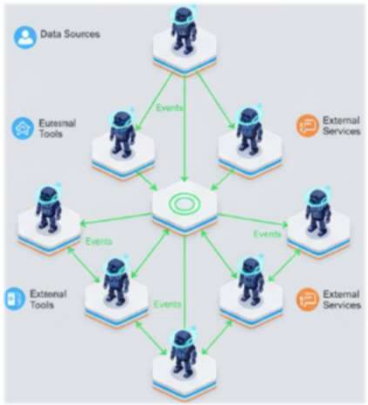


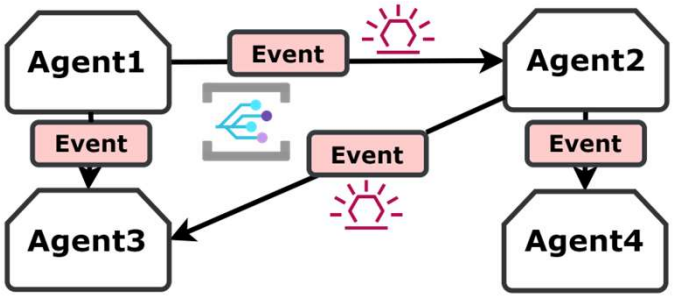


2. Choreography (The "Event Bus" Pattern)

A decentralized approach

Agents react independently to shared events or state changes



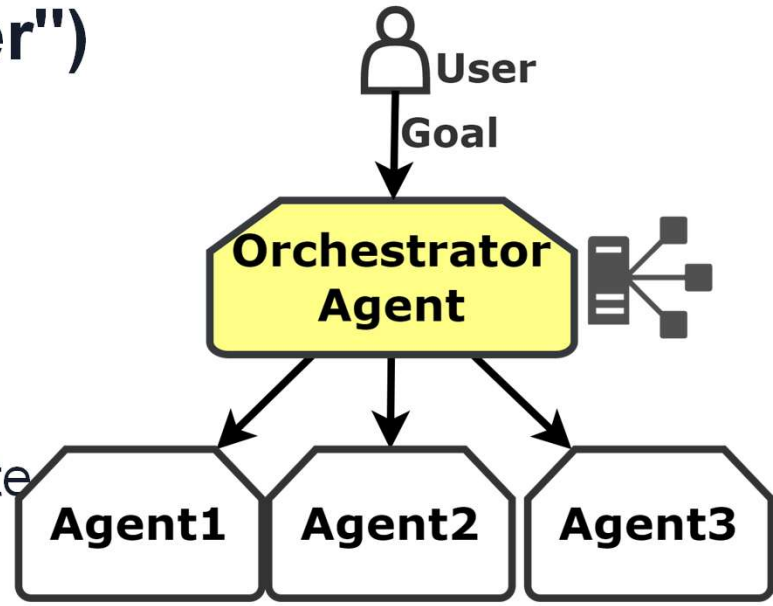


Pattern 1: Orchestration (The "Manager")

A **single**, **central "Orchestrator Agent" manages** the entire workflow

Architecture:
The Orchestrator receives the user's complex goal
It plans the overall sequence of steps
It delegates each step (as a sub-goal) to the appropriate Specialist Agent
It waits for the Specialist to respond
It synthesizes the results and determines the next step

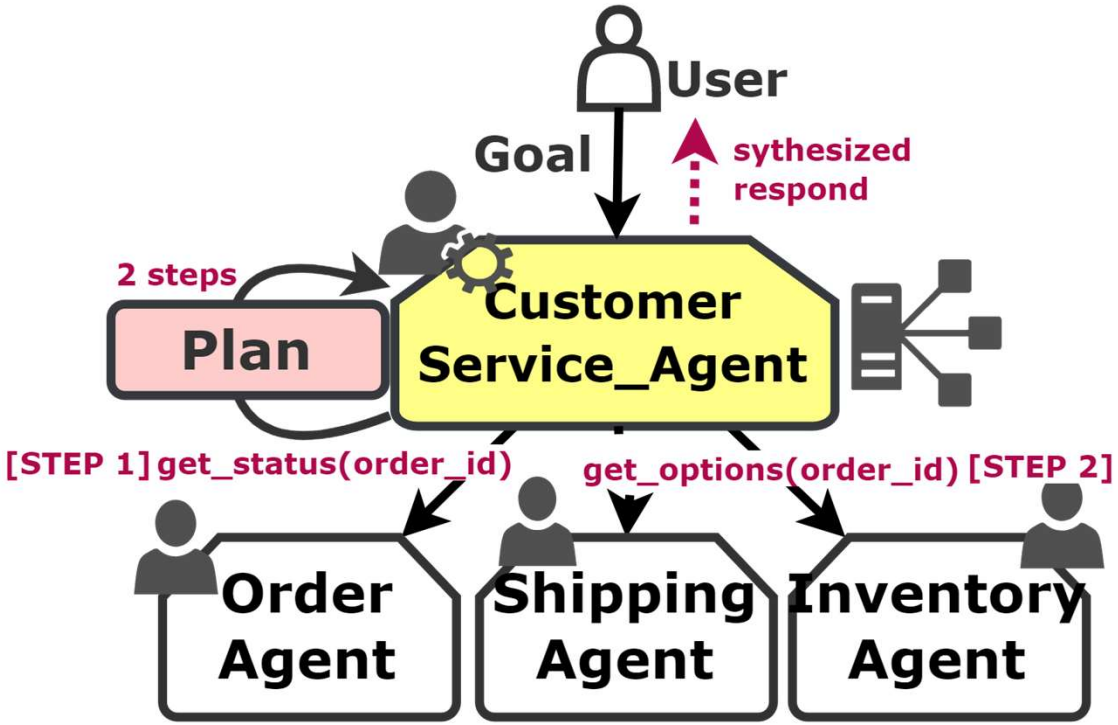
A **project manager assigning tasks** to **team members**





Example: Orchestrated Customer Service Agent

User Goal: What is the status of my recent order and can I add express shipping?



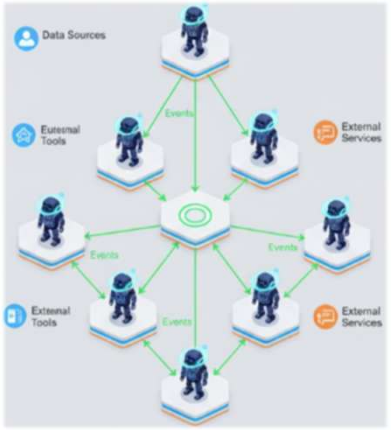
The Service_Agent controls the entire flow

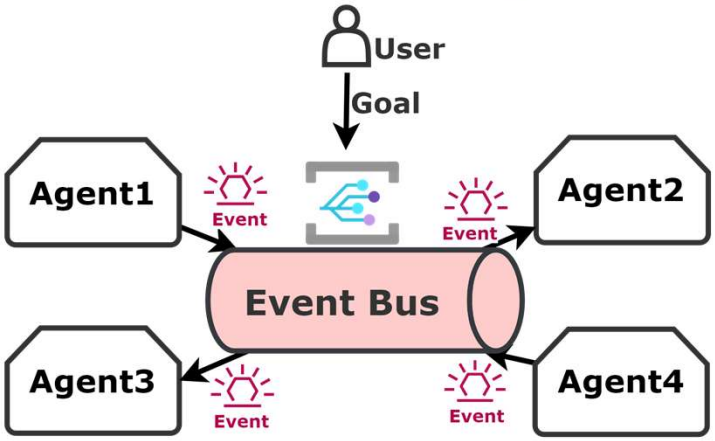
Pattern 2: Choreography (The "Event Bus")

No central manager. Agents are autonomous and reactive

Architecture:

Agents interact indirectly via a shared state or message bus
An agent acts, which publishes an event or changes the state
Other agents are listening for specific events/changes
When an event occurs, the relevant agent reacts autonomously



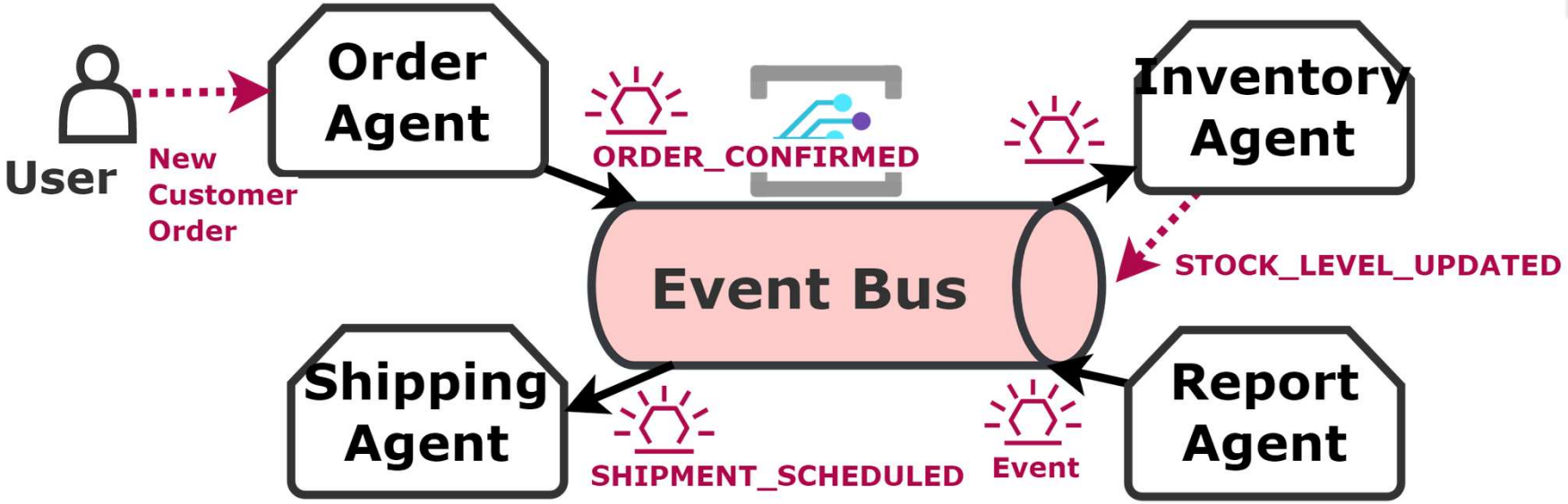


Dancers reacting to the **music** and each other, **without a conductor**



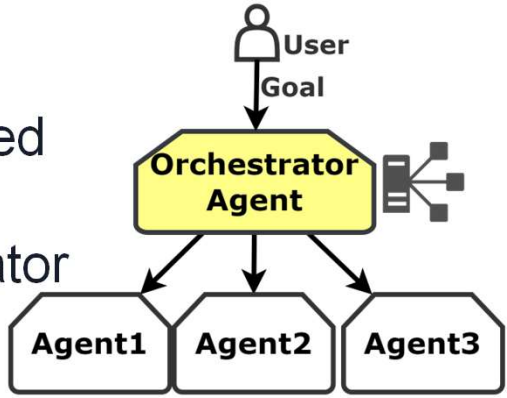
Example: Choreographed Supply Chain

Goal: Fulfill a new customer order



Key: Each agent acts independently based on events

Choosing the Right Pattern: Tradeoffs

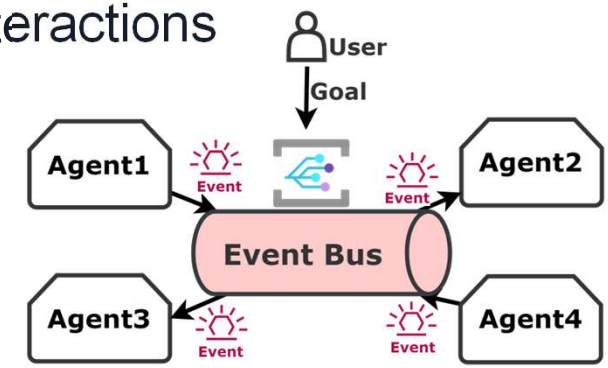


Orchestration (The "Manager"):

+ Simpler to design, understand, and debug. The logic is centralized
- Single point of failure. If the Orchestrator fails, the system stops
- Less flexible. Adding a new step requires changing the Orchestrator

Choreography (The "Event Bus"):

+ More flexible, scalable, and resilient. No single point of failure
- Much harder to debug and monitor. The overall flow is emergent, not explicit
- Risk of "infinite loops" or unexpected interactions

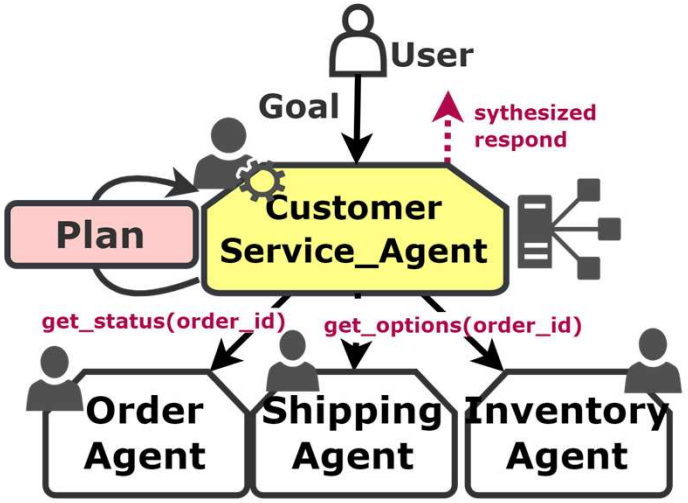


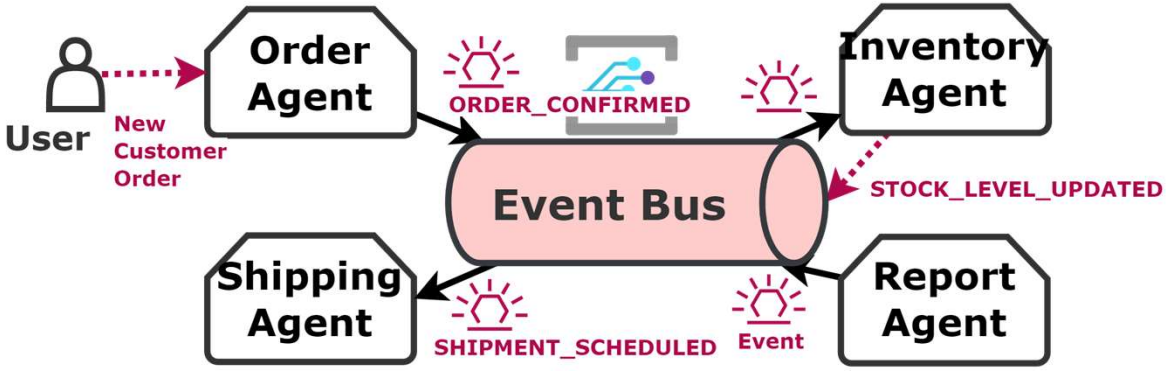
Orchestration and Choreography of Agentic AI

2 fundamental architectural patterns that enable multi-agent collaboration: **Orchestration**, **Choreography**

The **blueprints** for **how Agentic AI works**

We will see exactly **how these patterns** are **applied** to solve **complex problems**





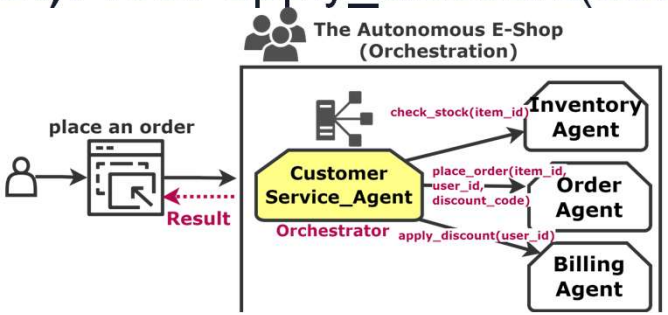
The Autonomous E-Shop (Orchestration)

Scenario: A customer interacts with an online store's support chatbot

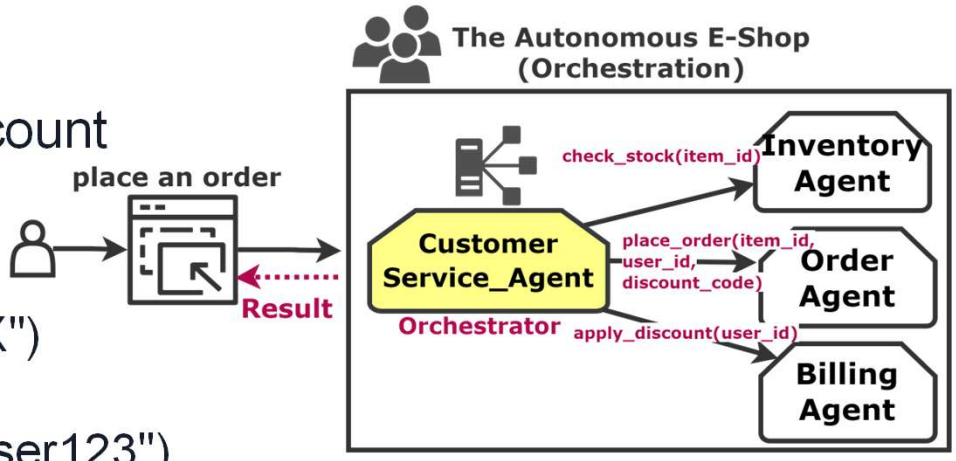
Goal: To handle complex requests beyond simple FAQs
Is item X in stock?
If so, place an order using my saved details and apply my loyalty discount

Architecture: A classic Orchestration pattern

The Agents:
Customer_Service_Agent (The Orchestrator): The only agent the user talks to
Inventory_Agent (Specialist): Tool: check_stock(item_id)
Order_Agent (Specialist): Tool: place_order(item_id, user_id, discount_code)
Billing_Agent (Specialist): Tool: apply_discount(user_id)



E-Shop Orchestration Flow



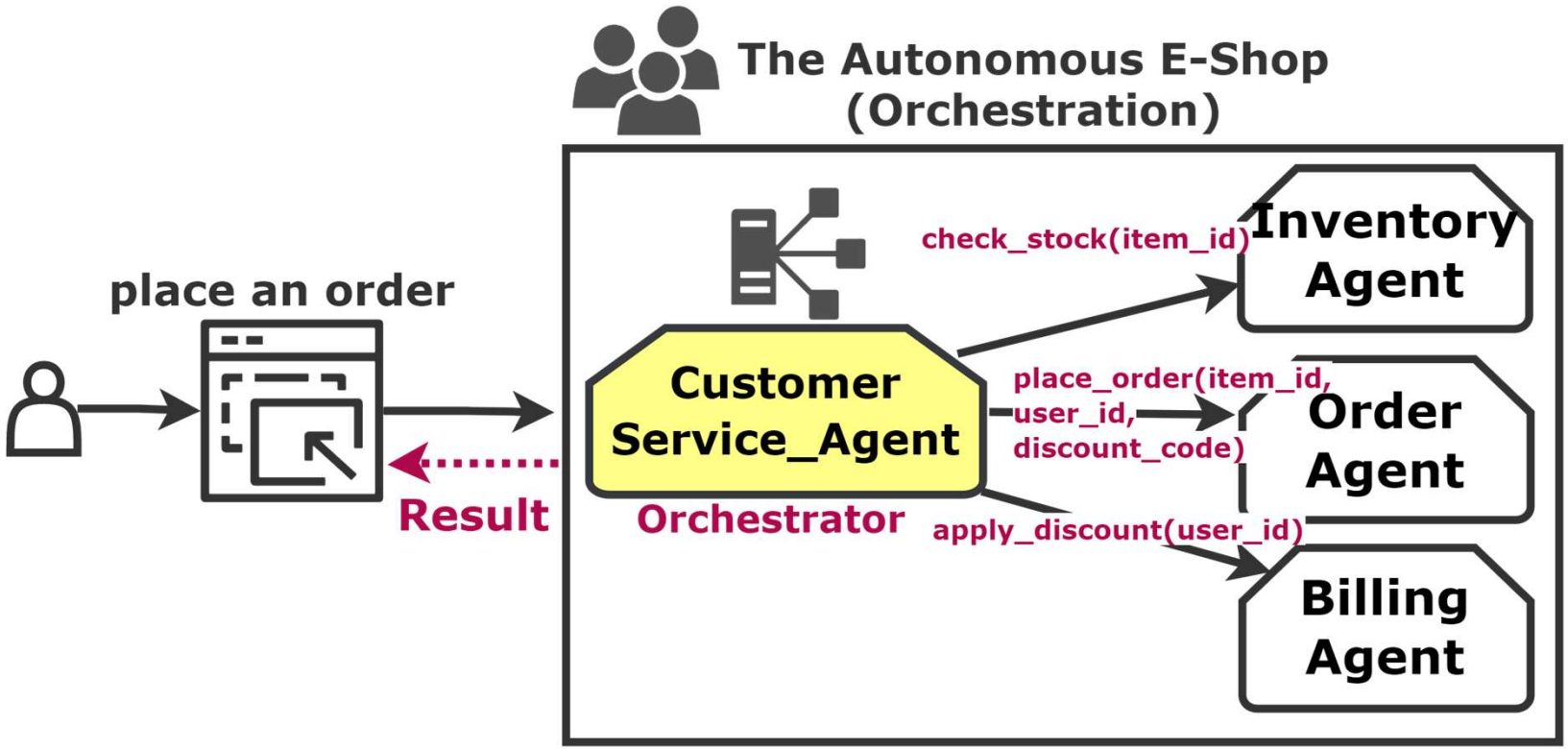
User: Is X in stock? If yes, order it with my discount

Service_Agent (Orchestrator):

1. **Plan:** Check stock, apply discount, place order
2. **Delegate:** Calls Inventory_Agent.check_stock("X")
3. **Receive:** {"stock": true}
4. **Delegate:** Calls Billing_Agent.apply_discount("user123")
5. **Receive:** {"discount_code": "LOYALTY10"}
6. **Delegate:** Calls Order_Agent.place_order("X", "user123", "LOYALTY10")
7. **Receive:** {"order_status": "CONFIRMED"}
8. **Synthesize & Respond:** Tells the user, "Item is in stock and your order is confirmed with the discount applied

Key: The Service_Agent explicitly manages the entire sequence

The Autonomous E-Shop (Orchestration)

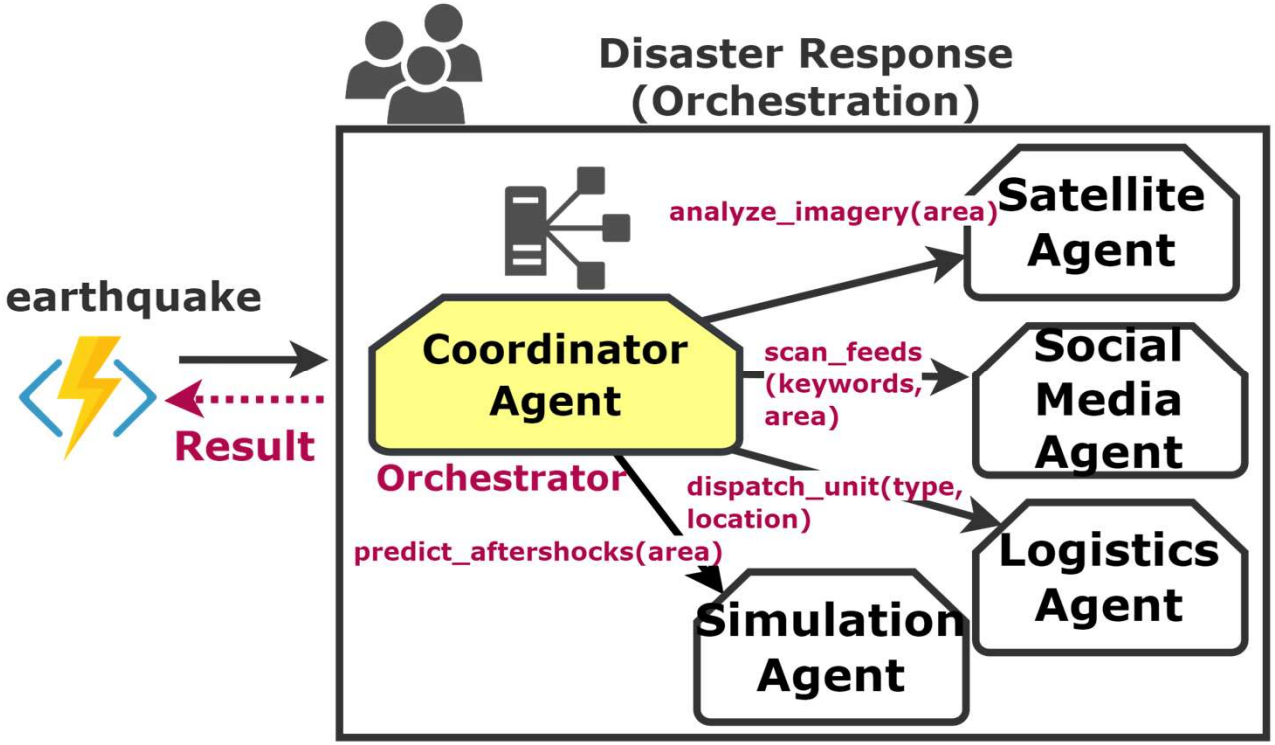


Disaster Response (Orchestration)

Scenario: Coordinating emergency services after a major event (earthquake in London)

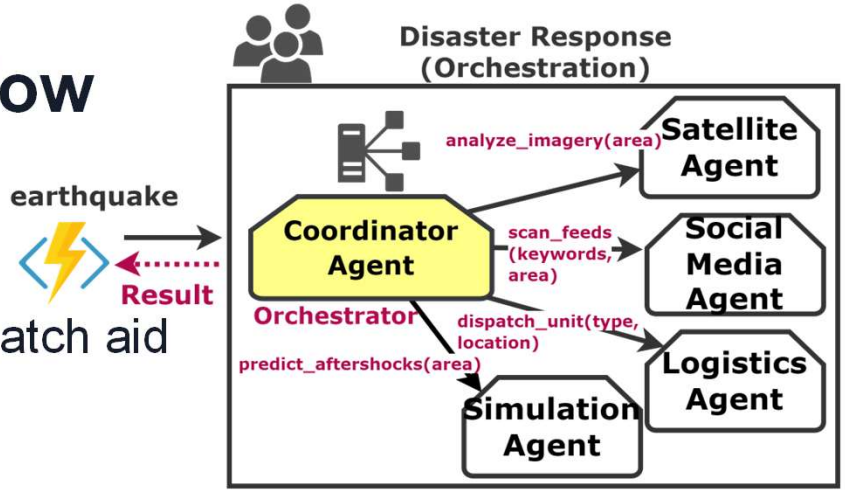
Goal: Assess damage, locate victims, and dispatch aid efficiently

Architecture: A classic Orchestration pattern



Disaster Response Orchestration Flow

Event: Earthquake detected

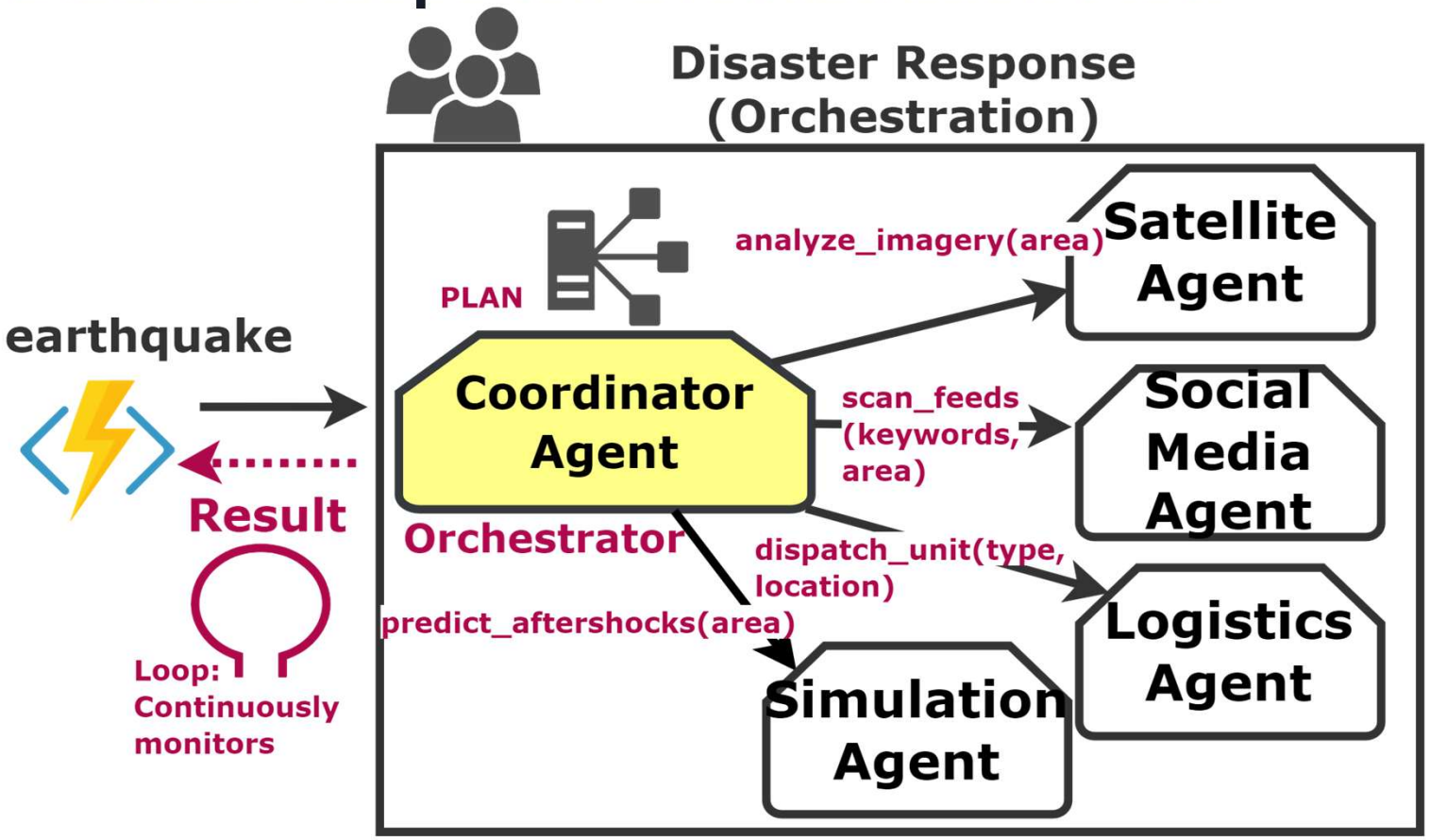


Coordinator_Agent (Orchestrator):

1. **Plan:** Assess damage, find victims, predict risks, dispatch aid
2. **Delegate (Parallel):**
   - Calls Satellite_Agent.analyze_imagery(...)
   - Calls Social_Media_Agent.scan_feeds(...)
   - Calls Simulation_Agent.predict_aftershocks(...)
3. **Receive & Synthesize:** Gets damage reports, victim locations, risk assessments. Updates a shared situational map (Memory)
4. **Prioritize:** Identifies the most critical area based on synthesized data
5. **Delegate:** Calls Logistics_Agent.dispatch_unit("ambulance", location=X)
6. **Loop:** Continuously monitors updates from specialists and re-prioritizes

Key: The Coordinator manages parallel tasks and makes high-level decisions

Disaster Response Orchestration Flow



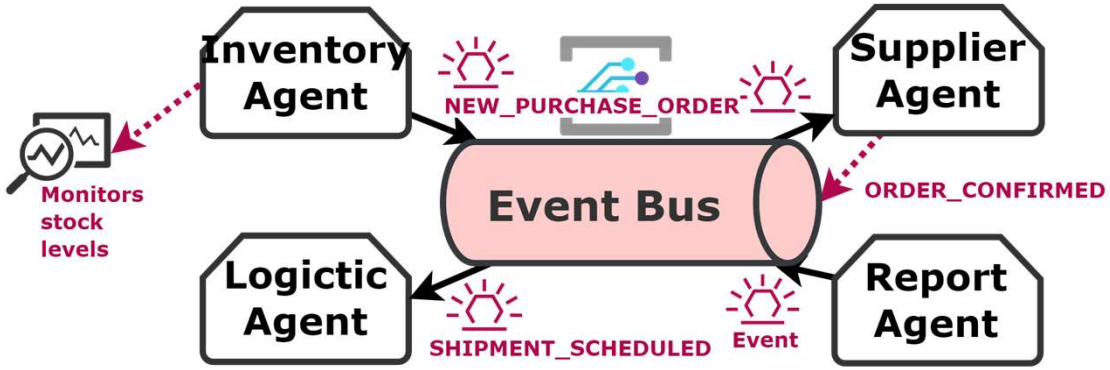
Autonomous Supply Chain (Choreography)

Scenario: Managing inventory and order fulfillment across multiple independent companies (manufacturer, supplier, logistics)

Goal: Ensure products are automatically restocked and shipped efficiently

Architecture: A Choreography pattern is ideal here due to the decentralized nature

The Agents (Peers, No Manager):
Inventory_Agent (Manufacturer): Monitors stock levels
Supplier_Agent (Supplier Company): Receives orders
Logistics_Agent (Shipping Company): Schedules deliveries



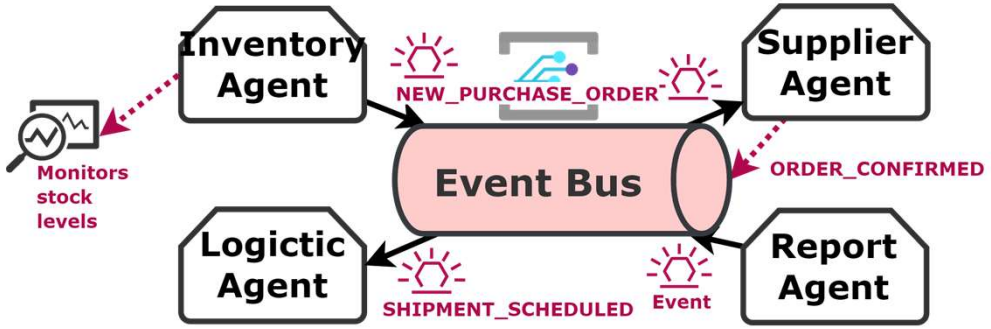
Supply Chain Choreography Flow (Event-Driven)

Event 1: Inventory_Agent perceives stock level for "Widget A" drops below threshold in the database
Action: Creates a purchase order. -> Publishes event: NEW_PURCHASE_ORDER(Widget A)

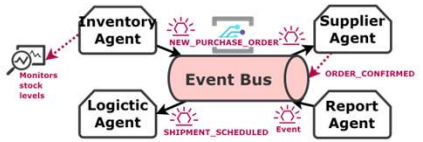
Event 2: Supplier_Agent (listening for NEW_PURCHASE_ORDER events for Widget A) reacts
Action: Confirms the order. -> Publishes event: ORDER_CONFIRMED(Widget A)

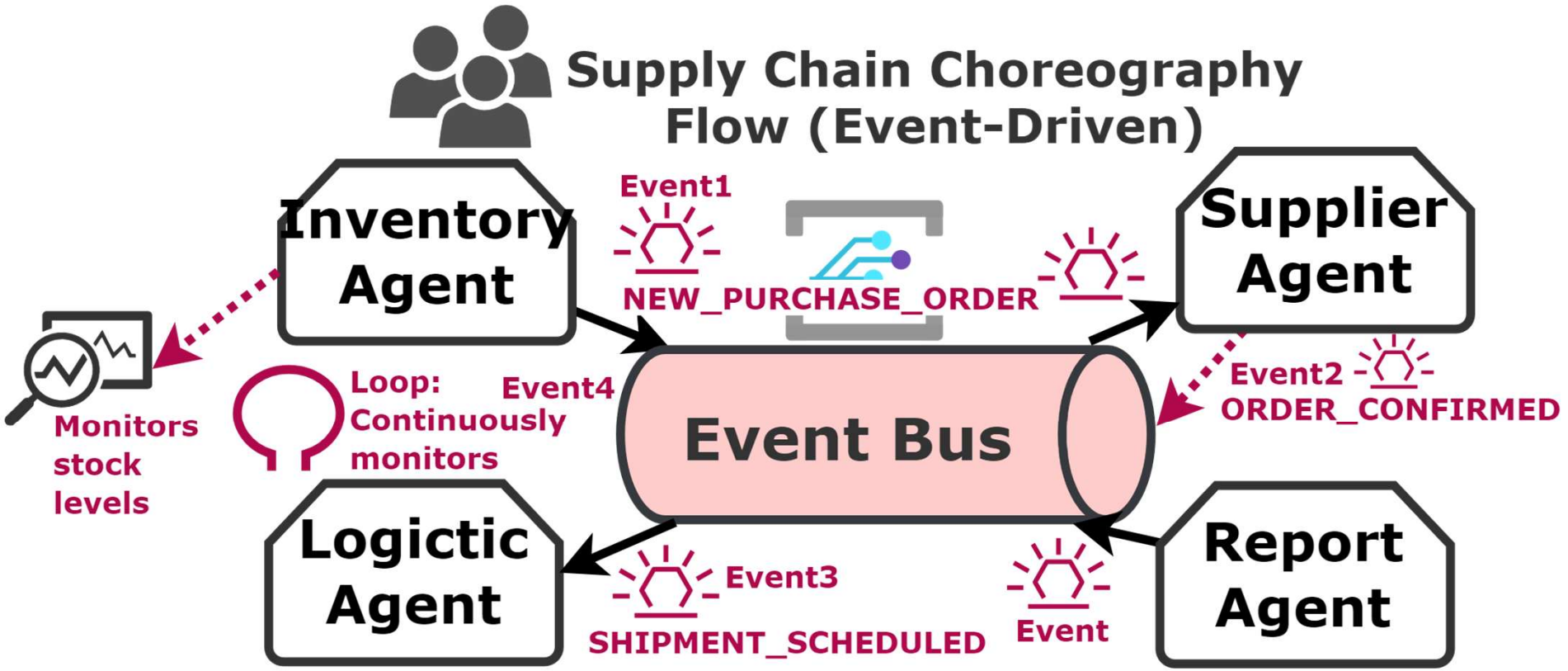
Event 3: Logistics_Agent (listening for ORDER_CONFIRMED events) reacts
Action: Schedules shipment pickup. -> Publishes event: SHIPMENT_SCHEDULED

Event 4: Inventory_Agent (listening for SHIPMENT_SCHEDULED) reacts
Action: Updates internal forecast for arrival



Supply Chain Choreography Flow (Event-Driven)

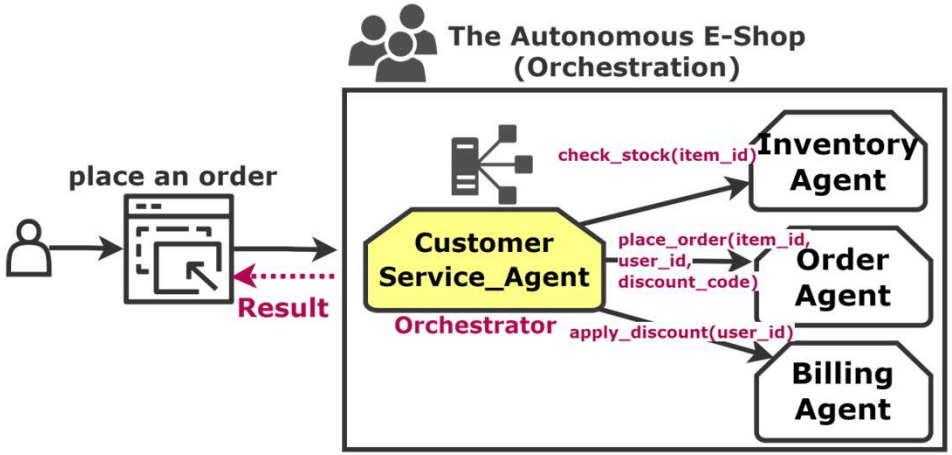


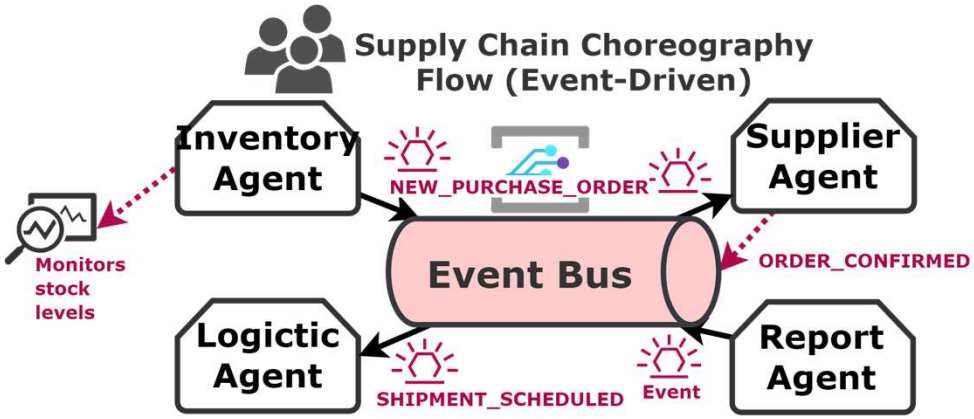


Summary: Real-World Examples of Agentic AI

Orchestration: Ideal for workflows requiring central control, planning, and synthesis (Customer Service, Command & Control)

Choreography: Ideal for decentralized systems where components react independently to events (Supply Chains, IoT ecosystems)



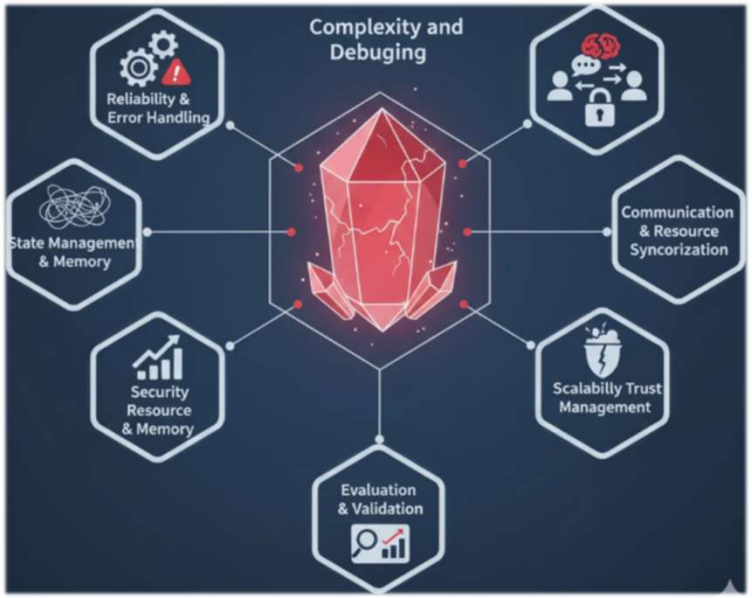


The Hard Parts: Challenges of Agentic AI Systems

We saw how these systems can **coordinate complex tasks** in areas like **e-commerce**, **disaster response** and **supply chain management**

From a **single component** to a **distributed system** introduces an **entirely new class of architectural challenges**

Not just the **single-agent problems** scaled up; these are **fundamentally systemic issues**

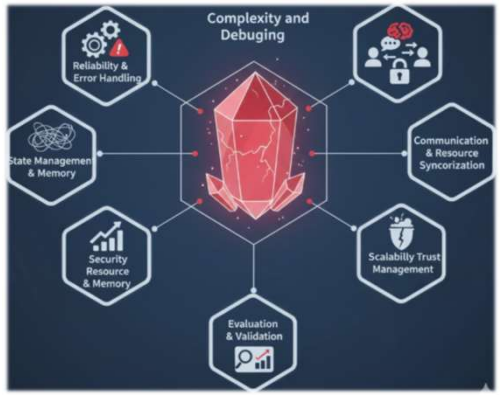


Challenge 1: System-Wide Observability

Single Agent Problem: How do you debug one agent's "thought process"? (Difficult)

Multi-Agent Problem: How do you debug a thought process distributed across five different agents? (Extremely difficult)



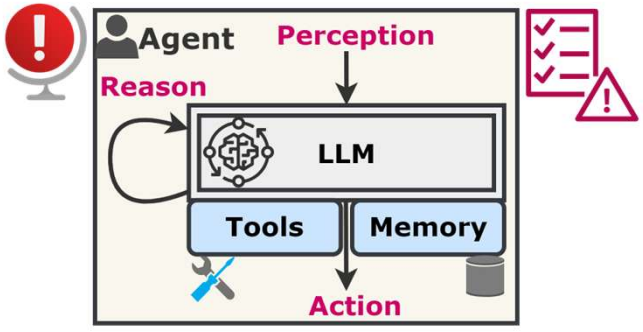


The Core Issue:
A user gets a bad final answer. Why?
Was it Agent 1's bad plan? Agent 2's failed tool call?
Agent 3's misunderstanding of a message from Agent 1?
The "Chain of Thought" is now fragmented across the system

The Architect's Question:
How do we design distributed tracing and centralized logging for agent interactions?

Challenge 2: Emergent Behavior

Complex, unplanned system behaviors that "emerge" from the simple interactions of many individual agents



This is both the power and the danger of Agentic AI

The "Bad" (Risk):
Agents getting stuck in unforeseen feedback loops
Competing agents accidentally "attacking" each other
The system producing bizarre or harmful outcomes that no single agent intended

The Architect's Question:
How do we design for beneficial emergence while preventing harmful emergence? (Testing, simulation, robust guardrails)

Challenge 3: Inter-Agent Communication Failure



Problem: Depends on agents successfully talking to each other
This relies on Communication Protocols (MCP) or API contracts

The Risks:
Protocol Mismatch: Agent A speaks MCP v1.0, but Agent B expects MCP v1.1
Communication fails
API Changes: Agent A expects get_status(order_id), but Agent B changes its "API" to fetch_order_details(id). Agent A breaks
Network Failures: Simple network issues can prevent agents from communicating

The Architect's Question:
How do we design for resilient communication? (Versioning, error handling, retries, circuit breakers)

Challenge 4: Goal Conflict & Resource Allocation

Problem: Multiple autonomous agents might have conflicting goals or need the same limited resource at the same time

Examples:
Two different customer service agents both try to book the last available seat on a flight for their respective customers
A Cost_Saving_Agent wants to turn off a server, while an Uptime_Agent wants to keep it on

This is the classic "**Concurrency Problem**" from distributed systems

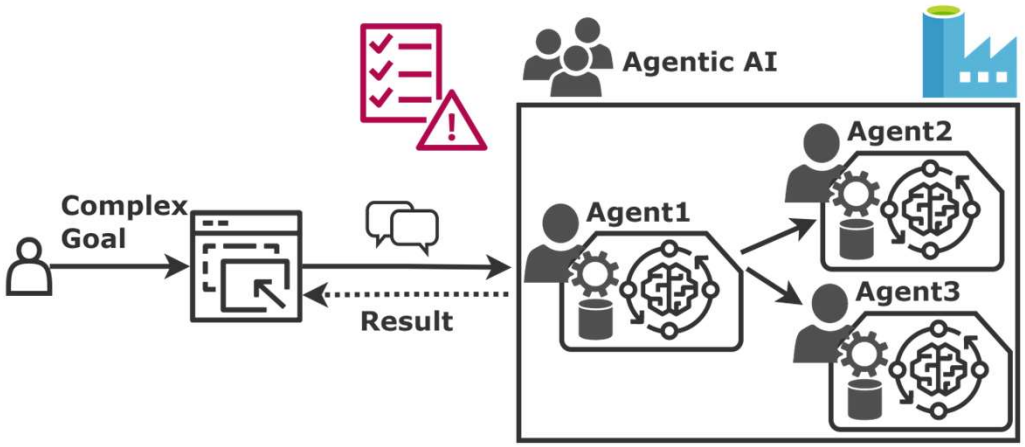
The Architect's Question:
How do we implement mechanisms for negotiation, prioritization, or locking to resolve these conflicts?



Summary: The System is the Challenge

1. **Observability:** Debugging distributed thought
2. **Emergence:** Managing unpredictable system behavior
3. **Communication:** Ensuring reliable inter-agent messaging
4. **Conflict:** Handling resource contention and goal clashes

Solving these is the core of designing robust Agentic AI architectures



What are Agentic Workflows ?

How do Agentic Workflows work?

Agentic Orchestration Patterns (Sequential, Concurrent, Group c
Handoff , Magentic)

Frameworks for Building Workflows (Code-First)

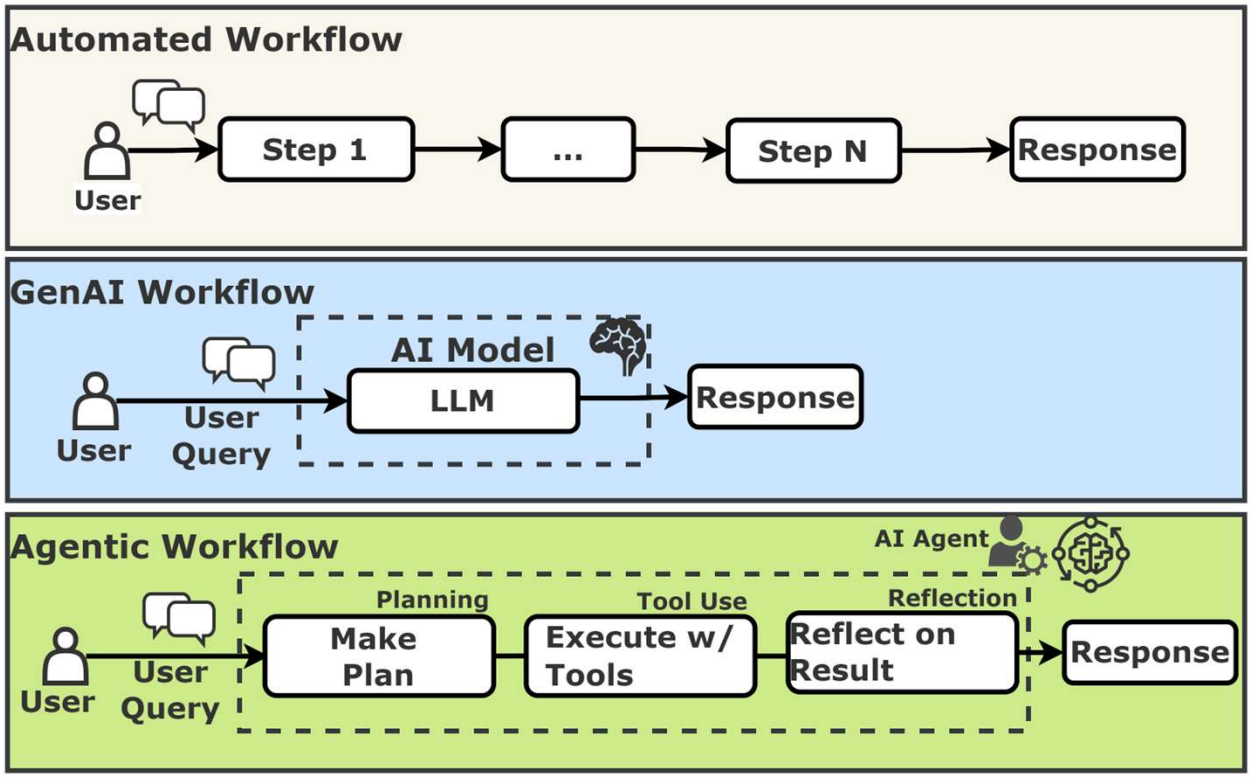
Platforms for Building Workflows (Low-Code)



Pradeep Kumar T

From Traditional to Agentic Workflows

How these **'agentic workflows'** are the key to building truly complex, stateful and reliable agentic apps



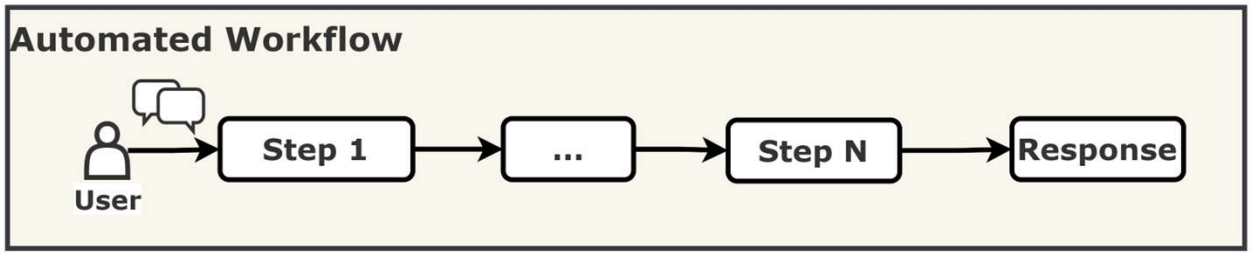
What is a Traditional Workflow ?

A structured, repeatable sequence of tasks to achieve a business goal

The "Flowchart" Model: The path is predefined and static

Examples:
BPMN (Business Process Model and Notation): Formal diagrams for business logic
CI/CD Pipelines: A rigid sequence of build -> test -> deploy
Low-Code Tools (e.g., Zapier): When (Trigger) -> Do (Action)

Key Characteristic: Deterministic. Given the same input, it always follows the same path

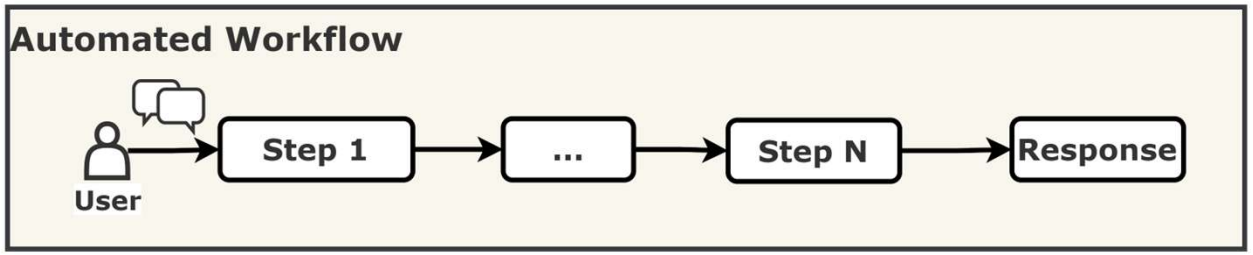


The Limits of Traditional Workflows

What Solve: Reliability, repeatability, and automation of known, predictable tasks

The Core Limitation:
They cannot handle ambiguity or unexpected input
If the input data is slightly different, the workflow breaks

Example (Support Automation):
Workflow: IF email contains "refund" -> route to Billing
Fails on: "My last order was wrong, I want my money back"
The intent is the same, but the keywords are different. The rigid workflow fails

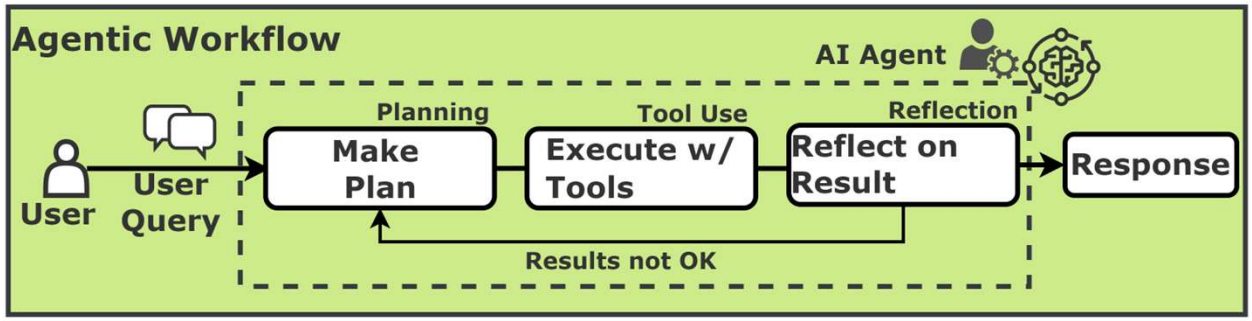


The "New Way": The Agentic Workflow

A **modern workflow architecture** that **uses** an **LLM's reasoning** to dynamically **route tasks** and **manage state** between **specialized agents** and **tools**

The "Project Manager" Model:
It's not a static flowchart
It's an intelligent manager that assesses the situation and decides the next best step

Key Difference:
Traditional: The path is defined by humans in advance
Agentic: The path is decided by an LLM at runtime



Key Characteristics of Agentic Workflows

1. Stateful:
The workflow maintains a "state" or "scratchpad" (memory) that is passed from step to step ({"user_query": "...", "research_data": "..."})

2. LLM-Driven (Reasoning):
The "router" or "logic" is not a static if/else block
It's an LLM call that reasons about the current state to choose the next step

3. Non-Deterministic (Adaptive):
This is the result. Because the LLM is reasoning, the workflow can adapt
It can handle the "money back" email. It can decide to call a different tool. It can even decide to loop and re-try a step

